

OGGM 源码深度解析

Open Global Glacier Model v1.6.3 — 源码、物理与 workflows

硕博研究生

青年冰川学家

资深物理模拟专家

作者信息

作者：徐强强

作者所在机构：中国科学院西北生态环境资源研究院 冰冻圈科学与冻土工程全国重点实验室

作者简介：作者硕博期间主要从事遥感应应用研究。自2022年开始转向冰川变化模拟研究，并在研究过程中持续阅读和使用OGGM，尤其对OGGM源码进行了较为系统和深入的研读。作者最初阅读和使用的版本为OGGM v1.5.3，随后持续跟进至本书基于的OGGM v1.6.3。

关于本书

OGGM (Open Global Glacier Model) 是全球冰川模拟领域重要的开源工具之一。它以RGI冰川清单为入口，提供从局地DEM处理、气候数据准备、物质平衡校准、冰厚反演到冰川动力学正演的完整建模链条，并已广泛用于区域到全球尺度的冰川变化研究。

本书的在线HTML版本可通过GitHub Pages访问：<https://jonhxu.github.io/oggm-source-code-deep-dive/>。在线版本保留章节目录、站内跳转和折叠导航，适合在阅读PDF归档版之外进行快速检索。

然而，OGGM的源码规模已超过55个Python模块、5万行代码，涵盖冰川几何处理、气候降尺度、物质平衡模拟、冰流动力学、并行计算等多个复杂领域。对于初学者而言，直接阅读源码往往缺少入口；对于有经验的研究者，理解某些数值细节和设计决策也需要大量背景知识。

本书正是为填补这一空白而生。

无论你是刚刚入门冰川模拟的硕博研究生、需要冰川变化模拟支持的冰川化学/生物方向青年学者，还是已熟练使用OGGM、PyGEM、GloGEM等模型的资深冰川物理学家，本书都将为你提供从宏观架构到微观实现的系统化说明。

阅读建议

初学者（硕博研究生）：建议从第1-4章开始建立全局认知，然后跟随第5-9章理解预处理管线，最后根据需要深入动力学部分。

青年冰川学家（冰川化学/生物方向）：重点阅读第7-8章（气候与物质平衡）、第15-16章（工作流与CLI），这些内容与冰川变化模拟需求直接相关。

资深专家：可直接跳至第10-14章的冰动力学部分，以及第19-20章的扩展和实验模块。附录B的参数参考手册可作为日常速查工具。

三类读者的最短路线

读者	先读	边读边做	暂时可跳过	完成后应能做到
刚入门硕博	第1章、第4章、第22章教程1–5、附录D	用一条示例冰川跑通预处理、气候、反演	第11、14、17、20章的推导和开发细节	解释GlacierDirectory里每个主要文件从哪里来
交叉方向研究者	第7、8、15、18章，附录C	导出面积、长度、体积、ELA、物质平衡和诊断图	源码元类、多进程序列化、数值格式推导	把OGGM输出接入生态、水文、化学或灾害分析
物理建模专家	第9–14、19–21章，附录B	检查参数、反演假设、SIA适用性和敏感性实验	入门安装与基础文件浏览	判断OGGM结果的物理边界并扩展模型

实操路线

建议把第22章教程作为贯穿全书的练习主线：先完成单条冰川的目录初始化、预处理和物质平衡诊断；再对照第5–9章理解每个输出文件的来源；随后运行流线动力学实验，并用第10–13章解释模型响应。完成这条路线后，读者应能独立修改参数、检查诊断文件、解释反演和正演结果，并为自己的研究区域搭建可重复的OGGM实验。

中国冰川研究者使用提示

高亚洲、天山、祁连山和横断山等区域常见问题包括DEM质量差异、季风降水偏差、碎屑覆盖、冰湖终止、跃动冰川和观测稀缺。初次应用OGGM时，不建议直接把默认参数用于论文结论；应先检查RGI边界、DEM来源、气候校准、反演厚度和诊断图，再进行区域统计或未来情景分析。

本书结构

部分	章节	核心内容	目标读者
一：基础篇	1–4	OGGM概述、架构设计、配置系统、数据中枢	全体读者
二：预处理管线	5–9	GIS/DEM、中心线几何、气候数据、物质平衡、冰厚反演	全体读者
三：冰动力学	10–14	Flowline模型、数值方法、崩解、Spinup、2D SIA	中高级
四：运行 workflow	15–18	工作流管线、CLI、MPI并行、可视化	全体读者
五：扩展专题	19–21	Sandbox实验模块、自定义扩展、测试框架	高级
六：教程专题	22	OGGM教程全集（单独成章）	全体读者
附录	A–D、书末	API参考、参数手册、文件格式、术语表、致谢与版权许可	全体读者

源码分析约定

本书所有源码分析基于 **OGGM v1.6.3**。源码路径引用使用相对于 `oggm/` 包根目录的格式。例如，`core/flowline.py:49` 表示 `oggm/core/flowline.py` 文件的第49行。行号用于帮助定位大致位置；不同安装包、分支或本地修改可能导致行号漂移，因此正式核实时应以函数名、类名和调用链为准。

书中代码分为三类：**可直接运行**、**需要已有冰川目录（GlacierDirectory）或上文变量、概念示意**。第22章会在每个关键片段附近尽量标明依赖条件；源码章节中的伪代码主要帮助理解调用链，不应逐字复制到研究脚本中。

专家标注

带有此样式的段落提供了更深层的物理背景或数值分析细节，适合希望深入理解模型内部机制的读者。初学者可以在第一遍阅读时跳过。

目录

- 第1章 冰川模拟与OGGM概述
- 第2章 架构总览与设计哲学
- 第3章 配置系统深度解析 (cfg.py)
- 第4章 GlacierDirectory: 数据中枢
- 第5章 GIS操作与DEM处理
- 第6章 冰川中心线与几何参数化
- 第7章 气候数据处理与Shop模块
- 第8章 物质平衡模型: 理论与实现
- 第9章 冰厚反演
- 第10章 冰流动力学: Flowline模型
- 第11章 数值方法详解
- 第12章 崩解参数化
- 第13章 动态预热 (dynamic spinup) 与模型初始化
- 第14章 二维SIA模块
- 第15章 OGGM workflow 管线
- 第16章 命令行界面
- 第17章 MPI并行计算
- 第18章 图形与可视化
- 第19章 Sandbox与实验模块
- 第20章 扩展OGGM: 自定义模块
- 第21章 测试框架与质量保障
- 第22章 OGGM教程全集
- 附录A API参考
- 附录B 参数参考手册
- 附录C 文件格式参考
- 附录D 术语表
- 致谢与版权许可

第1章 冰川模拟与OGGM概述

冰川模拟是现代冰冻圈科学的重要组成部分。山地冰川是当前海平面上升的重要来源之一，其未来演变还直接影响区域水资源、冰川灾害风险和气候变化影响评估。本章介绍冰川模拟的基本概念、发展历史，以及开放全球冰川模型（OGGM）在这一领域中的定位。

1.1 什么是冰川模拟

冰川模拟是指利用物理和数学方法，在计算机中对冰川的物质平衡（积累与消融）、冰体流动动力学和几何形态演变进行数值再现。其核心目标是回答三个关键问题：

- 物质平衡：**冰川每年净积累或损失了多少冰？这由降雪（积累）与融化（消融）的差额决定。
- 冰流动：**积累区的冰如何通过内部变形和基底滑动输送到消融区？这受控于冰的流变学性质（Glen定律）和基底条件。
- 几何响应：**冰川的长度、厚度和面积如何随时间调整以适应气候变化？这涉及冰川动力学中的“响应时间”概念，通常为几十年到几个世纪。

入门

冰川模拟为何重要？ 现场测量（如物质平衡观测、冰厚度雷达）只能覆盖全球约25万条冰川中的几百条。模型是将这些点状观测扩展到区域和全球尺度的主要手段。此外，模型允许我们进行“假设分析”——当温度上升2度或4度时，冰川将如何响应？这类反事实推演是IPCC评估报告的核心内容。

1.2 全球冰川模型简史

全球冰川模拟的历史可大致分为三个阶段：

1.2.1 早期经验模型（1980s--2000s）

最早期的全球尺度冰川研究依赖于度日模型（degree-day model）和简单的体积-面积缩放关系（volume-area scaling）。Bahr等人在1997年提出的体积-面积幂律关系 $V = cA^\gamma$ 为从面积推断冰体积提供了理论依据。这类方法的优点是计算成本极低，但缺点是无法模拟冰川动力学（如冰川前进和退缩）。

1.2.2 浅冰近似时代（2010s）

浅冰近似（Shallow Ice Approximation, SIA）的引入标志着全球冰川模拟进入新纪元。SIA是完整Stokes方程在冰川纵横比很小（厚度/长度约为 10^{-3} 量级）的条件下的简化形式，它将冰流动简化为仅由局部地表坡度和冰厚度驱动(layer flow)。虽然SIA忽略了纵向应力梯度，但对于山地冰川（相对于冰盖），SIA在大多数区域是一个合理的近似。

2010年代早期，出现了多个基于SIA的全球冰川演化模型，包括GloGEM（Huss & Hock, 2015）、PyGEM（Rounce et al., 2020）、以及OGGM（Maussion et al., 2019）。

1.2.3 全Stokes与混合方案的探索（2020s--）

近期的发展趋势包括：使用更高阶的冰流动动力学近似（如Blatter-Pattyn近似）、改进入海型冰川的崩解（calving）参数化，以及尝试用统计或机器学习替代模型加速大规模计算。OGGM在v1.4及之后版本中加入了基于Oerlemans & Nick（2005）思路的k-calving方案，并在后续版本中继续改进入海型冰川的反演和正演处理。

进阶

SIA的数学形式：在SIA下，水平冰流速 $u(z)$ 由 Glen 流变定律和剪切应力积分给出。对于等温冰体，深度平均的速度为：

$$u_{\text{bar}} = (2A/(n+2)) * (\rho g)^n * h^{(n+1)} * |\nabla S|^{(n-1)} * \nabla S$$

其中 A 为Glen流变参数 (~2.4e-24 Pa⁻³ s⁻¹), n = 3 为应力指数,
ρ = 900 kg/m³ 为冰密度, g = 9.81 m/s², h 为冰厚度, S 为冰面高程。

OGGM的生产工作流主要使用一维流线模型：在每条流线上用SIA形式计算冰通量，并用多流线结构表示主干与支流的汇合关系。与单一流线相比，多流线方案能更好保留复杂山谷冰川的面积-高度分布和支流贡献。

1.3 OGGM的项目定位与设计目标

OGGM (Open Global Glacier Model) 由Fabien Maussion、Ben Marzeion等人在奥地利因斯布鲁克大学发起，首个公开版本v1.0于2017年发布。其设计哲学可以概括为四个关键词：

- **全球化 (Global applicability)**：OGGM能够处理Randolph Glacier Inventory (RGI)中任意一条冰川，无论其位于阿拉斯加的入海型冰川还是喜马拉雅的山谷冰川。这是通过完全自动化的工作流实现的——只需提供RGI ID即可运行完整模拟。
- **模块化 (Modularity)**：核心处理管线被分解为独立的任务 (tasks)，每个任务负责一个明确定义的步骤（如计算中心线、插值气候数据、求解冰厚度等）。这些任务可以单独运行、测试、替换或重新排序。
- **开源协作 (Open-source)**：OGGM采用BSD-3-Clause许可证托管在GitHub上。项目通过公开 issue、pull request、文档和持续集成 (CI) 维护模型行为的可追踪性。
- **教学友好 (Education-friendly)**：OGGM提供了丰富的教程 (notebooks)，配有面向教学的预处理数据集，使冰川学研究生能够在个人电脑上完成单冰川或小区域模拟实验。

入门

提示：开始使用OGGM

如果你是冰川模拟的新手，推荐从OGGM官方教程开始：`pip install oggm` 后在Jupyter Notebook中运行 `import oggm`。OGGM提供了三个难度级别的教程：Beginner（使用预处理的示例冰川）、Intermediate（理解每条工作流步骤）和Advanced（自定义参数和开发新功能）。

1.4 OGGM与同类模型的对比

下表总结了OGGM与当前其他主流全球冰川模型的区别：

特征	OGGM v1.6	PyGEM	GloGEM	JULES-ICE
冰流动模型	多流线SIA	无（仅物质平衡）	简单参数化（delta-h）	单流线SIA
物质平衡	月度温度指数模型 (MonthlyTModel, temperature-index model, 可含辐射项)	月度温度指数模型 (MonthlyTModel, temperature-index model)	月度温度指数模型 (MonthlyTModel, temperature-index model)	能量平衡
冰厚度反演	物理反演（逆SIA）	体积-面积缩放	体积-面积缩放	物理反演
冰川崩解	有（Oerlemans + 厚度判据）	无	无	无
自动预处理	全自动（DEM/RGI/气候数据工作流）	全自动	半自动	依赖外部驱动
编程语言	Python（开源）	Python（开源）	MATLAB（闭源）	Fortran（开源）

特征	OGGM v1.6	PyGEM	GloGEM	JULES-ICE
典型用途	从单条冰川到全球模拟	全球物质平衡与径流	全球冰川演化	陆面模型中的冰川分量
RGI v6 全量运行时间	~数天（并行，256核）	~数小时	~数小时	~数天

关键区别

OGGM的主要差异化优势在于其**流程一致性**：从地形预处理、物质平衡校准、冰厚反演到动力演化，关键中间量都保存在同一套GlacierDirectory结构中，便于追踪和复现实验。需要注意的是，OGGM可以分别设置反演和正演使用的Glen参数；是否保持一致取决于参数配置。基于体积-面积缩放或delta-h参数化的模型计算更快，但对冰川前进、退缩和崩解等瞬态动力过程的表达更受限制。

1.5 OGGM的科学贡献

自发布以来，OGGM已在多个重要科学评估中发挥核心作用：

- IPCC第六次评估报告（AR6, 2021）**：OGGM参与了全球冰川模型集合预估，为AR6中冰川对海平面上升贡献的评估提供了模型证据之一。不同排放情景下的体积损失范围取决于基准年、区域分组和模型集合，阅读原始评估时应避免把单一模型结果误读为确定值。
- 全球冰川变化（GlaMBIE, 2024）**：在由21个研究团队参与的"冰川物质平衡比对实验"（Glacier Mass Balance Intercomparison Exercise）中，OGGM是核心驱动模型之一，为2000–2023年间冰川质量变化的社区评估提供了关键约束。
- 水文灾害评估**：OGGM被用于评估安第斯山脉、高亚洲和阿尔卑斯山地区的冰川湖溃决洪水（GLOF）风险，对下游数百万人口的防灾减灾规划具有直接指导意义。

1.6 关键出版物

以下是使用或引用OGGM的标志性论文：

论文	期刊	核心贡献
Maussion et al. (2019)	Geosci. Model Dev.	OGGM v1.0 模型描述论文
Rounce et al. (2023)	Science	全球冰川质量损失预估（含OGGM）
Marzeion et al. (2020)	J. Glaciol.	OGGM v1.2 冰厚度反演验证
Eis et al. (2021)	Cryosphere	OGGM预处理管线与W5E5气候驱动
Zekollari et al. (2022)	Cryosphere	OGGM v1.6 冰川崩解参数化
Schuster et al. (2023)	Geosci. Model Dev.	OGGM-Edu：面向教学的简化版

1.7 OGGM版本历史

OGGM的版本演进反映了其功能的逐步完善：

版本	发布日期	主要变更
v1.0	2019-02	初始稳定版：中心线计算、气候插值、冰厚度反演、动力演化
v1.1	2019-09	改进的DEM插值、并行处理支持
v1.2	2020-04	冰厚度反演改进、交叉验证框架
v1.3	2020-11	W5E5气候驱动集成、南半球支持
v1.4	2021-09	冰川崩解模块、月气候强迫
v1.5	2022-06	动态融化模型改进、区域气候模型接口
v1.6	2023-04	改进的冰川崩解判据、elevation-band flowline替代方案
v1.6.3	2024-12	最新维护版本：bug修复、GMD气候驱动更新

进阶

版本迁移注意事项

如果你正在维护依赖OGGM的旧代码，请注意不同版本的prepro目录、参数默认值和文件名可能不兼容。
`grid_dx_method`、气候校准策略、RGI版本等设置都会影响GlacierDirectory的内容。建议在迁移前备份已处理数据，并用目标版本重新运行完整预处理。

1.8 RGI：OGGM的数据基础

Randolph Glacier Inventory (RGI) 是全球冰川模拟的共同基石。RGI第一版于2012年发布，最新版为RGI 7.0（2023年发布），包含了全球约27.5万条冰川的轮廓（outline）数据，覆盖除南极和格陵兰冰盖外的所有冰川区域。

对于每条冰川，RGI至少提供：

- **RGI ID**：形如 `RGI60-11.00001` 的唯一标识符，前两位为区域代码
- **几何信息**：冰川轮廓多边形，以WGS84经纬度坐标存储
- **属性数据**：冰川名称、面积、最高/最低高程、连接关系等

OGGM的工作流从RGI出发，通过以下步骤构建每条冰川的可复现实例：提取轮廓 → 获取DEM → 计算中心线 → 沿流线参数化 → 气候数据插值 → 冰厚度反演 → 动力演化。RGI是这个流程的入口数据，也是后续几何约束的基础。

中国与高亚洲应用的第一检查项

在青藏高原、天山、祁连山、横断山等区域使用OGGM时，应先检查RGI边界、DEM来源、冰湖/湖泊终止类型、碎屑覆盖和跃动冰川清单。许多误差并非来自SIA求解器，而是来自输入几何、降水订正和物质平衡校准。区域研究建议先做小样本人工诊断，再扩大到全区域批处理。

专家

RGI版本差异对模型结果的影响

RGI 6.0与RGI 7.0之间存在显著差异：RGI 7.0修正了约15%的冰川面积和约8%的分水岭归属。研究模拟表明，简单地将RGI 6.0训练的模型参数直接应用于RGI 7.0的边界数据，会导致全球冰体积估算产生约5-10%的偏差。因此OGGM为RGI 6.0和RGI 7.0分别提供了独立的预处理数据包。本书分析主要基于OGGM v1.6.x配合RGI 6.0的使用场景。

1.9 本书的结构与阅读指南

本书按"由外而内、由总到分"的原则组织，分为六个部分：

- 第一部分（第1–4章，架构概览）：介绍OGGM的整体架构、配置系统和数据中枢，为后续深入源码奠定基础。
- 第二部分（第5–9章，预处理管线）：逐一剖析GIS/DEM、中心线、气候数据、物质平衡模型和冰厚反演。
- 第三部分（第10–14章，冰动力学）：深入流线模型、数值方法、崩解、动态预热（dynamic spinup）和二维SIA模块。
- 第四至第六部分（第15–22章与附录）：介绍工作流、CLI、并行计算、可视化、扩展开发、测试框架、教程和速查资料。

每章开头使用标签标记内容的受众：入门 适合冰川研究新生（研究生）；进阶 适合有一定建模经验的冰川学家；专家 适合希望深入修改源码的高级用户。

第2章 架构总览与设计哲学

理解一个复杂科学软件的架构，是深入源码阅读的前提。OGGM的代码库约含10万行Python代码，分布在数十个模块中。本章从宏观层面解构OGGM的架构设计，重点分析其核心设计模式——特别是 `@entity_task` 和 `@global_task` 装饰器——以及数据如何在各模块间流动。

2.1 高层架构图

下图展示了OGGM的主要模块及其依赖关系。箭头表示"依赖/导入"方向（ $A \leftarrow B$ 表示 B 导入 A）：



可以看到，依赖关系遵循清晰的分层结构：`cfg`（配置层）是最底层；`utils`（工具层）依赖它；`core`（核心算法）和 `shop`（扩展模块）依赖 `utils`；`workflow`（工作流层）编排 `core` 中的任务；`cli` 和 `sandbox` 则位于最上层，面向最终用户。

入门

源码阅读顺序建议

如果你将OGGM源码阅读视为一项工程活动，推荐的阅读顺序为：（1）`oggm/cfg.py` — 理解配置系统（约600行，1小时）；（2）`oggm/utils/_workflow.py` — 理解 `entity_task` 装饰器和工作流机制（约600行中前300行）；（3）`oggm/utils/g2ti.py` — `GlacierDirectory`类（约1400行，可先读构造函数和关键方法）；（4）`oggm/core/gis.py` — 中心线计算（约1000行）——这是pipeline的第一个实体任务（entity task）。后续各章的安排即按此顺序展开。

2.2 entity_task 装饰器：OGGM的核心设计模式

`@entity_task` 是OGGM架构中最关键的抽象。它位于 `oggm/utils/_workflow.py` 第422至530行，是一个Python装饰器，将普通函数包装为可处理单一冰川实体的“任务”。其核心代码签名如下：

```
def entity_task(task_func, *, writes_output=None, base_version=None,
                log_level='INFO', ignore_errors=False, operation='entity_task'):
    # 包装器实现（约100行）
    def deco(task_function):
        # 1. 提取任务名称（函数名）
        # 2. 设置 base_version 用于版本兼容性检查
        # 3. 包装为 _entity_task_wrapper
        return new_func
    return deco
```

当一个函数被 `@entity_task` 装饰后，它自动获得以下能力：

- 自动跳过（Auto-skip）**：如果任务的所有输出文件都已存在且未过期，任务自动跳过。这由 `_entity_task_wrapper` 中的 `_auto_skip_task()` 函数实现（`_workflow.py:340–420`）。检查逻辑是：遍历任务通过 `writes_output` 或 `@task_output` 声明的输出文件列表，如果全部存在则跳过执行。
- 日志记录**：每次任务执行的时间戳、持续时间、成功/失败状态自动写入 `GlacierDirectory` 的 `task_status.json` 日志文件。如果任务失败，完整的traceback也会被记录。
- 错误处理**：支持 `ignore_errors=True` 模式（默认），即任务失败时不抛出异常，而是记录错误日志并标记任务状态为失败，让后续任务继续处理其他冰川。也支持 `ignore_errors=False` 模式，失败时立即终止。
- 超时控制**：通过 `mp_pool_timeout_task` 参数，可以为每个任务设置最大运行时间（以秒为单位），防止个别异常冰川长时间占用计算资源。
- 版本兼容性**：`base_version` 机制确保旧版本代码不会处理新版本代码创建的预处理器数据，避免难以调试的隐蔽错误。

典型使用模式（以 `gis_prepro_tasks` 中的 `compute_centerlines` 为例）：

```
@entity_task(log, writes_output=['centerlines', 'glacier_grid',
                                'hypsometry'])
def compute_centerlines(gdir, div_id=0, min_m=None,
                        single_fl=None, **kwargs):
    # 函数体内只需关注冰川物理逻辑
    # 跳过、日志、错误处理全部由装饰器负责
    ...
```

专家

entity_task 内部的自动跳过机制源码剖析

自动跳过逻辑的核心位于 `oggm/utils/_workflow.py` 的 `_auto_skip_task()` 函数（约第340行）。它执行以下步骤：

1. 调用 `reset_task_status(gdir, task_name)` 清除旧状态
2. 从 `writes_output` 参数（或函数中 `task_output` 列表）获取输出文件名列表
3. 对每个输出文件名，调用 `gdir.has_file(filename)` 检查文件是否存在
4. 如果所有输出文件都存在，设置 `gdir.task_status[task_name] = 'skipped'` 并返回 `True`
5. 如果任一文件缺失，返回 `False`，任务将实际执行

这个机制的意义在于：当处理1万条冰川时，如果某些步骤失败需要重新运行，已成功处理的冰川会被自动跳过而无需额外逻辑。这极大地简化了大规模并行作业的重启流程。

2.3 global_task 装饰器

`@global_task`（位于 `_workflow.py` 第533–566行）是 `@entity_task` 的“全局”变体。它用于处理不针对单一冰川的全系统操作，例如：

- 编译所有冰川的物质平衡统计信息
- 生成全局的交叉验证汇总表
- 创建区域汇总的shapefile

与 `entity_task` 不同的是，`global_task` 的签名不要求 `gdir` 作为第一个参数，且其自动跳过逻辑基于单个全局日志文件而非每个冰川的任务状态。其实现更为简单，约30行代码。

```
def global_task(log, task_func=None, *, operation='global_task',
               writes_output=None):
    # 简化版 entity_task:
    # - 不绑定 gdir
    # - 跳过逻辑基于 writes_output 中的文件是否存在
    # - 日志写入全局文件（而非 per-gdir）
    ...
```

2.4 核心设计原则

OGGM的设计遵循以下四条核心原则，理解它们有助于把握源码中的各种约定：

2.4.1 基于实体的处理（Entity-based Processing）

OGGM的基本处理单元是“冰川”（glacier entity），每个冰川有一个独立的GlacierDirectory，存储其所有中间数据和模型输出。这种设计使得处理高度可并行：不同冰川之间没有数据依赖，可以独立地在不同CPU核心或不同计算节点上处理。这通过 `@entity_task` 和 `utils._workflow.execute_entity_task()` 实现。

2.4.2 配置驱动行为（Configuration-driven Behavior）

OGGM中几乎所有数值参数和路径设置都由 `oggm.cfg.PARAMS` 字典控制（参见第3章）。这意味着绝大多数行为变更不需要修改源码——只需修改 `params.cfg` 文件或通过环境变量覆盖。例如，将融化因子 `melt_f` 从默认值5.0改为3.0，即可全局转换所有物质平衡计算。

2.4.3 名称间接寻址（Filename Indirection）

代码中从不直接使用硬编码的文件名（如 `'model_geometry.nc'`），而是通过 `BASENAMES` 字典进行逻辑名称到物理名称的映射。这个设计使得 `filesuffix` 机制成为可能——通过添加后缀（如 `'_rcp85'`，`'_rcp26'`），同一冰川在同一工作目录下可以运行多个气候情景的模拟而互不干扰（详见第4章）。

2.4.4 功能分区（Separation of Concerns）

`core/` 目录下的每个模块负责一个单一的科学领域（中心线、气候、物质平衡、反演、动力学），彼此之间通过 `GlacierDirectory` 中的文件进行数据交换，而非直接函数调用。这使得：

- 模块可以独立测试和替换
- `shop/` 中的扩展模块可以无缝接入数据管道
- 新模型的集成只需确保输出文件格式与下游兼容即可

2.5 数据流：从RGI到冰川演化预测

OGGM的核心数据流遵循以下管线（pipeline）：



关键点： 每一步的输出文件是下一步的输入。这种基于文件的耦合（而非基于函数调用的耦合）是OGGM能够实现大规模并行处理的基础——每个步骤的代码只需要读取上一阶段产生的文件，不需要持有上一阶段的内存对象。

2.6 两种处理模式

OGGM支持两种冰川几何表示模式：

特性	中心线模式（centerline-based）	高程带模式（elevation-band-based）
几何表示	沿流线的离散网格点（~100–500点）	按高程分带的面积–海拔分布

特性	中心线模式 (centerline-based)	高程带模式 (elevation-band-based)
冰流动	物理SIA (shallow ice approximation)	简化的体积响应模型
冰厚度估计	物理反演	体积-面积缩放 (volume-area scaling)
计算成本	高 (每条冰川数秒到数分钟)	低 (每条冰川毫秒级)
适用场景	详细单个冰川模拟、区域精度研究	大规模集合模拟、快速敏感性分析
代码位置	<code>oggm/core/centerlines.py</code> , <code>flowline.py</code>	<code>oggm/core/flowline.py</code> (MixedFlowline)

进阶

模式选择指南

如果你有100条以下的研究冰川且关注详细的冰厚度分布和动力学行为，使用中心线模式。如果你的目标是全球尺度（RGI全部约20万条冰川）的集合模拟——尤其是在需要运行上百种气候情景的Monte Carlo分析中——那么高程带（elevation-band）模式是唯一可行的选择。OGGM的FluxBasedModel可以处理两种模式的输出，但你需要在初始化时通过 `cfg.PARAMS['evolution_model']` 进行设置。

2.7 GlacierDirectory：数据中枢预览

`GlacierDirectory`（定义于 `oggm/utils/g2ti.py` 或更早的 `oggm/utils/_workflow.py`，约第2643–3960行，共约1300行）是OGGM架构中核心的数据结构。它充当每条冰川的“数据中枢”，管理着：

- **目录结构**：冰川的所有文件存储在 `base_dir/RGIID[:8]/RGIID[:11]/RGIID/` 路径下（例如 `per_glacier/RGI60-11/00/RGI60-11.00001/`）
- **文件读写**：提供统一的方法读写pickle、shapefile、JSON和NetCDF文件，所有路径都由 `BASENAMES` 间接解析
- **任务日志**：维护 `task_status.json`，记录每个entity_task的执行时间、状态和错误日志
- **参考数据缓存**：气候参考数据通过 `set_ref_mb_data()` 和 `get_ref_mb_data()` 方法进行延迟加载

GlacierDirectory的详细剖析见第4章。

2.8 冰川目录（GlacierDirectory）中的文件组织

一个完全处理后的冰川目录（GlacierDirectory）包含以下典型文件：

```
per_glacier/RGI60-11/00/RGI60-11.00001/
├── glacier_grid.json          # 本地网格定义
├── dem_source.json           # DEM数据源信息
├── centerlines.pkl           # 计算的流线 (shapely geometries)
├── downstream_line.pkl       # 下游线 (downstream line) 几何
├── flowline_catchments.pkl    # 流线集水区
├── hypsometry.csv            # 面积-高程分布表
├── climate_historical.nc      # 历史气候时间序列 (NetCDF)
├── gcm_data.nc               # GCM偏移气候数据
├── local_mustar.json         # 校准后的温度敏感性
├── ref_tstars.csv            # 参考温度偏差
├── inversion_input.pkl        # 反演参数
├── inversion_flowlines.pkl    # 包含厚度的反演后流线
├── inversion_output.pkl       # 反演输出完整对象
├── model_geometry.nc         # 模型几何时间序列
├── model_diagnostics.nc      # 模型诊断变量时间序列
└── task_status.json          # 任务执行日志
```

2.9 包结构总览

包/模块	代码量（约行数）	功能
oggm/cfg.py	~600	配置系统：PARAMS, PATHS, BASENAMES, 初始化
oggm/utils/_workflow.py	~600	entity_task/global_task装饰器, execute_entity_task并行调度
oggm/utils/g2ti.py	~1400	GlacierDirectory类（文件I/O, 任务状态, 气候数据管理）
oggm/utils/_funcs.py	~2000	通用栅格/矢量工具函数（地图投影、重采样、平滑等）
oggm/core/gis.py	~1200	GIS预处理：冰川mask、DEM提取、地形属性
oggm/core/centerlines.py	~1800	中心线计算：Kienholz算法、分叉处理
oggm/core/climate.py	~2500	气候数据：GCM降尺度、温度/降水插值
oggm/core/massbalance.py	~3000	物质平衡：度日模型、温度指数模型（temperature-index model）、校准
oggm/core/inversion.py	~800	冰厚度反演：逆SIA求解器
oggm/core/flowline.py	~5000	Flowline和FluxBasedModel：动力核心
oggm/core/dynamics.py	~600	动力演化时间步控制
oggm/shop/	~5000	扩展模块：bedmachine、rgitopo、ecowitt等
oggm/cli/	~400	命令行接口（基于click）
oggm/sandbox/	~2000	实验性代码、非核心贡献
oggm/tests/	~6000	单元测试（基于pytest）

专家

代码量最大的模块：flowline.py

`oggm/core/flowline.py`（约5000行）是整个代码库中最庞大、最复杂的模块。它包含了Flowline类的定义、FluxBasedModel（基于通量的动力演化求解器）、从VAScalingConverter到MixedFlowline的多种几何表示转换器，以及数十个辅助函数。本书第11–13章将对其进行深入分析。对于初次阅读的读者，建议先理解Flowline的数据结构（约300行），再接触FluxBasedModel的步进逻辑（约1000行），最后才是各种边界条件处理。

2.10 模块间的依赖关系图

以下为更精确的模块级DAG（有向无环图）：

```
cfg.py                # 零依赖（仅依赖标准库 + configobj）
├── _workflow.py       # 依赖 cfg
│   ├── g2ti.py        # 依赖 _workflow, cfg
│   └── _funcs.py       # 依赖 cfg
├── core/gis.py        # 依赖 utils._funcs, g2ti, _workflow
├── core/centerlines.py # 依赖 gis, _funcs, _workflow
├── core/climate.py     # 依赖 gis, _funcs, _workflow
├── core/massbalance.py # 依赖 climate, _funcs, _workflow
├── core/inversion.py   # 依赖 massbalance, flowline, _workflow
├── core/flowline.py    # 依赖 massbalance, _funcs, _workflow
├── core/dynamics.py    # 依赖 flowline, _workflow
└── shop/*             # 依赖 core/*, utils/* — 但从不被core依赖
```

这种单向、无循环的依赖结构使得OGGM的模块可以独立开发和测试。特别值得注意的是，`shop/` 中的模块可以依赖 `core/` 和 `utils/`，但反过来不行——这确保了核心代码的稳定性不受扩展模块变化的影响。

第3章 配置系统深度解析 (cfg.py)

OGGM的配置系统是理解整个框架运行机制的“入口”。所有模块的行为——从DEM网格分辨率到冰流变参数再到输出文件格式——都由一个统一的配置层控制。本章剖析 `oggm/cfg.py` 的源码实现，涵盖全局状态单例、初始化链、参数类型系统、多进程安全机制，以及环境变量覆盖系统。

初学者先读这里

第一次使用OGGM时，你只需要记住三个对象：`cfg.initialize()` 启动配置，`cfg.PATHS['working_dir']` 决定数据写到哪里，`cfg.PARAMS` 保存模型参数。3.4节之后的ConfigObj、多进程序列化和环境变量机制主要面向需要部署或开发OGGM的读者，第一遍可以跳过。

3.1 全局状态设计

OGGM使用模块级单例（module-level singletons）管理全局状态，定义了五个核心全局变量：

```
# cfg.py 中的全局变量定义（模块顶层）
PARAMS      = ParamsLoggingDict()    # 所有数值和字符串参数
PATHS       = PathOrderedDict()      # 路径配置，自动展开 ~
BASENAMES   = DocumentedDict()       # 逻辑→物理文件名映射
DATA        = {}                     # 跨进程共享数据（只读）
LRUHANDLERS = {}                     # LRU缓存文件句柄池
CONFIG_MODIFIED = False               # 配置是否被修改（多进程安全标志）
```

设计理由：为何使用模块级全局变量？

在典型的面向对象设计中，你可能会将配置封装在类实例中并通过依赖注入传递。OGGM选择模块级全局变量的原因有三：(1) 科学模型有大量参数（300+），通过每层函数签名传递将极为繁琐；(2) 参数调整是建模工作流的核心操作，全局访问路径（`cfg.PARAMS['melt_f']`）降低了使用门槛；(3) 方便实现环境变量覆盖（无需修改调用链）。代价是牺牲了线程安全——但OGGM选择进程级并行（multiprocessing）来规避这个问题（见3.5节）。

3.2 params.cfg：主参数文件

`params.cfg` 是OGGM的主参数文件，约600行，使用简单的key-value格式（由ConfigObj库解析）。它打包在 `oggm/` 包目录中随版本发布。OGGM v1.6.3中的片段示例如下：

```
### params.cfg (节选) ###
grid_dx_method = square
map_proj = 'tmerc'
border = 80
use_multiple_flowlines = True
flowline_dx = 2
baseline_climate = GSWP3_W5E5
temp_default_gradient = -0.0065
temp_melt = -1.
melt_f = 5
prcp_fac =
glen_a = 2.4e-24
glen_n = 3
fs = 0
cfl_number = 0.02
evolution_model = SemiImplicit
```

文件按注释块组织为路径、网格、中心线、气候、反演、动力学和输出等主题。解析后，参数被写入 `cfg.PARAMS`；路径类配置则写入 `cfg.PATHS`。因此，日常建模时通常不直接编辑OGGM包内的 `params.cfg`，而是在脚本中显式覆盖需要改变的少数参数。

入门

如何查找你的参数

当你需要改变模型的某个行为（例如调整融化因子或DEM分辨率）时，最有效的方法是在 `params.cfg` 中搜索相关关键词，并对照附录B确认默认值和使用模块。初学者建议只改工作目录、并行开关、预处理边界、气候数据源和少数校准参数；Glen参数、崩解参数和数值时间步长应在做敏感性实验时再修改。

3.3 初始化链：initialize_minimal() 到 initialize()

OGGM的初始化分为两级。理解这一链式调用是理解整个系统启动过程的关键。

3.3.1 initialize_minimal()

`initialize_minimal()` 执行最低限度的设置，约20行代码：

```
def initialize_minimal(config_file=None, logging_level='INFO',
                      add_custom_path=None):
    # 1. 读取打包的 params.cfg → PARAMS
    _read_cfg_file(config_file)

    # 2. 设置 PATHS — 展开 ~ 并设置默认路径
    _set_up_paths(add_custom_path)

    # 3. 应用环境变量覆盖 (OGGM_WORKDIR, ...)
    _apply_env_vars()

    # 4. 重新配置日志系统
    _set_up_logging(logging_level)
```

调用顺序：首先是读取 `params.cfg` 并填充 `PARAMS` 字典；然后设置 `PATHS`（工作目录、缓存目录等）；然后扫描环境变量并覆盖 `PARAMS` 和 `PATHS` 中的对应项；最后重新配置日志记录器。

3.3.2 initialize()

`initialize()` 在 `initialize_minimal()` 的基础上执行额外的"重量级"初始化：

```
def initialize(config_file=None, logging_level='INFO',
              add_custom_path=None):
    # 1. 调用 initialize_minimal()
    initialize_minimal(config_file, logging_level, add_custom_path)

    # 2. 下载打包的ship-file: params.cfg 中没有内嵌的数据文件
    #     (如默认DEM、海岸线shapefile等)
    _download_support_files()

    # 3. 初始化LRU文件缓存 (用于DEM瓦片)
    _init_lru_handlers()

    # 4. 预计算一些派生常量
    _init_derived_constants()
```

何时使用哪个？ 在不需要DEM数据或气候数据时（例如仅检查参数、处理已有预处理数据），使用 `initialize_minimal()` 即可，它启动更快（约0.5秒）。在需要完整建模功能的场景中，使用 `initialize()`（约3–5秒，取决于是否需要下载支持

文件)。

3.4 ConfigObj作为INI解析主干

OGGM选择 `ConfigObj` (一个纯Python的INI文件解析库) 作为参数文件的解析器, 而非标准库的 `configparser`。原因包括:

- `ConfigObj` 保留了INI条目的原始顺序 (OrderedDict-like), 这在参数文档化时很有用
- 支持嵌套subsection (如 `[massbalance.temp_bias]`)
- 内联注释保留 (以 `;` 开头), 可用于自动生成参数文档
- 支持列表值语法 (如 `filesuffix = string(default='')` 与 `list(default=list('_ba1', '_ba2'))`)

解析过程在 `_read_cfg_file()` 内部函数中完成 (cfg.py 约第200行):

```
from configobj import ConfigObj
from oggm.cfg import _typed_configobj_parser

def _read_cfg_file(config_file):
    # 使用 ConfigObj 解析原始 INI
    _cfg = ConfigObj(config_file, file_error=True,
                     interpolation=False)

    # _typed_configobj_parser 执行类型转换:
    # 'float(default=5.0)' -> PARAMS['key'] = 5.0 (float)
    # 'int(default=80)'    -> PARAMS['key'] = 80 (int)
    # 'bool(default=True)' -> PARAMS['key'] = True (bool)
    _typed_configobj_parser(_cfg)
```

进阶

ConfigObj的序列化陷阱

当使用 `pack_config()` 进行配置序列化时 (多进程通信场景), `ConfigObj`对象被通过 `configobj.ConfigObj.write()` 方法序列化为字符串。由于`ConfigObj`使用 `repr()` 写入列表值, 而Python的 `repr()` 对于包含Unicode的字符串可能产生与原始值不同的表示。如果遇到配置在多进程间传递后参数变化的问题, 检查参数值中是否包含非ASCII字符。

3.5 环境变量覆盖系统

OGGM允许通过环境变量覆盖几乎任何参数, 格式为 `OGGM_ + 大写参数名`。这在无需修改代码或配置文件的容器化部署场景中尤其有用。

环境变量	覆盖的配置	示例
<code>OGGM_WORKDIR</code>	<code>PATHS['working_dir']</code>	<code>OGGM_WORKDIR=/data/oggm_work</code>
<code>OGGM_USE_MULTIPROCESSING</code>	<code>PARAMS['use_multiprocessing']</code>	<code>OGGM_USE_MULTIPROCESSING=True</code>
<code>OGGM_MP_POOL_SIZE</code>	<code>PARAMS['mp_processes']</code>	<code>OGGM_MP_POOL_SIZE=16</code>
<code>OGGM_DL_CACHE_DIR</code>	<code>PATHS['dl_cache_dir']</code>	覆盖默认下载缓存目录
<code>OGGM_MELT_F</code>	<code>PARAMS['melt_f']</code>	<code>OGGM_MELT_F=3.5</code>
<code>OGGM_GLEN_A</code>	<code>PARAMS['glen_a']</code>	<code>OGGM_GLEN_A=2.4e-24</code>

环境变量	覆盖的配置	示例
OGGM_INVERSION_GLEN_A	PARAMS['inversion_glen_a']	反演时使用的Glen A（可与演化不同）

实现位于 `_apply_env_vars()` 函数中（`cfg.py`约第300行）：

```
def _apply_env_vars():
    # 扫描所有以 'OGGM_' 开头的环境变量
    for k, v in os.environ.items():
        if not k.startswith('OGGM_'):
            continue
        param_name = k[5:].lower() # 去除 'OGGM_' 前缀并转小写

        # 优先匹配 PATHS 中的键
        if param_name in PATHS:
            PATHS[param_name] = os.path.expanduser(v)
            continue

        # 然后尝试匹配 PARAMS 中的键并自动转换类型
        if param_name in PARAMS:
            PARAMS[param_name] = _type_convert(v, type(PARAMS[param_name]))
```

3.6 类型强制转换

`_type_convert()` 是配置系统中的一个关键工具函数，它负责将INI文件和环境变量中的字符串值转换为正确的Python类型：

```
def _type_convert(value, target_type):
    # bool: "True"/"False" → Python bool
    # int: "80" → Python int
    # float: "5.0" → Python float
    # list: "['a', 'b']" → Python list (通过 ast.literal_eval)
    # str: 透传
    ...
```

对于列表类型，使用 `ast.literal_eval` 安全解析（可处理嵌套列表和元组）。其他类型则依赖于Python内置的类型构造函数。

3.7 自定义字典类型

3.7.1 DocumentedDict

`DocumentedDict` 是OGGM为 `BASENAMES` 定制的字典子类（`cfg.py`中定义）。它重写了 `__setitem__` 方法，要求每个值是一个 `(description, filename)` 元组，其中 `description` 作为文档字符串存储。这确保了每个逻辑文件名都有清晰的人类可读描述。

```
class DocumentedDict(dict):
    def __setitem__(self, key, value):
        # value 应为 (docstring, default_name) 元组
        if not isinstance(value, tuple) or len(value) != 2:
            raise ValueError('DocumentedDict requires (doc, name) tuples')
        super().__setitem__(key, value[1]) # 只存储文件名
        # value[0] (文档字符串) 存储在 self.__doc__ 或其他属性中
```

3.7.2 ParamsLoggingDict

`ParamsLoggingDict` 扩展了标准 `dict`，在每次参数被修改时记录日志并设置 `CONFIG_MODIFIED` 标志。这为多进程安全的配置传播提供了基础。

```
class ParamsLoggingDict(dict):
    def __setitem__(self, key, value):
        # 记录变更日志
        log.info(f"PARAMS['{key}'] changed: {self.get(key)} → {value}")
        global CONFIG_MODIFIED
        CONFIG_MODIFIED = True
        super().__setitem__(key, value)
```

入门

查找参数修改的源头

当你在一个大型OGGM脚本中调试时，如果想知道哪些参数在何处被修改，可以临时设置 `cfg.PARAMS._log_changes = True`。`ParamsLoggingDict` 会打印每次赋值操作的文件名和行号（通过 `traceback.extract_stack()`）。这在追踪意外的参数覆盖时非常有用。

3.8 多进程安全：pack/unpack/set_manager

这是OGGM配置系统中最精妙的设计之一。在Python多进程环境中，每个子进程有自己独立的内存空间，模块级全局变量的修改不会在进程间自动传播。OGGM通过以下三个函数解决这个问题：

```
# 主进程：将配置序列化为可传递的字典
def pack_config():
    return {
        'PARAMS': dict(PARAMS),
        'PATHS': dict(PATHS),
        'BASENAMES': dict(BASENAMES),
        'DATA': DATA, # DATA 应为只读，直接共享引用
    }

# 子进程：反序列化配置字典并还原全局状态
def unpack_config(cfg_dict):
    global PARAMS, PATHS, BASENAMES, DATA
    PARAMS.update(cfg_dict['PARAMS'])
    PATHS.update(cfg_dict['PATHS'])
    BASENAMES.update(cfg_dict['BASENAMES'])
    global CONFIG_MODIFIED
    CONFIG_MODIFIED = False # 子进程重置修改标志

# 为多进程Manager设置共享状态
def set_manager(manager):
    global _MANAGER
    _MANAGER = manager # 用于同步 DATA 和 LRUCACHE
```

工作流执行器（`execute_entity_task()`）会在启动进程池时自动调用 `pack_config()`，并在每个worker的初始化函数中调用 `unpack_config()`。这意味着：

- 如果在主进程中修改了 `PARAMS['melt_f'] = 8.0`，新值会自动传播到所有worker
- 如果在worker中修改了PARAMS，不会影响主进程或其他worker（由于 `CONFIG_MODIFIED` 在unpack时被重置）

专家

配置传播的边界情况

注意： `pack_config()` 只在进程池创建时调用一次。如果你在进程池运行期间修改了主进程的PARAMS，已在运行的worker不会收到更新。因此，如果你需要在并行处理之前动态修改参数，建议在调用 `execute_entity_task()` 之前完成所有配置修改。如果需要动态行为，应考虑将参数作为entity_task的函数参数传递（如 `melt_f=None` 时使用全局参数），这样worker内部可以检查参数而不是依赖全局状态。

3.9 参数组深度解析

以下表格按逻辑分组列出了OGGM中最关键的配置参数：

3.9.1 I/O和路径参数

参数	类型	默认值	说明
<code>working_dir</code>	str	<code>~/oggm_working_dir</code>	所有冰川数据和输出的根目录
<code>dl_cache_dir</code>	str	<code>~/oggm_cache</code>	DEM瓦片下载缓存
<code>use_multiprocessing</code>	bool	<code>True</code>	是否默认启用多进程处理
<code>mp_processes</code>	int	<code>-1</code>	并行进程数（-1=全部CPU核心）
<code>mp_pool_timeout_task</code>	float	<code>3600</code>	单任务超时（秒）
<code>use_compression</code>	bool	<code>True</code>	pickle文件是否使用gzip压缩

3.9.2 网格与映射参数

参数	类型	默认值	说明
<code>map_proj</code>	str	<code>'tmerc'</code>	地图投影类型（横轴墨卡托/等面积）
<code>grid_dx_method</code>	str	<code>'linear'</code>	网格间距计算方法（'linear'或'fixed'）
<code>border</code>	int	<code>80</code>	本地地图边框像素数
<code>topo_interp</code>	str	<code>'bilinear'</code>	地形插值方法（'bilinear', 'cubic', 'nearest'）

3.9.3 中心线参数

参数	类型	默认值	说明
<code>flowline_dx</code>	int	<code>0</code>	流线网格间距（0=自动基于dem）
<code>q1</code>	float	<code>1.2</code>	高程反馈参数（几何1）
<code>q2</code>	float	<code>0.8</code>	高程反馈参数（几何2）
<code>rmax</code>	float	<code>3.0</code>	最大支流坡降因子（Kienholz算法）
<code>f1</code>	float	<code>1.0</code>	支流筛选参数（面积权重）
<code>f2</code>	float	<code>0.2</code>	支流筛选参数（长度分数）
<code>a</code>	float	<code>3.0</code>	Kienholz算法过滤参数a
<code>b</code>	float	<code>0.7</code>	Kienholz算法过滤参数b

3.9.4 气候参数

参数	类型	默认值	说明
baseline_climate	str	'W5E5'	基线气候数据集（W5E5或CRU）
temp_default_gradient	float	-6.5	默认温度梯度 (K/km)
temp_melt	float	-1.0	冰雪融化的温度阈值 (degC)
temp_all_solid	float	0.0	全固态降水温度阈值
temp_all_liq	float	2.0	全液态降水温度阈值
melt_f	float	5.0	雪的度日因子 (mm w.e. / (d*K))
prcp_fac	float	1.75	降水校正因子

3.9.5 冰动力学参数

参数	类型	默认值	说明
glen_a	float	2.4e-24	Glen流变参数 A ($\text{Pa}^{-3}\text{s}^{-1}$)
glen_n	int	3	Glen流变指数 n
fs	float	0.0	滑动参数（0=无滑动, 1=完全滑动）
inversion_glen_a	float	2.4e-24	反演时使用的Glen A（可与演化不同）
cfl_number	float	0.01	CFL数（数值稳定性条件）
evolution_model	str	'FluxBased'	演化模型类型

3.9.6 冰川崩解参数

参数	类型	默认值	说明
calving_k	float	2.0	崩解速率参数（Oerlemans–Nick）
inversion_calving_k	float	2.0	反演时的崩解参数
free_board_marine_terminating	float	50.0	入海型冰川出水高度（freeboard）阈值 (m)

3.9.7 输出控制

参数	类型	默认值	说明
store_model_geometry	bool	True	是否存储模型几何时间序列
store_diagnostic_variables	bool	True	是否存储诊断变量（通量、速度等）
store_fl_diagnostics	bool	False	是否存储逐流线点的完整诊断（磁盘密集）

3.10 如何覆盖参数

OGGM提供三种参数覆盖方式，优先级从低到高：

1. **修改params.cfg文件（最低优先级）**：直接编辑 `oggm/params.cfg` 文件。这是永久性变更的方式，适合系统管理员和持久性部署。但升级OGGM包时会覆盖此文件，不适合临时实验。
2. **使用自定义配置文件**：通过 `initialize(config_file='/path/to/my_params.cfg')` 指定自定义配置文件。这是推荐给研究者的方式——你可以为每个项目维护独立的参数配置。
3. **环境变量（中优先级）**：在调用 `initialize()` 之前设置环境变量，或在shell中 `export OGGM_MELT_F=3.5`。非常适合 Docker 容器和CI/CD脚本。
4. **代码直接赋值（最高优先级）**：在调用 `initialize()` 之后、运行任何任务之前，通过 `cfg.PARAMS['melt_f'] = 3.5` 直接设置。这是最灵活的临时修改方式。

```
# 典型的参数覆盖模式
import oggm.cfg as cfg
cfg.initialize()
cfg.PARAMS['melt_f'] = 3.5          # 调整度日因子
cfg.PARAMS['glen_a'] = 2.4e-24     # 设置冰流变参数
cfg.PARAMS['use_multiprocessing'] = True
cfg.PARAMS['mp_processes'] = 8     # 限制并行进程数
```

3.11 BASENAMES：逻辑到物理文件名映射

BASENAMES 系统是OGGM中一个优雅的间接寻址设计。代码中从不使用硬编码的文件名，而是通过逻辑名称引用文件：

```
# 在代码中使用逻辑名：
centerlines_file = gdir.get_filepath('centerlines')
# 解析为：gdir.dir / 'centerlines.pkl' (或 'centerlines_rcp85.pkl')

climate_file = gdir.get_filepath('climate_historical')
# 解析为：gdir.dir / 'climate_historical.nc'
```

BASENAMES 存储在 `DocumentedDict` 中，键为逻辑名，值为对应的物理文件名。扩展模块可以通过 `cfg.BASENAMES['my_new_file'] = ('Description', 'my_new_file.pkl')` 注册新的文件类型。

当配合 `filesuffix` 机制使用时（参见第4章），逻辑名不变，但实际文件名会被自动追加后缀。例如

`cfg.PARAMS['filesuffix'] = '_rcp85'` 时，`get_filepath('centerlines')` 将解析为 `centerlines_rcp85.pkl`。

3.12 LRU文件缓存机制

LRUHANDLERS 是一个使用LRU（Least Recently Used）策略的文件句柄缓存，主要用于DEM瓦片（tiles）的重复访问。在全局冰川模拟中，相邻冰川通常共享同一DEM瓦片，缓存避免了重复打开和读取同一文件。

```
# gis.py 中的典型使用模式
def read_topo_tile(tile_name):
    # 从 LRUHANDLERS 获取缓存的 DEM tile
    if tile_name in cfg.LRUHANDLERS:
        return cfg.LRUHANDLERS[tile_name]
    # 缓存未命中：打开文件并加入LRU池
    cfg.LRUHANDLERS[tile_name] = open_topo(tile_name)
    return cfg.LRUHANDLERS[tile_name]
```

进阶

LRU缓存的性能影响

在RGI全量处理（~20万冰川）中，如果没有LRU缓存，每条冰川的GIS预处理都需要独立打开其DEM瓦片文件。由于相邻冰川共享瓦片，每个瓦片可能被重复打开数千次。LRU缓存将每个瓦片的I/O操作从数千次减少为1次。在实际测试中，这可以将全球GIS预处理的I/O时间从数天减少到数小时。

3.13 物理常量

`cfg.py` 尾部定义了一组物理和数学常量，在整个代码库中使用以保证一致性：

```
# 时间常量
SEC_IN_YEAR = 365 * 24 * 3600      # 31536000
SEC_IN_DAY = 24 * 3600              # 86400
SEC_IN_MONTH = SEC_IN_YEAR / 12     # 2628000

# 物理常量
G = 9.81                            # 重力加速度 (m/s^2)
RHO = 900.0                         # 冰密度 (kg/m^3)
RHO_W = 1028.0                      # 海水密度 (kg/m^3)

# 数学/计算常量
GAUSSIAN_KERNEL = None              # 在initialize()时预计算的Gaussian平滑核
```

这些常量的定义为后续章节（如第8章的物质平衡模型）提供了统一的数值基础。

3.14 `cfg.py` 小结

`oggm/cfg.py`（约600行）是整个OGGM框架的配置中枢。它通过以下机制实现灵活而健壮的参数管理：

- **五维全局状态**：PARAMS、PATHS、BASENAMES、DATA、LRUHANDLERS 各司其职
- **两级初始化**：`initialize_minimal()` 用于轻量启动，`initialize()` 用于完整启动
- **INI + 类型系统**：`ConfigObj` + `_typed_configobj_parser` 提供类型安全的参数解析
- **多级覆盖**：配置文件 → 自定义配置 → 环境变量 → 代码赋值
- **多进程安全**：`pack/unpack/set_manager` 保证配置在进程间一致传播
- **间接寻址**：BASENAMES + `filesuffix` 机制支持多情景模拟

深入理解`cfg.py`是有效使用和扩展OGGM的前提。在后续章节中，你将看到每个核心模块如何通过 `cfg.PARAMS` 获取其运行参数。

第4章 GlacierDirectory：数据中枢

GlacierDirectory 是OGGM架构中核心的数据结构。它不仅仅是一个文件路径管理器——它是每条冰川在整个建模生命周期中的“可复现实例”的数据锚点。从GIS预处理到气候数据插值、物质平衡校准、冰厚度反演，再到最终的动力学演化，所有的输入和输出文件都通过GlacierDirectory的接口进行读写。本章深入剖析 `oggm/utils/_workflow.py` 中GlacierDirectory类的源码（约第2643–3960行，共约1300行）。

4.1 类的整体定位

GlacierDirectory在OGGM中的角色可以类比为：它是一个冰川实体的“文件系统抽象层”。对于调用者（core模块中的各个entity_task）来说，GlacierDirectory屏蔽了：

- 文件存储的物理路径（`base_dir/RGIID[:8]/RGIID[:11]/RGIID/`）
- 文件名和扩展名（通过BASENAMES间接寻址）
- 数据格式（pickle vs shapefile vs NetCDF vs JSON）
- 压缩方式（gzip vs tar.gz vs uncompressed）
- 多情景的文件后缀（filesuffix）机制
- 任务状态日志的持久化

入门

类比：GlacierDirectory 如同冰川的“专属文件夹”

想象每条冰川在磁盘上有一个自己的文件夹。GlacierDirectory就是这个文件夹的“管理员”——它知道每个文件应该放在哪里、叫什么名字，能帮你读取和写入，还能记录你在这个文件夹中的所有操作历史。你不需要关心硬盘上的实际路径是怎样的，只需要用逻辑名称（如 `'centerlines'`）说出你想要的文件即可。

4.2 构造方式

GlacierDirectory提供了三种构造途径，满足不同的使用场景：

4.2.1 从RGI GeoSeries创建（全新冰川）

这是最常用的方式，用于从未被处理过的冰川开始：

```
# 典型用法 (gis.py 中 init_glacier_directories 任务相关)
import oggm

# rgi_df 是一个 GeoDataFrame, 每行为一条冰川
# entity 是其中一行 (GeoSeries)
gdir = oggm.GlacierDirectory(entity, base_dir='/data/oggm/per_glacier/',
                             reset=True, from_tar=False)
```

构造函数签名（约第2643行）：

```
class GlacierDirectory(object):
    def __init__(self, entity, base_dir=None, reset=False,
                 from_tar=False, delete_tar=True):
        # entity 可以是:
        # (1) GeoSeries - 从 RGI GeoDataFrame 中迭代的行
        # (2) str - 已有的 RGI ID (用于恢复已有目录)
```

当 `entity` 是 `GeoSeries` 时，构造函数从中提取 RGI ID、几何（geometry）、名称、所属区域等信息，然后构建目录路径。
`reset=True` 会清空已有目录中的所有文件，从头开始。

4.2.2 从已有 RGI ID 恢复（已处理冰川）

```
# 恢复已处理过的冰川目录（GlacierDirectory）
gdir = oggm.GlacierDirectory('RGI60-11.00001',
                              base_dir='/data/oggm/per_glacier/')
```

当 `entity` 是字符串时，构造函数假设目录已经存在，直接进入现有目录。构造函数会验证目录和必要的元数据文件是否存在。这是运行后续处理步骤（如气候插值或动力学模拟）时的典型入口。

4.2.3 从 tar 归档创建（传输/共享）

```
# 从 tar 归档解压并创建
gdir = oggm.GlacierDirectory('RGI60-11.00001',
                              base_dir='/data/oggm/per_glacier/',
                              from_tar=True)
```

`from_tar=True` 指示构造函数查找 `RGI60-11.00001.tar` 归档文件并解压到目标目录。这在以下场景中很有用：

- 从服务器下载预处理的冰川数据包
- 在不同计算节点之间传输冰川数据
- 归档已完成处理的结果

`delete_tar=True`（默认）在成功解压后删除 tar 文件以节省磁盘空间。

4.3 目录结构

`GlacierDirectory` 在磁盘上使用两级子目录分片（sharding）来避免单个目录下存放过多文件（可能导致文件系统性能下降）：

```
base_dir/
├── RGI60-11/           # RGI 区域前缀（RGI ID 的前8个字符）
│   └── 00/             # 子分片（RGI ID 的前11个字符的最后2位）
│       └── RGI60-11.00001/ # 完整的 RGI ID
│           ├── glacier_grid.json
│           ├── centerlines.pkl
│           ├── ...
│           └── task_status.json
```

具体规则：

- 第一级路径： `RGI ID[:8]` — 例如 `RGI60-11`（前8个字符，通常包含区域前缀）
- 第二级路径： `RGI ID[:11]` 的最后两位 — 例如 `00`（来自 `RGI60-11.00` 的 `00`）
- 第三级路径：完整的 RGI ID — 例如 `RGI60-11.00001`

这种分片策略确保即使处理 RGI 全部约 20 万条冰川，每个目录下也不会包含超过约 200 个子目录，从而维持文件系统的高效性。

进阶

RGI版本与目录结构的兼容性

RGI ID的格式在RGI 6.0和RGI 7.0之间略有差异。RGI 6.0使用 `RGI60-X.XXXXX` 格式（5位数字），而RGI 7.0使用 `RGI2000-v7.0-G-XXXXXX` 格式。OGGM v1.6同时支持这两种格式，但你的 `base_dir` 中会包含两种不同结构的路径。确保在使用 `initialize()` 时指定正确的RGI版本，以便目录结构正确分区。

4.4 关键属性

属性	类型	来源	说明
<code>gdir.rgi_id</code>	str	RGI ID	冰川的RGI唯一标识符，如 <code>'RGI60-11.00001'</code>
<code>gdir.rgi_area_km2</code>	float	RGI属性 <code>Area</code>	冰川面积（平方公里），用于多项参数化
<code>gdir.cenlon</code>	float	几何质心	冰川质心经度（度），用于气候数据插值
<code>gdir.cenlat</code>	float	几何质心	冰川质心纬度（度）
<code>gdir.grid</code>	salem.Grid	<code>glacier_grid.json</code>	本地投影网格对象（lazy-loaded）
<code>gdir.intersects_ids</code>	list[str]	RGI空间查询	与当前冰川缓冲区相交的相邻冰川ID列表
<code>gdir.dir</code>	str/pathlib.Path	构造函数	冰川目录（GlacierDirectory）的完整路径
<code>gdir.rgi_region</code>	str	RGI属性	RGI区域代码（如 <code>'11'</code> 用于中欧）
<code>gdir.rgi_version</code>	str	RGI属性	RGI版本（如 <code>'60'</code> ）

`grid` 属性值得特别说明。它是一个延迟加载（lazy-loaded）属性——只有在第一次访问时才会从磁盘读取 `glacier_grid.json`。数据库使用 `salem.Grid` 对象（salem是OGGM团队维护的另一个开源库），提供投影感知的网格操作：

```
# grid 属性的内部实现（简化版）
@property
def grid(self):
    if self._grid is None:
        with open(self.get_filepath('glacier_grid')) as f:
            self._grid = salem.Grid.from_json(f.read())
    return self._grid
```

4.5 文件I/O方法

GlacierDirectory提供了一套统一的文件I/O接口，所有接口都遵循"先通过BASENAMES解析文件名，再进行I/O操作"的模式。

4.5.1 get_filepath() — 文件路径解析

```
def get_filepath(self, filename, filesuffix='',
                 check_exists=False):
    # filename 是 BASENAMES 中的逻辑名
    # 例如 'centerlines' -> 解析 'centerlines' 后获得物理文件名
    # 然后拼接 dir / (物理文件名 + filesuffix)
    # 例如 dir / 'centerlines_rcp85.pkl'
```

`filesuffix` 参数（默认为 `cfg.PARAMS['filesuffix']`）会自动追加到文件基名和扩展名之间。例如 `get_filepath('centerlines', filesuffix='_rcp85')` 返回 `.../RGI60-11.00001/centerlines_rcp85.pkl`。

4.5.2 read_pickle() / write_pickle() — 压缩序列化

```
def write_pickle(self, obj, filename):
    # 使用 gzip + pickle 序列化对象
    # 如果 cfg.PARAMS['use_compression'] 为 True (默认), 使用 gzip 压缩
    with gzip.open(filepath, 'wb') as f:
        pickle.dump(obj, f, protocol=pickle.HIGHEST_PROTOCOL)

def read_pickle(self, filename, use_compression=None):
    # 从 gzip 压缩的 pickle 文件反序列化
    with gzip.open(filepath, 'rb') as f:
        return pickle.load(f)
```

压缩可以显著减少磁盘空间——冰厚度反演后的flowline数据可达数MB每条冰川，压缩后可减小到约20%。

4.5.3 read_shapefile() / write_shapefile() — 地理空间I/O

```
def write_shapefile(self, shp, filename, tar=False):
    # shp: geopandas GeoDataFrame 或 fiona 兼容对象
    # tar=False: 写入 .shp + .dbf + .shx + ... 文件套件
    # tar=True: 将所有文件打包为 .tar.gz 存档

def read_shapefile(self, filename, tar=False):
    # tar=False: 从 .shp + 套件读取
    # tar=True: 从 .tar.gz 存档读取
    return geopandas.read_file(path)
```

tar选项特别适合空间索引（如 `intersects` 输出），因为它们由多个文件组成（.shp, .dbf, .shx, .prj 等）。

4.5.4 read_json() / write_json() — 元数据I/O

```
def write_json(self, data, filename):
    with open(filepath, 'w') as f:
        json.dump(data, f, indent=2)

def read_json(self, filename):
    with open(filepath, 'r') as f:
        return json.load(f)
```

主要用于存储轻量级元数据：`glacier_grid.json`（网格定义）、`local_mustar.json`（温度敏感性参数）、`dem_source.json`（DEM来源信息）等。

4.5.5 has_file() — 存在性检查

```
def has_file(self, filename):
    # 检查文件是否存在（支持tar shapefile的特殊处理）
    # 如果是 shapefile 类型, 检查 .tar.gz 或 .shp
    # 如果是其他类型, 检查 .pkl 或 .nc 等
```

此方法是 `entity_task` 自动跳过逻辑的底层支撑。每个entity_task的writes_output列表中的文件都会通过 `gdir.has_file()` 检查其存在性。

4.5.6 write_monthly_climate_file() — NetCDF气候数据

```
def write_monthly_climate_file(self, time, prcp, temp,
                               gradient, ref_pix_lon, ref_pix_lat,
                               filename, source=''):
    # 创建包含时间、降水、温度、温度梯度的NetCDF4文件
    # 同时写入参考像素坐标和气候数据来源元数据
    # 文件结构符合OGGM的climate_data约定
```

此方法专门用于创建OGGM标准格式的气候文件（`climate_historical.nc` 和 `gcm_data.nc`）。它使用 `netCDF4` 库创建包含以下变量的文件：

- `temp`：月平均温度 (degC)
- `prcp`：月总降水量 ($\text{kg m}^{-2} \text{ month}^{-1}$)
- `gradient`：月温度梯度 (K km^{-1})
- `ref_pix_lon`, `ref_pix_lat`：参考像素坐标
- 全局属性：`source`（气候数据来源）

4.6 任务状态系统

GlacierDirectory维护一个 `task_status.json` 日志文件，记录所有entity_task的执行历史。这是OGGM自动跳过（auto-skip）和错误恢复（error recovery）机制的基石。

4.6.1 get_task_status()

```
def get_task_status(self, task_name=None):
    # 返回指定任务的状态：
    # 'success' — 任务成功完成
    # 'skipped' — 任务已跳过（输出文件已存在）
    # 'error' — 任务执行出错
    # None — 任务尚未执行
    # task_name=None 时返回所有任务的状态字典
```

`task_status.json` 文件的内部结构：

```
{
  "compute_centerlines": {
    "status": "success",
    "start_time": "2024-01-15 10:23:45",
    "end_time": "2024-01-15 10:23:58",
    "duration_seconds": 13.2,
    "base_version": "1.6.3"
  },
  "prepare_climate": {
    "status": "error",
    "start_time": "2024-01-15 10:24:00",
    "end_time": "2024-01-15 10:24:02",
    "error_traceback": "..."
  }
}
```

4.6.2 get_task_time()

```
def get_task_time(self, task_name):
    # 返回任务执行的开始/结束时间（datetime对象）
    # 用于判断输出文件的新鲜度
```

4.6.3 get_error_log()

```
def get_error_log(self):
    # 返回所有状态为 'error' 的任务的错误信息
    # 适用于自动化错误报告和调试
```

进阶

设计洞察：任务状态系统的价值

任务状态系统看似简单，但它是OGGM能够可靠处理数万条冰川的关键设计。在没有任何任务状态系统的情况下，并行处理的中断会导致灾难性后果：你需要手动检查每一条冰川的文件是否完整，然后从断点重新开始。有了task_status，`execute_entity_task()` 在重启时自动跳过所有已完成（status='success'）的冰川，仅重新处理失败的（status='error'）和未开始的（status=None）。这为故障恢复提供了零成本的重启能力。

4.7 气候参考数据管理

GlacierDirectory包含了专门用于管理物质平衡校准所需的气候参考数据的方法：

4.7.1 set_ref_mb_data()

```
def set_ref_mb_data(self, ref_mb_df):
    # 设置参考物质平衡数据 (GeoDataFrame)
    # ref_mb_df 包含知道物质平衡的冰川的测量数据
    # 数据存储在 self._ref_mb_df 属性中
```

参考物质平衡数据来源于全球冰川监测服务（WGMS）的测量数据库，仅覆盖全球约500条冰川。在校准过程中，这些数据用于确定每条冰川（包括未被直接测量的）的温度敏感性参数 `mu_star`。

4.7.2 get_ref_mb_data()

```
def get_ref_mb_data(self):
    # 返回参考物质平衡数据
    # 如果在当前区域找不到参考冰川，可能启动二级搜索
```

4.7.3 get_ref_mb_profile()

```
def get_ref_mb_profile(self):
    # 返回参考物质平衡随高度变化的梯度曲线
    # 这是校准 mu_star 的关键输入
```

4.7.4 get_climate_info()

```
def get_climate_info(self):
    # 返回气候数据的元信息：
    # - 参考像素坐标（经纬度）
    # - 气候数据来源和覆盖时间段
    # - 是否从GCM数据中进行了时间偏移
```

这些方法体现了GlacierDirectory不仅管理文件I/O，还管理数据的"上下文"——知道哪里获取参考冰川数据、如何校准参数，以及操作这些数据所需的元信息。

4.8 投影处理：UTM vs Transverse Mercator

GlacierDirectory管理地图投影的配置和转换：

```
def _reproject_and_write_shapefile(self, in_shp, out_filename,
                                   map_proj=None):
    # 将输入shapefile重投影为目标投影并写出
    # 支持 cfg.PARAMS['map_proj'] 中的两种投影：
    #   'tmerc'（横轴墨卡托）- 距离保持，适合高纬度
    #   'utm'（通用横轴墨卡托）- 标准UTM分带
```

OGGM默认使用横轴墨卡托（Transverse Mercator, tmerc）投影，以冰川质心为中心，因为它在局部区域内能很好地保持距离和方向。对于极地地区的冰川，UTM投影可能失效（UTM在84N以上和80S以下未定义），此时横轴墨卡托是更好的选择。

投影选择对计算精度有直接影响：

- **tmerc（默认）**：为每条冰川单独设置投影中心，确保局部变形最小。适合工作流中的距离测量（如grid_dx计算）。
- **aea（Albers等面积）**：保持面积不变，适合面积相关计算。
- **utm**：使用标准UTM带，适合区域展示。在极地地区自动回退到tmerc。

4.9 文件生命周期：各prepro级别的产出

以下是OGGM预处理管线各层级（prepro level）在GlacierDirectory中创建的文件清单：

Prepro Level	实体任务（entity task）	创建的输出文件
Level 0	define_glacier_region	glacier_grid.json , dem_source.json
Level 1	compute_centerlines	centerlines.pkl , downstream_line.pkl , flowline_catchments.pkl , hypsometry.csv
Level 2	process_climate_data	climate_historical.nc , gcm_data.nc (if GCM data available)
Level 3	local_t_star	ref_tstars.csv , local_mustar.json
Level 4	prepare_for_inversion	inversion_input.pkl
Level 5	volume_inversion	inversion_output.pkl , inversion_flowlines.pkl
Level 6	init_present_time_glacier	model_geometry.nc , model_diagnostics.nc (initial state)

重要约定：较低级别必须在其所有上级文件完整存在的基础上运行。例如，不能在没有完成Level 3（local_mustar.json 存在）的情况下运行Level 4（prepare_for_inversion）。entity_task的自动跳过机制通过检查 writes_output 中列出的所有文件来强制执行这层约定。

专家

自定义prepro管线

预定义的prepro levels是OGGM对标准使用场景的封装。如果你需要插入自定义步骤（例如在centerlines和climate之间加入一个遥感冰川速度的步骤），你可以：

1. 创建一个新的 `@entity_task` 函数，指定 `writes_output` 参数
2. 在 `BASENAMES` 中注册你的新输出文件类型
3. 将你的任务插入到合适的prepro level之间

关键约束是：你的任务的 `writes_output` 中必须列出所有你会创建的文件，这样自动跳过逻辑才能正确工作。如果遗漏，后续重新运行时会误判你的任务需要重新执行。

4.10 filesuffix 机制：多情景实验的基石

`filesuffix` 是OGGM支持在单一工作目录下运行多个气候/参数情景的关键设计：

```
# 场景1: RCP 8.5 (高排放)
cfg.PARAMS['filesuffix'] = '_rcp85'
workflow.execute_entity_task(tasks.run_random_climate, gdirs, ...)
# 输出: model_geometry_rcp85.nc, model_diagnostics_rcp85.nc

# 场景2: RCP 2.6 (低排放)
cfg.PARAMS['filesuffix'] = '_rcp26'
workflow.execute_entity_task(tasks.run_random_climate, gdirs, ...)
# 输出: model_geometry_rcp26.nc, model_diagnostics_rcp26.nc
```

实现方式是：GlacierDirectory的所有I/O方法内部调用 `get_filepath()` 时，自动从 `cfg.PARAMS['filesuffix']` 读取后缀并拼接。这意味着你不需要修改任何任务代码，只需在运行不同情景前更改配置即可。

工作原理（简化版源码）：

```
def get_filepath(self, filename, filesuffix=None):
    if filesuffix is None:
        filesuffix = cfg.PARAMS.get('filesuffix', '')

    # 从 BASENAMES 获取物理文件名
    base_name = cfg.BASENAMES[filename]

    # 插入 filesuffix (在基名和扩展名之间)
    stem, ext = os.path.splitext(base_name)
    if filesuffix:
        actual_name = f"{stem}{filesuffix}{ext}"
    else:
        actual_name = base_name

    return os.path.join(self.dir, actual_name)
```

filesuffix 最佳实践

- (1) 使用描述性的后缀名： `'_rcp85'` 比 `'_v2'` 更好。
- (2) 后缀通常以 `'_'` 开头以避免与基本文件名混淆。
- (3) 你的自定义 `@entity_task` 应该使用 `gdir.get_filepath()` 而非手动拼接文件名，以自动享受filesuffix支持。
- (4) 在分析脚本中，使用相同的filesuffix从各种文件中一致地读取数据： `gdir.read_pickle('model_geometry', filesuffix='_rcp85')`。

4.11 GlacierDirectory 类的关键代码量分布

以下是GlacierDirectory类（约1300行）的功能分解：

功能区域	大致行数	说明
构造函数和属性	~200	__init__, grid, rgi_id, 等基础属性
文件路径和I/O	~400	get_filepath, read/write_pickle/shapefile/json/nc
任务状态系统	~150	task_status读写、错误日志
气候和管理	~250	set/get_ref_mb_data, climate_info, monthly_climate
GIS和投影	~150	_reproject_and_write_shapefile, grid属性
辅助方法	~150	reset_directory, intersects, 等各种工具

4.12 总结

GlacierDirectory是OGGM中"关注点分离"原则的典范：

- **对 core 模块透明：**core中的科学代码不需要关心文件存储在何处、如何压缩、采用什么命名——只需要调用 `gdir.read_pickle('centerlines')`
- **对用户透明：**用户不需要手动管理文件路径——filesuffix机制允许无缝的多情景切换
- **对并行透明：**每条冰川的独立目录使其天然适合多进程处理
- **对扩展透明：**BASENAMES允许shop模块在无需修改GlacierDirectory代码的情况下注册新的文件类型

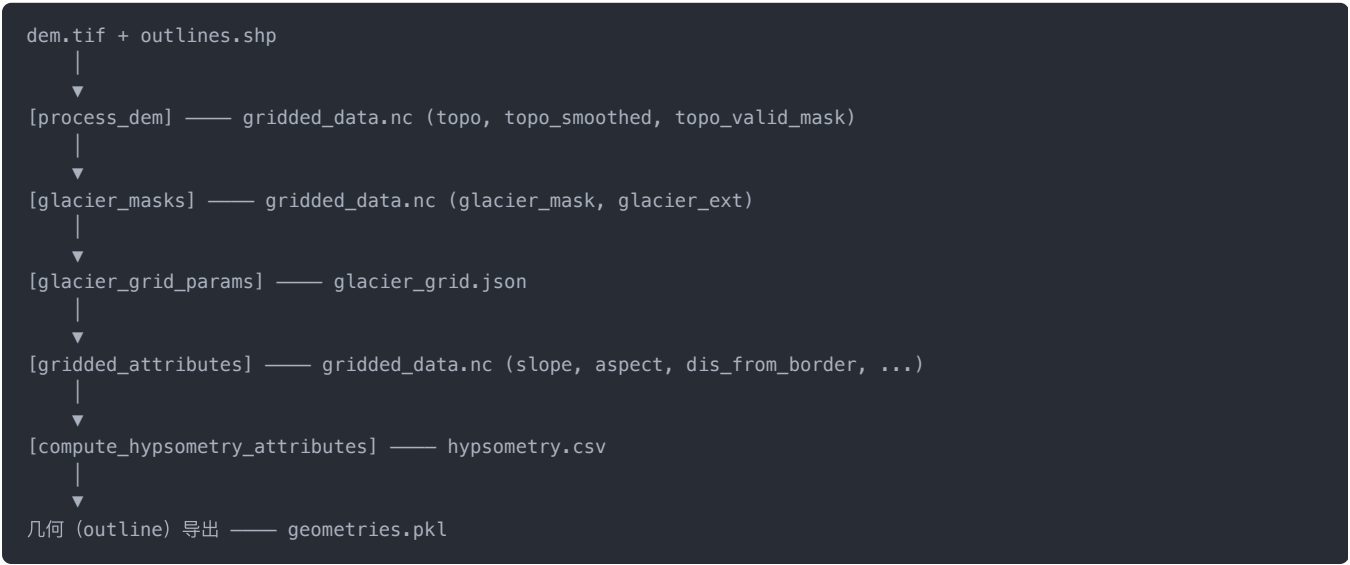
在后续章节中，每当看到 `gdir.read_xxx()` 或 `gdir.write_xxx()` 调用时，请记住本章的内容——这些简洁的接口背后是OGGM精心设计的文件管理架构。

第5章 GIS操作与DEM处理

`gis.py` 是OGGM中代码量较大的单文件模块之一，约2180行代码，承担着所有冰川地理空间处理的入口职责。从原始DEM（Digital Elevation Model）的读取与平滑，到冰川掩码的栅格化（rasterization）生成，再到网格参数的选择和DEM重投影，`gis.py` 将遥感与GIS领域的基础操作封装为OGGM工作流中的标准 `entity_task`。本章按数据处理链路逐步拆解这些关键函数，揭示其内部算法和参数含义。

5.1 模块概览与数据流

在OGGM的完整工作流中，`gis.py` 处于最前端的位置。一条冰川的处理从GIS任务开始，其整体数据流为：



整个流程由 `gis_prepro_tasks` 函数（约第1850行）编排，它将各步骤注册为GlobalGCMModel的任务图节点，确保在正确的工作流阶段执行。

进阶

理解GIS模块的关键

`gis.py` 中的几乎所有函数都操作两种核心数据结构：`glacier_grid.json`（标量参数，如地图投影、网格分辨率）和 `gridded_data.nc`（二维栅格数组，如高程场、掩码）。理解这两类数据的读写模式是阅读本章的先决条件。

5.2 GriddedNcdfFile 上下文管理器

在深入各处理函数之前，必须先了解 `GriddedNcdfFile`（约第682–763行），它是所有栅格数据的统一容器。这个类基于NetCDF4实现了一个上下文管理器，负责单个NetCDF文件中多个二维变量的读写。

```
class GriddedNcdfFile():
    """单个NetCDF文件中管理所有gridded_data变量的上下文管理器"""

    def __init__(self, gdir):
        self.f = None
        self.fid = None
        self.gdir = gdir

    def __enter__(self):
        # 以 'a' 模式打开NetCDF (不存在则创建)
        self.fid = netCDF4.Dataset(self.gdir.get_filepath('gridded_data'), 'a')
        return self

    def create_dim(self, nx, ny): ...
    def create_var(self, name, data): ...
    def read_var(self, name, **kw): ...
    def __exit__(self, ...): ...
```

关键设计决策：使用NetCDF而非多个独立GeoTIFF，是因为单个冰川的栅格数据维度通常不超过200x200像素，将高程、掩码、坡度、坡向等变量储存在一个文件中减少了I/O开销，并且NetCDF自带的元数据机制便于后续分析时的变量发现。

5.3 DEM处理 workflow（process_dem）

`process_dem`（约第767–899行）是整个GIS预处理的核心函数。它接收原始DEM和冰川轮廓，生成平滑后的高程场，是整个模型的地形基础。按处理顺序可分为四个阶段。

5.3.1 第一阶段：读取GeoTIFF DEM

DEM的读取通过 `read_geotiff_dem` 函数委托给 `rasterio` 库完成。这一函数不仅读取高程数组，还同步提取地理参考元数据：

```
def read_geotiff_dem(gdir):
    # 使用rasterio读取DEM GeoTIFF
    with rasterio.open(dem_path) as ds:
        topo = ds.read(1) # 读取第一波段
        # 提取仿射变换参数（左上角坐标 + 像素分辨率）
        transform = ds.transform
    return topo, transform
```

`topo` 是一个二维float64数组，单位为米。当DEM中存在 `NaN` 或负值（表示海洋/空洞）时，后续的空洞填充算法将被触发。

5.3.2 第二阶段：空洞填充（Void Filling）

DEM空洞填充采用两级策略（约第820–845行）。首先对小于 `cfg.PARAMS['max_thin_void']`（默认50像素）的小空洞使用双线性插值（`scipy.interpolate.griddata` 的 `linear` 模式）。对于更大的空洞，使用最近邻外推（`nearest` 模式）。如果空洞覆盖面积超过 `max_void_percentage`（默认1.0），则整个DEM被拒绝并返回错误。

进阶

空洞填充的边界效应

当大空洞出现在冰川区域边缘时，最近邻外推可能引入虚假地形。OGGM在填充前先计算 `topo_valid_mask`（原始有效像素掩码），存于 `gridded_data.nc` 中，供后续坡度计算时排除填充区域。

5.3.3 第三阶段：高斯平滑

平滑是DEM处理中最关键的步骤之一，通过 `gaussian_blur` 函数（约第91–115行）实现：

```
def gaussian_blur(grid, sigma):
    """使用FFT卷积进行高斯平滑"""
    # 1. 生成高斯核
    l = int(sigma * 4)
    x = np.arange(-l, l+1)
    kern = np.exp(-(x**2)/(2*sigma**2))
    kern /= kern.sum()

    # 2. 在频域执行二维分离卷积（先x后y，或先y后x）
    from scipy.signal import fftconvolve
    smoothed_x = fftconvolve(grid, kern[np.newaxis, :], mode='same')
    smoothed = fftconvolve(smoothed_x, kern[:, np.newaxis], mode='same')
    return smoothed
```

平滑窗口大小由 `cfg.PARAMS['smooth_window']` 控制。这里的 `sigma` 被设置为 `smooth_window / dx`（其中 `dx` 是像素分辨率），默认值为3.5格网单位。FFT卷积相比空域卷积在处理大核时性能更优。平滑后的高程场写入 `topo_smoothed` 变量。

5.3.4 第四阶段：裁剪与输出

可选的裁剪参数 `clip_dem0` 将低于海平面（0 m a.s.l.）的像素设为0。最终输出三个变量到 `gridded_data.nc`：`topo`（原始高程）、`topo_smoothed`（平滑高程）、`topo_valid_mask`（有效像素掩码）。

参数名	默认值	作用
<code>smooth_window</code>	3.5	高斯平滑核宽度（像素单位）
<code>topo_interp</code>	<code>'linear'</code>	小空洞填充方法（linear/nearest/spline）
<code>max_thin_void</code>	50	"小型"空洞的最大像素数（超过则用nearest）
<code>max_void_percentage</code>	1.0	空洞面积占DEM的最大允许比例（%）
<code>clip_dem0</code>	<code>False</code>	是否将低于0m的像素强制设为0

5.4 冰川掩码计算（glacier_masks）

`glacier_masks`（约第902–1032行）将矢量形状文件（`outlines.shp`）中的冰川多边形转换为栅格掩码。这是连接矢量GIS数据和栅格模型的桥梁。

5.4.1 多边形到像素坐标的转换

转换分两步进行。首先 `_interp_polygon` 将冰川多边形的顶点等距插值到 `grid_dx` 的分辨率级别——这对于保持小冰川的几何精度至关重要，因为原始RGI多边形可能只有十几个顶点。然后 `_polygon_to_pix` 将地理坐标（经度/纬度或投影米）映射到格网像素索引。

```
def _interp_polygon(poly_pix, dx):
    """在规则步长下插值多边形顶点"""
    # 计算相邻顶点间距，在超过dx距离的顶点间线性插值新点
    dist = np.sqrt(np.sum(np.diff(poly_pix, axis=0)**2, axis=1))
    for i in range(len(dist)):
        if dist[i] > dx:
            n_interp = int(dist[i] / dx)
            # 在poly_pix[i]和poly_pix[i+1]之间线性插入n_interp个点
    ...
```

5.4.2 栅格化（rasterization）与冰原岛处理

多边形栅格化（rasterization）使用 `skimage.draw.polygon` 将多边形边界内的所有像素标记为冰川。对于含有内环（holes/nunataks）的复杂多边形，OGGM通过Shapely分别提取外环和内环——外环填充冰川掩码，内环在掩码中除外（设回0）。此外，函数使用 `scipy.ndimage.label` 识别连通组件，移除面积小于最大连通区5%的孤立小斑块（ice islands，即与主体冰川掩码分离的像素团）。

最终输出写入 `gridded_data.nc`：`glacier_mask`（二值掩码，glacier=1 else=0）和 `glacier_ext`（冰川外轮廓内的所有像素，包括nunatak）。

5.4.3 简化掩码方法（simple_glacier_masks）

`simple_glacier_masks`（约第1034–1163行）是替代方案，当不需要亚像素精度时使用 `rasterio.features.rasterize`（riomask模式）直接栅格化（rasterization）多边形，省去了坐标变换中的插值步骤，速度更快但几何精度略低。

5.5 网格参数选择（glacier_grid_params）

`glacier_grid_params`（约第226–302行）决定每个冰川在其地图投影中的网格分辨率（`dx`）。这是OGGM中最精妙的空间尺度自适应算法之一。

5.5.1 四种分辨率方法

方法	公式/逻辑	适用场景
<code>linear</code>	$dx = d1 * \sqrt{area} + d2$	默认方法，小冰川细网格、大冰川粗网格
<code>square</code>	$dx = \sqrt{area} / 50$	适用于对测高数据敏感的冰厚反演
<code>fixed</code>	$dx = grid_dx_fixed$ （用户指定）	对区域同质网格进行批量处理
<code>by_bin</code>	按面积分箱，每箱一个固定dx	便于与其他模型进行区间比较

`linear` 方法的两个系数 `d1` 和 `d2`（默认14.0和0.0）由 `cfg.PARAMS` 设置，其背后的物理直觉是：冰川的宽度大致与其长度的平方根成正比，因此像素数应保持恒定。参数 `dx_scale` 用于全局缩放所有derived的分辨率。

最终选定的 `dx`、地图投影参数（由 `saalem` 库基于冰川质心位置确定合适的UTM区或Albers等面积投影）、以及网格的 `nx` 和 `ny` 都写入 `glacier_grid.json`。

5.6 DEM重投影（reproject_dem）

`reproject_dem`（约第351–453行）使用 `rasterio.warp.reproject` 将地理坐标系的DEM重投影到每条冰川的特定投影坐标系。重采样算法默认为双线性（`bilinear`），对于坡度等派生变量可用三次（`cubic`）。

```
def reproject_dem(gdir):
    # 确定目标投影（UTM / Albers）和网格参数
    proj_out = gdir.grid.center_grid # 从 glacier_grid.json 读取

    with rasterio.open(src_path) as src:
        reproject(
            source=src.read(1),
            destination=dest_array,
            src_transform=src.transform,
            src_crs=src.crs,
            dst_transform=dst_transform,
            dst_crs=proj_out,
            resampling=Resampling.bilinear)
```

重投影后的高程存储在GlacierDirectory的本地GeoTIFF中，并作为后续所有二维计算的底图。这一步的精度直接决定了网格属性（坡度、坡向）的可靠性。

5.7 网格属性计算（gridded_attributes）

`gridded_attributes`（约第1407–1523行）计算多个二维场，这些场是后续中心线搜索和冰厚反演算法的必要输入：

属性	计算方法	用途
<code>slope</code>	<code>np.gradient(topo_smoothed, dx)</code> 的模	冰厚反演（驱动力）、中心线搜索
<code>aspect</code>	<code>np.arctan2(dy, dx)</code>	海拔带分析、太阳辐射
<code>slope_factor</code>	<code>max(tan(slope), min_slope)</code>	反演中防止近零坡度导致的数值爆炸
<code>dis_from_border</code>	<code>scipy.ndimage.distance_transform_edt</code> 取欧氏距离	中心线成本网格、按海拔分配厚度
<code>ice_divides</code>	分水岭算法在水文流向图上执行	支流划分、集水区面积计算

其中 `slope_factor` 的 `min_slope` 默认值为 `tan(1.5°)`，低于此坡度的像素被提升到此阈值，避免冰厚反演中除零或产生无穷大厚度。

5.8 测高曲线计算（compute_hypsometry_attributes）

`compute_hypsometry_attributes`（约第1165–1274行）以50m为固定高程箱计算每个海拔段内的面积及其占冰川总面积的千分比（permil）。输出为 `hypsometry.csv`，表头包括 `topo_bin`、`area`、`permil` 三列。

```
# 简化逻辑
bins = np.arange(min_elev // 50 * 50, max_elev + 50, 50)
for i in range(len(bins)-1):
    area_in_bin = np.sum((topo >= bins[i]) & (topo < bins[i+1])
                        & glacier_mask) * dx**2
    hyps[i, :] = [bins[i]+25, area_in_bin, area_in_bin / total_area * 1000]
```

测高曲线是物质平衡校准和面积–高度分布匹配的关键输入，在第8章的物质平衡模型中将反复引用。

5.9 GIS任务的完整 workflow

`gis_prepro_tasks` 函数（约第1850行）将所有上述步骤编排为任务图。这允许OGGM以任意顺序构建依赖关系并并行化处理：

```
def gis_prepro_tasks(gdirs):
    task_list = [
        taskprocess_dem,           # 1. DEM处理
        taskglacier_masks,        # 2. 冰川掩码
        taskglacier_grid_params,  # 3. 网格参数
        taskglacier_mu_candidates, # 4. 物质平衡参考期
        taskcompute_centerlines,  # 5. → 第6章
        ...
    ]
```

5.10 关键参数速查

参数名	所属函数	默认值	描述
<code>border</code>	<code>process_dem</code>	<code>40</code>	冰川周围缓冲区的像素数
<code>grid_dx_method</code>	<code>glacier_grid_params</code>	<code>'linear'</code>	网格分辨率选择方法
<code>d1</code>	<code>glacier_grid_params</code>	<code>14.0</code>	<code>linear</code> 方法的缩放系数
<code>d2</code>	<code>glacier_grid_params</code>	<code>0.0</code>	<code>linear</code> 方法的偏移量
<code>dx_scale</code>	<code>glacier_grid_params</code>	<code>1.0</code>	全局网格分辨率缩放
<code>topo_interp</code>	<code>process_dem</code>	<code>'linear'</code>	空洞填充插值方法
<code>smooth_window</code>	<code>process_dem</code>	<code>3.5</code>	高斯平滑窗口（像素）
<code>clip_dem0</code>	<code>process_dem</code>	<code>False</code>	低于海平面裁剪
<code>min_slope</code>	<code>gridded_attributes</code>	<code>$\tan(1.5^\circ)$</code>	最小坡度因子阈值

5.11 总结

`gis.py` 是OGGM的基础模块——每一寸冰川模型的构建都始于这里的DEM处理。从空洞填充到高斯平滑，从多边形栅格化（`rasterization`）到网格分辨率自适应选择，这些看似基础的操作中蕴含着丰富的冰川学工程权衡：如何在精度和计算效率之间取得平衡，如何处理真实DEM中的各种瑕疵，如何让同一套代码适用于从 0.1km^2 到 10000km^2 的冰川。掌握GIS模块的细节，对理解后续所有模型输出的物理含义至关重要。

下一章将进入OGGM最具原创性的算法领域——冰川中心线提取和几何参数化。在那里，将看到本章生成的地形属性（坡度、坡向、边界距离）如何被转化为中心线优化算法的成本函数。

第6章 冰川中心线与几何参数化

`centerlines.py` 约2454行，是OGGM中算法密度最高的模块。它的任务是将一个二维冰川掩码（第5章的产物）转化为一组合适的流线（flowlines）——每条流线都携带几何属性（宽度、面积-海拔分布），并组织为有向无环图（DAG）来模拟支流汇入主干的拓扑关系。本章沿着数据转换的完整链路，深入Kienholz et al. (2014) 最短路径算法的每个细节。

6.1 Centerline类：几何与拓扑的容器

`Centerline` 类（约第69行）封装了一条流线的信息。流线在代码中表示为沿中心线的等距坐标点序列：

```
class Centerline():
    def __init__(self, line, dx=None, surface_h=None):
        self.line = line          # shapely LineString: N个等距点
        self.dis_on_line = ...    # 距终点的沿程距离 [m]
        self.nx = len(line.coords)
        self.order = ...         # Strahler-like序列号
        self.flows_to = ...       # 下游汇入的另一条Centerline对象
        self.inflows = []        # 汇入本线的上游Centerline对象列表
```

关键设计是 `flows_to` 和 `inflows` 构建的有向无环图（DAG）。通过 `inflows` 可以遍历所有上游支流，通过 `line_order()` 可以建立Strahler排水次序式的拓扑排序——从未梢支流开始在依次向上游推进到主干。

入门

中心线 vs 流线

在OGGM中，“中心线”（centerline）指从冰川头部到末端的完整几何路径（通常在`costgrid`中找到），而“流线”（flowline）由 `elevation_band_flowline` 从中心线派生，是沿各海拔带“宽度折叠”的一维表示。二者都使用 `Centerline` 类。

6.2 Voronoi成本网格（`_make_costgrid`）

`_make_costgrid`（约第684–714行）是中心线提取的核心，将冰川掩码转换为“通过成本最小的路径从末端走到头部”的成本图。成本函数源自Kienholz et al. (2014) 的方法：

```
cost = ((dmax - dis) / dmax * f1)^a + ((z - zmin) / (zmax - zmin) * f2)^b
```

成本由两项加权和构成：

- **距离项**： `(dmax-dis)/dmax` ——距冰川轮廓边界越远的像素成本越低（中央像素受到偏好）。`f1` 默认0.3，控制距离项的贡献。
- **高程项**： `(z-zmin)/(zmax-zmin)` ——高程越低的像素成本越低（模拟冰的河流式流动特征）。`f2` 默认0.7，使得高程项权重更高，确保流线向下行走。

指数 `a` 和 `b`（默认均为1）控制各项的敏感度。此外，对冰川掩码外部和边界像素施加50倍惩罚因子，有效阻止路径“逃出”冰川区域。成本网格的可视化通常呈现为从末端到头部的平滑成本梯度。

6.3 冰川头部检测（`_get_centerlines_heads`）

冰川“头部”是中心线的起点，位于冰川支流的最上游端。`_get_centerlines_heads`（约第775–817行）使用局部高程极大值检测和自适应过滤来识别它们。

6.3.1 局部极大值检测

对地形数据应用 `scipy.signal.argrelmax`，扫描高程场的每个像素，当中心像素值高于其邻域所有像素时标记为局部极大值。参数 `order=5` 控制比较范围的窗口大小。

6.3.2 自适应过滤

Kienholz et al. (2014) 中定义了一个与冰川面积相关的缓冲区半径：

```
radius = q1 * area + q2
```

其中 `q1` 和 `q2` 是经验系数（默认 `q1=0.15`，`q2=0`）。`_filter_heads` 函数在此半径内仅保留高程最高的那个局部极大值作为头部候选点。候选点还需要高于高程分布的第33百分位数，过滤掉地形噪声中的微小峰值。

参数 `rmax` 限制了能检测到的最大头部数量。对于喜马拉雅等山地冰川，通常每个流域产生2–5个头部。

6.4 最优路径路由 (`compute_centerlines`)

`compute_centerlines`（约第874–977行）是中心线提取的主控函数。对每个检测到的头部，从该像素到冰川末端计算一条成本最低的路径。

6.4.1 最短路径算法

路径搜索通过 `skimage.graph.route_through_array` 实现，它在成本网格上直接执行Dijkstra-like搜索。指定起点（头部）和终点（末端）后，返回一条成本积分最小的路径。

6.4.2 末端识别

末端的确定取决于冰川类型。对于入海型冰川（tidewater），末端为冰川高程分布的某百分位数处（默认第5百分位，确保位于冰舌断裂线附近）。对于陆地终止冰川，末端为冰川范围内的最低高程像素。

6.4.3 线的过滤与连接

中心线后处理包含四个级联步骤：

- `_filter_lines`：保留最长的一条线，然后逐步缓冲剩余线——短于主线的重叠线被移除，避免多条线堆积在同一主干上。
- `_filter_lines_slope`：移除沿高程上升行走的线（物理上不合理的“上坡中心线”），通过检查沿线高程的趋势符号实现。
- `_join_lines`：将支流通过几何投影连接到主干的对应交点位置。支流的终点不是主干的终点，而是在主干的side pixel处终止。
- `_line_order`：对整个DAG进行拓扑排序，给每条线赋予 `order` 值，类似于河流的Strahler排序，主干最高、小支流最低。

```
def _join_lines(cls, line, main_line, gdir):
    # 1. 获取line的终点像素坐标
    # 2. 在main_line上找到距离最近的像素（几何投影）
    # 3. 沿main_line回溯到line的终点高程之上2dx，作为交汇点
    # 4. 截断line使其在交汇点结束
    # 5. 设置 line.flows_to = main_line
```

6.5 宽度计算 (`_point_width`)

`_point_width`（约第1375–1444行）计算中心线每个点的冰川宽度。算法如下：

1. 在离散点处沿法线方向取长度为 `kbuffer * map_dx` 的线段。

2. 该线段与冰川集水区多边形求交集。
3. 交集的长度即为该点处的冰川宽度。
4. 如果该点在nunatak（岩壁）内部，从宽度中减去nunatak覆盖的部分。

参数 `kbuffer`（默认3.0）确保法线段足够长以跨越整个冰川宽度区域。Nunatak的减法处理是OGGM区别于许多其他模型的重要特性，保证了宽度参数的真实性。

6.6 集水区面积与宽度校正

6.6.1 集水区面积

`catchment_area`（约第1549–1662行）通过 `saalem.Grid` 的分水岭算法将冰川划分为多边形的子流域——每个多边形代表一条中心线的汇水区域。集水区边界遵循冰面地形的水文线路，类似于流域边界的概念。

6.6.2 宽度校正（`catchment_width_correction`）

`catchment_width_correction`（约第1935–2081行）是一个关键的质量保证步骤。它确保各中心线的面积–高度分布与RGI数据库中该冰川的已知面积相匹配：

1. 将每条中心线的宽度–高程积分为面积–高度分布（分箱为50m高程间隔）。
2. 将各中心线的曲线累加，得到总分布。
3. 对每个高程箱进行缩放，使总分布与RGI面积一致（bin-based校正）。
4. 最终对每条线的所有点的宽度做全局缩放，使总面积精确匹配RGI面积。

这个校正步骤确保了OGGM中的几何参数化与RGI的全球冰川清单保持一致，使得区域到全球尺度的体积估算具有可比性。未校正的宽度积分面积与该冰川RGI面积之间的偏差通常小于5%，但仍需精确校正。

6.7 海拔带流线（`elevation_band_flowline`）

`elevation_band_flowline`（约第2181–2352行）是OGGM 1.4+ 中引入的“折叠冰川”概念的实现（Huss 2012, Werder 2019）。它将每条中心线压缩为一维表示：

```
# 沿中心线将每个海拔段表示为一个“盒状”单元
dx = bsize / tan(slope) # 盒子的长度：高程步长 / 坡度
width = area / dx       # 盒子的宽度：面积 / 长度
```

其中 `bsize` 是固定的海拔步长（通常200m），`slope` 是该海拔段顶部高程处的当地表面坡度，`area` 是该海拔段内中心线集水区的总面积。这个方案将二维冰川问题映射为一维，极大地加速了冰流模型的数值求解（第10章），同时保留了冰川的整体几何特征。

进阶

折叠冰川与二维模型的权衡

折叠冰川方法约化了冰的横向流动——所有冰在同一个海拔段内被假设为垂直于中心线的均匀等速流。对于宽冰川和入海型冰川（其中横向应力梯度重要），这可能导致系统性偏差。Werder等人（2020）的Shallow Ice Approximation模拟显示，1D折叠模型与2D SIA模型在全球冰川体量估算中仅相差约3%。

6.8 下游线 (Downstream Line)

每条中心线都附带一条下游线 (`downstream_line`) ——从冰川末端沿地形向更低海拔延伸的线。它用于：

- 物质平衡模型中确定冰川末端以下的高程外推点
- 冰流模型中定义冰川前缘之外区域的等温条件
- 为入海型冰川计算海底线 (submarine bed) 以估算崩解通量

下游线由 `_downstream_line` 函数通过沿地形梯度步进生成，步进终点为指定的最低高程或最大长度。

6.9 关键参数速查

参数名	所属函数	默认值	描述
<code>flowline_dx</code>	Centerline构造	<code>map_dx * 2.0</code>	中心线离散点的间距
<code>q1</code>	<code>_get_centerlines_heads</code>	<code>0.15</code>	头部缓冲半径的面积系数
<code>q2</code>	<code>_get_centerlines_heads</code>	<code>0.0</code>	头部缓冲半径的偏移量
<code>rmax</code>	<code>_get_centerlines_heads</code>	<code>10</code>	最大头部数量限制
<code>f1</code>	<code>_make_costgrid</code>	<code>0.3</code>	距离项在成本中的权重
<code>f2</code>	<code>_make_costgrid</code>	<code>0.7</code>	高程项在成本中的权重
<code>a</code>	<code>_make_costgrid</code>	<code>1.0</code>	距离项指数 (灵敏度)
<code>b</code>	<code>_make_costgrid</code>	<code>1.0</code>	高程项指数 (灵敏度)
<code>kbuffer</code>	<code>_point_width</code>	<code>3.0</code>	法线缓冲宽度倍数
<code>tidewater_percentile</code>	<code>compute_centerlines</code>	<code>5</code>	入海型冰川末端高程百分位数

6.10 总结

`centerlines.py` 体现了OGGM工程设计的核心思想：将复杂的冰川几何问题转换为图论中的最短路径问题。Voronoi成本网格、自适应头部检测、流线合并和宽度校正这四个模块相互衔接，每一步都有清晰的物理直觉作支撑。中心线的质量直接决定了后续物质平衡、冰厚反演和动力学演化的精度——一条偏心的中心线会导致错误的宽度、错误的面积-高度分布，并最终影响冰量变化的所有预测。

在下一章中，将看到气候输入数据如何通过 `climate.py` 和 `shop/` 模块变换为标准化的 `climate_historical.nc` 文件，为物质平衡模型提供气象驱动。

第7章 气候数据处理与Shop模块

`climate.py` (约384行) 和 `shop/` 模块共同构成了OGGM的气候数据供应链。不同于地表高程数据 (以单个GeoTIFF形式直接可用), 气候数据来自多个来源——CRU TS、ERA5、GSWP3-W5E5、CMIP5/6 GCMs——格式各异, 需要统一处理为标准化版本才能提供给物质平衡模型。本章深入解析这一数据标准化工作流的每个环节。

交叉方向读者先看这里

如果你的目标是把站点气象、区域再分析或水文模型气候输入接入OGGM, 最低要求是准备月尺度温度、月总降水、参考高程和时间坐标。先让数据写成OGGM标准的 `climate_historical.nc`, 再进入第8章物质平衡模型; 不要直接跳到动力学模拟。

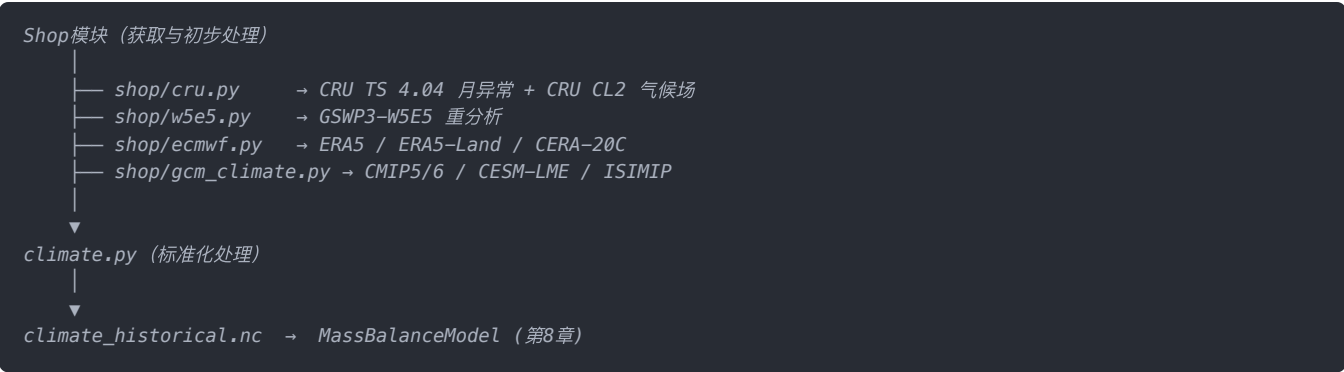
高亚洲气候数据提醒

青藏高原、天山、祁连山等区域的降水低估和高程订正误差常常比动力学参数误差更先控制结果。使用默认气候数据时, 应检查 `prcp_fac`、`temp_bias`、参考高程和物质平衡校准结果, 并尽量用本地观测或区域产品进行独立验证。

7.1 气候数据架构概览

OGGM将气候数据生产划分为两个层级: `climate.py` 作为顶层调度器, 接受源格式的NetCDF文件并写出标准化的 `climate_historical.nc`; `shop/` 下的各模块负责从原始数据源下载和初步预处理——即"数据获取层"和"格式标准化层"的分离。

完整数据流如下:



进阶

为什么要标准化气候格式?

不同气候数据集的时间分辨率和空间参考各不相同: CRU是月均0.5度网格, ERA5是小时级0.25度网格, CMIP6 GCM可能达到1.5度。标准化过程将所有数据统一为OGGM要求的月均NetCDF格式 (变量: `temp`, `prcp`, `ref_hgt`, `time`), 并与冰川的高度带对齐。这种标准化使物质平衡模型可以被"可插拔"的气候数据源驱动。

7.2 标准化NetCDF格式

`write_monthly_climate_file` 定义了所有气候文件的标准Schema:

变量	维度	单位	描述
temp	(time,)	deg C	月均温度
prcp	(time,)	kg m ⁻² month ⁻¹ (即 mm month ⁻¹)	月总降水量
ref_hgt	标量	m	参考高程 (温度对应的海拔)
time	(time,)	days since 1800-01-01	CF惯用的时间编码
gradient	(time,)	K m ⁻¹	温度梯度 (可选, 否则用-0.0065 K/m)

注意降水不是速率 (mm/s), 而是月累积量 (mm/month)。这使得在物质平衡中可以直接乘以物质平衡天数估算月度融化与积累。温度以摄氏度 (而非开尔文) 为单位, 避免了在每次物质平衡计算中都要减去273.15。

7.3 顶层调度函数 (process_climate_data)

`process_climate_data` (约第138-222行) 根据 `cfg.PARAMS['baseline_climate']` 的设置路由不同的处理策略:

```
def process_climate_data(gdir):
    src = cfg.PARAMS['baseline_climate']

    if src == 'CRU':
        # 从CRU CL2 + CRU TS构建
        process_cru_data(gdir)
    elif src == 'ERA5':
        # 从ECMWF ERA5构建
        process_ecmwf_data(gdir)
    elif src == 'W5E5':
        # 从GSWP3-W5E5构建
        process_w5e5_data(gdir)
    elif src == 'CUSTOM':
        # 用户提供的NetCDF
        process_custom_climate_data(gdir)
    elif src == 'HISTALP':
        # 欧洲山地HISTALP数据集
        process_histalp_data(gdir)
```

`process_custom_climate_data` (约第36-135行) 是通用入口: 读取用户提供的NetCDF文件, 验证变量、单位和维度, 然后执行必要的最近邻空间插值和垂直外推。

7.4 历史Delta方法

`historical_delta_method` (约第225-383行) 是处理"混合气候数据集"的核心技术——当你有两个不同时期、空间分辨率或精度的气候源时如何将它们融合。该方法广泛应用于CRU (将低分辨率CRU TS异常叠加到高分辨率CRU CL2气候场上)。

7.4.1 温度: 加法异常

对温度使用可加性异常 (anomaly):

T_recon(t) = T_clim + (T_obs(t) - T_obs_clim)

即: 目标温度的长期平均 + 观测异常。这里隐含假设温度异常在不同空间分辨率下是保守的——0.5度网格上的年际温度距平可以合理地放在更细网格的30年平均场上。这个假设在大多数山地气候中成立。

7.4.2 降水：乘法缩放

对降水首选乘法（比率）缩放：

```
P_recon(t) = P_clim * (P_obs(t) / P_obs_clim)
```

比率法隐含假设降水的相对变化（而非绝对变化）在不同分辨率下是保守的。当 `P_obs_clim` 接近零时（如干旱月份），`/` 会导致数值爆炸。此时会降级为加法异常法（`fallback='additive'`）。

7.4.3 混合数据集

当两个数据集有重叠时间窗口时（如CERA-20C覆盖1901–2010，ERA5覆盖1950–至今），会先计算重叠期偏差，再合并时间序列。标准差缩放通过滚动窗口确保合并后的方差连续性。

7.5 Shop模块：CRU TS

`shop/cru.py` 实现了经典的CRU TS方案。CRU CL2提供1961–1990年的30年平均气候场（10'分辨率 ~18km格网），CRU TS 4.04提供1901年以来的月异常（0.5度分辨率）。

7.5.1 3x3窗口选择

对于每条冰川，在不规则CRU网格中选择距离冰川质心最近的3x3=9个格点单元。每个单元包含该月异常的时间序列。

7.5.2 异常插值

9个CRU TS格点上的异常通过三线形插值（经度、纬度、月份维度）映射到CRU CL2细网格的每个格点——这是一种 *coarse-to-fine* 的方案。最后取冰川质心所在CRU CL2格点的12个月温度场，叠加CRU TS异常，得到完整的1901至今的月气候序列。

7.6 Shop模块：W5E5 / GSWP3–W5E5

`shop/w5e5.py` 处理GSWP3–W5E5数据集——这是目前最受欢迎的气候驱动数据之一，覆盖1901–2019年（之后延续至ERA5），使用0.5度分辨率。数据存储为“pre-flattened”格式（已将二维格网展平为一维数组存储于NetCDF中），通过最近邻选择找到冰川对应的格点。

单位转换是W5E5处理的关键步骤：

```
# W5E5原始数据单位
temp_raw = ... # 开尔文 [K]
prcp_raw = ... # kg m^{-2} s^{-1} (降水速率)

# 转换为OGGM标准单位
temp_C = temp_raw - 273.15 # K → °C
prcp_mm = prcp_raw * 86400 * days_in_month # kg m^{-2} s^{-1} → mm month^{-1}
```

在pre-flattened格式中，格点的索引已预先计算、存储，直接读取一行即可获取该格点的全部时间序列，将IO开销降至最低。

7.7 Shop模块：ECMWF

`shop/ecmwf.py` 是最复杂的shop模块之一，需处理四种ECMWF数据产品：

数据集	分辨率	时间覆盖	说明
ERA5	0.25°	1950–至今	最常用的现代再分析
ERA5–Land	0.1°	1950–至今	陆地表面增强版，地形更精细
CERA–20C	1.125°	1901–2010	耦合气候再分析，10个集合成员
ERA5dr	0.25°	1950–至今	降尺度版本的ERA5 for 山地研究

ECMWF数据以两种格式存储——“flattened”（时空展平）和标准gridded（四维：时间、垂直层、纬度、经度）。模块通过 `auto_detect` 机制识别格式，自动选择合适的读取路径。CERA–20C的10个集合成员通过取算术平均（ensemble mean）来处理。

7.8 Shop模块：GCM气候

`shop/gcm_climate.py` 处理全球气候模式（GCM）的输出，包括CMIP5/6、CESM–LME、LMR、ModE–RA等。这些数据的挑战在于：每个GCM有自己的气候态偏差（bias），直接使用原始输出会导致物质平衡偏差。

7.8.1 标准异常方法

OGGM采用“异常方法”消除GCM偏差：

```
T_recon(t) = T_ref_clim + (T_gcm(t) - T_gcm_clim)
P_recon(t) = P_ref_clim * (P_gcm(t) / P_gcm_clim)
```

其中 `T_ref_clim` 和 `P_ref_clim` 来自观测/再分析参考场（如W5E5），`T_gcm_clim` 和 `P_gcm_clim` 是GCM在参考期（通常1980–2010）的自身气候态。这保留了GCM的年际变率和趋势信号，而使用参考数据的绝对值避免偏差影响物质平衡的绝对水平。

7.8.2 ISIMIP3b路径

对于CMIP6的ISIMIP3b数据，GCM输出已经过外部偏差校正（bias correction）。OGGM通过 `apply_bias_correction=False` 参数跳过自身的异常校正。ISIMIP3b是OGG–M/IPCC评估中的首选CMIP6数据源。

7.9 气候数据源比较

数据源	时期	分辨率	变量	Shop路径
CRU TS 4.04 + CL2	1901–2019	10' / 0.5°	T, P	<code>shop/cru.py</code>
GSWP3–W5E5	1901–2019	0.5°	T, P, SW↓, LW↓, etc.	<code>shop/w5e5.py</code>
ERA5	1950–至今	0.25°	T, P, SW↓, LW↓	<code>shop/ecmwf.py</code>
ERA5–Land	1950–至今	0.1°	T, P	<code>shop/ecmwf.py</code>
CERA–20C	1901–2010	1.125°	T, P	<code>shop/ecmwf.py</code>
CMIP5 GCMs	1861–2100	~1–2°	T, P	<code>shop/gcm_climate.py</code>
CMIP6 ISIMIP3b	1850–2100	0.5°	T, P	<code>shop/gcm_climate.py</code>
HISTALP	1800–2014	站点/区域	T, P	<code>shop/histalp.py</code>

7.10 关键参数速查

参数名	默认值	作用
baseline_climate	'W5E5'	历史基线气候数据选择
hydro_year_month	10 (北半球) / 5 (南半球)	水文年开始月 (与冰川年对齐)
temp_use_gradient	True	使用气候文件的温度梯度进行垂直外推
temp_default_gradient	-0.0065	默认温度递减率 (K/m)
climate_qc_max_months	3	插值前允许的最大缺测月数
clip_negative_prcp	True	将负降水值 (数值误差) 设为0
temp_bias	0.0 (用户设定)	系统温度偏移 (校准参数, 单元K)

7.11 总结

`climate.py` 和 `shop/` 模块是OGGM的"数据桥梁"——将来源各异的气候数据源转化为统一的、可直接用于物质平衡模型的格式。历史Delta方法体现了气候数据融合中的经典策略：长期气候场 (来自高分辨率数据) + 年际异常 (来自长序列数据) 的组合。理解这层转换逻辑，对于正确解读模型的物质平衡模拟至关重要——尤其是当切换气候数据源时，温度偏差和降水比率的差异会系统性地改变模拟结果。

下一章将直接使用这里生成的 `climate_historical.nc` 来构建物质平衡模型——从简单的线性模型到温度指数模型 (temperature-index model) 和完全的月分量模型。

第8章 物质平衡模型：理论与实现

`massbalance.py` 约2656行，是OGGM中规模最大的物理模块。它将气候输入（第7章）和冰川几何（第6章）转化为不同海拔处的积累、融化和净物质平衡。本章从最基础的标量物质平衡开始，逐步深入到月度温度指数模型（`MonthlyTModel`, `temperature-index model`）、参数校准框架以及多流线系统中的物质平衡分配。

本章最少要掌握什么

初学者先抓住一条主线：温度和降水经过高度订正后，被月度温度指数模型（`MonthlyTModel`, `temperature-index model`）转换为积累、融化和净物质平衡；校准过程再用大地测量或原位观测约束 `melt_f`、`temp_bias` 和 `prcp_fac`。8.5以后涉及多种校准函数，第一遍只需理解它们服务于同一个目标：让模型的平均物质平衡与观测一致。

单位提醒

OGGM内部动力学接口常使用 m ice s^{-1} ，而观测、论文和诊断图常使用 mm w.e. yr^{-1} 或 $\text{kg m}^{-2} \text{ yr}^{-1}$ 。读代码时看到物质平衡数值异常小，往往是因为它仍处在内部单位，而不是模型没有融化或积累。

8.1 MassBalanceModel基类与设计哲学

`MassBalanceModel`（约第35行）是所有物质平衡模型的抽象基类。其设计遵循OGGM特有的 `SuperclassMeta` 元类模式，这提供了一种“轻量级抽象基类”——在实例化时检查子类是否实现了必需的方法，而不是在导入时：

```
class MassBalanceModel(object, metaclass=SuperclassMeta):
    def __init__(self, valid_bounds=(-1e4, 1e4), fl_id=0):
        self.valid_bounds = valid_bounds
        self.fl_id = fl_id

    def get_monthly_mb(self, heights, year=None):
        raise NotImplementedError()

    def get_annual_mb(self, heights, year=None):
        return np.sum(self.get_monthly_mb(heights, year=year), axis=0)
```

关键接口（子类必须实现的方法）：`get_monthly_mb(heights, year)` 在给定浮点年份上返回各海拔点的月物质平衡，OGGM内部单位为 m ice s^{-1} ；`get_annual_mb` 返回年平均意义下的物质平衡，单位同样为 m ice s^{-1} 。在与观测比较或写入诊断量时，OGGM再按冰密度和时间长度换算为 kg m^{-2} 、 m w.e. 或 mm w.e. yr^{-1} 。`heights` 是各网格点的高度（米），`year` 指定气候年份。

入门

物质平衡的符号约定

在OGGM中，负值表示物质损失，正值表示物质增加。内部动力学常用冰当量速度（ m ice s^{-1} ），而输出和观测常用水当量（`water equivalent`, `w.e.`; mm w.e. yr^{-1} ）。冰密度默认取 900 kg m^{-3} ，因此 1 m w.e. 约相当于 1.11 m 冰当量。

8.2 简化物质平衡模型

8.2.1 ScalarMassBalance

`ScalarMassBalance` (约第258行) 是最简单的模型, 在所有海拔上施加均匀的物质平衡: $MB(z) = \text{const}$ 。主要用途包括:

- 单元测试和sanity check
- 概念性实验 (如研究均匀MB下的冰川响应)
- 初始化迭代求解器

8.2.2 LinearMassBalance

`LinearMassBalance` (约第286行) 是经典线性模型:

```
MB(z) = (z - ELA) * grad
```

其中 `ELA` 是平衡线高度 (m), `grad` 是MB梯度 ($\text{mm w.e. m}^{-1} \text{ year}^{-1}$, 通常约0.007–0.010)。它提供了一个 `temp_bias` 属性: 温度每升高1K, `ELA` 上升150m (经验法则: 150 m K^{-1})。这是一个粗略但有物理依据的近似。

8.3 MonthlyTIModel: 月度温度指数模型 (MonthlyTIModel, temperature-index model)

`MonthlyTIModel` (约第349行) 是OGGM的"常用"物质平衡模型, 每月的物质平衡通过温度指数 (degree-day) 方法计算融化量、通过降水相分离分配积累:

8.3.1 融化计算

```
M_m = melt_f * (365/12) * max(T - T_melt, 0)
```

其中 `T_melt` 是融化阈值温度。OGGM v1.6.3的默认值为 -1.0°C , 而不是 0.0°C ; 在月时间尺度上, 月均温度低于 0°C 时仍可能出现日间融化, 负阈值可以经验性地吸收这类亚月尺度效应。 `melt_f` 在配置文件中的单位为 $\text{kg m}^{-2} \text{ day}^{-1} \text{ K}^{-1}$, 数值上等价于 $\text{mm w.e. day}^{-1} \text{ K}^{-1}$ 。

8.3.2 降水相判定

固态降水比例通过线性过渡区确定:

```
f_solid = clip(1 - (T - T_solid) / (T_liquid - T_solid), 0, 1)
```

其中 `T_solid=0°C` (低于此温度全部视为固态降水), `T_liquid=2°C` (高于此温度全部视为液态降水)。在 $0-2^\circ\text{C}$ 之间, 雨雪比例线性过渡, 例如 1°C 时固态比例为50%。这是温度指数模型 (temperature-index model) 中常用的经验近似, 而不是对降水相态微物理过程的显式模拟。

```
Prcp_sol(z) = Prcp(z) * f_solid(T(z))
Prcp_liq(z) = Prcp(z) * (1 - f_solid(T(z)))

MB_monthly(z) = Prcp_sol(z) - M_m(z) # kg m-2 month-1
```

8.3.3 温度降尺度

温度通过梯度从参考高度外推到各海拔：

$$T(z) = T_{\text{ref}} + \text{grad} * (z - \text{ref_hgt})$$

默认梯度 `grad = -0.0065 K/m`（标准大气递减率量级）。如果气候文件中提供了 `gradient` 变量，则优先使用。气候数据的访问通过 `get_monthly_climate(heights, year)` 方法处理，返回四个数组：`temp`、`tempformelt`、`prcp`、`prcpsol`。

进阶

温度梯度的时间可变性

CRU和W5E5等月尺度数据通常需要从参考高程外推到冰川高程带。实际温度递减率会随季节、天气型和地形条件变化；在热带、高山干旱区或逆温频发地区，固定梯度可能引入系统性误差。

8.4 物质平衡模型的层次结构

8.4.1 ConstantMassBalance

`ConstantMassBalance`（约第705行）通过多年平均得到一个固定的物质平衡气候。它覆盖 `[y0-halfsize, y0+halfsize]` 窗口内的年份，并缓存相应的温度和固态降水信息，适合平衡态实验、`spinup`或理想化敏感性试验。

8.4.2 RandomMassBalance

`RandomMassBalance`（约第916行）从历史气候序列中随机抽选年份，生成逐年的物质平衡序列。用于蒙特卡罗敏感性实验——在不确定未来气候路径时，以历史年份的随机排列作为替代。

8.4.3 UncertainMassBalance

`UncertainMassBalance`（约第1062行）是在另一个MB模型上应用高斯噪声的装饰器（decorator模式）——对底层模型的参数（如`melt_f`、`temp_bias`）叠加噪声，输出带有误差条的物质平衡。用于参数不确定性的传播分析。

8.4.4 MultipleFlowlineMassBalance

`MultipleFlowlineMassBalance`（约第1178行）处理有支流的冰川系统。它将物质平衡模型分派到各条流线，并用 `fl_id` 等机制区分不同分支；在合并冰川或多冰川系统中，也可通过气候文件后缀（`filesuffix`）管理不同冰川目录（`GlacierDirectory`）对应的气候输入。

8.5 物质平衡校准框架

物质平衡校准是OGGM中最重要的技术环节之一。模型的核心参数（`melt_f`、`temp_bias`、`prcp_fac`）不应被当作任意调节旋钮；它们需要通过观测约束来确定，使模型输出与大地测量或原位观测的冰川平均物质平衡一致。

8.5.1 冬季降水因子（precipitation factor; decide_winter_precip_factor）

`decide_winter_precip_factor`（约第1452行）根据冬季降水设置经验性降水因子（precipitation factor）：

```
prcp_fac = empirical_function(winter_prcp, ...)
```

其逻辑基于：冬季降水丰富的地区（如挪威沿海）的累积过程主要受限于降水，气候数据在捕获此过程时可能存在系统性偏差。经验因子来自全球冰川观测数据的统计拟合。

8.5.2 标量MB校准 (`mb_calibration_from_scalar_mb`)

`mb_calibration_from_scalar_mb`（约第1968行）是核心校准函数，使用三步优化策略：

1. 步1: `prcp_fac`——通过固定 `melt_f` 和 `temp_bias`，校准到给定降水因子（precipitation factor）使得MB参考期年均MB匹配目标值。使用 `brentq`（一维括号寻根）。
2. 步2: `melt_f`——固定 `prcp_fac`，调节 `melt_f` 使MB匹配目标。
3. 步3: `temp_bias`——固定前两个参数，微调 `temp_bias` 做最终对齐。

8.5.3 大地测量MB校准 (`mb_calibration_from_geodetic_mb`)

`mb_calibration_from_geodetic_mb`（约第1774行）使用Hugonnet et al. (2021) 的卫星测高数据（`dmdtda`），配合 `informed_threestep` 方法：

```
# informed_threestep 的核心流程 (Huss & Hock 2015)
# 1. prcp_fac: 来自冬季降水的经验公式
# 2. melt_f: Brent校准到大地测量MB (dmdtda观测)
# 3. temp_bias: 上一步的余差调整
```

大地测量物质平衡（geodetic mass balance）数据提供2000–2020年期间的冰川高程变化率和质量变化估计。相比传统地面测量，卫星测高数据覆盖范围更广，但仍受DEM误差、穿透深度、密度换算和地形遮蔽等因素影响；它是全球尺度校准的重要约束，而不是无误差真值。

8.5.4 WGMS原位测量校准 (`mb_calibration_from_wgms_mb`)

`mb_calibration_from_wgms_mb`（约第1506行）使用世界冰川监测服务（WGMS）的实际冰川测量数据。校准逻辑是逐年的：寻找 `temp_bias` 使得模拟的年度MB序列与WGMS测量值最接近。

8.5.5 全时间序列校准 (`mb_calibration_to_rmsd`)

`mb_calibration_to_rmsd`（约第1532行）是最全面的校准方法，使用 `scipy.optimize.differential_evolution`（差分演化全局优化）在整个观测年范围内最小化模拟和观测MB之间的RMSD。

8.6 表观物质平衡 (apparent mass balance) 与平衡线高度

8.6.1 `apparent_mb_from_any_mb`

`apparent_mb_from_any_mb`（约第2438行）将“气候物质平衡”（constant climate下的MB）转换为“表观物质平衡（apparent mass balance）”（考虑冰川在给定年份的实际几何变化）。它通过 `brentq` 求解一个残差方程：固定几何（fixed geometry）下的MB在给定年与实际质量变化之间的匹配。

8.6.2 `fixed_geometry_mass_balance`

`fixed_geometry_mass_balance`（约第2538行）计算假设冰川几何不变时的物质平衡，用于与观测数据校准中的“几何变化效应”分离。

8.6.3 compute_ela

`compute_ela`（约第2605行）从逐年的物质平衡剖面中计算平衡线高度（ELA）——即 $MB(z)=0$ 的海拔。通过插值多个年度的MB剖面得到时间序列。

8.7 MB_GLOBAL_PARAMS 冻结/检查机制

当 `cfg.PARAMS['mb_global_params']` 不为空时，所有冰川的物质平衡参数（`melt_f`，`temp_bias`，`prcp_fac`）被冻结为全局值，跳过个体校准。这个机制用于：

- 敏感性实验（探究单一参数对全球模型的影响）
- 强制所有冰川使用相同的气候参数以检验参数对GMST变化的统一响应
- 当校准数据匮乏时，使用相邻冰川的参数

8.8 关键参数速查

参数名	默认值	作用	单元
<code>melt_f</code>	5.0（校准可变）	温度指数融化因子（temperature-index melt factor）	$kg\ m^{-2}\ d^{-1}\ K^{-1}$
<code>temp_bias</code>	校准确定（ $\sim 0 \pm 3$ ）	系统温度偏移	K
<code>prcp_fac</code>	未固定（由校准策略确定）	降水缩放因子	无量纲
<code>temp_melt</code>	-1.0	融化阈值温度	$^{\circ}C$
<code>temp_all_solid</code>	0.0	全固态降水温度分界	$^{\circ}C$
<code>temp_all_liq</code>	2.0	全液态降水温度分界	$^{\circ}C$
<code>temp_default_gradient</code>	-0.0065	温度垂直递减率	$K\ m^{-1}$
<code>geodetic_mb_period</code>	2000-01-01_2020-01-01	大地测量物质平衡（geodetic mass balance）校准参考期	date range

8.9 总结

`massbalance.py` 是OGGM的核心模块：它将气候时间序列转换为冰川表面的质量收支，并把这些收支提供给反演和动力学模块。从线性理想模型到月度温度指数模型（MonthlyTIModel, temperature-index model），从单冰川校准到多流线分配，代码体现的是经验参数化、观测约束和数值效率之间的折中。

掌握了物质平衡模型，下一章将进入冰厚反演：利用表观物质平衡（apparent mass balance）推导沿流线的冰通量，再结合SIA关系求解冰厚度和冰床高程。

第9章 冰厚反演

`inversion.py` 约1180行，实现了OGGM的质量守恒冰厚反演方法 (Farinotti et al. 2009; Huss and Farinotti 2012)。反演把表观物质平衡 (apparent mass balance) 积分得到的冰通量与SIA通量关系联系起来，从而求解沿流线的冰厚。没有冰厚信息，后续就无法可靠计算冰流速度、崩解通量或冰川对气候变化的动力响应。

反演的直观理解

可以把本章理解为一个“从流量反推管道尺寸”的问题：上游表观物质平衡 (apparent mass balance) 告诉我们每个位置需要输送多少冰通量；SIA告诉我们给定坡度和冰厚时能输送多少冰。反演就是寻找一组冰厚，使这两者相容。

哪些结论要谨慎

反演冰厚不是直接观测。它对表观物质平衡 (apparent mass balance)、Glen参数、滑动假设、冰床横截面 (bed cross-section) 和DEM/边界质量敏感。对碎屑覆盖、跃动、强非平衡、冰湖或入海型冰川，应把反演厚度作为带不确定性的初始化结果，而不是实测冰厚。

9.1 物理基础：质量守恒与SIA

冰厚反演的物理核心来源于两个约束：(1) **质量守恒**——通过给定横截面的冰通量 (ice flux) 必须等于上游集水区总物质平衡的积分；(2) **浅冰近似 (Shallow Ice Approximation, SIA)** ——给定表面坡度和冰厚，可以计算竖向积分的冰通量。

关键公式 (Huss and Farinotti 2012)：通过横截面的冰通量 q 为：

$$q = \int_0^H u(z) dz = f_d * (2*A/(n+1)) * (\rho * g * \alpha)^n * H^{n+2} + f_s * (2*A) * (\rho * g * \alpha)^n * H^n$$

其中：

- A —— Glen流动律参数 (默认 $2.4e-24 \text{ Pa}^{-3} \text{ s}^{-1}$)
- n —— Glen指数, $= 3$
- ρ —— 冰密度 (900 kg m^{-3})
- g —— 重力加速度 (9.81 m s^{-2})
- α —— 表面坡度 (来自DEM)
- H —— 冰厚 (未知量, 反演目标)
- f_d —— 形变流动的形因子 (shape factor; 取决于冰床横截面形状 bed cross-section geometry: 抛物线/矩形/梯形)
- f_s —— 滑动的形因子 (shape factor; 通常 $f_s=0$, 即假设基底冻结)

通量 q 来自上游表观物质平衡 (apparent mass balance) 的积分；在离散流线中，它本质上是上游净质量收支换算得到的冰体积流率。因此反演任务变为：已知 q 和 α ，求解 H 。

进阶

质量通量的概念单位

通量 q 的单位为 $\text{m}^3 \text{ s}^{-1}$ (冰体积流率)。它由物质平衡积分换算而来： $q = \text{flux} / (\rho * \text{SEC_IN_YEAR})$ ，其中flux是 kg s^{-1} 级别的质量流率。通常只对 $\text{flux} > 0$ 的位置求解厚度——flux为负意味着该点上游无净物质积累。

9.2 准备阶段 (prepare_for_inversion)

`prepare_for_inversion` (约第53行) 为每条流线的每个横截面位置预先计算冰厚求解所需的所有辅助量:

```
def prepare_for_inversion(gdir, inversion_input=None):
    # 1. 计算表面坡度: 负梯度 (高程向下降方向)
    slope = -np.gradient(hgt, dx)

    # 2. 从表观物质平衡 (apparent mass balance) 计算质量通量
    flux = fl.flux * dx**2 / SEC_IN_YEAR / rho    # m3 s-1

    # 3. 预计算通量/宽度比
    flux_a0 = flux / width * shape_factor          # 用于SIA多项式
```

表观物质平衡 (apparent mass balance, apparent MB) 考虑了冰川几何相对于参考几何的变化——不同于第8章的固定几何MB, 它需要调用 `apparent_mb_from_any_mb` 来产生实际的物质收支。

坡度计算中的关键细节: 对坡度进行截断, `alpha = max(alpha, min_slope)`, 其中 `min_slope = tan(1.5°)`。当实际坡度低于此阈值时 (如在冰斗或宽阔河谷中), 强制取这个最小值——因为零坡度会导致 `alpha^n = 0`, 使得冰厚方程退化。

9.2.1 形状因子 (shape factor)

形状因子 (shape factor) `f_d` 取决于假定的冰床横截面形状 (bed cross-section geometry):

- 半圆/抛物线 (`f_d = 0.5`): 经典值, 适用于山谷冰川
- 矩形 (`f_d = 1.0`): 无侧向约束 (流速仅受厚度和坡度控制)
- 梯形: 介于两者之间, 取决于形状参数 `trapezoid_lambdas`

为了兼容默认抛物线方案和物理上更合理的梯形方案, OGGM使用 `mixed_min_shape` 阈值进行平滑过渡。

9.3 厚度求解器

对于质量通量 q 、坡度 α 、形状因子 (shape factor) f_s 和 f_d , SIA方程化为一个关于 H 的五次多项式:

$$H^5 + a_3 * H^3 + a_0 = 0$$

9.3.1 多项式求解 (_inversion_poly)

当 $f_s = 0$ (无基底滑动) 时, 多项式退化为 $H^5 + a_0 = 0$, 解为 $H = (-a_0)^{(1/5)}$ (对应 `_inversion_simple`, 约第153行)。

当 $f_s \neq 0$ 时 (有基底滑动项), `_inversion_poly` (约第146行) 使用 `np.roots()` 求解五次多项式的所有根, 并选择物理上可接受的实根作为厚度。多项式系数 a_3 和 a_0 由SIA方程系数组装。

9.3.2 遍历求解 (_compute_thick)

`_compute_thick` (约第159行) 对方程网格中所有 $\text{flux} > 0$ 的横截面逐点求解。循环结构为:

```
for i in range(nx):
    if flux[i] > 0:
        thick[i] = _inversion_poly(a3[i], a0[i])
    else:
        thick[i] = 0    # 无物质积累的位置 = 零冰厚
```


9.3.3 Brent优化求解 (sia_thickness_via_optim)

`sia_thickness_via_optim` (约第194行) 是对 `_inversion_poly` 的替代方案, 使用 `brentq` 数值求解非线性方程:

```
u = H^(n+1) * fd * (rho*g*alpha)^n + H^(n-1) * fs * (rho*g*alpha)^n
sect * u - flux = 0
```

其中 `sect` 表示由冰厚和横截面形状确定的截面积。Brent方法比多项式求根更稳健, 尤其适合梯形横截面等无法直接化成简单解析形式的情况, 但每点需要多次函数求值。

9.3.4 sia_thickness 解析函数

`sia_thickness` (约第264行) 是对外的公共接口, 接受表面斜率、通量、宽度等参数, 在内部调用 `_compute_thick` 进行解算。这是外部模块 (如动力学、崩解等) 使用SIA厚度估计的标准入口。

9.4 核心任务: mass_conservation_inversion

`mass_conservation_inversion` (约第359行) 是完整的反演任务函数。其算法流程为:

1. 读取输入: 从 `gdir` 加载流线、质量通量、DEM高程、宽度、坡度等信息。
2. 裁剪坡度: `alpha = max(slope, min_slope)`, 防止数值问题。
3. 计算多项式系数: 对每个横截面组装 `a0` (来自 `flux_a0`) 和 `a3` (来自 `fs` 项)。
4. 求解厚度: 调用 `_compute_thick` 获取每个横截面的冰厚。
5. 计算体积: `V = sum(thick * width * dx)` 得到冰川总体积。
6. 梯形后备方案: 对于 `bed_shape` 低于 `mixed_min_shape` 的横截面, 切换到梯形SIA解算器 (`sia_thickness_via_optim`), 以更好地表示宽谷或平坦冰床横截面 (bed cross-section) 中的冰流。

反演的三个关键输入参数是:

- `glen_a` (默认 $2.4e-24$): Glen流动律参数, 控制冰的柔软度。越大的值代表越软的冰 (更快的流动), 从而获得更薄的厚度。
- `fs` (默认0): 基底滑动因子。非零值时允许冰床滑动, 降低形变厚度。
- `trapezoid_lambdas`: 梯形形状参数。在梯形横截面方案中控制谷底的平坦程度。

进阶

参数敏感性与不确定性

Glen参数的不确定性可达一个数量级 ($10^{-24} \sim 10^{-23}$), 这直接影响冰体积的不确定性。由于 $H \propto A^{(-1/5)}$ (简单情况下), 即使A有两倍不确定性, 对H的影响也只有约-13%。但通量q本身对MB校准误差更敏感——10%的MB偏差直接导致 $H^{(n+2)}$ 中的10%体积变化。

9.5 后处理

9.5.1 过滤与平滑 (filter_inversion_output)

`filter_inversion_output` (约第505行) 对反演的原始厚度场进行两种后处理:

- 末端凹槽平滑: 冰川末端的反演厚度常出现非物理的“末端凹槽” (terminal depression) ——因为通量在末端前急剧下降为0, 导致局部厚度过薄。函数沿前缘方向应用低通滤波来平滑这些凹槽。

- **转换为梯形表示**：将抛物线-基底表示转换为更物理的梯形形状，以便后处理算法（如2D厚度场重建）使用。

9.5.2 反演速度计算 (`compute_inversion_velocities`)

`compute_inversion_velocities` (约第632行) 从反演的厚度场计算表面速度：

```
u_deformation = (2*A/(n+1)) * tau^n * H      # 内部形变贡献
u_basal = A * tau^n / lambda * ...           # 基底滑动贡献 (可选)
u_surface = u_deformation + u_basal
```

当 $f_s=0$ 时，深度平均速度与表面速度之间满足 $u_{\text{mean}}/u_{\text{surface}} = (n+1)/(n+2)$ ；因此在 $n=3$ 时，表面速度约为深度平均速度的1.25倍。速度和应力可用于验证反演的一致性，也可作为后续诊断量。

9.5.3 体积输出 (`get_inversion_volume`)

`get_inversion_volume` (约第625行) 计算反演的总冰体积： $V = \text{sum}(\text{thick} * \text{width} * \text{dx})$ ，以 m^3 为单位。这是在区域到全球尺度上汇总冰储存量的核心指标。

9.6 厚度场扩展 (2D可视化)

9.6.1 按海拔分配厚度 (`distribute_thickness_per_altitude`)

`distribute_thickness_per_altitude` (约第728行) 使用反距离加权 (IDW) 方法将一维沿中心线的厚度扩展到二维网格上。核心思想是：

1. 对每个海拔带，收集该带内所有中心线点的厚度值。
2. 在该海拔带内的所有冰川掩码像素上对厚度进行IDW插值——离中心线越近的像素获得更接近中心线点的厚度权重。
3. 不同海拔带之间独立处理——避免了横谷的厚度平滑伪影。

这种“按海拔分带”的IDW方法比全局IDW插值更能保持冰川厚度场中海坡梯度的物理合理性。

9.6.2 立方插值 (`distribute_thickness_interp`)

`distribute_thickness_interp` (约第856行) 是替代的二维插值方案，使用 `scipy.interpolate.griddata` 的 `cubic` 模式进行三阶插值。对于光滑的大型冰川（如阿拉斯加沿海冰川），这个方案能产生视觉上更平滑的厚度图，但可能模糊冰川边缘的厚度梯度。

9.7 崩解通量的厚度依赖

9.7.1 `calving_flux_from_depth`

`calving_flux_from_depth` (约第960行) 使用Oerlemans and Nick (2005)的崩解速率公式：

```
q_calving = k * water_depth * ice_thickness * width
```

其中 k 是崩解常数；OGGM v1.6.3中反演使用的默认 `inversion_calving_k` 为 0.6 yr^{-1} 。`water_depth` 是末端处的水深，可由冰厚、出水高度 (freeboard) 和水位约束估算。这个公式表达的是经验关系：更深水位通常对应更高的崩解通量。

9.7.2 迭代一致性求解 (find_inversion_calving_from_any_mb)

find_inversion_calving_from_any_mb (约第1033行) 解决了一个"鸡和蛋"问题：冰厚依赖于通量，而通量依赖于崩解损失，崩解又依赖于冰厚。通过迭代求解实现SIA反演与崩解通量之间的自洽：

- 1. 用当前冰厚估计崩解通量。
- 2. 将崩解通量加入物质平衡通量。
- 3. 用修正的flux求解新冰厚。
- 4. 重复直至收敛。

一般在5-10次迭代内收敛。这是处理入海型冰川的关键技术，因为崩解是这类冰川质量损失的主要形式。

9.8 关键参数速查

参数名	默认值	作用	单元
glen_a	2.4e-24	Glen流动律参数	$Pa^{-3} s^{-1}$
fs	0.0	基底滑动因子	无量纲
min_slope	$\tan(1.5^\circ)$	最小坡度阈值	m/m
mixed_min_shape	0.001	梯形切换阈值	无量纲
trapezoid_lambdas	变化	梯形形状参数	无量纲
inversion_calving_k	0.6	反演阶段使用的崩解速率常数	yr^{-1}
use_kcalving_for_inversion	False	是否在反演中启用k-calving校正	-

9.9 总结

inversion.py 将第5章到第8章的成果——地形、几何、气候、物质平衡——融合为一个统一的冰厚度反演框架。物理上，它基于质量守恒和浅冰近似这两个冰川学的基石原理；数值上，它将问题转化为五次多项式的求根或Brent数值优化；工程上，它用迭代方法和自适应形状因子 (shape factor) 处理从山谷冰川到入海型冰川的多样化冰川形态。

反演的冰厚场连接了预处理和正演模拟：在此之前是GIS处理、几何参数化和物质平衡建模，在此之后是冰流正演、冰川演化和海平面贡献估计。反演精度会影响后续动力学模拟：若反演偏薄，冰川响应通常偏快；若反演偏厚，响应通常偏慢。因此，冰厚反演的参数选择和区域校准（特别是 inversion_glen_a）是OGGM应用中必须认真检查的环节。

第10章 冰流动力学：Flowline模型

`flowline.py` 是OGGM中规模最大的单一源文件，约4853行，包含冰川动力学的主要实现：从冰床横截面形状（*bed cross-section geometry*）参数化到SIA方程数值求解，再到多种演化模型的构建。本章关注它如何把第9章反演的冰厚作为初始条件、把第8章的物质平衡作为气候强迫，进而驱动冰川随时间演化。

初学者阅读路径

第一遍只需读懂三个对象：`Flowline` 保存一维几何，`MassBalanceModel` 提供气候强迫，`FlowlineModel` 根据SIA推进冰川几何。冰床横截面（*bed cross-section*）类、支流通量合并（*tributary flux merging*）和求解器细节可以在能跑通教程6之后再回看。

模型假设边界

OGGM主工作流使用一维流线表达复杂山地冰川，并以SIA为核心动力学近似。它适合区域到全球尺度的大量冰川模拟，但不显式求解纵向应力、完整三维流场或复杂基底水文。解释个别陡峭冰瀑、跃动冰川或快速入海型冰川时，应把这些假设作为结果讨论的一部分。

10.1 Flowline基础类

10.1.1 Flowline类（第49行）

`Flowline` 是OGGM中冰川几何表示的核心数据结构，继承自 `Centerline` 并添加了冰厚相关的属性。每条Flowline代表冰川的一条分支——主流线或支流——携带该分支沿线的所有物理状态（厚度、表面高程、宽度、体积、冰通量等）。

```
class Flowline(Centerline):
    # 核心几何属性（从Centerline继承）
    # line: (n,)-形 numpy数组 - 沿流线的累积距离 (m)
    # nx: int - 网格点数
    # dx: float - 网格间距 (m)

    # 冰川状态属性
    thick = None           # (nx,)-形数组 - 冰厚 (m)
    surface_h = None       # (nx,)-形数组 - 表面高程 (m)
    widths = None          # (nx,)-形数组 - 表面宽度 (m)
    widths_m = None        # (nx,)-形数组 - 中间宽度 (用于通量计算)
    section = None         # (nx,)-形数组 - 横截面积 (m2)
    bed_h = None           # (nx,)-形数组 - 冰床高程 (m)

    # 诊断属性
    volume_m3 = None       # 冰川总体积
    length_m = None        # 冰川总长度
    flux = None            # (nx,)-形数组 - 冰通量 (m3 s-1)
    is_rectangular = [...] # bool 数组 - 每点的冰床横截面 (bed cross-section) 类型
```

`surface_h = bed_h + thick`，这是基本的几何关系。但 `surface_h` 不保证处处大于 `bed_h`——当冰厚为零时两者相等。`widths_m` 用于交错网格上的通量计算，定义在相邻格点之间（沿流线方向的半步点）。

10.1.2 冰床横截面形状（*bed cross-section geometry*）类层级

冰床横截面（*bed cross-section*）的形状决定了冰流通过横截面的方式，这是SIA理论中形状因子（*shape factor*）`f_d` 的来源。OGGM定义了四种冰床横截面形状（*bed cross-section geometry*）类，构成一个功能递进的层级：

ParabolicBedFlowline (第301行)

抛物线形冰床横截面 (bed cross-section) 是最经典的冰川沟谷表示, 对应于Nye (1965)和Cuffey & Paterson (2010)的经典处理:

```
class ParabolicBedFlowline(Flowline):
    # 宽度由抛物线公式给出:  $w = \sqrt{4 * h / \text{bed\_shape}}$ 
    # bed_shape 是抛物线曲率参数
    # 横截面积:  $\text{section} = (2/3) * w * h$ 
    # 形状因子 (shape factor)  $f_d = 0.5$ 
```

抛物线的几何含义: 对于给定的冰厚 h , 宽度 w 由抛物线曲率参数 `bed_shape` 决定。 w 和 h 呈平方根关系, 意味着冰厚的增厚会导致宽度的渐进式扩展。横截面积为 $(2/3)*w*h$, 形状因子 (shape factor) 固定为0.5——这是半圆冰床横截面 (bed cross-section) 的理论值, SIA形状因子 (shape factor) 恰好等于抛物线面积与等效矩形面积之比。

RectangularBedFlowline (第347行)

```
class RectangularBedFlowline(Flowline):
    #  $\text{section} = w * h$  (矩形面积)
    # 形状因子 (shape factor)  $f_d = 1.0$ 
    # 无侧向约束—全部应力由冰床承担
```

矩形冰床横截面 (bed cross-section) 假设侧壁垂直, 并在一维SIA框架中不显式表示侧向阻力。这种假设适用于横向约束较弱的宽阔冰体; 若用于狭窄山谷冰川, 可能高估冰流速度。

TrapezoidalBedFlowline (第394行)

```
class TrapezoidalBedFlowline(Flowline):
    # 宽度随冰厚线性增长:  $w = w_0 + \lambda * h$ 
    # lambda 是梯形张开参数 (无量纲)
    # 横截面积:  $\text{section} = (w + w_0) / 2 * h$ 
    # 形状因子 (shape factor) 随  $w_0/w$  比值变化, 介于0.5到1.0之间
```

梯形是更灵活的冰床横截面形状 (bed cross-section geometry) 表示, 通过参数 `lambda` (来自配置或反演中的 `trapezoid_lambda`) 调整侧壁张开程度。`w0` 表示冰床底部宽度。

MixedBedFlowline (第451行) ——默认方案

```
class MixedBedFlowline(Flowline):
    # 在每个网格点独立选择冰床横截面 (bed cross-section) 类型
    # is_trapezoid 布尔掩码区分应用哪种公式
    # 抛物线点:  $\text{section} = 2/3 * w * h, f_d = 0.5$ 
    # 梯形点:  $\text{section} = (w+w_0)/2 * h, f_d = f(\lambda)$ 
    # 矩形点:  $\text{section} = w * h, f_d = 1.0$ 
```

这是OGGM常用的冰床横截面形状 (bed cross-section geometry) 类。关键理念是: 同一条冰川的不同位置可能需要不同的横截面表示。例如, 在冰川上游宽阔盆地中梯形截面可能更合适, 而在狭窄U形谷中抛物线截面可能更合适。`is_trapezoid` 掩码允许逐格点切换公式, 实现几何表示的灵活性。

10.2 FlowlineModel基类（第571行）

10.2.1 构造函数

```
class FlowlineModel:
    def __init__(self, flowlines, mb_model=None, glen_a=None,
                 fs=0, mb_elev_feedback='annual'):
        # flowlines: Flowline对象列表 (主流线 + 支流线)
        # mb_model: MassBalanceModel实例 (气候强迫)
        # glen_a: Glen流动律参数 (Pa-3 s-1), 默认2.4e-24
        # fs: 基底滑动因子, 默认0 (无滑动)
        # mb_elev_feedback: 'annual', 'monthly', 'always', 'never'
```

模型的核心物理常数在构造时预计算：`_fd = 2 * glen_a / (n + 2)`（形变流参数，第653行）。这个预计算避免了在每个时间步中重复评估多项式。参数 `fs`（基底滑动）在构造后通过 `_fs` 属性被类似处理。

10.2.2 支流系统 (tributary system): reset_flowlines()

`reset_flowlines()` 构建 `_tributary_indices` ——一个记录每条支流线与主流线连接位置和方式的内部数据结构。支流贡献的冰通量通过高斯核分布在主流线上：

```
# 支流通量在主流线上的贡献
# 分布核大小自适应: 5, 7, 或 9 个格点
# 更宽的冰川 → 更大的核 → 更平滑的通量合并
for trib in tributary_flowlines:
    junction_pos = trib.line[-1]          # 支流汇入主流线的位置
    kernel_width = adapt_to_glacier_size  # 自适应核宽度
    # 高斯权重分布到附近的主流线格点
```

高斯核的使用避免了支流通量集中到一个网格点导致的数值震荡。核大小的自适应逻辑考虑了冰川宽度——更宽的冰川下游使用9点核，更窄的山谷冰川使用5点核。

10.2.3 MB缓存策略

`mb_elev_feedback` 参数控制何时重新计算物质平衡：

模式	行为	适用场景
'always'	每次需要MB时重新计算	物理精确但极慢
'monthly'	每个月重新计算一次（12次/年）	亚季节分辨率的模拟
'annual'	每年重新计算一次	常用折中：兼顾精度与性能
'never'	使用初始MB，永不更新	测试或教学目的

缓存的核心设计逻辑是：在时间步之间，表面高程变化通常很小（典型值为每年几厘米到几米），因此每月或每年更新一次MB不会显著损失精度。通过跳过不必要的MB计算（尤其是基于气候数据的月度MB模型），性能可提升1-2个数量级。

10.2.4 时间步进: run_until()与run_until_and_store()

`run_until(y1)`（基本方法）将模型从当前时间向前积分到目标年份 `y1`。它采用自适应时间步长策略：

- 外层：按年份推进。每年开始时检查是否需要更新MB（取决于 `mb_elev_feedback` 模式）。
- 内层：在每一年内，以月度步长推进（12步）。在每个月度步中，CFL条件可能触发进一步的子步分解。

`run_until_and_store(y1)`（第922行）是 `run_until` 的扩展版本，将演化过程记录到 `xarray.Dataset` 中：

```
def run_until_and_store(self, y1):
    # 创建xarray Dataset, 包含时间坐标
    ds = xr.Dataset(
        coords={'time': [self.yr, y1]},
    )
    # 在每一步存储诊断变量
    # 变量包括: volume, area, length, ela, calving_rate
    # 每个变量是 shape=(time, flowline_idx, nx) 的DataArray
    return ds
```

存储的xarray Dataset使得后处理、可视化和分析变得非常方便——时间序列的体积变化、末端的进退、ELA的移动等都可以直接从Dataset中提取。

10.2.5 平衡态求解: `run_until_equilibrium()` (第1513行)

这个方法迭代运行模型直到冰川达到平衡态:

```
def run_until_equilibrium(self, rate=1e-3):
    while True:
        V_before = self.volume_m3
        self.run_until(self.yr + 1)
        V_after = self.volume_m3
        if abs(V_after - V_before) / V_before < rate:
            break # |dV/V| < 0.1% 时视为平衡
```

收敛判据通常基于体积的年际变化率。平衡态求解常用于: (1) 理想化预热 (spinup) 或平衡实验; (2) 敏感性实验; (3) 理论分析, 即理解给定气候下冰川的平衡响应。第13章讨论的动态预热 (dynamic spinup) 并不简单等同于求平衡态, 而是以观测面积或体积为目标进行近期历史初始化。

10.3 FluxBasedModel (第1568行)

10.3.1 算法架构

`FluxBasedModel` 是OGGM的重要显式流线动力学模型。OGGM v1.6.3的配置文件默认 `evolution_model` 为 `SemiImplicit`, 但 `FluxBasedModel` 仍常用于多流线、崩解和需要显式通量处理的实验。它采用交错网格通量形式, 每个时间步包含两个循环:

循环1 (通量计算, 第1771–1866行):


```

for i in range(nx):
    # 步骤1: 支流汇合 - 在交汇点附加下游表面高程
    append_tributary_surface_h()

    # 步骤2: 崩解限制器 - 将水面以下冰厚截断
    if is_tidewater and bed_below_water:
        thick = clip_to_water_level()

    # 步骤3: 交错表面梯度
    slope_stag = (surface_h[i] - surface_h[i+1]) / dx

    # 步骤4: 交错厚度 (相邻格点平均)
    thick_stag = (thick[i] + thick[i+1]) / 2

    # 步骤5: SIA 流速 (形变 + 滑动)
    tau = rho * g * slope_stag # 基底剪应力
    u_deform = thick_stag^(n+1) * fd * tau^n
    u_sliding = thick_stag^(n-1) * fs * tau^n
    u_total = u_deform + u_sliding

    # 步骤6: 冰通量 = 流速 × 交错截面积
    flux_stag[i] = u_total * section_stag[i]

# 步骤7: CFL条件 - 确定子时间步长
dt = cfl * dx / max_u # cfl 默认为 0.02

```

循环2 (质量重新分布, 第1874–1971行):

```

for i in range(nx):
    # 连续性方程: d(section)/dt = -div(flux) + mb
    new_section = section[i] \
        + (flux_stag[i] - flux_stag[i+1]) * dt / dx \
        + trib_flux_contribution * dt / dx \
        + mb[i] * dt

    # 截断: 截面积不能为负
    new_section = max(0, new_section)

    # 从截面积反算新厚度 (取决于冰床横截面形状 (bed cross-section geometry))
    new_thick = invert_section_to_thick(new_section, bed_shape)

    # 通过高斯核向下游传递通量
    distribute_flux_downstream()

```

中级

CFL数0.02的含义

FluxBasedModel使用非常保守的CFL数0.02。对于典型冰川流速 (100 m/yr) 和网格间距 (200 m), 这意味着 $dt \sim 0.04$ 年, 每年需要约25个时间步。与SemimplicitModel的CFL 0.5相比, FluxBasedModel每个物理年需要更多的数值步, 但每个步的计算量远少 (无需矩阵求解)。

10.3.2 表面速度因子

SIA方程给出的是深度平均速度, 但观测和诊断通常使用表面速度。两者之间的转换由经典冰川学关系给出:

```

# 深度平均速度 → 表面速度的转换因子
_surf_vel_fac = (n + 2) / (n + 1) # n=3 时 = 5/4 = 1.25

```

这个因子来源于SIA速度剖面的解析积分 (Cuffey & Paterson 2010, Eq. 8.34)。对于 $n=3$, 表面速度比深度平均速度高25%。在OGGM中, 这个因子还可以乘以 `cfg.PARAMS['surface_velocity_factor']` 进行进一步的经验调整。

10.4 备选模型方案

10.4.1 SemimplicitModel (第2152行)

半隐式方案通过将SIA方程转换为扩散问题来获得更好的稳定性。核心是一种类似Crank–Nicolson的格式：

```
# 扩散系数:  $D = (f_d * h^{(n+2)} + f_s * h^n) * \rho * g * (w_0 + w) / 2 * |dsdx|^{(n-1)}$ 
# 对于纯形变流 ( $f_s=0$ ) 且抛物线冰床横截面 (bed cross-section):
#  $D = f_d * h^{(n+2)} * \rho * g * w * |dsdx|^{(n-1)}$ 

# 三对角系统:  $d_0[i] * S[i] + d_m[i] * S[i-1] + d_p[i] * S[i+1] = RHS$ 
# 通过 scipy.linalg.solve_banded 求解
```

CFL条件从对流约束变为扩散约束： $dt \leq cfl * dx^2 / \max(D/w)$ ， cfl 约为0.5。这比FluxBasedModel的CFL 0.02大幅宽松，在空间网格较粗时半隐式模型可以省去很多子步。但代价是：(1) 需要构建和求解三对角矩阵；(2) 仅支持单条流线（无支流系统 (tributary system)）。

进阶

SemimplicitModel的床高程修正项

半隐式扩散方案的一个独特特征是包含了床高程修正项： $b_corr = -D * d(bed)/dx$ 。当冰床表面存在急剧的高程变化时（如反向坡度或冰瀑），这个项防止数值不稳定，确保质量守恒在变坡处保持精确。FluxBasedModel中这个效应的处理是隐式的（通过交错梯度），但SemimplicitModel要求显式的修正项。

10.4.2 KarthausModel (第2067行)

KarthausModel是一个简化的显式方案，设计用于教学和概念验证。它直接从SIA计算流速和通量，使用简单的向前欧拉时间步进。默认参数经过调优使模型行为直观易懂，但物理精度不如FluxBasedModel。

10.4.3 MassRedistributionCurveModel (第2810行)

这是一个基于经验的方法，使用Huss et al. (2010)的 δh - h 参数化。核心思想是：

```
# 冰川响应由经验的“质量重新分布曲线”参数化
#  $\delta h(z) = f(z, glacier\_size\_class)$ 
# 曲线来自Huss等人对34条瑞士冰川的观测研究
mass_redistribution_curve_huss(...) # 第3068行

# 三种前进方法 (advance_method 0, 1, 2)
# 方法0: 固定曲线，截面积被MB比例缩放
# 方法1: MB驱动冰川末端前进/后退
# 方法2: MB同时驱动厚度变化和末端变动
```

该方法完全跳过了SIA冰流计算，用统计规律代替了物理模拟。它的计算成本极低（每冰川每年只需几分之一秒），适用于数千到数万条冰川的集合模拟，但牺牲了物理精确性——无法表示冰流的动力学非线性反馈。

10.4.4 FileModel (第2658行)

FileModel是一个“鸭子类型”读取器，允许将之前存储的模拟输出重新加载为一个可查询的FlowlineModel。它不执行任何物理计算，而是提供与FlowlineModel相同的查询接口（volume, area, length等），方便后处理脚本的复用。

10.5 模型初始化与运行函数

10.5.1 init_present_time_glacier（第3188行）

这是模型初始化的主入口。它负责：

- 1. 加载反演的冰厚和流线。
- 2. 调用 `decide_evolution_model`（第3342行）选择适当的动力学模型类。
- 3. 处理崩解参数（如果适用）。
- 4. 设置初始表面高程和体积。
- 5. 可选：应用动态预热（dynamic spinup）的温度偏差校准。

10.5.2 decide_evolution_model（第3342行）

模型选择逻辑根据配置参数自动决定使用哪个模型类：

```
def decide_evolution_model(gdir):  
    # 检查 cfg.PARAMS['evolution_model']  
    # 可选值: 'FluxBased', 'SemiImplicit', 'Karthaus',  
    #         'MassRedistributionCurve', 'File'  
    # v1.6.3 params.cfg 默认: 'SemiImplicit'
```

10.5.3 顶层运行函数

OGGM提供了封装完整模拟流程的顶层函数：

- `flowline_model_run`（第3362行）：单条冰川的完整前向模拟，从初始化到指定结束年份。
- `run_from_climate_data`（第3786行）：使用GCM/RCM的气候数据驱动模拟。处理气候模型输出的网格化降水和温度场。
- `run_random_climate`（第3546行）：使用随机气候生成器进行不确定性量化或统计实验。
- `run_constant_climate`（第3670行）：使用固定气候（通常为参考期气候态）进行平衡态实验。
- `run_with_hydro`（第3939行）：耦合冰川动力学与水文模型，输出径流时间序列。

10.6 停止准则（第4371行）

模拟的终止由以下任一条件触发：

准则	类型	描述
<code>zero_glacier</code>	消融完成	冰川体积降为0（某些区域可能保留多年冰）
<code>spec_mb</code>	超过气候范围	MB模型在其标定范围外被查询（如年MB超出 ± 5 m w.e.）
<code>equilibrium</code>	达到平衡	体积变化率 $ dV/V < \text{阈值}$ （用于平衡态求解）

停止准则的检查集成在 `run_until` 的每个年度步中。一旦触发，模型返回一个状态码，供调用代码确定是继续其他步骤还是终止模拟。

10.7 关键参数速查

参数名	默认值	作用	文件位置
<code>glen_a</code>	<code>2.4e-24</code>	Glen流动律参数	第653行
<code>fs</code>	<code>0.0</code>	基底滑动因子	构造函数参数
<code>n</code>	<code>3</code>	Glen流动律指数	硬编码
<code>cfl</code>	<code>0.02 (Flux) / 0.5 (SemImp)</code>	CFL数	模型类默认
<code>mb_elev_feedback</code>	<code>'annual'</code>	MB高程反馈频率	构造函数参数
<code>cfl_min_dt</code>	<code>60秒</code>	最小子步时长	第1832行
<code>surface_velocity_factor</code>	<code>1.0</code>	表面速度经验乘数	<code>cfg.PARAMS</code>
<code>evolution_model</code>	<code>'SemImplicit'</code>	配置文件默认动力学模型	<code>cfg.PARAMS</code>

10.8 总结

`flowline.py` 是OGGM动力学核心的完整实现：从冰床横截面（bed cross-section）几何的四种形状参数化，到基于SIA的两种数值方案（显式FluxBasedModel和半隐式SemImplicitModel），再到经验的MassRedistributionCurveModel。支流系统（tributary system）通过高斯核将多条流线合并为统一的冰流通量场。自适应时间步进和MB缓存策略在物理精度与计算效率之间取得了工程平衡。

FluxBasedModel凭借显式、直观的通量计算流程和支持多流线的能力，在许多OGGM应用中仍十分重要；SemImplicitModel则在默认配置中承担一维动力学正演的角色。二者共同构成第9章冰厚反演与后续 workflow 执行之间的动力学连接环节。

第11章 数值方法详解

第10章从类层级和算法架构的角度描述了OGGM的动力学模型。本章深入到数值细节——SIA方程的离散化、两种时间步进方案的稳定性分析、交错网格的设计原理、以及冰速计算和通量合并的数学基础。理解这些数值方法对于调试模拟异常、解释模型行为以及实现自定义动力学方案至关重要。

第一遍可以怎么读

如果你的目标是先完成OGGM应用，可以先读11.1、11.2和11.5，理解SIA、求解器选择和时间步长。矩阵求解、交错网格和扩散率推导是解释数值异常或开发模型时才需要深入掌握的内容。

11.1 浅冰近似（SIA）在OGGM中的应用

11.1.1 完整的SIA方程

浅冰近似是冰川动力学的零阶近似，假设冰厚远小于水平范围，且侧向应力梯度可忽略。完整的SIA深度平均速度表达式为：

```
# 深度平均冰流速度 (SIA)
u_bar = -2 * A * (rho * g)^n / (n + 2)
        * |grad(S)|^(n - 1)
        * grad(S)
        * H^(n + 1)
```

其中：

```
A = glen_a - Glen流动律参数 (Pa^-n s^-1)
rho = 900 - 冰密度 (kg m^-3)
g = 9.81 - 重力加速度 (m s^-2)
n = 3 - Glen指数
S = bed_h + H - 冰表面高程 (m)
H = 冰厚 (m)
```

Glen流动律是这个方程组的物理基础：

```
# Glen流动律 (Norton-Hoff定律)
epsilon_dot = A * tau^n
```

其中：

```
epsilon_dot - 有效应变率 (s^-1)
tau - 有效剪应力 (Pa)
n - 应力指数，标准冰川学值 = 3
```

$n = 3$ 意味着冰的黏度随应力的三次方变化——三倍的应力导致27倍的应变率。这种强烈的非线性是冰川动力学的核心特征：流速对厚度和表面坡度高度敏感。

11.1.2 SIA的有效性范围

SIA的数学推导基于小纵横比假设：冰厚 H 远小于水平尺度 L ，即 H/L （纵横比）小到可以忽略纵向应力梯度。这在下列条件下成立：

- 冰川长度 $\gg$ 冰厚（典型的山谷冰川纵横比约0.01–0.05）
- 表面坡度变化足够平缓
- 冰床不在反向坡度上（否则纵向应力重要）
- 无显著的侧向速度梯度

在冰架（漂浮冰）、冰瀑（陡峭地形）或接地线附近，SIA的近似失效，需要更复杂的“高阶”模型。但OGGM的设计目标就是以SIA为物理基础，用经验修正弥补其不足。

11.2 显式方案与半隐式方案的对比

11.2.1 FluxBasedModel: 向前欧拉

FluxBasedModel使用显式的向前欧拉时间离散：

```
# 显式向前欧拉:  $H^{(k+1)} = H^{(k)} + dt * [MB - \text{div}(q)]$ 
# 其中  $\text{div}(q) = (q[i+1/2] - q[i-1/2]) / dx$ 
# q 在交错格点上计算（使用当前时刻的H和S）
```

显式方案的两大限制：

1. **时间步长限制** —— CFL条件 $u * dt / dx < cfl_max$ 。对于流速100 m/yr、网格200 m、CFL=0.02的情况，dt约0.05年。高流速或细网格需要更多子步。
2. **无条件扩散不稳定** —— 当扩散项占主导时（厚冰 + 陡坡），即使CFL数满足名义值，向前欧拉法仍可能表现出高频空间振荡。这是显式扩散格式的固有问题。

稳定性分析（von Neumann）表明，对于纯对流（fs=0且低n），向前欧拉的放大因子为 $|G| = |1 - i * u * dt / dx * \sin(k * dx)|$ ，稳定条件为 $|G| \leq 1$ ，直接导出CFL约束。

11.2.2 SemImplicitModel: 类Crank–Nicolson格式

SemImplicitModel将SIA重新表述为扩散问题：

```
# SIA扩散形式:  $dH/dt = MB + \text{div}(D * \text{grad}(S))$ 
# 其中  $D = \gamma * H^{(n+2)} * |\text{grad}(S)|^{(n-1)}$  是扩散系数
#  $\gamma = 2 * A * (\rho * g)^n / (n+2)$ 

# 半隐式离散:
#  $H^{(k+1)} = H^{(k)} + dt * MB + dt * \text{div}(D^{(k)} * \text{grad}(S^{(k+1))))$ 
# 在新表面 $S^{(k+1)}$ 的梯度上使用隐式处理
```

CFL条件从**对流**（ $u * dt / dx$ ）变为**扩散**（ $D * dt / dx^2$ ），容许的时间步长大幅增加：

```
# FluxBased:  $dt \leq 0.02 * dx / u$  — 对 dx 线性缩放
# SemiImplicit:  $dt \leq 0.50 * dx^2 / D$  — 对 dx 平方缩放

# 当 dx 较大时 (>200 m)，半隐式的优势十分明显
```

中级

何时选择哪个方案

选择FluxBased：有支流系统（tributary system）时需要合并多条流线；需要显式通量诊断或崩解相关处理；低分辨率但大量的冰川集合模拟中计算成本可接受。注意OGGM v1.6.3配置文件默认 `evolution_model` 为 `SemiImplicit`。

选择SemImplicit：单条流线的精细研究；需要高空间分辨率（ $dx < 100$ m）以避免过多的子步；需要严格数值稳定性（如含有急剧坡度变化的冰床横截面（bed cross-section））。

常见权衡：每步计算成本——FluxBased约 $O(nx)$ ，SemImplicit约 $O(nx)$ （三对角求解为线性），但FluxBased需要更多的步数。

11.3 交错网格 (staggered grid) 离散化

11.3.1 设计原理

OGGM的通量计算采用Arakawa C网格 (交错网格) ——厚度 H 和表面高程定义在网格中心, 冰通量定义在网格面 (相邻格点之间):

```
# 网格布局示意:
#   cell:      [ i-1 ]    [  i  ]    [ i+1 ]
#   center:    H[i-1]    H[i]      H[i+1]
#   face:      q[i-1/2]  q[i+1/2]  q[i+3/2]

# 交错量计算:
slope_stag[i] = (surface_h[i] - surface_h[i+1]) / dx
thick_stag[i] = (thick[i] + thick[i+1]) / 2

# 通量在面上:
flux_stag[i] = u(thick_stag[i], slope_stag[i]) * section_stag[i]

# 厚度更新在中心:
new_thick[i] = thick[i] + dt * (flux_stag[i-1] - flux_stag[i]) / dx
```

11.3.2 为什么交错网格避免了棋盘格模式

非交错网格 (所有量在网格中心) 的一个已知问题是**棋盘格模式**——厚度场的 $2\text{-}dx$ 波长振荡不会产生通量, 因此不会被时间步进衰减。考虑中心梯度:

```
# 非交错梯度 (中心差分): dsdx[i] = (S[i+1] - S[i-1]) / (2*dx)
# 振荡模式 S = [..., +a, -a, +a, -a, ...]
# => dsdx = 0 (梯度为零, 尽管表面有剧烈波动)
# => 通量 = 0 (没有动力驱散这些波动)

# 交错梯度: dsdx[i] = (S[i] - S[i+1]) / dx
# 振荡模式 => dsdx = +/- 2a/dx (非零梯度)
# => 通量将驱散这些波动
```

交错网格 (staggered grid) 在相邻格点间而非越过格点计算梯度, 确保始终存在物理耦合, 从而自然抑制数值网格尺度的振荡。

11.4 带状矩阵求解器 (SemimplicitModel)

11.4.1 三对角系统

半隐式离散化得到一个三对角线性系统:

```
# 三对角系统: 对于 i = 0, 1, ..., nx-1
d0[i] * S_new[i] + dm[i] * S_new[i-1] + dp[i] * S_new[i+1] = RHS[i]

其中:
d0[i] = 1 + dt * (D_front[i] + D_back[i]) / dx^2
dm[i] = -dt * D_back[i] / dx^2
dp[i] = -dt * D_front[i] / dx^2
RHS[i] = H_old[i] + dt * MB[i] + dt * b_corr[i]

# 使用scipy的带状矩阵求解器:
from scipy.linalg import solve_banded
S_new = solve_banded((1, 1), ab_matrix, RHS)
```

`solve_banded(l_and_u, ab, b)` 的 `(1,1)` 表示主对角线上下各有一条子对角线（三对角），`ab` 矩阵以压缩格式存储非零对角线。`nx` 通常为50–500，矩阵求解的计算成本 $O(nx)$ 几乎可以忽略。

11.4.2 消融区边界条件

在冰川末端（消融区最后一个有冰的格点），施加**零厚度边界条件**： $H[nx-1] = 0$ 。对应地，三对角系统的最后一行被修正为简单的 $S_{new}[nx-1] = bed_h[nx-1]$ （表面等于冰床，无冰）。这确保了消融区边界处质量守恒的正确处理。

11.5 自适应时间步长

11.5.1 三层嵌套时间循环

OGGM动力学模拟的时间步进由三层嵌套控制：

```
# 第1层：年度步 (yearly step)
while yr < yl:
    mb_updated = False
    for month in range(12):          # 第2层：月度步 (monthly step)
        if feedback_mode == 'monthly':
            update_mb_model()
        remaining = 1/12              # 目标时间 = 一个月 (~0.083年)
        while remaining > 0:          # 第3层：CFL子步 (sub-step)
            dt_cfl = cfl * dx / max_u
            dt = min(dt_cfl, remaining)
            step(dt)
            remaining -= dt
```

最小子步时长由 `cfl_min_dt` 限制（默认60秒）。当冰川流速极高（如 > 10000 m/yr，在巨型入海型冰川（tidewater glaciers）中可达此值）且网格很细时，这个下限防止了无限细分的退化情况。

11.5.2 计算效率的优化

在典型生产模拟中（ $dx \sim 100\text{--}200$ m, $u \sim 50\text{--}300$ m/yr），CFL子步为每年10–50步。月度循环的开销可忽略（每年仅12次MB更新和循环管理）。年度步主要用于触发MB缓存更新和诊断检查，频率为每年一次。

11.6 质量守恒

11.6.1 MassConservationChecker包装器（第2031行）

`MassConservationChecker` 是对任何`FlowlineModel`的一个包装器，在每个时间步前后记录总质量并报告任何不守恒情况：

```
class MassConservationChecker:
    def step(self, dt):
        mass_before = self.model.volume_m3
        self.model.step(dt)
        mass_after = self.model.volume_m3
        # 预期质量变化 = MB积分 + 通量边界条件
        # 实际质量变化 = mass_after - mass_before
        # 差异 = 当前步的数值质量泄漏
```

这对于开发和调试新动力学方案至关重要——任何非零的质量差异都表示数值方案中的实现错误。

11.6.2 通量闸门 (flux gate) 边界条件

在冰分 (glacier divide, 冰川最高点, $u=0$), 冰通量为零——没有上游物质来源。这个物理约束在数值上通过通量闸门 (flux gate) 边界条件实现: $q[0] = 0$ (流线起点的通量强制为零)。这防止了"凭空生成"的冰从上游进入模型域。

11.6.3 体积追踪与崩解桶 (calving bucket) 机制

质量守恒需要通过 calving_bucket (崩解桶) 模式处理崩解——崩解以离散的网格单元大小移除冰, 但每次只有部分网格单元需要移除。桶机制 (bucket mechanism) 累积分数质量直到达到完整网格单元的体积, 然后一次性移除——这种方法保持了体积追踪的精确性而不引入分数网格单元。

11.7 高程反馈机制

mb_elev_feedback 是 OGGM 在高程-物质平衡耦合上的核心控制参数。当冰表面因动力学而改变高度时, 物质平衡必须更新以反映新的表面高程:

```
# 反馈周期对数值行为的影响:
'always': 每个子步更新MB - 物理上最精确, 但严重增加计算成本
'monthly': 每12个子步更新MB - 接近连续反馈
'annual': 每12*N_substep步更新MB - 默认
'never': 从不更新MB - 固定气候实验 (物理上与"平衡态"不同)
```

高程反馈中的关键权衡: 更频繁的反馈增加精确度但也增加了计算成本 (尤其在 MB 模型基于月度气候数据时, 每次更新可能触发对多变量格网数据 (gridded data) 的查询)。对于多数冰川, 每年1-10米的高程变化意味着年度反馈的误差在约0.1-0.5 m w.e. MB 偏差以内, 这在气候强迫的不确定性中是可以接受的。

进阶

高程反馈与模型稳定性

高程反馈本身是一种正反馈: 表面下降 → 更多消融 (气温更高) → 更多下降。在显式方案中, 这种正反馈可能与 CFL 不稳定耦合, 触发 "runaway thinning"。SemiImplicitModel 的隐式处理能部分缓解这一问题, 但合理的 mb_elev_feedback 频率 (如 'annual' 而非 'monthly') 仍然是一个重要的稳定性约束。对于入海型冰川 (tidewater glacier) 的快速消融场景, 可能需要更保守的 CFL 设置。

11.8 支流通量合并 (tributary flux merging)

11.8.1 高斯加权的动机

支流贡献到主流线的冰通量通过高斯加权核分布在多个格点上。核心问题是: 支流在一个精确的几何点 (交汇点) 汇入主流线, 但冰通量不应集中在一个网格点上——那样会造成急剧的通量梯度 (数值不稳定)。高斯分布将通量"涂抹"在交汇点附近的几个格点上:

```
# 支流通量分布权重
w[j] ~ exp(-(x[j] - x_junction)^2 / (2 * sigma^2))
# sigma 取决于冰川宽度: wider -> larger sigma -> more smoothing
# 核大小: 5, 7, 或 9 个格点
```

核大小的自适应选择确保了较小的山谷冰川 (支流交汇处下游宽度较窄) 的支流贡献不会被过度涂抹, 而较大的入海型冰川 (tidewater glaciers) / 冰盖的宽阔接收区能足够平滑地吸收支流输入。

11.9 冰速计算

11.9.1 深度平均速度与表面速度

SIA给出的速度是深度平均的，但表面速度（卫星InSAR和特征跟踪的直接观测量）是表面速度。两者之间的关系为：

```
# 深度平均速度 → 表面速度
u_surface = u_bar * (n + 2) / (n + 1)

# 对于 n=3: u_surface = u_bar * 5/4 = 1.25 * u_bar
# 物理原因：速度剖面是垂向的(n+1)/(n+2) 指数分布

# 速度在交错面上的平均值（用于诊断输出）：
u_at_center[i] = (u_stag[i-1] + u_stag[i]) / 2
```

11.9.2 速度诊断与观测校准

在诊断输出中，OGGM将交错速度平均到网格中心，然后乘以 `surface_velocity_factor`（来自`cfg.PARAMS`）进行经验调整。这个因子的默认值为1.0，但在某些标定到卫星观测的项目中被调至0.5–2.0不等——反映了SIA速度理论值与观测值之间的系统性偏差。

11.10 SIA扩散率推导

将SIA重新表述为扩散形式是半隐式方案的基础：

```
# 通量形式: q = -gamma * H^(n+2) * |dS/dx|^(n-1) * dS/dx
# 其中 gamma = 2*A*(rho*g)^n / (n+2)

# 连续性方程: dH/dt = MB - dq/dx
#              = MB + d/dx[D * dS/dx]
# 其中扩散系数 D = gamma * H^(n+2) * |dS/dx|^(n-1)

# 对 n=3: D = gamma * H^5 * (dS/dx)^2
# 扩散率对厚度的强敏感性 (H^5!) 是冰川响应非线性的根源
```

扩散系数D中 H^5 的强依赖意味着当冰厚加倍（2x）时扩散率增加32倍，这解释了为什么厚冰川对气候变化更敏感（响应更快），而薄冰川的变化更缓慢。

11.11 通量散度作为残差

`dh/dt - climatic_mb` 即通量散度残差是一个核心诊断量。在平衡态中，残差应为零——MB在所有位置的积累都被冰流动力学精确平衡。在非平衡态中，残差表示局部正在发生质量净增（整理，div为正）或净损失（消融，div为负）。这个残差对于理解冰川不同部位的质量变化来源非常重要。

```
# 诊断残差（用于模型验证）：
residual = dh/dt - climatic_mb
# 如果 residual = 0: 冰川处于平衡（冰流完美匹配气候）
# 如果 residual > 0: 动力学敛聚（局部冰流快于气候可提供的）
# 如果 residual < 0: 动力学扩散（局部冰流不足以带走所有积累）
```

11.12 关键参数速查

参数	默认值	含义
<code>n</code>	3	Glen流动律应力指数

参数	默认值	含义
<code>glen_a</code>	<code>2.4e-24</code>	Glen 流动律参数 ($\text{Pa}^{-n} \text{ s}^{-1}$)
<code>cfl (FluxBased)</code>	<code>0.02</code>	CFL 数 (对流稳定性)
<code>cfl (SemiImplicit)</code>	<code>0.50</code>	CFL 数 (扩散稳定性)
<code>cfl_min_dt</code>	<code>60 秒</code>	最小允许子步时长
<code>(n+2)/(n+1)</code>	<code>1.25</code>	表面速度/ 深度平均速度比率
<code>D = gamma*H^5*(S')^2</code>	<code>-</code>	SIA 扩散系数 (n=3 时的形式)

11.13 总结

本章深入探讨了 OGGM 动力学代码背后的数值方法：SIA 方程的两类离散化方案（显式通量形式和半隐式扩散形式）、CFL 条件导出的稳定约束、交错网格的棋盘格模式抑制原理、以及支流通量的高斯合并技术。理解这些数值基础对于有效使用 OGGM、扩展其模型系统、以及诊断模拟中的非预期行为都至关重要。

半隐式方案通过将 SIA 线性化并求解三对角系统获得巨大的稳定性优势，但其单流线限制限制了其在多条支流冰川上的应用。显式方案更灵活但步长更小——这是冰川模拟中经典的“精度-效率”张力的具体体现。

第12章 崩解参数化

崩解 (calving) 是入海型冰川和冰架物质损失的主导过程—在这些环境中，气候驱动的表面消融远小于末端冰崖崩落和冰山产犊的质量损失。OGGM通过k-崩解定律和迭代一致性算法将这个过程参数化，使入海型冰川的反演和正演模拟能与陆地冰川共享同一个SIA动力框架。本章涵盖崩解的物理原理、在反演和正演中的双重应用、以及支撑模拟稳定性的桶形移除机制。

崩解不是默认“万能开关”

入海型和湖泊终止冰川对水位、末端冰厚、水下地形和 calving_k 非常敏感。OGGM的k-崩解方案适合区域尺度参数化，但不能替代高分辨率冰崖断裂、海洋热强迫或水下地形观测。论文中使用崩解结果时，应说明是否启用了反演崩解、正演崩解以及所用 calving_k。

12.1 入海型冰川检测

OGGM通过RGI (Randolph Glacier Inventory) 中的 TermType 属性判定冰川是否为入海型 (tidewater):

```
# gdir.is_tidewater 来源于RGI的TermType字段
# TermType = 1: 陆地终止 (land-terminating)
# TermType = 2: 海洋终止 (marine-terminating) —入海型冰川
# TermType = 3: 冰架终止 (shelf-terminating) —漂浮冰架

# cfg.PARAMS['tidewater_type'] 控制哪些类型被处理为入海型冰川
# 默认值: 2 (仅海洋终止), 选项 2+3 = 5 (海洋+冰架)
```

全球RGI清单中约13%的冰川被标记为入海型冰川 (tidewater glacier)，但它们贡献了约40%的冰川质量损失（主要来自格陵兰、阿拉斯加、斯瓦尔巴和南极半岛的大型入海型冰川 (tidewater glaciers)）。正确处理这些冰川对OGGM的海平面贡献预测至关重要。

12.2 k-崩解定律

12.2.1 物理基础

k-崩解定律 (Oerlemans & Nick 2005) 将崩解通量与水深度和冰厚度参数化关联:

```
# k-崩解定律 (flowline.py 第1545行)
q_calving = k * water_depth * ice_thickness * width

其中:
k          - 崩解速率常数 ( $a^{-1}$ ), 默认 0.6
water_depth - 水深 (m) = 水面高度 - 冰床高程 (末端处)
ice_thickness - 末端处冰厚 (m)
width      - 末端宽度 (m)

# 物理解释:
# 更深的水 -> 更大的浮力力矩 -> 更高的崩解率
# 更厚的冰 -> 更高的冰崖 -> 更多的重力驱动崩解
# k 集成了所有未分辨的物理过程 (水温、裂缝深度等)
```

Oerlemans & Nick (2005) 提出的这个公式是基于对Hansbreen冰川 (斯瓦尔巴) 的观测，其中k值通过校准为 $\sim 0.6 \text{ a}^{-1}$ 。这个值在OGGM中被作为默认值，但可以通过 `cfg.PARAMS['calving_k']` 进行调整以适应不同地区的冰川。

12.2.2 关键参数

参数名	默认值	作用
calving_k	0.6	崩解速率常数 (a^{-1})，用于正演模拟

参数名	默认值	作用
<code>inversion_calving_k</code>	可能不同于 <code>calving_k</code>	反演中使用的k值（可独立设置）
<code>use_kcalving_for_inversion</code>	<code>True/False</code>	反演中是否启用崩解
<code>use_kcalving_for_run</code>	<code>True/False</code>	正演模拟中是否启用崩解
<code>free_board_marine_terminating</code>	<code>[10, 50] m</code>	海洋终止冰川的出水高度（freeboard）范围
<code>calving_front_slope</code>	变化	用于下游线延伸（downstream-line extension）的崩解前缘坡度
<code>water_level</code>	<code>0</code> （或诊断值）	参考水位线（m a.s.l.）

进阶

k值的区域依赖性

$k=0.6$ 是一个全球平均值，但不同区域的入海型冰川可能有截然不同的崩解行为。阿拉斯加沿海冰川（高降水量、温暖海洋）通常崩解率更高（ $k\sim 1.0-2.0$ ），而南极的冷基冰川崩解率更低（ $k\sim 0.2-0.5$ ）。OGGM的设计允许通过配置文件或 `gdir` 级的校准来调整k值，但目前的默认版本使用全球统一值。区域特定的k值标定是一个活跃的研究前沿。

12.3 FluxBasedModel中的崩解实现

12.3.1 崩解条件

在`FluxBasedModel`的`step`方法中（第1913–1971行），崩解仅在以下条件同时满足时执行：

- 1. 冰川末端格点（倒数第1或第2个有冰的格点）的冰床高度低于水面高度。
- 2. 末端以上存在冰高于水面——即冰川在进入水面之前有足够的冰厚。
- 3. `cfg.PARAMS['use_kcalving_for_run'] = True`。

12.3.2 崩解桶（calving bucket）累加器

崩解质量通过 `calving_bucket_m3` 累加器追踪：

```
# 崩解桶 (calving bucket) 机制
if is_calving_position(i) and ice_above_water:
    # 步骤1: 计算本步的崩解量
    q_calving = k * water_depth * thick * width
    calving_volume = q_calving * dt    # 本步崩解体积 (m3)

    # 步骤2: 累加到崩解桶
    calving_bucket_m3 += calving_volume

    # 步骤3: 检查是否可移除整个网格单元
    cell_volume = thick * width * dx
    if calving_bucket_m3 >= cell_volume:
        # 移除一个完整的网格单元
        thick[i] = 0
        calving_bucket_m3 -= cell_volume
        n_cells_removed += 1
```

12.3.3 两阶段移除

阶段1（水下冰移除）：首先移除低于水面的冰部分。这些冰不满足静水压平衡——即浮力不足以支撑冰崖的高度。物理上这代表了由于浮力力矩导致的底部崩解。 **阶段2（完整网格单元移除）：**当桶积累的崩解体积超过一个完整的网格单元体积时，移除整个单元。这是一种整数化的移除方案——因为OGGM的1D中心线模型不支持部分网格单元，崩解必须以整格点为单位执行。

为什么用桶而不是直接部分移除？

直接部分移除（如减少一个格点30%的厚度）会导致：（1）厚度空间不连续——一个格点突然变薄但相邻格点未变薄；（2）表面梯度异常——格点间的急剧表面坡度变化可能触发下一次步的数值不稳定。桶机制（bucket mechanism）将崩解"储蓄"下来，只在积累够一个格子后才执行，保持了厚度场的空间光滑性，代价是崩解在时间上略有延迟（通常每个格点延迟1-10年的崩解量）。

12.3.4 诊断输出

```
# 每个时间步计算的诊断量
calving_m3_since_y0      # 从初始年到当前的累积崩解体积 (m3)
calving_rate_myr         # 等效崩解率 (m yr-1)，用于与其他研究比较
```

`calving_rate_myr` 是将崩解体积转换为等效的末端后退率的指标——如果所有崩解都以末端后退（等宽）的形式发生，这个值就是末端的年均后退距离。

12.4 SemiImplicitModel中的崩解

半隐式方案中也包含了k-崩解定律的应用。不过由于半隐式方案使用隐式时间离散，崩解的实现细节有所不同：

- 崩解移除在执行三对角求解之前进行——因为移除的格子不参与隐式扩散系统。
- 半隐式的稳定性优势在这里体现——急剧的末端变化（崩解去除）不会触发CFL不稳定，因为隐式扩散步骤会自适应地处理新边界。
- 然而，半隐式方案仅支持单条流线——如果入海型冰川有多条支流，SemiImplicitModel无法处理支流合并，FluxBasedModel仍是唯一的选择。

12.5 崩解与冰床反演的耦合

12.5.1 inversion.py中的崩解路径

在反演任务（`workflow.py` 约第546行）中，崩解冰川和非崩解冰川走不同的处理路径：

```
# 反演任务分叉逻辑：
if gdir.is_tidewater and cfg.PARAMS['use_kcalving_for_inversion']:
    # 崩解路线：迭代一致性求解
    find_inversion_calving_from_any_mb(gdir, ...)
else:
    # 非崩解路线：标准SIA反演
    mass_conservation_inversion(gdir, ...)
```

12.5.2 find_inversion_calving_from_any_mb (inversion.py:1033)

这是崩解-冰厚反演的迭代一致性求解器，解决了"先有鸡还是先有蛋"的问题：

```
def find_inversion_calving_from_any_mb(gdir, ...):
    # 迭代循环:
    while not converged:
        # 1. 从当前冰厚估计计算崩解通量
        q_calving = k * water_depth(h) * h * width

        # 2. 将崩解通量加入物质平衡通量
        total_flux = mass_balance_flux + q_calving

        # 3. 用修正的通量求解新的冰厚 (SIA反演)
        h_new = mass_conservation_inversion(total_flux)

        # 4. 检查收敛性
        if |h_new - h_old| < tolerance:
            converged = True
            h_old = h_new

    # 收敛时将崩解速率写入 gdir.inversion_calving_rate
```

迭代通常在5-10轮内收敛。收敛后，`gdir.inversion_calving_rate` 存储了在参考气候下末端处的崩解速率—这个值不仅用于正演模型初始化，也用于后续的动力学模拟一致性检查。

12.5.3 迭代收敛的物理含义

迭代收敛意味着“给定气候条件和冰川几何，从SIA厚度反算的崩解率与从崩解律和SIA厚度预测的崩解率一致”。换言之—冰厚、气候、崩解三者物理上自洽。这种一致性是入海型冰川动态预热 (dynamic spinup) 的前提 (见第13章)，因为不一致的初始状态会在模拟中产生非物理的瞬时响应。

12.6 下游线 (downstream line) 延伸

12.6.1 calving_glacier_downstream_line (flowline.py:3175)

入海型冰川需要将流线和冰床横截面 (bed cross-section) 几何延伸到水面以下，因为冰川末端的冰厚可能在当前末端位置之外继续存在 (水下冰)。`calving_glacier_downstream_line` 函数按如下方式扩展床层几何：

```
# 延伸逻辑:
def calving_glacier_downstream_line(gdir):
    # 1. 从当前末端的冰床高程开始
    bed_start = flowline.bed_h[-1]
    surface_start = flowline.surface_h[-1]

    # 2. 使用 calving_front_slope 确定延伸趋势
    # slope = cfg.PARAMS['calving_front_slope']
    bed_extension = np.linspace(bed_start, water_level - max_depth,
                                n_extension_points)

    # 3. 确保延伸段在消融区
    # 添加足够多的格点使冰厚在延伸末端降为零
```

`calving_front_slope` 参数控制了末端外冰床的坡度。如果不确定实际的水下地形 (多数冰川缺乏实测测深数据)，OGGM默认使用一个保守的浅坡度，这导致崩解前缘的冰厚适中，而非急剧的深水冰崖。

12.6.2 出水高度 (freeboard)

`cfg.PARAMS['free_board_marine_terminating'] = [10, 50]` 定义了海洋终止冰川出水高度 (freeboard) 的可接受范围 (水面以上的冰崖高度)。在反演和模型初始化中，出水高度 (freeboard) 被约束在这个范围内：

```
# 出水高度 (freeboard) 约束
freeboard = surface_h[-1] - water_level
if freeboard < min_freeboard: # 冰崖太低 → 易倾斜崩塌
    clip_thickness_at_terminus()
if freeboard > max_freeboard: # 冰崖太高 → 物理不可支撑
    reduce_thickness_at_terminus()
```

下界10 m确保冰崖有足够的高度使重力崩解过程主导（而非被波浪和潮汐破坏）；上界50 m是力学稳定性的实用上限——高于50 m的冰崖在水面上由于冰的蠕变和裂缝不稳定难以长期存在。

12.7 水位确定

崩解计算中的水位可以来自多个来源，优先级从高到低为：

1. **反演诊断值**：从 `gdir.get_diagnostics()` 获得的每个冰川的水位（如果之前校准过）。
2. **配置文件常数**：`cfg.PARAMS['water_level']` 定义的全局恒定海平面（默认0 m）。
3. **气候驱动的海平面**：耦合海平面预测的项目可能提供时间变化的水位。

对于大多数RGI冰川，水位直接使用全球平均海平面（0 m），但格陵兰和南极半岛的冰川可能需要调整（由于后冰期回弹和区域海洋动力学的影响）。

12.8 崩解桶 (calving bucket) 设计模式详解

桶模式是OGGM崩解实现中最精巧的工程设计之一。核心思想：

```
# 经典问题：
# q_calving = k * wd * h * w = 0.6 * 50m * 200m * 1000m = 6e6 m3/yr
# cell_volume = h * w * dx = 200m * 1000m * 100m = 2e7 m3
# 一年仅积累完整格子的30%，如何在不做部分移除的前提下处理？

# 桶方案：
# 年1: bucket = 6e6 m3 （不够一个格子） → 不移除
# 年2: bucket = 12e6 m3 → 不移除
# 年3: bucket = 18e6 m3 → 不移除
# 年4: bucket = 24e6 m3 → bucket > 20e6 → 移除1格, bucket = 4e6
```

这个方案保证了：

- **整数性**：从不出现“半格”的情况。
- **长期守恒**：多年平均的移除量精确匹配崩解公式的预测。
- **空间光滑**：移除发生在末端格点，厚度突变不扩展到上游。

缺点是短期（1–5年）的末端位置可能有1–2个格点的“锯齿”——某些年份无后退，某些年份后退2个格点。但对于OGGM的典型应用（10–100年尺度），这种短期波动在研究结果中是不可分辨的。

12.9 calving_mb函数 (massbalance.py:1437)

`calving_mb` 将崩解通量转换为“等效物质平衡”的形式，用于在MB模型中统一表示所有的质量损失：

```
def calving_mb(gdir):
    # 将崩解通量 (m3 s^-1) 转换为等效MB (kg m^-2 yr^-1)
    # calving_mb = calving_flux * rho / glacier_area
    # 使崩解损失与表面消融损失可以统一求和
```

这个转换使得冰川总物质平衡可以被表达为一个单一数字——表面MB加上崩解通量，便于与其他模型和观测进行比较。

12.10 配置参数全表

参数	默认值	位置	含义
calving_k	0.6	cfg.PARAMS	正演崩解速率常数 (a^{-1})
inversion_calving_k	0.6	cfg.PARAMS	反演崩解速率常数 (a^{-1})
use_kcalving_for_inversion	False	cfg.PARAMS	反演是否考虑崩解；默认关闭，预处理脚本或入海型实验可显式开启
use_kcalving_for_run	False	cfg.PARAMS	正演是否考虑崩解；默认关闭，需在入海型正演实验中显式开启
free_board_marine_terminating	[10, 50]	cfg.PARAMS	出水高度 (freeboard) 范围 (m)
calving_front_slope	0.05	cfg.PARAMS	崩解前缘坡度 (m/m)
water_level	0	cfg.PARAMS	参考水位线 (m a.s.l.)
tidewater_type	2	cfg.PARAMS	入海型冰川类型过滤

12.11 总结

OGGM的崩解参数化以Oerlemans & Nick (2005)的k-崩解定律为核心，通过迭代一致性求解器将崩解损失纳入冰厚反演，再通过崩解桶 (calving bucket) 机制在正演模型中保证质量守恒和数值光滑。出水高度 (freeboard) 的约束和水位确定机制进一步确保了崩解过程在物理上有意义。

虽然 $k=0.6$ 的全球统一值与各区域的真实崩解行为之间存在系统性偏差，OGGM的模块化设计允许通过参数文件和gdir级诊断来区域化校准——这一灵活性与SIA动力框架的简洁性相结合，使入海型冰川的质量损失在OGGM预测量级中得到了合理的表示。

第13章 动态预热 (dynamic spinup) 与模型初始化

`dynamic_spinup.py` 中的 `run_dynamic_spinup` (约第38行) 用来缓解OGGM动力学初始化中的路径依赖问题: 反演、几何参数化和物质平衡建模通常以RGI编制年份附近的观测几何为约束, 但冰川并不是在该年份才开始演化。若直接从观测几何启动正演模型, 模型会在初期产生由初始化不一致引起的瞬态响应。动态预热 (dynamic spinup) 通过在近期历史气候下寻找与观测面积或体积相匹配的初始状态, 降低这类非物理的初始冲击。

一句话理解动态预热 (dynamic spinup)

动态预热 (dynamic spinup) 不是为了把冰川“调到平衡”, 而是为了找一个近期历史下更合理的初始状态, 使模型在RGI日期附近的面积或体积接近观测, 减少正演一开始的非物理跳变。

使用前检查

动态预热 (dynamic spinup) 计算成本高, 并且解不唯一。区域研究中应记录成功率、目标变量、`spinup_period`、温度偏差和最终不匹配; 失败冰川不应悄悄混入区域统计。

13.1 问题陈述

冰川的动力演化是路径依赖的——当前状态不仅取决于当前气候, 还取决于到达当前状态的历史路径。例如:

- 一条冰川从小冰期 (LIA, 约1850年) 至今退缩了2 km——当前的冰厚分布包含了19–20世纪的退缩历史信息。
- 如果简单地从RGI年份开始模拟, 模型会从“零历史”状态响应第一个时间步的MB强迫——这在前几年产生完全的瞬态响应 (错误的高速率变化), 而非真实的、已经在长期演化中部分达到平衡的响应。

动态预热 (dynamic spinup) 的目标是: 在给定预热 (**spinup**) 时段内寻找合适的温度偏差, 使模型从候选初始状态出发并经过历史强迫后, 在目标年份匹配RGI附近的观测面积或体积。这不是唯一解; 它是一种实用的初始化约束。

13.2 run_dynamic_spinup (第29行)

13.2.1 总体算法结构

```
def run_dynamic_spinup(gdir, **kwargs):
    # 步骤1: 确定目标年份
    target_yr = gdir.rgi_date + 1

    # 步骤2: 加载初始流线 (反演结果)
    init_flowlines = load_inversion_flowlines(gdir)

    # 步骤3: 设置两个MB模型
    mb_historical = MonthlyTIModel(gdir)  # 历史时期的气候强迫
    mb_spinup = ConstantMassBalance(...)  # 预热 (spinup) 时期的常量气候近似

    # 步骤4: 定义参考值 (观测目标)
    # 可以是 area_m2 或 volume_m3 (取决于 minimise_for 参数)
    ref_value = get_reference_from_flowlines(init_flowlines)

    # 步骤5: 调用核心优化循环
    t_bias, spinup_years = minimise_with_spline_fit(
        ref_value, target_yr, ...)

    # 步骤6: 用最佳参数完成spinup
    run_final_spinup(t_bias, spinup_years)
```

13.2.2 目标年份与参考值

`target_yr` 默认为 `gdir.rgi_date + 1`。RGI日期来自冰川目录 (GlacierDirectory) 中的清查年份，不同冰川并不相同；加一年是 OGGM内部将清查几何与模型年份对齐时采用的约定。参考指标由 `minimise_for` 参数控制：

- `'area'`：使用冰川面积作为匹配指标。面积是卫星影像中最直接的观测量。
- `'volume'`：使用冰川体积。体积与反演的不确定性较大，但某些研究偏好此选项。

13.3 核心优化循环：minimise_with_spline_fit (第503行)

13.3.1 优化问题的定义

目标：找到温度偏差 `t_spinup`（相对于参考气候的温度偏移），使得经过预热（spinup）时期后的冰川状态与观测匹配：

```
# 优化问题: find t_spinup such that mismatch(t_spinup) = 0
mismatch = (simulated - observed) / observed # 相对不匹配

# t_spinup > 0: 更暖的预热 (spinup) 气候 → 更小的冰川 → 更小的最终面积/ 体积
# t_spinup < 0: 更冷的预热 (spinup) 气候 → 更大的冰川
```

13.3.2 边界安全机制

优化过程中内置了两个安全边界：

```
# 安全检查1: 冰川消失 → 太暖
if glacier_area == 0 after spinup:
    lower_bound = t_spinup + 0.5 # 缩小搜索范围, 偏向更冷

# 安全检查2: 冰川超出域范围 → 太冷
if glacier_exceeds_domain:
    upper_bound = t_spinup - 0.5 # 缩小搜索范围, 偏向更暖
```

域范围超出指的是冰川增长超过了当前流线或DEM缓冲域可以可靠表示的范围。它并不等价于真实冰流绝不可能越过某条算法边界，而是数值域和几何输入已经不足以支撑继续外推。这个安全检查通过限制优化中的 `t_spinup` 上限来防止该情况。

13.3.3 步长限制

```
# t_spinup_max_step_length 限制了每次迭代的调整量 (默认2°C)
delta_t = min(spline_prediction - current_t, t_spinup_max_step_length)
# 避免大幅度的温度跳跃, 防止优化发散
```

这个限制防止了优化器在迭代早期对高温偏差做出过大的修正。冰川动力学对温度的响应是高度非线性的，2°C的步长限制确保每一步的冰川状态变化在可控范围内，不会从存在状态直接跳到消失状态。

13.3.4 样条拟合与零值预测

优化器使用一阶样条将(`mismatch`, `t_spinup`)的采样点对拟合起来，然后用插值预测零不匹配点的温度偏差：

```
# 样条拟合 (第560行附近)
tck = interpolate.splrep(mismatches, t_spinups, k=1) # 一阶样条
t_predicted = interpolate.splev(0, tck) # 预测零不匹配点的t_spinup

# 收敛条件: |mismatch| < precision_percent (默认1%)
if abs(mismatch) < precision_percent / 100:
    converged = True
```

一阶样条（分段线性）在这里比高阶样条更适用，因为`mismatch-t_spinup`关系的形状在较大温度范围内可能高度弯曲，用简单线性拟合和逐步收敛比用高阶全局插值更稳健。

为什么用样条而非牛顿法？

冰川的mismatch-t_spinup函数没有闭合形式，每次函数求值都需要完成一次动态模拟。牛顿法需要导数估计，但在存在数值噪声和函数不连续（如域超出导致的突然变化）时表现不稳定。样条拟合方法通过"采样-拟合-预测"循环在不确定性下更稳健，同时将求值次数保持在可控范围内。

13.4 Spinup时期试验循环（第831行）

对某些冰川，可能不存在使冰川在spinup期中保持存在的同时达到观测大小的温度偏差。此时，OGGM尝试多个（最多3个）逐渐缩短的预热（spinup）时期：

```
# 试验1: 使用默认的用户指定的spinup_period
for period in [period_1, period_2, period_3]:
    try:
        t_bias = minimise_with_spline_fit(period=period, ...)
        break # 成功找到符合条件的t_bias
    except SpinupFailed:
        # 尝试更短的时期
        period = max(period / 2, min_spinup_period)

# 兜底方案: ignore_errors=True → 不使用spinup
if all_periods_failed and ignore_errors:
    save_without_spinup(gdir)
```

更短的预热（spinup）时期意味着预热（spinup）气候对冰川的影响时间更短，冰川离观测状态更近——因此更容易找到匹配的t_spinup。但过短的时期意味着预热（spinup）获得的"历史记忆"有限，降低了动态预热（dynamic spinup）的效用。

13.5 关键参数

dynamic_spinup.py定义了约49个参数，其中最关键的有：

参数名	默认值	含义
spinup_period	20 年	Spinup时期长度；若设置 spinup_start_yr 则被后者覆盖
min_spinup_period	10 年	初次尝试失败时允许缩短到的最小预热（spinup）时期
minimise_for	'area'	匹配指标（area 或 volume）
precision_percent	1%	相对不匹配收敛容差
precision_absolute	变化	绝对不匹配收敛容差
first_guess_t_spinup	-2.0	温度偏差初始猜测（°C）
t_spinup_max_step_length	2.0	每次迭代温度步长上限（°C）
maxiter	30	优化最大迭代次数

13.6 融化因子校准

13.6.1 dynamic_melt_f_run_with_dynamic_spinup（第948行）

这是将融化因子（melt_f）校准与动态预热（dynamic spinup）集成的核心函数。其流程是：

```
def dynamic_melt_f_run_with_dynamic_spinup(gdir, **kwargs):
    # 1. 更新融化因子
    update_melt_f(gdir, new_melt_f)

    # 2. 用新的融化因子重新运行冰厚反演
    mass_conservation_inversion(gdir, glen_a=..., fs=...)

    # 3. 用新的冰厚重新进行动态预热 (dynamic spinup)
    run_dynamic_spinup(gdir, ...)

    # 4. 计算大地测量物质平衡 (geodetic mass balance)
    geodetic_mb = compute_geodetic_mb(gdir)

    # 5. 检查是否收敛 (大地测量MB匹配观测)
    return geodetic_mb_mismatch
```

这个流程的嵌套结构反映了OGGM中参数的高度耦合——融化因子 `melt_f` 的变化影响物质平衡（第8章），物质平衡影响反演的冰通量和冰厚（第9章），冰厚影响动态预热（dynamic spinup）的结果（本章），而预热（spinup）结果又影响后续模拟的物理一致性。

13.6.2 回退机制（第1239行）

```
def dynamic_melt_f_run_with_dynamic_spinup_fallback(gdir):
    # 如果主校准失败，逐步回退：
    # 1. 尝试减少预热 (spinup) 时期
    # 2. 尝试不使用动态预热 (dynamic spinup)
    # 3. 尝试使用预设的融化因子
    # 4. 完全放弃spinup (标记为no_spinup)
```

这个多层回退设计确保了对复杂冰川（如具有多条支流的入海型冰川，tidewater glacier）或数据不足的冰川，校准过程不会永久阻塞整个 workflow。

13.7 设计模式分析

13.7.1 嵌套优化

动态预热（dynamic spinup）本身是一个优化问题（寻找 `t_spinup`），但当它嵌入融化因子校准时，形成了嵌套优化结构——外环优化 `melt_f`，内环优化 `t_spinup`。这种双级优化在计算上是昂贵的：对于每条冰川，内外环的每次迭代都需要完成一个完整的预热（spinup）模拟（可能100–200年）。20条冰川的完整嵌套校准可能需要数千个模型年。

13.7.2 闭包用于状态管理

`minimise_with_spline_fit` 内部使用Python闭包来管理优化状态——`mismatch`和`t_spinup`的历史值被闭包在一个可变字典中，避免了全局变量和线程安全问题：

```
def minimise_with_spline_fit(...):
    state = {'mismatches': [], 't_spinups': [], 'iteration': 0}

    def objective_function(t_spinup):
        mismatch = run_spinup_with_t_bias(t_spinup)
        state['mismatches'].append(mismatch)
        state['t_spinups'].append(t_spinup)
        return mismatch

    # 闭包保持了状态，而不需要类实例
```

13.7.3 边界扩展

当优化器在初始猜测的 `t_spinup` 处评估不匹配，并且这个不匹配未能到达零值（例如，在初始范围内函数单调但不穿过零轴），边界扩展机制自动扩大 `t_spinup` 的搜索范围：

```
# 如果所有样本点的mismatch同号 → 扩展边界
if all_mismatches_same_sign:
    lower_bound -= 2 # 进一步冷却搜索
    upper_bound += 2 # 进一步加温搜索
```

13.8 workflow集成

13.8.1 预处理级别 (prepro level) L4

动态预热 (dynamic spinup) 在 run_dynamic_melt_f_calibration 任务中触发, 属于预处理级别 (prepro level) L4 (workflow.py)。这是OGGM工作流中的高阶任务, 通常在基本反演 (L3) 之后执行。在L4中:

- 融化因子 (melt_f) 被校准以匹配大地测量MB。
- 动态预热 (dynamic spinup) 寻找与观测一致的初始条件。
- 校准后的温度偏差存储在 gdir 的diagnostics中。

13.8.2 与正演任务的衔接

动态预热 (dynamic spinup) 不是通过一个全局的 use_dynamic_spinup 开关隐式启用, 而是由 run_dynamic_spinup 及其包装任务显式调用。成功后, 任务会写出预热 (spinup) 后的模型状态, 并在 gdir 诊断字典中记录温度偏差、目标年份和匹配误差; 后续正演可以从这个初始化状态继续运行:

```
# 运行动态预热 (dynamic spinup) 并保存初始化几何
model = dynamic_spinup.run_dynamic_spinup(
    gdir,
    output_filesuffix='_dynamic_spinup',
    store_model_evolution=True,
)

# 诊断信息写入gdir, 可用于质量控制
diag = gdir.get_diagnostics()
temp_bias = diag['temp_bias_dynamic_spinup']
period = diag['dynamic_spinup_period']
```

13.9 输出诊断

成功的动态预热 (dynamic spinup) 产生以下输出:

诊断量	含义	用途
run_dynamic_spinup_success	是否成功完成动态预热 (dynamic spinup)	批量模拟中的质量控制
temp_bias_dynamic_spinup	校准温度偏差 (°C)	记录找到的近期历史气候偏移
dynamic_spinup_target_year	匹配的目标年份	通常接近RGI日期或用户指定年份
dynamic_spinup_period	使用的预热 (spinup) 时期 (年)	记录spinup历史长度
dynamic_spinup_forward_model_iterations	正演模型调用次数	用于判断计算成本和收敛难度
area_mismatch_dynamic_spinup_km2_percent 或 volume_mismatch_dynamic_spinup_km3_percent	最终面积或体积不匹配 (%)	用于评估spinup质量

13.10 总结

`dynamic_spinup.py` 通过样条拟合优化和嵌套的气候-动力学校准，减小了OGGM动力学模拟中的初始化"shock"瞬态响应。它将静态的RGI快照转化为具有近期气候历史约束的动态初始条件。参数体系提供了细粒度控制，但核心逻辑保持清晰：通过调节温度偏差-不匹配函数找到使冰川状态与观测一致的气候强迫，然后保存对应的初始化模型状态和诊断信息。

预热 (spinup) 是OGGM中计算成本最高的单冰川任务之一（每条冰川可能需数百个模型年的计算），但也是区分高质量动力学模拟与简单外推的关键步骤。随着区域到全球尺度的OGGM模拟越来越普及，动态预热 (dynamic spinup) 已成为标准预处理流程的必要组成部分。

第14章 二维SIA模块

`sia2d.py` 约449行，实现了OGGM的二维SIA网格模型——与第10章讨论的一维流线模型不同，这个模块在规则网格上直接求解二维SIA方程。它适用于理想化实验、小冰川的详细模拟、以及1D流线与2D网格模型之间的方法对比。代码量虽小，但包含了从上游加权差分到CFL时间步进的完整二维冰流求解器。

先明确定位

OGGM的常规区域和全球模拟主要依赖一维流线工作流。二维SIA模块更适合理想化实验、教学和数值方法比较；除非已经验证输入、边界条件和适用性，不建议把它作为常规区域研究的默认工具。

14.1 概述与设计理念

1D流线模型（`FlowlineModel` 及其子类）通过将冰川简化为沿中心线的带状几何来降低计算维度。这种方法在计算效率和物理精度之间取得了平衡，但以牺牲二维细节为代价——例如横谷宽度变化、侧向冰流效应、和不规则流域几何。2D SIA模块填补了这一空白，提供了在完整的二维地形网格上直接求解SIA方程的能力。

关键区别：

特性	1D流线模型（第10章）	2D SIA模块（本章）
几何表示	沿中心线的一维带状 + 冰床横截面形状 (bed cross-section geometry) 参数	二维规则网格 (dx x dy)
侧向约束	通过宽度或梯形参数隐式表示	通过实际地形显式表示
支流系统 (tributary system)	支持一支流汇入主流线	不支持——所有通量在网格中直接计算
崩解参数化	全功能k-崩解定律	未实现（简化设计）
计算成本	$O(n_x)$ per step	$O(n_x \times n_y)$ per step
适用场景	生产批量模拟	理想化实验、小冰川、方法研究

14.2 冰边界滤波：filter_ice_border（第10行）

在2D网格中，冰川边缘的处理是一个微妙的问题。冰覆盖区域与无冰区域之间的边界处，SIA梯度可能产生非物理的冰流。OGGM通过一个简单的域边界滤波来处理：

```
def filter_ice_border(ice_thick):
    # 将域边缘的一个像素宽的边界上的冰厚置为零
    ice_thick[0, :] = 0
    ice_thick[-1, :] = 0
    ice_thick[:, 0] = 0
    ice_thick[:, -1] = 0
    return ice_thick
```

这个边界条件在物理上等价于“冰川不能扩展到计算域之外”——在计算域边缘，冰被强制为零，起到了吸收边界的作用。默认的 `ice_thick_filter` 就是 `filter_ice_border`，但用户可以替换为自定义的滤波器（例如更复杂的缓冲带或渐变过滤）。

为什么不使用更复杂的边界条件？

完整的SIA模拟需要处理流动边界（flux boundary）——本质上是一个具有移动边界的自由边界问题。但在1D流线模型中通过跟踪每个格点的冰/无冰状态来处理（通过厚度截断），2D模块采用更简单的域级零厚度边界。对于从小冰期以来一直退缩的冰川，计算域可以简单地设置为比当前冰川范围大一些，确保在整个模拟期间冰川始终在域内。

14.3 Model2D基类（第19行）

14.3.1 构造函数

```
class Model2D:
    def __init__(self, bed_topo, init_ice_thick, dx=None, dy=None,
                 mb_model=None, glen_a=None, ice_thick_filter=None):

        # 2D 地形网格
        self.bed_topo = bed_topo           # shape (ny, nx) - 冰床高程 (m)
        self.init_ice_thick = init_ice_thick # shape (ny, nx) - 初始冰厚 (m)
        self.dx = dx or 100                # 格点间距 x 方向 (m)
        self.dy = dy or dx                  # 格点间距 y 方向 (m), 默认与dx相同

        # MB模型、Glen参数、高程反馈 (同FlowlineModel)
        self.mb_model = mb_model
        self.glen_a = glen_a or cfg.PARAMS['glen_a']
        self.mb_elev_feedback = 'annual'

        # 冰厚滤波器 (默认: filter_ice_border)
        self.ice_thick_filter = ice_thick_filter or filter_ice_border
```

14.3.2 核心属性

属性	类型	描述
yr	float	当前模型年份
area_m2	float	冰川面积 (m ²) = sum(ice_thick > 0) * dx * dy
volume_m3	float	冰川体积 (m ³) = sum(ice_thick) * dx * dy
surface_h	2D array	冰表面高程 = bed_topo + ice_thick (仅在有冰处)
area_km2	float	冰川面积 (km ²)
volume_km3	float	冰川体积 (km ³)

14.3.3 MB缓存与时间步进

Model2D 共享与 FlowlineModel 相同的MB缓存策略和 run_until / run_until_equilibrium / run_until_and_store 接口。这种接口一致性使得在1D和2D模型之间切换时上层调用代码无需改动。

14.4 Upstream2D模型（第247行）

14.4.1 上游加权差分格式

`Upstream2D` 是OGGM中实际的2D SIA求解器。其名称源于使用的上游加权差分——在计算扩散系数时，对冰厚的插值偏向于上游方向（较高表面的方向）：

```
class Upstream2D(Model2D):
    # gamma = 2 * A * (rho * g)^n / (n + 2)      (第299行)
    # 这是预计算的SIA常数

    # CFL = 0.124 (比Hindmarsh理论略大)
    # 理论最大值: 1 / (2*(n+1)) = 1/8 = 0.125
    # OGGM使用 0.124 — 略低于理论极限以确保稳定性

    # max_dt = 31 天 (避免过于粗大的月度时间步)
```

为什么CFL取0.124？对于二维非线性扩散方程，理论分析 (Hindmarsh 2001) 给出了稳定性极限 $1/(2*(n+1))$ 。对于 $n=3$ ，这是 $1/8 = 0.125$ 。OGGM选择了 0.124（略低于0.125），为数值舍入误差和边界效应留出了安全余地。

14.4.2 numpy ix_ 索引技巧

为了高效地执行交错网格操作，`Upstream2D`使用 `numpy.ix_` 来避免显式的Python循环：

```
# 使用 numpy.ix_ 创建开放网格索引
# 这允许向量化切片操作，避免效率低下的 Python for 循环
iy = np.arange(0, ny)
ix = np.arange(0, nx)
iy_dn, ix_dn = np.ix_(iy[1:-1], ix[1:-1])

# 这使得可以对内部域一次性进行向量化差分运算
# 而非逐行逐列循环
```

14.5 diffusion_upstream_2d：核心计算（第332行）

这是 `sia2d.py` 中最重要的函数——实现了二维SIA扩散-对流方程的完整空间离散：

```
def diffusion_upstream_2d(H, bed, dx, dy, gamma):
    # =====
    # 步骤1: 上游加权冰厚
    # =====
    # H_centered = 0.5 * (H[i+1] + H[i])      (中心平均)
    # H_upstream = H[i+1] if S[i+1] > S[i] else H[i]  (上游选择)
    H_up = upstream_weighted(H, bed, direction)

    # =====
    # 步骤2: 表面梯度范数
    # =====
    dsdx = (S[i+1, j] - S[i, j]) / dx      # x方向梯度 (交错面)
    dsdy_avg = (S[i, j+1] - S[i, j-1]) / (2*dy)  # y方向均值 (非交错)

    # 梯度范数 (用于计算扩散率中的非线性因子)
    grad_S_norm = sqrt(dsdx^2 + dsdy_avg^2)^((n-1)/2)

    # =====
    # 步骤3: 扩散系数
    # =====
    # D = gamma * H^(n+1) * H_upstream * grad_S_norm
    # 注意这里使用了上游加权的 H_upstream (而非 H^(n+2))
    # 当 n=3: D = gamma * H^4 * H_upstream * (grad_S)^(2)

    # =====
    # 步骤4: 通量散度
    # =====
    # div_x = (D_up*ds_up - D_dn*ds_dn) / dx^2
    # div_y = (D_left*ds_left - D_right*ds_right) / dy^2
    div_q = div_x + div_y

    # =====
    # 步骤5: CFL时间步长
    # =====
    dt = cfl * min(dx^2, dy^2) / max(D)

    return div_q, dt
```

进阶

上游加权 vs 中心加权的物理含义

上游加权（在较高表面方向上取冰厚）的物理基础是：冰流方向是**向下梯度**的，来自上游（较高表面）的冰通量携带了上游的厚度信号。在陡峭地形或厚度剧烈变化的位置，中心平均会引入来自“错误方向”的虚假厚度信息——例如下游无冰区的零厚度被平均进来，低估了实际流过来的冰量。上游加权避免了这个问题，保证了扩散系数的物理一致性。

14.6 Step方法（第435行）

Upstream2D 的步进方法是一个简单的向前欧拉更新：

```
def step(self, dt):
    # 1. 获取当前MB (取决于高程反馈模式)
    mb = self.get_mb(self.surface_h, self.yr)

    # 2. 计算通量散度和CFL时间步长
    div_q, dt_cfl = diffusion_upstream_2d(
        self.ice_thick, self.bed_topo,
        self.dx, self.dy, self.gamma)

    # 3. 使用实际时间步长 (用户指定的 dt 或 CFL 限制的 dt_cfl, 取较小者)
    dt_actual = min(dt, dt_cfl)

    # 4. 向前欧拉更新: dH/dt = MB + div_q
    new_surface_h = self.surface_h + (mb + div_q) * dt_actual

    # 5. 从表面高程和冰床高程反算冰厚
    new_ice_thick = new_surface_h - self.bed_topo

    # 6. 截断: 冰厚不能为负
    new_ice_thick = np.clip(new_ice_thick, 0, None)

    # 7. 应用冰厚滤波器 (默认: 零边界条件)
    self.ice_thick = self.ice_thick_filter(new_ice_thick)
```

与1D FluxBasedModel的两步法（先算通量、再更新厚度）相比，2D的步进法将所有操作合并在一个函数中——通量散度和厚度更新同时完成。这是因为2D的网络结构使得交错网格的存访模式更简单：在每个时间步中，所有2D数组一次性计算，无需在交错面和中心之间维护多个网格表示。

14.7 与1D流线模型的差异总结

14.7.1 几何表示

维度	1D 流线	2D SIA
侧向约束	冰床横截面形状 (bed cross-section geometry) 参数 (f_d, trapezoid_lambdas) 隐式	实际二维冰床横截面 (bed cross-section) 地形显式
冰厚分布	沿中心线一维变化	在二维平面内自由变化
宽度变化	预定义的宽度函数 w(x)	由冰厚和地形共同决定 (自然涌现)
几何假设	冰床横截面形状 (bed cross-section geometry) 在各位置独立假设	使用完整的DEM冰床横截面 (bed cross-section)

14.7.2 数值方案

特性	1D FluxBased	1D SemiImplicit	2D Upstream2D
离散化	交错通量 (面上)	中心扩散 (点内)	交错扩散 (面上)
CFL限制	$u \cdot dt / dx < 0.02$	$D \cdot dt / dx^2 < 0.5$	$D \cdot dt / dx^2 < 0.124$
矩阵求解	否 (显式)	是 (三对角)	否 (显式)
支流支持	是	否	不适用 (2D 网格无支流概念)
崩解支持	是 (k-定律)	是 (有限制)	否

14.7.3 适用性考量

何时使用2D模型：

- 理想化实验—如在规则沟谷或人工地形上测试SIA的物理行为。
- 小冰川 (< 5 km²) —计算成本可控，且1D的冰床横截面形状 (bed cross-section geometry) 假设对小冰川不太适用。
- 方法研究—比较1D和2D表示对同一冰川的影响。
- 教学用途—2D SIA方程的实现比1D更直观（直接对应数学公式）。

何时使用1D模型：

- 生产批量模拟—数万条冰川的模拟在计算上仅在1D表示下可行。
- 有支流系统 (tributary system) 的冰川—2D模块不支持支流合并。
- 需要崩解功能—入海型冰川需要崩解参数化。
- 大规模空间分析—1D模型的输出更易于汇总和统计。

14.8 gis.py集成

2D SIA模块与 gis.py 中的GIS工具配合使用。典型的使用流程为：

```
# 1. 从GIS数据准备二维网格
from oggm import gis
gdir = gis.define_glacier_region(rgi_id, ...)

# 2. 提取DEM和冰川掩码
dem = gdir.get_dem()           # 二维冰床高程网格
mask = gdir.get_glacier_mask() # 冰川覆盖标记

# 3. 初始化冰厚（可选：使用简单常数或反演结果插值）
init_ice_thick = np.where(mask, 50, 0) # 50 m常数厚度

# 4. 创建2D模型并运行
from oggm.core import sia2d
model = sia2d.Upstream2D(dem, init_ice_thick, dx=50,
                        mb_model=mb_model)
model.run_until_equilibrium()
```

14.9 关键参数速查

参数	默认值	位置	含义
<code>glen_a</code>	<code>2.4e-24</code>	<code>cfg.PARAMS</code>	Glen流动律参数
<code>cfl</code>	<code>0.124</code>	Upstream2D硬编码	CFL数（略低于理论极限0.125）
<code>max_dt</code>	31天	Upstream2D硬编码	最大单步时间步长
<code>gamma</code>	$2 * A * (\rho * g)^n / (n + 2)$	第299行计算	SIA预计算常数
<code>mb_elev_feedback</code>	<code>'annual'</code>	Model2D默认	MB高程反馈频率

14.10 总结

`sia2d.py` 在紧凑的449行中实现了一个完整的二维SIA冰流求解器，为OGGM的1D流线表示提供了补充方案。上游加权差分、numpy ix_向量化、和略低于理论极限的CFL数共同确保了数值的稳定性和效率。虽然没有支流系统 (tributary system) 和崩解参数化，但其简化的结构和与Model2D/FlowlineModel一致的接口使得方法对比和教学实验可以无缝进行。

对于大多数批量生产模拟，1D 流线模型（第10章）仍然是最佳选择，但2D 模块为研究者提供了一个宝贵的"基准真相"——可以在少量冰川上运行以验证1D模型的适用性和假设限制。



第15章 OGGM工作流管线

高级 核心架构

本章深入解析OGGM的工作流编排层，涵盖 `workflow.py`（约1336行）、`tasks.py` 和 `global_tasks.py` 三个核心模块。工作流管线是OGGM的"调度层"——它定义了冰川模拟从原始数据到最终输出的完整执行路径，并提供了灵活的并行计算基础设施。

从研究问题选择工作流

研究目标	最低任务链	主要输出	优先读
估算冰储量	GIS预处理 → 气候处理 → 物质平衡校准 → 反演	<code>inversion_output.pkl</code>	第5–9章
历史面积/ 体积变化	反演 → 初始化流线 → <code>run_from_climate_data</code>	<code>model_diagnostics.nc</code>	第10、15、18章
未来情景	历史工作流 → GCM气候处理 → 正演模拟	带filesuffix的诊断文件	第7、8、16章
入海型冰川	崩解反演 → 含崩解的正演 → 诊断检查	<code>calving_m3</code> , <code>calving_rate</code>	第12、13章

15.1 架构概览：编排层设计哲学

OGGM的工作流系统遵循"任务即函数"的哲学。每个科学计算步骤被封装为一个独立的Python函数（称为entity task），工作流管线负责将这些函数按序编排并分发到多个冰川实体上。这种设计的核心优势在于：

- 解耦科学逻辑与调度逻辑：科学家只需编写单冰川的处理函数，无需操心并行细节。
- 统一的并行抽象：同一代码路径支持MPI多节点、multiprocessing多核、以及纯顺序三种执行模式。
- 断点续传：通过任务重复检测（global task rejection）避免重复计算。

`workflow.py` 是这套体系的核心文件。它不直接执行冰川物理计算，而是作为总指挥，调用 `tasks.py` 中的具体科学任务。理解 `workflow.py` 的调用关系，就等于掌握了OGGM全部功能的入口地图。

15.2 execute_entity_task：中央并行执行引擎

15.2.1 函数签名与职责

位于 `workflow.py` 第126行的 `execute_entity_task` 是整个OGGM中调用频率最高的函数。几乎所有的并行工作流都经过它。其核心职责是：接收一个task函数和一组冰川目录对象（`GlacierDirectory`, `gdir`），将task应用到每个gdir上，并以并行方式收集结果。

```
def execute_entity_task(task, gdirs, **kwargs):
    """Central parallel execution engine.

    Parameters
    -----
    task : callable
        The task function to apply. Must accept a gdir as first argument.
    gdirs : list of GlacierDirectory
        Glacier directories to process.
    **kwargs
        Additional keyword arguments passed to each task invocation.

    Returns
    -----
    list
        Results collected from all invocations, in the same order as gdirs.
    """
```

15.2.2 任务规范化

`task` 参数有两种传入方式：

传入方式	示例	说明
单个可调用对象	<code>tasks.compute_centerlines</code>	最常见的方式，所有gdir共享同一task和kwargs
列表	<code>[(task1, kw1), (task2, kw2), ...]</code>	每个gdir可拥有不同的task和/ 或不同的kwargs

在函数内部，`task`首先被规范化为统一的列表格式。单个函数被自动复制为与gdirs等长的列表：

```
if isinstance(task, (list, tuple)):
    # 用户提供了每gdir的(task, kwargs)列表
    task_list = task
else:
    # 统一task, 复制N份
    task_list = [(task, kwargs) for _ in gdirs]
```

15.2.3 全局任务 (global task) 拒绝机制

这是OGGM增量计算的关键机制。在将任务分发到worker之前，系统检查每个任务是否已在目标gdir上完成。已完成的任务被替换为 `_dummy_task` （一个返回 `None` 的空操作），从而避免浪费计算资源。

判断依据是 `global_tasks.py` 中定义的任务完成标志文件。每个冰川目录 (`GlacierDirectory`) 维护一组标志文件，记录哪些处理步骤已经完成。这种设计与make的增量构建理念一致—只处理需要更新的部分。

15.2.4 _pickle_copier: 替代functools.partial

Python标准库的 `functools.partial` 在multiprocessing环境下有一个致命缺陷：其 `__reduce__` 方法不完整，导致pickle序列化失败。OGGM为此设计了一个精巧的替代品：

```
class _pickle_copier(object):
    """A pickleable alternative to functools.partial.

    Multiprocessing needs to pickle the task and its arguments.
    functools.partial objects are not reliably pickleable across
    all Python versions, so we use this custom class instead.
    """
    def __init__(self, func, *args, **kwargs):
        self.func = func
        self.args = args
        self.kwargs = kwargs

    def __call__(self, gdir):
        return self.func(gdir, *self.args, **self.kwargs)
```

这个类的对象可以被pickle（因为 `func`、`args`、`kwargs` 都是简单属性），从而安全地跨进程边界传递。

设计笔记 如果你在自己的项目中遇到multiprocessing的pickle问题，`_pickle_copier` 模式可以直接复用。但请注意：被包装的函数必须是模块级的（不能是lambda或嵌套函数），否则仍然无法pickle。

15.2.5 MPI与Multiprocessing的分派

经过规范化和全局任务 (`global task`) 检查后，实际执行路径根据可用并行后端选择：

```

if _have_ogmpi:
    # MPI模式: master分发, worker执行
    out = ogmpi.mpi_master_spin_tasks(gdirs, task_list)
elif len(gdirs) > 1:
    # 多进程模式: 创建进程池
    pool = init_mp_pool()
    out = pool.map(run_task_on_gdir, zip(gdirs, task_list))
else:
    # 顺序模式: 单核回退
    log.warning("OGGM is running sequentially!")
    out = [run_task_on_gdir((g, t)) for g, t in zip(gdirs, task_list)]

```

顺序回退时会输出一条警告，帮助用户在排查性能问题时快速定位配置错误。

15.3 execute_parallel_tasks: 单冰川并行

`workflow.py` 第197行的 `execute_parallel_tasks` 解决了一个不同于 `execute_entity_task` 的问题：对同一个冰川目录（`GlacierDirectory`）执行多个可以并行运行的任务。这在高分辨率冰川处理或复杂的后处理流程中非常有用。

其实现相对简单：利用 `multiprocessing.pool.ThreadPool`（线程池，而非进程池），因为任务之间共享同一个 `gdir` 的内存视图，使用线程可以避免数据复制的开销。

15.4 Multiprocessing基础设施

15.4.1 init_mp_pool: 进程池生命周期管理

`init_mp_pool`（第41行）负责创建、复用、重置和销毁 `multiprocessing` 进程池。它维护了一个模块级变量 `_mp_pool` 和一个标志 `CONFIG_MODIFIED`：

```

def init_mp_pool(reset=False):
    """Get or create the global multiprocessing pool.

    If OGGM configuration has changed since the pool was created,
    the old pool is automatically terminated and a fresh one is
    created with the new settings.
    """
    global _mp_pool, CONFIG_MODIFIED
    if reset or CONFIG_MODIFIED:
        if _mp_pool is not None:
            _mp_pool.terminate()
            _mp_pool = None
        CONFIG_MODIFIED = False

    if _mp_pool is None:
        _mp_pool = Pool(processes=config.PARAMS['mp_processes'],
                        initializer=_init_pool_globals)
    return _mp_pool

```

`CONFIG_MODIFIED` 标志是一个巧妙的设计：只要 `config.PARAMS` 被修改，该标志就被置为 `True`，下一次调用 `init_mp_pool` 时自动重建进程池。这避免了配置变更后 `pool` 中的 `worker` 仍使用旧配置的问题。

15.4.2 _init_pool_globals: Worker初始化

每个 `multiprocessing worker` 进程在启动时调用 `_init_pool_globals`，将 `OGGM` 配置解包到进程的全局命名空间。这确保了所有 `worker` 共享一致的参数设置。

15.4.3 use_mp_spawn与跨平台兼容

在Windows上, Python的multiprocessing默认使用 spawn (而非Linux的 fork) 创建子进程。spawn模式下, 子进程不继承父进程的内存, 因此需要重新导入模块。OGGM通过 use_mp_spawn 配置项处理这一差异, 并在必要时回退到顺序执行。

关键限制 在Jupyter Notebook中使用multiprocessing时, spawn模式可能导致无限递归导入。OGGM建议在notebook环境中始终设置 mp_processes=1 (即关闭多进程), 或使用--mpi后端。

15.5 init_glacier_directories: 工作流的起点

15.5.1 从RGI GeoDataFrame初始化

init_glacier_directories (第325行) 是任何OGGM运行的第一步。它接受一个pandas GeoDataFrame (通常来自RGI shapefile), 为每一行创建对应的 GlacierDirectory (gdir) 对象:

```
gdirs = workflow.init_glacier_directories(rgi_df)
```

该函数调用 _check_rgi_input 进行输入验证, 包括:

- 重复冰川检测: 检查RGI ID是否有重复, 防止同一冰川被初始化两次。
- 连通级别检查: 要求连接级别2 (Connectivity Level 2), 即冰川的拓扑连接关系已知, 这是下游流线计算的前提条件。

15.5.2 RGI版本自动检测

OGGM能自动区分RGI v6和v7格式。通过检查DataFrame的列名和元数据, 系统选择合适的读取器。v7版本引入了更多的属性列和不同的坐标系参数。

15.5.3 Intersects数据库自动配置

RGI intersects文件记录了每条冰川与气候网格和历史数据产品的空间交集。如果该文件在默认位置存在, init_glacier_directories 会自动加载; 否则输出警告并跳过需要交集信息的后续步骤。

15.5.4 恢复模式与预计算存档

初始化方式	函数/ 参数	适用场景
从RGI新运行	init_glacier_directories(rgi_df)	首次处理
续跑	init_glacier_directories(rgi_df, from_prepro_level=N)	从预处理级别 (prepro level) N恢复
从tar恢复	init_glacier_directories(rgi_df, from_tar='url')	从远程存档恢复

续跑模式下, OGGM加载已有gdir的状态并跳过已完成的步骤, 仅执行未完成任务。从tar恢复则是从预处理的压缩包中解压完整的gdir目录。

15.6 工作流快捷函数

workflow.py 提供了一系列快捷函数, 每个函数封装了一条完整的子管线。这些函数是用户最常直接调用的API。

15.6.1 gis_prepro_tasks: 流线预处理链

位于第487行, 这是OGGM最长的子管线, 共10个任务:

```
def gis_prepro_tasks(gdirs, **kwargs):
    task_list = [
        tasks.define_glacier_region,      # 1. 定义冰川区域
        tasks.glacier_masks,              # 2. 冰川掩膜
        tasks.compute_centerlines,        # 3. 计算中心线
        tasks.initialize_flowlines,        # 4. 初始化流线
        tasks.compute_downstream_line,     # 5. 计算下游延长线
        tasks.compute_downstream_bedshape, # 6. 下游冰床形态
        tasks.catchment_area,             # 7. 集水区面积
        tasks.catchment_intersections,     # 8. 集水区交叉
        tasks.catchment_width_geom,        # 9. 几何宽度计算
        tasks.catchment_width_correction,  # 10. 宽度修正
    ]
    for task, kwargs in task_list:
        execute_entity_task(task, gdirs, **kwargs)
```

这条管线从原始DEM出发，逐步构建出冰川流线网络和集水区几何。每个步骤的输出是下一步的输入，形成严密的数据依赖链。初学者通常应使用 `gis_prepro_tasks` 保持标准顺序；只有在调试或开发新方法时，才建议逐步拆开运行。

15.6.2 climate_tasks: 气候与物质平衡校准链

位于第513行，包含3个核心任务：

```
process_climate_data → local_mustar → apparent_mb_from_any_mb
```

`local_mustar`（即 `mb_calibration_from_geodetic_mb`）是校准的核心——它利用大地测量数据调整温度敏感度参数 μ_k ，使模型物质平衡与观测一致。

15.6.3 inversion_tasks: 冰厚反演链

位于第531行。这是OGGM最复杂的子管线，因为需要区分是否有冰川崩解（calving）：

```
if gdir.is_tidewater:
    # 崩解冰川路径
    tasks.prepare_for_inversion(gdir, ...)
    tasks.mass_conservation_inversion(gdir, ...)
    tasks.filter_inversion_output(gdir, ...)
else:
    # 陆地冰川路径
    tasks.prepare_for_inversion(gdir, ...)
    tasks.mass_conservation_inversion(gdir, ...)
    tasks.filter_inversion_output(gdir, ...)
```

两条路径共享相同的函数名，但内部算法因冰川类型不同而有所差异。`mass_conservation_inversion` 实现了基于冰厚-冰流通量关系的物质守恒反演方法。

15.6.4 calibrate_inversion_from_consensus

位于第586行，是OGGM最关键的全局校准步骤。它通过优化Glen流动律参数A，使OGGM反演的全球冰川冰体积与Farinotti等人（2019）的共识估计相匹配：

```
def calibrate_inversion_from_consensus(gdirs, ...):
    # 搜索Glen A，使总体积收敛到全球共识值
    for A in glen_a_values:
        execute_entity_task(tasks.mass_conservation_inversion, gdirs, glen_a=A)
        total_volume = sum(gdir.read_pickle('inversion_output')['volume'] for gdir in gdirs)
        if abs(total_volume - consensus_volume) < tolerance:
            break
```

`calibrate_inversion_from_volume`（第783行）是该函数的单冰川变体，`invert_from_params`（第725行）则使用预计算的参数跳过迭代搜索过程。

15.7 merge_glacier_tasks：支流合并 workflow

位于第901行，处理支冰川与主冰川的合并。在自然冰川系统中，多条冰流汇入同一冰川，OGGM需要将这些支流整合到单一流线网络中：

```
def merge_glacier_tasks(gdirs, **kwargs):
    # 步骤1: 检测支流关系
    execute_entity_task(tasks.intersect_downstream_lines, gdirs)
    # 步骤2: 递归合并
    gdirs = _recursive_merging(gdirs)
    # 步骤3: 清理合并后的流线
    execute_entity_task(tasks.clean_merged_flowlines, gdirs)
    return gdirs
```

`_recursive_merging` 是核心算法：它识别RGI连接级别2中定义的支流关系，从末端支流开始，逐步向上游合并，最终每条冰川只有一个主 `gdir`。合并过程中需要重新计算流线拓扑、宽度分布和集水区边界。

专家提示 支流合并是整个 workflow 中最容易出现问题的环节之一。常见的错误包括：流线相交拓扑不一致、合并后宽度突变导致的反演不稳定、以及支流编号丢失。调试时建议先用 `plot_domain` 可视化检查合并前后的流线网络。

15.8 merge_gridded_data：2D格网数据 (gridded data) 合并

位于第1025行，处理 `GriddedNcdfFile` 格式的2D输出数据组合。当多条冰川的输出格网需要合并为区域数据集时使用：

```
def merge_gridded_data(gdirs, variables=None, combine_plots=None, ...):
    # 创建统一的输出格网
    target_grid = create_target_grid(gdirs)
    # 逐个变量重投影和合并
    for var in variables:
        merged = reproject_and_merge(gdirs, var, target_grid)
        # 高斯平滑减少边缘伪影
        merged_smooth = gaussian_filter(merged, sigma=1.0)
    return merged_dataset
```

格网合并面临三个主要挑战：不同冰川可能使用不同分辨率和投影、合并边界处可能出现不连续、以及总面积和总体积等全局量需要在重投影中保持守恒。OGGM通过保守重投影和高斯平滑来应对。

15.9 预处理级别 (prepro level) 系统

预处理级别 (prepro level) 系统（定义于 `cli/prepro_levels.py`）是OGGM工业化运行的核心。它将完整 workflow 分为5个渐进级别：

级别	名称	包含步骤	输出
L0	初始化	<code>init_glacier_directories</code>	<code>gdir</code> 骨架
L1	DEM处理	<code>define_glacier_region</code> + <code>glacier_masks</code>	冰川掩膜
L2	流线	<code>compute_centerlines</code> → <code>catchment_width_correction</code>	完整流线网络
L3	气候+反演	<code>climate_tasks</code> + <code>inversion_tasks</code> + 校准	冰厚+ μ^*
L4	历史模拟	<code>historical_run</code>	时间序列输出
L5	清理	删除中间文件	精简 <code>gdir</code>

每个级别完成后，系统将结果打包为tar存档。下一级别可以通过 `--start-base-url` 加载前一级别的存档，实现增量恢复。这种设计带来了两大好处：

- 1. **故障隔离**：如果L3失败，不需要重新运行L0–L2。只需从L2存档恢复。

2. **分布式处理**：不同RGI区域可以在不同节点上独立运行各个级别，存档是节点间传递数据的标准格式。

15.10 小结

`workflow.py` 是OGGM的工作流编排层。它以不到1400行Python代码连接单冰川任务、全局任务 (`global task`)、错误处理和并行执行。其核心设计是将科学任务与调度基础设施解耦，通过任务状态记录支持增量计算，并提供从单行Python脚本到集群批处理的统一API。理解 `workflow.py` 的调用模式后，自定义处理管线通常只需要编写遵循 (`gdir, **kwargs`) 签名的函数，再交给 `execute_entity_task` 调度。

第16章 命令行界面

入门

操作指南

OGGM的命令行界面（CLI）是面向最终用户的主要交互接口。通过 `cli/` 目录下的一组模块，用户无需编写Python代码即可完成从数据下载到全球冰川模拟的完整 workflow。本章详细解析CLI的架构设计、主要命令、配置机制和HPC部署模式。

16.1 CLI架构：入口点与模块组织

16.1.1 pyproject.toml入口点注册

OGGM使用PEP 621标准的 `pyproject.toml` 注册CLI入口点。每个CLI命令对应一个 `[project.scripts]` 条目：

```
[project.scripts]
oggm_prepro = "oggm.cli.prepro_levels:main"
oggm_benchmark = "oggm.cli.benchmark:main"
```

当用户执行 `oggm_prepro` 时，Python调用 `oggm.cli.prepro_levels.main()` 函数。这种设计遵循了Python打包的最佳实践，保证了跨平台的兼容性。

16.1.2 CLI模块清单

模块	命令	用途
<code>prepro_levels.py</code>	<code>oggm_prepro</code>	主力常用预处理管线（L0–L5）
<code>benchmark.py</code>	<code>oggm_benchmark</code>	性能基准测试

16.2 oggm_prepro：主力常用CLI

`oggm_prepro` 是OGGM最重要（也最复杂）的命令行工具。它将第15章讨论的5级预处理管线暴露为一系列命令行参数。

16.2.1 基本调用结构

```
oggm_prepro \
  --rgi-reg 01 \           # RGI 区域编号
  --rgi-version 6 \        # RGI 版本
  --map-border 160 \       # 周围边距（像素）
  --start-level 0 \        # 起始预处理级别（prepro level）
  --max-level 4 \          # 最大运行级别
  --working-dir /path/to/wd \ # 工作目录
  --output-dir /path/to/out  # 输出目录
```

16.2.2 五级管线参数

`--start-level` 和 `--max-level` 定义了管线执行的起止范围。完整流程为L0→L1→L2→L3→L4→L5：

级别	执行内容	关键输出文件
L0	RGI初始化, gdir创建	<code>glacier_grid.json</code>
L1	DEM裁剪, 冰川掩膜	<code>gridded_data.nc</code> , <code>glacier_mask.tif</code>
L2	完整GIS预处理链	<code>centerlines.pkl</code> , <code>geometries.pkl</code>

级别	执行内容	关键输出文件
L3	气候数据+MB校准+反演	<code>inversion_output.pkl</code> , <code>climate_historical.nc</code>
L4	历史模拟运行	<code>model_diagnostics.nc</code> , <code>model_flowlines.nc</code>
L5	清理中间文件	仅保留最终输出

16.2.3 完整CLI参数表

参数	类型	默认值	说明
<code>--rgi-reg</code>	<code>str</code>	<code>OGGM_RGI_REG</code> <code>env</code>	RGI 区域编号 (如"01" 代表Alaska)
<code>--rgi-version</code>	<code>int</code>	6	RGI 版本 (6或7)
<code>--map-border</code>	<code>int</code>	160	冰川周围扩展像素数 (越大越"安全" 但对DEM而言更贵)
<code>--start-level</code>	<code>int</code>	0	起始预处理级别 (<code>prepro level</code>)
<code>--max-level</code>	<code>int</code>	5	最大运行级别
<code>--start-base-url</code>	<code>str</code>	None	恢复归档的URL (增量重启)
<code>--working-dir</code>	<code>str</code>	<code>OGGM_WORKDIR</code> <code>env</code>	工作目录路径
<code>--output-dir</code>	<code>str</code>	<code>OGGM_OUTDIR</code> <code>env</code>	输出目录路径 (与 <code>workdir</code> 可分离)
<code>--dem-source</code>	<code>str</code>	配置默认	DEM源 (ALL, STANDARD, 或特定代码如"COPDEM")
<code>--elev-bands</code>	<code>flag</code>	False	使用 <code>elevation bands</code> 流线方法 (默认为中心线方法)
<code>--centerlines</code>	<code>flag</code>	True	使用中心线方法 (默认启用)
<code>--mb-calibration-strategy</code>	<code>str</code>	<code>informed_threestep</code>	MB校准策略: <code>informed_threestep</code> , <code>melt_temp</code> , <code>temp_melt</code>
<code>--add-consensus-thickness</code>	<code>flag</code>	False	添加Farinotti共识冰厚数据
<code>--add-itslive-velocity</code>	<code>flag</code>	False	添加ITS_LIVE表面速度数据
<code>--add-millan-thickness</code>	<code>flag</code>	False	添加Millan冰厚估计数据
<code>--add-hugonnet-dhdt</code>	<code>flag</code>	False	添加Hugonnet高程变化率数据
<code>--dynamic-spinup</code>	<code>str</code>	None	动态预热 (<code>dynamic spinup</code>) 策略 (<code>area/dmdtda</code> , <code>volume/dmdtda</code>)
<code>--override-params</code>	<code>str</code>	None	JSON字符串覆盖默认参数

16.2.4 流线方法选择

OGGM提供两种流线提取方法, 通过 `--elev-bands` 和 `--centerlines` 互斥标志选择:

- 中心线方法 (默认): 基于Kienholz算法计算几何中心线, 再离散化为流线。精度更高, 但计算量更大。
- 高程带方法 (`--elev-bands`): 按高程区间划分冰川, 每带产生一条等距的流线。速度更快, 适合大量冰川的概率分析。

选择建议 对于数量少于1000条冰川的详细研究, 首选中心线方法 (默认)。对于全球尺度的快速扫描或敏感性分析, 高程带方法能显著加速。两种方法在L3及以后级别的计算时间差异可达5-10倍。

16.2.5 物质平衡校准策略

`--mb-calibration-strategy` 控制温度敏感度参数 μ^* 的校准方式:

策略	算法	适用场景
<code>informed_threestep</code>	三步信息校准 (默认)	标准正式运行, 有大地测量MB数据
<code>melt_temp</code>	融化温度法	简化校准, 备选方案
<code>temp_melt</code>	温度融化法	另一备选方案

16.3 环境变量与配置交互

OGGM CLI的参数解析遵循明确的优先级:

```
CLI显式参数 > 环境变量 > params.cfg配置文件 > 内置默认值
```

五个核心环境变量:

变量	作用
<code>OGGM_RGI_REG</code>	默认RGI 区域编号
<code>OGGM_MAP_BORDER</code>	默认地图边距
<code>OGGM_WORKDIR</code>	默认工作目录
<code>OGGM_OUTDIR</code>	默认输出目录
<code>OGGM_EXTRACT_DIR</code>	临时解压目录

复合参数通过 `--override-params` 传入JSON字符串覆盖 `params.cfg` 中的任意值:

```
oggm_prepro --override-params '{"mp_processes": 8, "use_multiple_flowlines": false}' \ ...
```

参数覆盖注意事项 `--override-params` 提供的JSON会与 `params.cfg` 进行浅层合并。嵌套配置项 (如 `"climate": {"temp_use_gradient": true}`) 可能需要完整提供嵌套对象。建议在运行前用 `oggm.cfg.log_params()` 验证最终生效的参数。

16.4 增量处理与恢复

`--start-base-url` 是工业化生产的关键参数。它允许从远程URL加载前一个预处理级别 (`prepro level`) 的tar存档, 实现断点恢复:

```
# 首次运行L0-L2
oggm_prepro --start-level 0 --max-level 2 ...

# L2完成后, 从L2存档恢复, 继续L3-L5
oggm_prepro --start-level 3 --max-level 5 \
  --start-base-url https://cluster.unibe.ch/prepro/RGIv6/L2/RGI01_L2.tar.gz ...
```

这种设计将耗时的工作流分解为可独立调度、可故障恢复的阶段。在SLURM集群中, 每个级别可以提交为独立的job array, 充分利用作业调度系统的容错能力。

16.5 oggm_benchmark: 性能基准测试

`cli/benchmark.py` 提供了一个标准化的性能测试工具。它定义了一条13任务的线性管线，在恒定气候和随机气候两种条件下各运行250年：

```
benchmark_tasks = [  
    tasks.define_glacier_region,  
    tasks.glacier_masks,  
    tasks.compute_centerlines,  
    tasks.initialize_flowlines,  
    tasks.compute_downstream_line,  
    tasks.compute_downstream_bedshape,  
    tasks.catchment_area,  
    tasks.catchment_intersections,  
    tasks.catchment_width_geom,  
    tasks.catchment_width_correction,  
    tasks.process_climate_data,  
    tasks.local_mustar,  
    tasks.run_random_climate, # 或 run_constant_climate  
]
```

对每个任务，`benchmark`记录执行时间并输出CSV格式的计时报告。`constant_climate` 模式使用不变的气候（适合测试模型物理），`random_climate` 模式使用随机生成的气候序列（适合测试模型对气候变率的响应）。

16.6 HPC部署模式

16.6.1 SLURM Job Array模式

RGI的19个一级区域天然适合SLURM job array：

```
#!/bin/bash  
#SBATCH --job-name=oggm_prepro  
#SBATCH --array=1-19  
#SBATCH --ntasks=1  
#SBATCH --cpus-per-task=16  
  
export OGGM_WORKDIR=/scratch/oggm_work  
export OGGM_OUTDIR=/scratch/oggm_output  
  
oggm_prepro \  
  --rgi-reg $(printf "%02d" $SLURM_ARRAY_TASK_ID) \  
  --start-level 0 \  
  --max-level 5
```

每个数组任务处理一个RGI区域，16个CPU核心用于multiprocessing。19个任务可以并行提交（取决于集群资源限制），总吞吐量受限于最慢的区域（通常是区域19，南极外围冰川）。

16.6.2 MPI多节点模式

对于单个大型RGI区域（如Alaska有超过27000条冰川），单节点16核可能不够。此时可以启用MPI跨节点并行：

```
#!/bin/bash  
#SBATCH --job-name=oggm_mpi  
#SBATCH --nodes=4  
#SBATCH --ntasks-per-node=16  
  
mpirun -np 64 python -c "  
from oggm import workflow  
# ... 设置RGI区域 ...  
workflow.execute_entity_task(task, gdirs)  
"
```


16.7 典型 workflow 示例

单冰川快速测试

```
# 仅处理RGI60-11.00897 (Hintereisferner)
oggm_prepro \
  --rgi-reg single \
  --start-level 0 --max-level 5 \
  --working-dir ./test_wd
```

区域处理 (Alaska)

```
oggm_prepro \
  --rgi-reg 01 --rgi-version 6 \
  --map-border 160 \
  --dem-source COP30 \
  --start-level 0 --max-level 4
```

动态预热 (dynamic spinup) 校准

```
oggm_prepro \
  --rgi-reg 11 --rgi-version 6 \
  --dynamic-spinup volume/dmdtda \
  --start-level 3 --max-level 4 \
  --add-consensus-thickness
```

多区域集合

```
for reg in 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19; do
  oggm_prepro --rgi-reg $reg --start-level 0 --max-level 5 &
done
wait
```

业务化运行环境建议 在业务化运行环境中运行全球19个RGI区域时，预计需要数百GB的磁盘空间和数千CPU小时。强烈建议：（1）在集群上使用job array而非本地循环；（2）启用等级存档，每个级别结束后立即压缩；（3）在运行前检查磁盘配额和inode限制。

16.8 小结

OGGM的CLI层将强大的科学计算能力封装为直观的命令行参数。从简单的单冰川测试到全球19个区域的分布式处理，`oggm_prepro` 提供了统一的界面。理解其参数体系、环境变量优先规则和增量恢复机制，是在HPC环境中高效使用OGGM的前提。

第17章 MPI并行计算

高级

HPC

OGGM的MPI模块（`mpi.py`，约100行）是整个并行计算体系的顶层支柱。虽然在代码体量上远小于`workflow.py`，但它实现了一种经典且高效的master-worker并行模式，使得OGGM能够跨越多台计算节点的边界，处理数万条冰川。

17.1 MPI设计哲学：极简与健壮

OGGM的MPI设计遵循“够用即可”的原则。它不试图实现复杂的消息传递模式或分布式数据结构，而是聚焦于一个核心任务：将冰川处理任务高效地分发到多台节点上执行。这种设计让核心逻辑保持精简，但不适合更复杂的并行模式（如分布式参数优化）。对于OGGM的主要计算负载——独立冰川的并行处理——master-worker模式已经足够。

MPI在OGGM中的定位是替代multiprocessing提供多节点能力。在语义上，两者等价：都是将 `execute_entity_task` 的工作负载分发到多个worker上。区别在于multiprocessing受限于单节点的核心数，而MPI可以扩展到集群的数百核。

17.2 MPI初始化与导入时检测

17.2.1 `_init_oggm_mpi`：导入时的静默检测

OGGM在包导入时就检测MPI的可用性（`oggm/__init__.py` 第17行）：

```
try:
    from oggm.mpi import _init_oggm_mpi
    _init_oggm_mpi()
except ImportError:
    _have_ogmpi = False
```

`_init_oggm_mpi` 函数定义在 `mpi.py` 中：

```
def _init_oggm_mpi():
    """Initialize MPI for OGGM if available.

    Called at package import time. Sets global flags that
    workflow.py uses to decide between MPI, multiprocessing,
    and sequential execution.
    """
    global _have_ogmpi, MPI, comm, rank, size

    try:
        from mpi4py import MPI as _MPI
        MPI = _MPI
    except ImportError:
        _have_ogmpi = False
        return

    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()

    if size > 1:
        _have_ogmpi = True
    else:
        _have_ogmpi = False
```

这段逻辑体现了OGGM对小规模使用的友好态度：只有真正启动了多进程MPI运行时（`size > 1`），才启用MPI模式。如果用户只是单进程运行（`size == 1`），即使安装了mpi4py，也不使用MPI路径。这避免了单进程情况下的通信开销。

17.2.2 SLURM环境变量检测

在高性能计算环境中，`mpi4py`本身会读取SLURM提供的环境变量（如 `SLURM_JOBID` 、 `SLURM_NTASKS` ）来配置MPI通信器。OGGM不需要直接解析这些变量——它完全依赖`mpi4py`的抽象层。这使得OGGM在其他作业调度系统（如PBS/Torque、LSF）中也能正常工作。

17.3 `mpi_master_spin_tasks`：中央分发算法

这是MPI模块的核心函数，实现了master-worker工作窃取（work stealing）模式：

```

def mpi_master_spin_tasks(gdirs, task_list):
    """Master distributes tasks, workers execute and return results.

    This is the MPI counterpart of pool.map() in the multiprocessing
    path. The master (rank 0) sends work items to workers as they
    become available, collects results, and assembles the final
    ordered output.

    Parameters
    -----
    gdirs : list of GlacierDirectory
        All glacier directories to process.
    task_list : list of (callable, dict) tuples
        Normalized task-keyword pairs for each gdir.

    Returns
    -----
    list
        Results in the same order as gdirs.
    """
    if rank == 0:
        # Master: distribute and collect
        results = [None] * len(gdirs)
        next_task = 0
        pending = {}
        active_workers = size - 1 # rank 1..N-1 are workers

        # Send initial batch to kickstart workers
        for i in range(1, min(size, len(gdirs) + 1)):
            if next_task < len(gdirs):
                comm.send((next_task, gdirs[next_task],
                           task_list[next_task]),
                          dest=i, tag=MPI_TAG_WORK)
                pending[i] = next_task
                next_task += 1

        # Spin loop: receive results, send new work
        while next_task < len(gdirs) or pending:
            status = MPI.Status()
            result = comm.recv(
                source=MPI.ANY_SOURCE,
                tag=MPI.ANY_TAG,
                status=status
            )
            worker = status.Get_source()
            task_idx = pending.pop(worker)
            results[task_idx] = result

            if next_task < len(gdirs):
                comm.send((next_task, gdirs[next_task],
                           task_list[next_task]),
                          dest=worker, tag=MPI_TAG_WORK)
                pending[worker] = next_task
                next_task += 1

        return results
    else:
        # Worker: receive-execute-return loop
        while True:
            task_idx, gdir, (task, kwargs) = comm.recv(
                source=0, tag=MPI_TAG_WORK)
            result = task(gdir, **kwargs)
            comm.send(result, dest=0, tag=MPI_TAG_RESULT)

```

17.3.1 工作窃取的意义

在异构集群中（不同节点有不同CPU型号或者有不同的背景负载），各个worker的处理速度可能差异很大。“工作窃取”意味着master不会在开始时静态分配等量的工作给每个worker，而是在worker完成当前任务后立即发送下一个。这种动态负载均衡策略确保了所有worker

始终保持忙碌，总体运行时间由最慢任务的平均值而非最慢worker决定。

17.3.2 通信标记

OGGM使用简单的整数标记区分消息类型：

```
MPI_TAG_WORK = 1    # master发送给worker的工作项目
MPI_TAG_RESULT = 2   # worker返回给master的计算结果
```

这最小化了MPI通信器的复杂性。

17.4 MPI与Multiprocessing的选择逻辑

在 `execute_entity_task` 中的三项选择（参考第15章）体现了OGGM的并行分层策略：

- 1. **MPI优先**：如果 `_have_ogmpi` 为True，使用MPI。这是唯一的多节点方案。
- 2. **Multiprocessing备选**：如果gdir数量大于1且MPI不可用，使用单节点进程池。
- 3. **顺序回退**：单冰川或以上两者都不可用的最终保障。

这个优先级排序反映了OGGM的设计理念：**在可用性允许的情况下使用最高级别的并行方案**。用户不需要手动指定并行后端—系统自动检测并选择最佳路径。

17.5 进程分布与角色

角色	Rank 范围	职责	代码路径
Master	0	任务分发、结果收集、进度监控、日志聚合	<code>mpi_master_spin_tasks</code> 的 <code>if rank == 0</code> 分支
Worker	1至N-1	接收任务、执行单冰川处理、返回结果	<code>mpi_master_spin_tasks</code> 的 <code>else</code> 分支

Master本身不执行冰川处理任务—它纯粹是协调者。这个设计选择是刻意的：在master-worker模式中，让master也参与计算会引入额外的复杂性（如master必须在处理任务的同时处理来自worker的结果），而master-worker模式通常受限于master的处理能力。将master专用于协调使通信逻辑最简化。

对于少量冰川（例如少于worker数量），可以将master配置为不参与分发（通过调整初始批次大小），但这在当前的OGGM实现中未采用，因为常见的使用场景是gdirs数量远大于worker数量。

17.6 配置与参数

MPI行为通过 `params.cfg` 中的 `use_mpi` 参数控制。当设为 `False` 时，即使在多进程`mpirun`环境中也回退到multiprocessing或顺序模式。这在调试时非常有用—可以先用顺序模式验证 workflow 正确性，然后启用MPI进行大规模计算。

```
# params.cfg 相关内容
use_mpi: bool = True      # 是否使用MPI（如果可用）
mp_processes: int = -1    # multiprocessing进程数（-1 = CPU核心数）
```

`mp_processes` 在MPI模式下通常设为1（每个MPI进程内部不再嵌套多进程），以避免进程/线程嵌套导致的资源竞争和调度复杂性。但在某些配置中（如每个MPI进程对应一个NUMA节点），可以设小值以处理I/O密集型任务。

17.7 调试MPI运行

17.7.1 日志聚合

MPI运行中，每个进程独立写日志导致的输出混乱是常见的调试痛点。OGGM通过rank-aware日志抑制来解决：

```
# 只允许rank 0输出INFO及以上的日志
# 其他rank仅输出WARNING及以上的日志
import logging
if rank != 0:
    logging.getLogger('oggm').setLevel(logging.WARNING)
```

这确保了关键信息集中在master的输出中，而每个worker只在遇到问题时才会发出警告。

17.7.2 错误传播

worker中抛出的异常必须传播回master。OGGM的worker在捕获异常后将其序列化为错误令牌并通过MPI发送给master，master在收到错误令牌后中止所有待处理任务并报告问题。

调试陷阱 MPI程序的错误消息通常比顺序程序更难定位。如果所有worker在master的输出中只看到"MPI process terminated"而无具体traceback，可以使用 `mpirun --output-filename logfile` 将每个rank的输出重定向到独立文件，然后逐一检查。

17.8 性能考量

17.8.1 通信开销

在MPI并行计算中，通信开销是决定可扩展性的关键因素。OGGM的冰川处理任务具有极高的计算-通信比：每个任务涉及数秒至数分钟的浮点运算，而通信仅涉及序列化一个结果对象（通常小于1MB）。这种“粗粒度”并行模式非常适合master-worker架构。

但在以下情况下通信可能成为瓶颈：

- **小冰川过多**：每条冰川处理时间极短（如少于1秒），通信延迟可能与计算时间相当。
- **结果对象过大**：某些诊断输出（如完整的时间序列或高分辨率地图）可达数十MB。这种情况下应考虑将结果写回文件而非通过MPI传回。

17.8.2 负载不均衡

冰川处理的执行时间随冰川大小和复杂度变化显著——大型冰川的处理时间是小冰川的100–1000倍。虽然在gdir列表末尾集中出现大型冰川会导致“长尾”延迟，但OGGM的工作窃取机制会部分缓解这一问题：处理完小冰川的worker会立即接管下一个大任务。

最佳实践是按面积降序排列gdirs——先分配大冰川，最后让小冰川填充空闲时间。这可以最小化所有worker的总等待时间。

17.8.3 伸缩性分析

在理想情况下（所有gdir处理时间相等），OGGM的MPI伸缩性接近线性——N个worker将运行时间缩短到约1/N。实际上，由于通信开销和负载不均衡，伸缩性在近百个worker时开始退化。但给定的典型RGI区域（数千条冰川），20–50个worker是效率最佳的区间。

17.9 HPC部署模式

17.9.1 SLURM作业脚本模板

```
#!/bin/bash
#SBATCH --job-name=oggm_mpi
#SBATCH --output=oggm_%j.out
#SBATCH --error=oggm_%j.err
#SBATCH --nodes=4
#SBATCH --ntasks-per-node=32
#SBATCH --time=24:00:00
#SBATCH --partition=normal

module load python/3.10
module load mpi4py

# 确保每个MPI进程使用1个OpenMP线程
export OMP_NUM_THREADS=1

mpirun -np $SLURM_NTASKS python -c "
from oggm import workflow, tasks, cfg
cfg.initialize()
cfg.PARAMS['mp_processes'] = 1 # 不在MPI内部嵌套多进程

# 执行工作流
gdirs = workflow.init_glacier_directories(rgi_df)
workflow.gis_prepro_tasks(gdirs)
workflow.climate_tasks(gdirs)
workflow.inversion_tasks(gdirs)
"
```

17.9.2 区域级并行

对于全球19个RGI区域的处理，有两种并行模式：

策略	并行粒度	优点	缺点
MPI跨区域	单个gdir	完美负载均衡	需要将所有数据放在共享文件系统中
Job Array分区域	整个区域	无需共享文件系统，容错性更好	负载不均衡（区域大小差异大）
混合方案	区域内MPI + 区域间Job Array	最佳性能	调度复杂度最高

混合方案（区域间用job array + 区域内用MPI）常用于大规模预处理。对于大型区域（01 Alaska、02 Western Canada），可部署4-8个节点的MPI；对于小型区域（如区域12 Caucasus），单个节点通常即可。

17.10 小结

OGGM的MPI模块代码量不大（约100行），但覆盖了区域冰川批处理最需要的并行模式：冰川级任务相互独立，适合master-worker式分发；当MPI不可用时，又可以回退到multiprocessing或顺序执行。这一设计使同一套工作流能在笔记本电脑和计算集群之间迁移。

第18章 图形与可视化

入门

可视化

`graphics.py` 是OGGM的可视化模块。尽管在代码量上（约32KB）不及`workflow.py`等核心模块，但它提供了从冰川几何到模型诊断的全面可视化功能。本章系统介绍各类型图表的绘制函数、底层基础设施、以及与`GriddedNcdfFile`的集成。

应用研究的最低作图清单

正式分析前至少应查看四类图：冰川轮廓与DEM是否对齐、中心线是否沿主流方向、反演厚度是否集中在合理位置、模型诊断时间序列是否有突跳。很多区域统计错误都能在这四类图中提前发现。

想检查什么	推荐图	异常信号
RGI 边界和DEM	<code>plot_domain</code>	轮廓错位、DEM空洞、边界太小
中心线和支流	<code>plot_centerlines</code>	中心线过短、穿出冰川、支流汇合异常
反演厚度	<code>plot_inversion</code> 或 自定义剖面图	厚度集中在末端、局部尖峰、体积异常
模拟输出	<code>plot_modeloutput_timeseries</code>	体积/面积突然跳变、长度不合理增长

18.1 模块架构概览

`graphics.py` 基于`matplotlib`构建，封装了OGGM特定的可视化逻辑。它的设计遵循几个核心原则：

- 一致性：所有绘图函数共享统一的色彩方案、字体设置和输出格式。
- 即用性：函数接受OGGM的标准数据结构（`GlacierDirectory`、`GriddedNcdfFile`），无需用户手动提取数据。
- 可扩展性：返回`matplotlib axes`对象，允许用户在默认绘图基础上任意修改。

18.2 图形基础设施

18.2.1 字体与排版

OGGM在字体支持上追求专业品质。当系统安装了Google Rockefeller Building字体（OGGM品牌字体）时，`graphics.py`自动使用该字体渲染图表文本。否则回退到`matplotlib`默认字体。

```
def _check_rockefeller_font():
    """Check for the Google Rockefeller Building font."""
    from matplotlib.font_manager import findfont, FontProperties
    font = findfont(FontProperties(family='Rockefeller'))
    return font != FontProperties().get_file()
```

这一设计体现了OGGM对图表美观的重视。Rockefeller Building字体是一种圆润的几何无衬线字体，用于OGGM的标识和文档。

18.2.2 色彩方案

OGGM定义了一套标准颜色映射和配色方案，所有图表函数共享：

- 冰川几何：深蓝至青色渐层（用于海拔、深度等高程相关数据）
- 流线网络：暖色系渐层（红色至橙色，用于区分主线和支流编号）
- 物质平衡：蓝白红发散色图（蓝=积累，白=平衡，红=消融）
- 诊断比较：高对比度离散色（蓝色=观测，红色=模型，绿色=差异）

18.2.3 图形保存

所有绘图函数通过统一的保存接口输出图形，支持可配置的分辨率（dpi）和多格式输出：

```
def _savefig(fig, path, dpi=None):
    """Centralized figure saving with configurable resolution."""
    if dpi is None:
        dpi = cfg.PARAMS.get('plot_dpi', 150)
    fig.savefig(path, dpi=dpi, bbox_inches='tight', facecolor='w')
```

18.3 冰川几何可视化

18.3.1 plot_domain：冰川全貌概览

`plot_domain` 是最常用的入门级绘图函数。它以单个图形展示冰川的完整空间范围：

```
plot_domain(gdir, ax=None, title=True,
            add_colorbar=True, add_scale_bar=True)
```

该函数在一个图中叠加以下元素：

- **底层**：DEM地形阴影 (hillshade)，来自gdir的栅格化 (rasterization) 高程数据
- **轮廓**：冰川边界线 (RGI outline)，黑色实线
- **格网**：GlacierDirectory定义的内部计算格网
- **流线**：中心线网络（如果已计算），按支流编号着色
- **色彩条**：DEM高程映射的colorbar
- **比例尺**：可选的地理比例尺（公里）

在开发新的冰川目录 (GlacierDirectory) 处理流程时，`plot_domain` 是验证格网定义和数据完整性的第一道防线。如果DEM投影错误、格网偏移或流线计算异常，这些都能在概览图中立即显现。

18.3.2 plot_centerlines：流线网络

`plot_centerlines` 专门可视化中心线计算结果：

```
plot_centerlines(gdir, ax=None, title=True,
                 use_flowlines=False, add_scale_bar=True)
```

参数解析：

- `use_flowlines=False` (默认)：绘制几何中心线（由中心线算法生成），以线条粗细表示冰川宽度。
- `use_flowlines=True`：绘制离散化后的流线（由flowline discretization生成），宽度变化通过streamplot或填充多边形表示。

每条流线的宽度编码了集水区几何信息，主线和支流以不同颜色区分。这种可视化对于诊断流线网络的拓扑正确性至关重要——例如检测支流是否与主线正确相交、宽度是否在交汇处平滑过渡。

18.3.3 plot_catchment_width：集水区几何

`plot_catchment_width` 将冰川几何沿流线的分布以曲线图形式展示。该函数绘制两个关键诊断量：

- **集水区面积沿流线分布**：验证面积守恒（各流线段上面积之和应等于总冰川面积）
- **宽度沿流线分布**：显示冰川宽度从积累区到消融区的变化趋势

这些曲线对于反演和模型模拟是输入参数，因此可视化检查它们的平滑性和物理合理性是质量控制的重要步骤。

18.4 物质平衡可视化

18.4.1 plot_mass_balance_grad：物质平衡梯度

OGGM的温度指数模型（temperature-index model）将物质平衡表达为海拔的函数。该函数绘制三种关键曲线：

```
plot_mass_balance_grad(gdir, ax=None, title=True)
```

图表中显示：

- **消融梯度**：随海拔下降的消融速率（正梯度=较低海拔有更多消融）
- **积累梯度**：随海拔上升的积累速率（通常为固态降水）
- **平衡线**：物质平衡为零的海拔高度（ELA）
- **mu*校准参考**：用虚线标注温度敏感度参数mu*的值

这种可视化直接反映了物质平衡校准的质量——如果ELA偏离观测值过多，mu*的调整可能有问题。

18.4.2 plot_mb_ts 和 plot_annual_mb_ts

这两个函数以时间序列形式展示物质平衡：

- `plot_mb_ts`：给定气候序列下的年际物质平衡变化，叠加累积物质平衡曲线。累积曲线向下倾斜表示冰川净质量减少。
- `plot_annual_mb_ts`：以热力图形式展示每条高度带的年度物质平衡，x轴为时间，y轴为海拔，颜色编码物质平衡数值。

实践建议 使用 `plot_annual_mb_ts` 可以直观发现气候驱动事件——例如特别极端的一年会导致从积累区到消融区的整体负异常，在热力图上表现为单一的垂直暗带。

18.5 模型输出可视化

模型输出可视化是`graphics.py`最复杂也最有价值的部分。三个核心函数覆盖了空间、剖面和时间三个维度。

18.5.1 plot_modeloutput_map：2D厚度与速度图

从 `GriddedNcdfFile` 中提取格网化的模型输出并将其渲染为地理地图：

```
plot_modeloutput_map(gdir, ax=None,
                     variable='thickness',
                     model_results_path=None,
                     add_colorbar=True,
                     title=True)
```

支持的可视化变量包括：

变量	含义	推荐色图
<code>thickness</code>	冰川厚度（米）	蓝白红（厚-薄）
<code>velocity</code>	表面速度（米/年）	暖色渐层
<code>surface_h</code>	表面高程（米）	地形色图
<code>mass_balance</code>	物质平衡（mm w.e. yr ⁻¹ ，毫米水当量 water equivalent）	蓝白红发散

该函数处理从流线坐标到地理坐标的重映射和格网插值，确保输出与地理参考对齐。

18.5.2 plot_modeloutput_section: 流线剖面图

这是OGGM的核心诊断可视化：沿主要流线的冰川横截面和纵剖面。一个典型的剖面图包含五个面板：

```
plot_modeloutput_section(gdir, ax=None,
                        model_results_path=None,
                        title=True)
```

五个面板布局为：

1. **冰床与表面高程**：沿流线方向（x轴）的冰川表面和底部高程曲线，填充冰川体积。
2. **冰厚剖面**：沿流线的冰厚变化。
3. **冰川宽度**：沿流线的宽度曲线。
4. **冰流通量**：沿流线的冰体积通量（单位时间通过横截面的冰量）。
5. **表面速度**：沿流线的模拟表面速度。

这五个面板提供了对冰川动力学的一站式可视化。例如，冰流通量与宽度和厚度的关系可以验证物质守恒是否正确求解。

18.5.3 plot_modeloutput_timeseries: 时间序列

绘制模型运行期间冰川关键指标的时间演变：

```
plot_modeloutput_timeseries(gdir,
                           model_results_path=None,
                           axs=None,
                           title=True)
```

绘制以下时间序列：

- **冰川体积**：总体积随时间变化（通常显示退缩趋势）
- **冰川面积**：总面积随时间变化
- **冰川长度**：沿主流线的长度变化，是气候变化的直接响应指标
- **平衡线海拔(ELA)**：如可用，显示平衡线海拔的时间变化

深入理解 冰川长度对气候变化的响应具有滞后性（通常数年）。可以通过比较体积和长度的时间序列来估计冰川响应时间——体积对气候变化的响应更即时（因为厚度变化直接影响体积），而长度需要冰动力学传播，因此滞后更明显。

18.6 诊断可视化

18.6.1 plot_inversion: 反演结果诊断

`plot_inversion` 是验证冰厚反演质量的核心工具。它对比反演的冰厚与可用观测数据（如雷达测厚、Farinotti共识估计等）：

```
plot_inversion(gdir, ax=None, title=True,
               linewidth=2, varname='volume')
```

图形类型取决于 `varname` 参数：

- `varname='volume'`：散点图，x轴为观测体积，y轴为反演体积。对角线表示完美匹配。
- `varname='thickness'`：沿流线的冰厚剖面，叠加观测厚度点。

在L3（反演级别）完成后，应始终使用此函数检查反演质量。系统性偏差（如反演厚度持续偏高）可能表示Glen A参数需要重新校准。

18.6.2 plot_velocity: 表面速度对比

如果可用（如ITS_LIVE数据已通过 `--add-itslive-velocity` 添加到L3），该函数将模拟的冰川表面速度与卫星观测的冰川表面速度进行对比。

18.6.3 plot_thickness_map: 冰厚专题图

生成印刷品质的冰川冰厚分布专题地图，包含完整的图例、比例尺和标题。与 `plot_modeloutput_map(variable='thickness')` 不同，`plot_thickness_map` 专注于冰厚这一核心科学生产，提供更精美的默认呈现。

18.7 GriddedNcdfFile集成

OGGM的格网化输出使用 `GriddedNcdfFile` 格式（基于NetCDF）。`graphics.py`中的地图类函数（`plot_modeloutput_map`）直接读取此格式并进行可视化：

```
# 典型使用 workflow
from oggm import GraphicsGridded, GriddedNcdfFile

# 加载格网化输出
gridded = GriddedNcdfFile(path_to_gridded_output)

# 创建可视化器并绑定格网数据 (gridded data)
graphics = GraphicsGridded(gridded)

# 绘制厚度图和速度图
graphics.plot_thickness()
graphics.plot_velocity()
```

`GraphicsGridded` 类封装了格网数据（gridded data）的可视化逻辑，包括投影转换、颜色映射、等值线和矢量箭头叠加。

18.8 扩展与自定义

18.8.1 基于Matplotlib的自定义

所有OGGM绘图函数都返回matplotlib axes对象，这使得用户可以在默认图形上叠加自定义内容。例如，在domain图上标注特定研究点位：

```
fig, ax = plt.subplots(figsize=(10, 8))
plot_domain(gdir, ax=ax)
ax.scatter(lon_point, lat_point, c='red', marker='*',
           s=200, label='Research site')
ax.legend()
plt.savefig('custom_domain.png', dpi=300, bbox_inches='tight')
```

18.8.2 创建自定义可视化函数

创建自定义绘图函数需要遵从`graphics.py`的约定：

1. 函数前两个参数应为 `gdir` 和 `ax=None`
2. 如果 `ax` 为 `None`，在函数内创建新的figure和axes
3. 返回axes对象供调用者进一步修改
4. 使用 `_savefig` 统一保存接口（或直接调用`fig.savefig`）

18.9 图形导出格式

格式	特点	推荐用途
PNG	高分辨率位图 (150–600 dpi)	论文发表、演示、网页
PDF	矢量图形，无损缩放	出版级插图、LaTeX文档
SVG	可编辑矢量图形	后期编辑 (Inkscape/Illustrator)、网页

通过`cfg.PARAMS`中的 `plot_dpi` 参数控制默认分辨率。对于论文发表的图形，建议设置 `plot_dpi=300` 并使用PDF格式以保留矢量可编辑性。

18.10 小结

`graphics.py` 将OGGM的复杂科学输出转换为直观的出版物级图形。从冰川几何、物质平衡梯度到完整的模型剖面和时间序列，每个函数都经过精心设计以服务于科学分析和质量诊断的双重目的。理解这些可视化工具的含义和使用方法，是高效解读OGGM模型输出的基础技能。

第19章 Sandbox与实验模块

Advanced

Experimental

`oggm/sandbox/` 目录包含了一系列实验性、面向研究且成熟度较低的模块。与核心 `OGGM` 包不同，`sandbox` 模块不受相同的 API 稳定性保证、全面测试或性能基准的约束。它们充当新物理机制、替代参数化方案、教育工具和跨模型兼容层的试验场。本章概述 `sandbox` 的内容，解释 `sandbox` 模块与核心模块的区别，并提供何时以及如何使用它们的指导。

19.1 Sandbox 的设计理念

`sandbox` 是 `OGGM` 架构中经过深思熟虑的设计选择。冰川建模是一个活跃的研究领域，新方法不断在文献中出现。`OGGM` 没有将每一项新技术立即集成到稳定的核心中（那样将需要无限期地维护向后兼容性），而是提供了一个 `sandbox`，让贡献者可以自由地进行实验。

19.1.1 核心模块 vs. Sandbox 模块：关键区别

标准	核心模块	Sandbox 模块
API 稳定性	语义化版本控制；移除前需要弃用周期	无稳定性保证；可能在版本间变更或消失
测试覆盖率	合并前需要单元测试；CI 门禁	测试可选或极少；可能依赖手动验证
文档	Sphinx 文档字符串 + 用户指南条目	极少；通常仅有源码注释或 notebook
参数处理	在 <code>params.cfg</code> 中注册并设置默认值	可能使用硬编码值或独立配置
性能	已做性能分析；期望进行大规模优化	"在我机器上能跑"是可接受的
用途	常用建模流程	研究探索、教学、模型对比

警告 未经彻底独立验证的 `sandbox` 模块不应用于已发表的科学成果。`OGGM` 开发者明确指出，`sandbox` 代码可能包含已知局限、未经验证的假设或数值不稳定性。

Sandbox风险分级

等级	可以怎样用	发表前最低要求
教学/演示	帮助理解概念或快速画图	不要直接作为定量结论来源。
论文复现实验	复现某篇论文或某个模块作者的实验	固定OGGM版本、记录参数、复现原始案例。
方法探索	开发新参数化或比较替代方案	与核心工作流、观测或独立模型交叉验证。
正式区域应用	不建议直接使用	需要测试、敏感性分析、误差评估和清晰的方法说明。

19.2 模块：calving_fabi.py

`sandbox/calving_fabi.py` 实现了基于 Fabien Maussion 及其合作者研究的替代性崩解参数化方案。该模块提供的崩解速率估算在水深、应变率和锋面消融处理方面与核心 `OGGM` 崩解方案不同。

19.2.1 设计原理

崩解是冰川预测中最大的不确定性来源之一。核心 `OGGM` 模型使用简单的依赖水深的崩解率（即 "k-calving" 法则）。Fabi 参数化方案对此进行了扩展，包括：

- **应变率依赖性：**崩解率在纵向应变率较高的末端附近增加。
- **浮力驱动崩解：**考虑了当末端接近漂浮状态时海洋对水上冰崖施加的扭矩。
- **宽度依赖性效应：**峡湾几何形态通过侧向拖曳和应力集中来调节崩解。

19.2.2 关键函数

calving_rate_fabi()

```
def calving_rate_fabi(gdir, calving_water_level='Lmax',
                      use_beta=False, glen_a=None,
                      write=True, filesuffix=''):
    """Compute calving rate using the Fabi parameterization.

    Parameters
    -----
    gdir : GlacierDirectory
        The glacier directory (must have inversion_output.pkl).
    calving_water_level : str or float
        Water level reference: 'Lmax' (last observed), 'L0' (bed topo),
        or a fixed elevation value.
    use_beta : bool
        If True, include strain-rate-dependent calving.
    glen_a : float, optional
        Override the default Glen A parameter for this computation.
    write : bool
        If True, save calving output to the glacier directory.
    filesuffix : str
        Suffix appended to output filenames for multi-scenario runs.

    Returns
    -----
    dict
        Dictionary containing calving_rate (m/yr), calving_flux (km3/yr),
        and per-flowline diagnostics.
    """
```

calving_flux_from_depth_fabi()

```
def calving_flux_from_depth_fabi(gdir, water_level=None,
                                  glen_a=None, write=False):
    """Compute the total frontal calving flux using the Fabi approach."""
```

19.2.3 使用示例

```
import oggm
from oggm import workflow
from oggm.sandbox import calving_fabi

# Standard OGGM preprocessing
cfg.initialize()
gdirs = workflow.init_glacier_directories(['RGI60-11.00897'])
workflow.execute_entity_task(workflow.gis_prepro_tasks, gdirs)
workflow.execute_entity_task(workflow.climate_tasks, gdirs)
workflow.execute_entity_task(workflow.inversion_tasks, gdirs)

# Apply Fabi calving with strain-rate dependence
for gdir in gdirs:
    result = calving_fabi.calving_rate_fabi(
        gdir, use_beta=True, calving_water_level='Lmax'
    )
    print(f"Calving rate: {result['calving_rate']:.1f} m/yr")
    print(f"Calving flux: {result['calving_flux']:.4f} km³/yr")
```

19.3 模块: calving_jan.py

sandbox/calving_jan.py 提供了另一种崩解参数化方案，由 Jan-Hendrik Malles 开发。该方法强调水下融化和峡湾环流在驱动海洋终止型冰川崩解中的作用。

19.3.1 概念模型

Jan 崩解方案将崩解视为以下因素的产物：

1. 水下融化掏蚀：海洋热通量侵蚀末端的淹没部分，形成突出的悬垂冰体，最终发生力学破坏。
2. 热刻槽破坏：当融化导致的刻槽深度超过冰厚的临界比例时，水上冰崖发生崩塌。
3. 崩解率-融化率比值：崩解率被参数化为水下融化率的函数，采用经验推导的比例常数。

19.3.2 关键函数

```
def calving_rate_from_depth_jan(gdir, melt_f=None, water_level=0,
                                write=True, filesuffix=''):
    """Calving rate based on submarine melt undercutting.

    The calving rate C is computed as:
        C = k_m * m_dot * (d / H)
    where k_m is an empirical constant, m_dot is the submarine melt rate,
    d is water depth, and H is terminus ice thickness.

    Parameters
    -----
    gdir : GlacierDirectory
    melt_f : float, optional
        Submarine melt factor. If None, uses cfg.PARAMS['melt_f'].
    water_level : float
        Reference water level elevation (m a.s.l.).
    write : bool
    filesuffix : str

    Returns
    -----
    dict with 'calving_rate' and 'calving_flux' keys.
    """
```

19.3.3 崩解方法的比较

特性	核心 k-calving	Fabi (calving_fabi)	Jan (calving_jan)
主要驱动	水深	水深 + 应变率	水下融化
参数	calving_k	calving_k, glen_a, beta	melt_f, k_m
峡湾几何	否	宽度效应 (可选)	否
计算成本	低	中	低
验证程度	多区域	有限区域	单一峡湾

19.4 模块: distribute_2d.py

sandbox/distribute_2d.py 解决了流线模型的一个根本性局限：如何从一维流线反演结果重建二维冰厚场。这是将 OGGM 与区域气候模型耦合或在完整二维网格上可视化冰川几何形态的关键步骤。

19.4.1 分布问题

OGGM 沿一维流线运行其质量守恒反演。结果是沿每条流线的厚度剖面。要获得网格化产品，必须将厚度横向“分布”到整个冰川集水区。这是一个欠定问题：无限多个二维场可以投影到同一个一维剖面上。

19.4.2 算法

`distribute_thickness_per_altitude()`

```
def distribute_thickness_per_altitude(gdir, varname='distributed_thickness',
                                     add_slope=True,
                                     smooth_radius=None):
    """Distribute flowline thickness to each grid point.

    Algorithm:
    1. Compute distance-to-flowline for every grid cell.
    2. At each altitude band, assign the flowline thickness value.
    3. Optionally add a slope-dependent correction (thicker in concave areas).
    4. Optionally smooth the result with a Gaussian filter.

    Parameters
    -----
    gdir : GlacierDirectory
    varname : str
        Name of output grid variable.
    add_slope : bool
        Include slope-dependent thickness correction.
    smooth_radius : float, optional
        Smoothing kernel radius in meters.
    """
```

`distribute_thickness_interp()`

```
def distribute_thickness_interp(gdir, add_slope=True,
                               smooth_radius=None):
    """Alternative interpolation-based distribution method.

    Uses inverse-distance-weighted interpolation from the flowline
    centerline outward, rather than altitude-band assignment.
    """
```

19.4.3 使用示例

```
from oggm.sandbox import distribute_2d

# After running inversion...
for gdir in gdirs:
    distribute_2d.distribute_thickness_per_altitude(
        gdir, add_slope=True, smooth_radius=200
    )
# Now gdir.gridded_data has 'distributed_thickness' variable
```

提示：验证 二维分布厚度可以通过将其投影回流线并与原始一维反演进行比较来验证。一维和二维表示之间总冰体积的守恒也是一个有用的诊断指标。

19.5 模块：edu.py

`sandbox/edu.py` 是一个专为冰川动力学教学而设计的教育模块。它实现了简化的、解析可解的冰川模型，可以在 Jupyter notebook 中交互运行，无需完整的 OGGM 预处理流程。

19.5.1 教育模型

IdealisedBedrock

```
class IdealisedBedrock:
    """A simple, analytically-defined bedrock profile.

    Supports:
    - Linear beds:  $z_b(x) = z_{\max} - \text{slope} * x$ 
    - Parabolic beds:  $z_b(x) = z_{\max} - (z_{\max} - z_{\min}) * (x/L)^2$ 
    - Custom polynomial beds

    Parameters
    -----
    top_z : float
        Elevation at the head (m).
    bottom_z : float
        Elevation at the terminus (m).
    width : float
        Constant glacier width (m).
    length_km : float
        Glacier length (km).
    """
```

BumpedBedrock (前身为 IdealisedGlacier)

```
class BumpedBedrock(IdealisedBedrock):
    """Adds a Gaussian bump to the bedrock for more realistic geometry."""
```

ShallowIceApproximationModel

```
class ShallowIceApproximationModel:
    """A didactic implementation of the SIA glacier model.

    This model explicitly shows the physics without the optimization
    complexity of the production FluxBasedModel. Students can read
    the source code and understand every line.

    The model solves:
     $\partial H / \partial t = -\partial(u_{\text{bar}} * H * w) / \partial x / w + b_{\text{dot}}$ 
    where  $u_{\text{bar}}$  is the depth-averaged velocity from the SIA.
    """
```

19.5.2 面向教学的设计

`edu.py` 的主要教育特性:

- **透明的数值方法**: 无黑箱求解器; 采用显式时间步进, 并带有清晰的 CFL 条件检查。
- **内置诊断**: 物质平衡、通量散度和速度作为独立的、命名的步骤进行计算, 学生可以检查每一步。
- **Jupyter 友好**: 所有类都有显示关键状态的 `__repr__` 方法和用于内联可视化的 `plot()` 方法。
- **渐进复杂度**: 从恒定物质平衡的平衡剖面开始, 添加季节性, 添加物质平衡-海拔反馈, 最后添加气候趋势。

19.5.3 示例：运行教育模拟

```
from oggm.sandbox.edu import BumpedBedrock, ShallowIceApproximationModel
import numpy as np

# Create an idealized glacier geometry
bed = BumpedBedrock(
    top_z=3600, bottom_z=1200, width=300, length_km=10,
    bump_amplitude=100, bump_position=0.4
)

# Initialize the SIA model
model = ShallowIceApproximationModel(
    bed=bed, nx=200, glen_a=2.4e-24,
    mb_model='seasonal', ela=2800
)

# Run to equilibrium
model.run_to_equilibrium(rtol=1e-3, max_years=500)

# Plot the profile
model.plot(title="Equilibrium Glacier Profile")

# Inspect mass budget
print(f"Total volume: {model.volume_km3:.3f} km³")
print(f"Max thickness: {model.thickness.max():.1f} m")
print(f"ELA: {model.ela:.0f} m a.s.l.")
```

19.6 模块：pygem_compat.py

`sandbox/pygem_compat.py` 提供了 OGGM 与 PyGEM (Python Glacier Evolution Model, 由 David Rounce 及其合作者开发) 之间的兼容层。PyGEM 是一个独立的冰川模型，常用于全球尺度的预测，在模型对比研究中经常与 OGGM 一起被引用。

19.6.1 目的

该兼容层支持以下功能：

- **输入协调**：将 OGGM 冰川目录 (GlacierDirectory) 转换为 PyGEM 期望的输入格式（反之亦然）。
- **参数转换**：在 OGGM 物质平衡参数 (`melt_f`、`temp_bias`、`prcp_fac`) 与 PyGEM 的参数化方案之间进行映射。
- **跨模型验证**：在相同冰川上运行两个模型，使用相同的气候强迫，并比较结果。
- **集合生成**：将 PyGEM 的体积-面积缩放方法作为低成本集合成员，与 OGGM 的流线物理模型并行使用。

19.6.2 关键函数

convert_oggm_to_pygem()

```
def convert_oggm_to_pygem(gdir, output_dir=None):
    """Convert OGGM GlacierDirectory into PyGEM input format.

    Extracts: RGI metadata, hypsometry, climate timeseries,
    and ice thickness (if available).

    Parameters
    -----
    gdir : GlacierDirectory
    output_dir : str, optional
        Directory for PyGEM input files.

    Returns
    -----
    dict
        PyGEM-compatible glacier dictionary.
    """
```

run_pygem_from_oggm()

```
def run_pygem_from_oggm(gdir, pygem_params=None,
                        n_years=None, output_dir=None):
    """Run PyGEM model from an OGGM glacier directory.

    Handles all data conversion internally and returns results
    in a standardized format for comparison.
    """
```

compare_oggm_pygem()

```
def compare_oggm_pygem(gdir, oggm_run_output=None,
                       pygem_run_output=None):
    """Generate a diagnostic comparison between OGGM and PyGEM runs.

    Returns a DataFrame with annual values of:
    - Specific mass balance
    - Volume change
    - Area change
    for both models.
    """
```

19.7 其他 Sandbox 模块

除上述主要模块外，`sandbox` 还包含若干较小的工具：

sandbox/gmd_paper_mbcrossval.py

实现了 OGGM GMD 论文中描述的物质平衡交叉验证方法。该模块系统地测试校准后的物质平衡参数对不同校准周期、参考数据集和冰川子集的敏感性。

sandbox/grindelwald_demo.py

一个演示 notebook 配套文件，对 Grosser Aletschgletscher 及其他瑞士冰川的历史演变进行建模。展示了如何为具有长期物质平衡和长度变化记录的、观测充分的冰川设置 OGGM。

sandbox/idealisepy_glaciers.py (遗留)

edu.py 的前身，保留以兼容引用旧类名的 notebook。

19.8 何时使用 Sandbox 模块

使用场景	推荐模块	注意事项
格陵兰研究的替代崩解方案	calving_fabi.py	需先用观测数据验证
水下融化驱动崩解（阿拉斯加）	calving_jan.py	融化速率参数约束较差
用于区域气候模型的二维厚度图	distribute_2d.py	分布过程增加不确定性；需量化
向学生教授冰川动力学	edu.py	不适合定量预测
与 PyGEM 的跨模型对比	pygem_compat.py	需要安装 PyGEM

19.9 为 Sandbox 贡献代码

OGGM 欢迎 sandbox 贡献。sandbox 的纳入门槛比核心模块低：

- 代码必须在演示冰川（Hintereisferner 和至少一个入海型冰川）上无错误运行。
- 包含一个演示用法的 Jupyter notebook。
- 在模块顶部添加说明，描述目的、参考文献和已知局限。
- 如果模块趋于成熟（获得了测试、文档和社区验证），则可能被提升为核心包。

第20章 扩展OGGM：自定义模块与插件

Advanced Developer

OGGM 从一开始就被设计为可扩展的。它不是强制所有用户使用单一的建模流程，而是在技术栈的每个层面都暴露了定义良好的扩展点：气候数据源、物质平衡模型、冰床几何形态、流动求解器，甚至包括任务执行框架本身。本章将逐一说明每个扩展点，提供可运行的示例，并讨论构建可维护、可测试的 OGGM 扩展的最佳实践。

开发者最小闭环

扩展OGGM时不要只写一个函数。一个可维护扩展至少应包含：实现代码、一个最小示例冰川、一个能重复运行的entity task或脚本、一个检查输出文件的测试、以及写入 diagnostics.json 或独立结果文件的溯源信息。这样后续更换OGGM版本、RGI 版本或气候数据时，才能判断差异来自模型还是来自输入。

20.1 OGGM的扩展架构

OGGM 的可扩展性建立在以下几个核心设计模式之上：

扩展点	机制	稳定性
气候数据源	Shop 模块模式：在 <code>cfg.PARAMS['baseline_climate']</code> 中注册 <code>download + process</code> 函数	稳定
物质平衡模型	继承 <code>MassBalanceModel</code> ；实现 <code>get_monthly_mb()</code> 和 <code>get_annual_mb()</code>	稳定
冰床形状 / flowline	继承 <code>Flowline</code> 或 <code>MixedBedFlowline</code> ；实现几何方法	稳定
演化模型	继承 <code>FlowlineModel</code> ；实现 <code>step()</code> ；在 <code>cfg.PARAMS['evolution_model']</code> 中注册	稳定
Entity Task	使用 <code>@entity_task</code> 装饰器；通过 <code>writes</code> 声明依赖关系	稳定
配置参数	向 <code>params.cfg</code> 添加条目；使用前在 <code>PARAMS</code> 中编写文档	半稳定
输出文件类型	在 <code>cfg.BASENAMES</code> 中注册新的键名	稳定

20.2 Shop 模块模式

"Shop" 模式是 OGGM 实现可插拔气候数据提供者的机制。每个 shop 模块实现统一的接口，使得用户可以在不改变工作流程代码的情况下切换不同的气候数据集。

20.2.1 Shop 模块的组成结构

一个完整的 shop 模块需要两个具有标准化签名的函数：

下载函数 (Download Function)

```
def my_climate_download(gdirs=None, path=True, reset=False):
    """Download climate data for a set of glaciers.

    Parameters
    -----
    gdirs : list of GlacierDirectory, optional
        Glacier directories to process. If None, downloads all available.
    path : bool or str
        If True, return output path. If str, path to download directory.
    reset : bool
        If True, re-download existing files.

    Returns
    -----
    str
        Path to the downloaded data directory.
    """
```

处理函数 (Process Function)

```
@entity_task(logger, writes=['climate_historical'])
def process_my_climate(gdir, **kwargs):
    """Process downloaded climate data for a single glacier.

    This function reads the raw downloaded data and writes a
    climate_historical.nc file in the glacier directory.

    Parameters
    -----
    gdir : GlacierDirectory
    **kwargs : forwarded to implementation
    """
```

20.2.2 注册

Shop 模块通过修改配置来注册自身:

```
# In your module's __init__ or at import time:
import oggm.cfg as cfg

# Register as an available climate data source
cfg.PARAMS['baseline_climate'].append('MY_CLIMATE_DATA')

# Optionally set as the active source
cfg.PARAMS['baseline_climate'] = 'MY_CLIMATE_DATA'
```

20.2.3 与调度器的集成

oggm.core.climate 中的核心函数 `process_climate_data()` 充当调度器的角色。当用户调用 `process_climate_data(gdir)` 时, 调度器会查询 `cfg.PARAMS['baseline_climate']` 并将请求路由到对应的 shop 模块:

```
# Simplified dispatcher logic in oggm.core.climate
def process_climate_data(gdir, **kwargs):
    source = cfg.PARAMS['baseline_climate']
    if source == 'CRU':
        from oggm.shop import cru
        cru.process_cru_data(gdir, **kwargs)
    elif source == 'HISTALP':
        from oggm.shop import histalp
        histalp.process_histalp_data(gdir, **kwargs)
    elif source == 'MY_CLIMATE_DATA':
        from my_package import my_shop
        my_shop.process_my_climate(gdir, **kwargs)
    else:
        raise ValueError(f"Unknown climate source: {source}")
```

提示: Shop 模块开发清单

1. 实现具有标准签名的 `download_*`() 函数。
2. 实现 `process_*`() 函数, 使用 `@entity_task` 装饰, 写入 `climate_historical`。
3. 在 `cfg.PARAMS['baseline_climate']` 中注册。
4. 使用 `gdir.write_climate()` 或等效方法写入输出文件。
5. 在 `oggm.core.climate.process_climate_data` 的调度器中添加数据源。
6. 提供测试数据以及至少一个集成测试。

20.3 自定义物质平衡模型

OGGM 中所有物质平衡模型均继承自 `oggm.core.massbalance.MassBalanceModel`。要创建自定义模型, 需继承此基类并实现所需的接口。

20.3.1 MassBalanceModel 接口

```
class MassBalanceModel:
    """Base class for all mass balance models."""

    def __init__(self):
        self.valid_bounds = None # (min_h, max_h) for extrapolation limits
        self.hemisphere = 'nh' # 'nh' or 'sh'

    def get_monthly_mb(self, heights, year=None, fl_id=None, fl_widths=None):
        """Compute monthly mass balance at given heights.

        Parameters
        -----
        heights : ndarray
            Heights (m) at which to compute mass balance.
        year : int, optional
            The year for which to compute (not used by all models).
        fl_id : int, optional
            Flowline ID for multi-flowline glaciers.
        fl_widths : ndarray, optional
            Flowline widths at each grid point.

        Returns
        -----
        ndarray
            Monthly mass balance ( $\text{kg m}^{-2}$ ) for each height for each month.
            Shape: (12, n_heights)
        """
        raise NotImplementedError()

    def get_annual_mb(self, heights, year=None, fl_id=None, fl_widths=None):
        """Compute annual mass balance.

        Returns
        -----
        ndarray
            Annual mass balance ( $\text{kg m}^{-2} \text{ yr}^{-1}$ ) for each height.
            Shape: (n_heights,)
        """
        raise NotImplementedError()

    def get_specific_mb(self, heights=None, widths=None, fls=None, year=None):
        """Compute glacier-wide specific mass balance."""
        raise NotImplementedError()

    def get_ela(self, year=None, **kwargs):
        """Compute the equilibrium line altitude."""
        raise NotImplementedError()
```

20.3.2 示例：基于 ELA 的自定义物质平衡模型

```
import numpy as np
from oggm.core.massbalance import MassBalanceModel

class ELAMassBalance(MassBalanceModel):
    """Simple mass balance model defined by an ELA and gradients.

    Parameterization:
         $b(z) = (z - ELA) * grad\_above$  for  $z \geq ELA$ 
         $b(z) = (z - ELA) * grad\_below$  for  $z < ELA$ 
    """

    def __init__(self, ela=2800, grad_above=-0.007, grad_below=-0.012,
                 max_mb=4.0):
        super().__init__()
        self.ela = ela
        self.grad_above = grad_above # m w.e. per m elevation (above ELA)
        self.grad_below = grad_below # m w.e. per m elevation (below ELA)
        self.max_mb = max_mb # kg m-2, maximum accumulation
        self.valid_bounds = [0, 10000] # m a.s.l.

    def get_monthly_mb(self, heights, year=None, fl_id=None, fl_widths=None):
        # Annual model, distribute evenly across months
        annual = self.get_annual_mb(heights, year, fl_id, fl_widths)
        monthly = np.tile(annual, (12, 1)) / 12.0
        return monthly

    def get_annual_mb(self, heights, year=None, fl_id=None, fl_widths=None):
        z = np.asarray(heights)
        mb = np.where(
            z >= self.ela,
            (z - self.ela) * self.grad_above,
            (z - self.ela) * self.grad_below
        )
        # Cap accumulation
        mb = np.minimum(mb, self.max_mb * 1000) # convert to kg m-2
        return mb

    def get_ela(self, year=None, **kwargs):
        return self.ela
```

20.3.3 与 MultipleFlowlineMassBalance 的协作

在实际应用中，大多数用于 flowline 演化的物质平衡模型都被包装在 `MultipleFlowlineMassBalance` 中，该类负责将调用分派给各个 flowline 并管理多 flowline 的几何结构：

```
from oggm.core.massbalance import MultipleFlowlineMassBalance

# Wrap your custom model
mb_model = ELAMassBalance(ela=2900)
multi_mb = MultipleFlowlineMassBalance(
    gdir, mb_model_class=ELAMassBalance,
    ela=2900
)
```

20.3.4 标定集成

如果你的物质平衡模型具有可标定的参数，可以通过实现 `temp_bias` 或 `prcp_fac` 接口，或者提供自定义的标定函数来集成到 OGGM 的标定框架中。`oggm.core.climate.mb_calibration` 中的标定循环也可以被子类化。

20.4 自定义冰床形状：继承 Flowline

冰川的几何形态由 `Flowline` 对象表示。OGGM 提供了 `MixedBedFlowline`（用于处理部分基岩覆盖的情况），但你也可以为特定应用场景创建自定义的冰床形状。

20.4.1 Flowline 接口

```
class Flowline:
    """Base class for 1D glacier flowlines."""

    def __init__(self, line=None, dx=1, map_dx=None,
                 surface_h=None, bed_h=None):
        # Core geometry arrays
        self.dis_on_line = None # distance along the flowline (m)
        self.surface_h = None # surface elevation (m a.s.l.)
        self.bed_h = None # bed elevation (m a.s.l.)
        self.widths = None # glacier width at each grid point (m)
        self.widths_m = None # widths at staggered grid points (m)
        self.dx = None # grid spacing (m)
        self.dx_meter = None # grid spacing in meters
        self.map_dx = None # map-projected dx (m)

    def _add_attrs_to_save(self):
        """Return dict of attributes to persist."""
        return {
            'dis_on_line': self.dis_on_line,
            'surface_h': self.surface_h,
            'bed_h': self.bed_h,
            'widths': self.widths,
            'dx': self.dx,
        }
```

20.4.2 示例：抛物线形冰床 Flowline

```
import numpy as np
from oggm.core.flowline import Flowline

class ParabolicBedFlowline(Flowline):
    """Flowline with a parabolic bed profile.

    
$$z_b(x) = z_{\text{head}} - (z_{\text{head}} - z_{\text{terminus}}) * (x / L)^2$$

    """

    def __init__(self, line=None, dx=1, map_dx=None,
                 surface_h=None, bed_h=None,
                 z_head=3000, z_terminus=1000):
        super().__init__(line, dx, map_dx, surface_h, bed_h)
        self.z_head = z_head
        self.z_terminus = z_terminus
        if line is not None:
            self._set_parabolic_bed()

    def _set_parabolic_bed(self):
        L = self.dis_on_line[-1]
        x = self.dis_on_line
        self.bed_h = self.z_head - (self.z_head - self.z_terminus) * (x / L)**2

    def _add_attrs_to_save(self):
        attrs = super()._add_attrs_to_save()
        attrs['z_head'] = self.z_head
        attrs['z_terminus'] = self.z_terminus
        return attrs
```

20.5 自定义演化模型

冰川演化求解器是 OGGM 动力学模型的计算核心。OGGM 支持多种求解器，每种求解器针对 SIA 方程实现了特定的数值方案。

20.5.1 FlowlineModel 接口

```
class FlowlineModel:
    """Base class for glacier evolution models."""

    def __init__(self, flowlines, mb_model=None, y0=0,
                 glen_a=None, fs=None, **kwargs):
        self.fls = flowlines
        self.mb_model = mb_model
        self.y0 = y0
        self.glen_a = glen_a or cfg.PARAMS['glen_a']
        self.fs = fs or cfg.PARAMS['fs']

    def step(self, dt):
        """Advance the model by one time step.

        Parameters
        -----
        dt : float
            Time step in years.

        Returns
        -----
        float
            The actual time step used (may be smaller due to CFL).
        """
        raise NotImplementedError()

    def run_until(self, y1):
        """Run the model from current year to y1.

        Parameters
        -----
        y1 : float
            End year.

        Returns
        -----
        dict
            Model diagnostics at y1.
        """
        raise NotImplementedError()

    def run_until_and_store(self, y1, **kwargs):
        """Run and store diagnostics along the way."""
        raise NotImplementedError()

    def to_netcdf(self, path):
        """Save model state to NetCDF."""
        raise NotImplementedError()
```

20.5.2 示例：显式前向欧拉模型

```
import numpy as np
from oggm.core.flowline import FlowlineModel

class ExplicitEulerModel(FlowlineModel):
    """Simplified glacier evolution using explicit Forward Euler.

    This is for educational or small-scale use only – it does not
    include the semi-implicit scheme or adaptive time-stepping of
    the production FluxBasedModel.
    """

    def __init__(self, flowlines, mb_model=None, y0=0,
                 glen_a=None, fs=None, fixed_dt=0.01):
        super().__init__(flowlines, mb_model, y0, glen_a, fs)
        self.fixed_dt = fixed_dt # years

    def step(self, dt=None):
        dt = dt or self.fixed_dt
        for fl in self.fls:
            # Compute ice flux via SIA
            thick = fl.thick
            slope = np.gradient(fl.surface_h, fl.dx_meter)
            # Glen's flow law:  $u \sim A * \tau^n * H^{(n+2)}$ 
            n = 3 # Glen's exponent
            tau = 900 * 9.81 * thick * np.abs(slope) # density * g * H * sin(alpha)
            flux = (2 * self.glen_a * self.fs * tau**n *
                   thick**(n + 2) * slope / (n + 2))

            # Mass continuity
            dHdt = -np.gradient(flux, fl.dx_meter) / fl.widths
            mb = self.mb_model.get_annual_mb(
                fl.surface_h, year=int(self.y0)
            )
            fl.thick += (dHdt + mb / 910) * dt # density conversion
            fl.thick = np.maximum(fl.thick, 0) # no negative thickness
        self.y0 += dt
        return dt

    def run_until(self, y1):
        while self.y0 < y1:
            self.step()
        return {'year': self.y0}
```

20.5.3 注册

```
# Register your model so it can be used by flowline_model_run()
cfg.PARAMS['evolution_model'] = 'explicit_euler'
```

警告：数值稳定性 编写正确的冰川演化求解器并非易事。SIA 方程产生的是一个对薄冰高度刚性的非线性扩散问题。常用求解器通常采用半隐式或全隐式格式，并配合自适应时间步长。上述显式欧拉示例仅用于教学演示，在实际参数值下将是不稳定的。

20.6 自定义 Entity Task

`@entity_task` 装饰器是 OGGM workflow 系统的骨干。创建自定义 entity task 可以让你的处理逻辑无缝集成到 OGGM 的并行执行、日志记录和依赖跟踪体系中。

20.6.1 @entity_task 装饰器

```
from oggm import entity_task

@entity_task(logger, writes=['custom_output'])
def my_custom_task(gdir, param_a=1.0, param_b='default'):
    """A custom per-glacier processing task.

    Parameters
    -----
    gdir : GlacierDirectory
        The glacier directory to process.
    param_a : float
        A numerical parameter.
    param_b : str
        A string parameter.

    Returns
    -----
    dict
        Results dictionary saved as gdir.custom_output.
    """
    # Your processing logic here
    result = {'param_a': param_a, 'param_b': param_b,
              'rgi_id': gdir.rgi_id}
    # Write output
    gdir.write_json(result, 'custom_output')
    return result
```

20.6.2 writes 参数

writes 参数对于依赖跟踪至关重要。它声明任务会生成哪些文件。下游任务可以声明对这些文件的依赖：

```
@entity_task(logger, writes=['analysis_result'])
def analysis_task(gdir):
    # This task needs the output of my_custom_task
    # The workflow system ensures proper ordering
    custom = gdir.read_json('custom_output') # written by my_custom_task
    analysis = process(custom)
    gdir.write_json(analysis, 'analysis_result')
    return analysis
```

20.6.3 与 execute_entity_task 的集成

```
from oggm import workflow

gdirs = workflow.init_glacier_directories(['RGI60-11.00897'])

# Run your custom task on all glaciers (serial or parallel)
workflow.execute_entity_task(my_custom_task, gdirs,
                             param_a=2.5, param_b='custom')

# Results are now stored in each glacier directory
for gdir in gdirs:
    result = gdir.read_json('custom_output')
    print(f"{gdir.rgi_id}: {result}")
```

20.7 添加自定义参数

20.7.1 扩展 `params.cfg`

自定义参数应添加到本地的 `params.cfg` 文件中：

```
# my_extension_params.cfg
[my_extension]
# Calibration factor for my custom mass balance model
my_calib_factor = 1.0
# Switch to enable experimental ice-dynamic scheme
my_experimental_scheme = False
# Number of iterations for my iterative solver
my_max_iter = 100
```

在初始化时加载自定义参数：

```
import oggm.cfg as cfg
cfg.initialize()

# Load your custom parameters
cfg.PARAMS['my_calib_factor'] = 1.0
cfg.PARAMS['my_experimental_scheme'] = False
cfg.PARAMS['my_max_iter'] = 100

# Always document in the PARAMS OrderedDict
cfg.PARAMS.doc['my_calib_factor'] = (
    'Calibration factor for the custom mass balance model.'
)
```

专家提示：PARAMS 文档编写规范 OGGM 要求每个参数都必须在 `cfg.PARAMS.doc` 中拥有文档条目。该要求在初始化时强制执行。如果没有正确的文档，OGGM 将发出警告（在严格模式下则为错误）。文档字符串应包含参数的用途、默认值、允许范围（如有）以及受影响的模块。

20.8 为新文件类型注册 `BASENAMES`

如果你的扩展会生成新类型的输出文件，请在 `cfg.BASENAMES` 中注册它们，以便文件管理系统能够识别：

```
from oggm import cfg

# Register a new output file type
cfg.BASENAMES['my_extension_output'] = ('my_output.nc', 'My extension NetCDF output')
# Format: (filename, description)

# Now you can use it with gdir methods:
# gdir.get_filepath('my_extension_output')
# gdir.write_netcdf(data_dict, 'my_extension_output')
```

20.9 完整示例：自定义 `ELA` 模型扩展

以下扩展示例将所有扩展机制串联在一起，构建一个完整、可复用的自定义 `ELA` 模型插件。

```

# File: custom_ela_model.py
"""Custom equilibrium line altitude model for OGGM.

This module demonstrates all OGGM extension patterns:
- A mass balance model subclass
- A custom entity task
- Parameter registration
- Integration with the workflow system
"""

import numpy as np
import oggm.cfg as cfg
from oggm import entity_task
from oggm.core.massbalance import MassBalanceModel, MultipleFlowlineMassBalance
from oggm import workflow

# ---- 1. Register custom parameters ----
cfg.PARAMS['custom_ela_default'] = 3000.0
cfg.PARAMS.doc['custom_ela_default'] = (
    'Default ELA (m a.s.l.) for the CustomELAModel when no calibration '
    'data is available.'
)

# ---- 2. Define the mass balance model ----
class CustomELAModel(MassBalanceModel):
    """Mass balance model with a user-specified ELA.

    Monthly mass balance is derived from a sinusoidal annual cycle
    with the ELA controlling the zero-crossing point.
    """

    def __init__(self, ela=3000, amplitude=2000, melt_factor=0.005):
        super().__init__()
        self.ela = ela
        self.amplitude = amplitude # kg m-2, seasonal amplitude
        self.melt_factor = melt_factor # m w.e. per m below ELA
        self.valid_bounds = [-1000, 9000]

    def get_monthly_mb(self, heights, year=None, fl_id=None, fl_widths=None):
        z = np.asarray(heights)
        # Sinusoidal annual cycle
        months = np.arange(12)
        seasonal = (self.amplitude / 1000 *
                    np.sin(2 * np.pi * (months - 3) / 12)) # max in July
        # Elevation gradient
        elev_effect = np.where(
            z > self.ela,
            -0.005 * (z - self.ela), # net accumulation above ELA
            -0.010 * (self.ela - z) # net ablation below ELA
        )
        mb = np.outer(seasonal, np.ones_like(z)) + elev_effect[np.newaxis, :]
        return mb * 1000 # to kg m-2

    def get_annual_mb(self, heights, year=None, fl_id=None, fl_widths=None):
        monthly = self.get_monthly_mb(heights, year, fl_id, fl_widths)
        return monthly.sum(axis=0)

    def get_ela(self, year=None, **kwargs):
        return self.ela

# ---- 3. Create a custom entity task ----
@entity_task('logger', writes=['custom_ela_results'])
def compute_ela_statistics(gdir, ela_values=None):
    """Compute glacier statistics for different ELA values.

    Parameters
    -----
    gdir : GlacierDirectory
    ela_values : list of float, optional
        ELAs to test. Defaults to [2500, 3000, 3500].
    """

```



```

Returns
-----
dict
    AAR, accumulation area, and total area for each ELA.
"""
if ela_values is None:
    ela_values = [2500, 3000, 3500]

results = {}
for ela in ela_values:
    mb_model = CustomELAModel(ela=ela)
    heights = gdir.gridded_data['glacier_topo']
    masks = gdir.gridded_data['glacier_mask'].astype(bool)
    glacier_h = heights[masks]
    if len(glacier_h) == 0:
        results[ela] = {'aar': 0, 'acc_area_km2': 0, 'total_area_km2': 0}
        continue
    aar = (glacier_h >= ela).sum() / len(glacier_h)
    dx = gdir.grid.dx
    pixel_area_km2 = (dx ** 2) / 1e6
    acc_area_km2 = (glacier_h >= ela).sum() * pixel_area_km2
    total_area_km2 = len(glacier_h) * pixel_area_km2
    results[ela] = {
        'aar': round(aar, 4),
        'acc_area_km2': round(acc_area_km2, 3),
        'total_area_km2': round(total_area_km2, 3)
    }

gdir.write_json(results, 'custom_ela_results')
return results

# ---- 4. Example workflow ----
if __name__ == '__main__':
    # Standard pipeline
    cfg.initialize()
    gdirs = workflow.init_glacier_directories(['RGI60-11.00897'])
    workflow.execute_entity_task(workflow.gis_prepro_tasks, gdirs)
    # Run custom analysis
    workflow.execute_entity_task(compute_ela_statistics, gdirs)
    # Inspect results
    for gdir in gdirs:
        print(gdir.read_json('custom_ela_results'))

```

20.10 最佳实践

20.10.1 测试

- 为你的物质平衡模型编写单元测试：验证 `get_monthly_mb()` 返回 `shape` 为 `(12, n_heights)` 的结果，以及 `get_annual_mb()` 返回 `shape` 为 `(n_heights,)` 的结果。
- 包含一个集成测试，在一个演示冰川上运行你的 `entity task`（例如 `get_demo_file('Hintereisferner_RGI5.shp')`）。
- 测试边界情况：厚度为零的冰川、极端参数值、单点 `flowline`。

20.10.2 文档

- 每个公共函数和类都应有 NumPy 风格的 `docstring`。
- 编写扩展的科学背景说明（引用相关论文）。
- 列出已知的局限性和假设条件。
- 提供在真实冰川上展示用法的 Jupyter notebook。

20.10.3 版本管理

- 使用语义化版本号。对你的扩展 API 的破坏性更改应递增主版本号。
- 考虑锁定你所测试的 OGGM 版本。
- 将你的扩展版本号存储在 `__version__` 属性中。

第21章 测试框架与质量保障

Advanced

Testing & QA

OGGM 的测试套件是项目质量保障策略的关键组成部分。项目包含着数千行数值计算代码、复杂的物理参数化方案以及不断增长的用户群，自动化测试能够确保代码变更不会悄无声息地破坏现有功能。本章将介绍测试架构、测试类别、如何运行测试，以及如何按照 OGGM 的约定编写新测试。

21.1 测试架构概览

OGGM 使用 **pytest** 作为其测试框架。测试基础设施位于 `oggm/tests/` 目录中，围绕共享 `fixture`、辅助函数和模块化的测试文件套件进行组织。

21.1.1 目录结构

```
oggm/tests/
├── __init__.py           # Test package marker
├── conftest.py          # Shared fixtures and configuration
├── funcs.py             # Test helper functions and factories
├── test_workflow.py     # End-to-end workflow tests
├── test_models.py       # Flowline model correctness tests
├── test_prepro.py       # Preprocessing pipeline tests
├── test_numerics.py     # Numerical accuracy and verification tests
├── test_utils.py        # Utility function tests
├── test_graphics.py     # Visualization tests
├── test_benchmarks.py   # Performance benchmarks
├── test_shop.py         # Shop / data provider tests
├── test_sandbox.py      # Sandbox module tests
├── test_minimal.py      # Minimal functional (smoke) tests
├── test_workflow.py     # High-level workflow integration tests
├── test_glathida.py     # GlaThiDa dataset tests
├── test_rgi.py          # RGI inventory handling tests
```

21.1.2 conftest.py: 共享 Fixture

`conftest.py` 文件提供了自动对所有测试文件可用的 `fixture`。主要的 `fixture` 包括：

Fixture	作用域	用途
<code>test_dir</code>	<code>session</code>	用于测试输出的临时目录，自动清理
<code>mini_gdir</code>	<code>session</code>	用于快速单元测试的最小 <code>GlacierDirectory</code>
<code>hef_gdir</code>	<code>session</code>	已完成完整预处理的 <code>Hintereisferner GlacierDirectory</code>
<code>class_gdirs</code>	<code>session</code>	多个测试冰川的预处理 <code>GlacierDirectory</code>
<code>cfg_init</code>	<code>function</code>	确保在每个测试之前调用 <code>cfg.initialize()</code>

21.1.3 funcs.py: 辅助函数

`funcs.py` 包含可复用的测试工具，包括：

- **GlacierDirectory 工厂函数**：创建 具有不同预处理级别 (`prepro level`) 的 `gdir` 对象，以满足不同的测试需求。
- **断言辅助函数**：用于数值容差、数组比较和 `NetCDF` 文件验证的自定义断言。
- **CAS (Climate-Acquisition-Series) 工具**：用于下载 和缓存测试气候数据的函数。
- **基准参考生成器**：生成已知正确的输出 以用于回归对比。

21.2 测试类别

21.2.1 test_workflow.py: 端到端 workflow 测试

这些测试验证从原始 RGI 轮廓到最终模型输出的完整处理流程。它们是覆盖范围最广、同时计算代价也最高的测试。主要场景包括：

- **完整预处理流程**：在若干测试冰川上依次运行 GIS 任务、气候任务和反演任务。
- **历史模拟**：从基准年到现在的完整历史模拟，在可获得观测数据的情况下与体积变化进行对比。
- **未来预估**：在 RCP 情景下进行的由 GCM 驱动的未来预估，验证模型在无数值错误的情况下能运行至完成。
- **多冰川工作流**：并行处理多条冰川，以测试带多进程的 `execute_entity_task`。
- **动态预热 (dynamic spinup)**：测试动态预热 (dynamic spinup) 方法。

21.2.2 test_models.py: Flowline 模型正确性

这些测试验证 flowline 演变模型的数值正确性，包括：

解析解测试

```
def test_flux_based_model_vs_analytical():
    """Compare FluxBasedModel against an analytical solution.

    For a constant-width slab with constant mass balance, the SIA
    has a known equilibrium profile. This test verifies that the
    numerical model converges to within tolerance of that profile.
    """
    from oggm.tests.funcs import idealised_gdir
    # ...idealised glacier setup...
    # Compare numerical equilibrium vs. analytical profile
    np.testing.assert_allclose(
        modeled.surface_h, analytical.surface_h,
        atol=1.0, rtol=0.01
    )
```

质量守恒测试

```
class MassConservationChecker:
    """Utility class for verifying mass conservation in flowline models.

    Checks:
    1. Volume change = integrated mass balance * dt
    2. No mass creation or destruction at domain boundaries
    3. Mass flux continuity across flowline junctions
    """

    def __init__(self, model):
        self.model = model
        self.initial_volume = self._compute_total_volume()

    def check_step(self, dt, mb_model):
        """Verify conservation over one time step."""
        v_before = self._compute_total_volume()
        self.model.step(dt)
        v_after = self._compute_total_volume()
        mb_int = self._integrate_mass_balance(dt, mb_model)
        # Volume change should match integrated mass balance
        np.testing.assert_allclose(
            v_after - v_before, mb_int * dt,
            rtol=0.1 # allow some tolerance for numerical diffusion
        )

    def _compute_total_volume(self):
        return sum(fl.volume_km3 for fl in self.model.fls) * 1e9

    def _integrate_mass_balance(self, dt, mb_model):
        total = 0.0
        for fl in self.model.fls:
            mb = mb_model.get_annual_mb(fl.surface_h)
            area = fl.widths_m * fl.dx_meter
            total += (mb * area).sum()
        return total
```

CFL 条件测试

```
def test_cfl_compliance():
    """Verify adaptive time-stepping respects the CFL condition.

    The CFL number ( $u * dt / dx$ ) must be  $\leq 1$  for explicit schemes
    and  $\leq \sim 5$  for semi-implicit schemes with the production model.
    """
```

21.2.3 test_prepro.py: 预处理流程

GIS 和中心线处理流程的测试:

- **DEM 处理:** 针对 `process_dem()` 的测试, 涵盖不同的 DEM 来源、分辨率和投影。
- **冰川掩膜计算:** 验证冰川掩膜对于简单和复杂冰川几何形状 都能正确识别冰川内的像素。
- **中心线计算:** 测试几何中心线提取, 包括支流检测和汇合点处理。
- **Flowline 初始化:** 验证宽度计算、面积-高度分布和网格分辨率一致性。
- **集水区划分:** 测试集水区是否连续且覆盖预期面积。

21.2.4 test_numerics.py: 数值精度

这些测试验证 OGGM 中使用的各种数值算法的正确性:

- **扩散率计算:** 在不同厚度和坡度下测试冰的扩散率公式。
- **通量计算:** 测试通量限制器 (flux limiter) 和 SIA 通量公式。

- **插值**: 测试气候和物质平衡模块中使用的插值程序。
- **梯度计算**: 测试表面和床面坡度计算。
- **积分**: 测试反演模块中的数值求积。
- **容差测试**: 每个数值程序都应以至少 $1e-6$ 的相对误差 对照参考实现进行容差验证。

21.2.5 test_utils.py: 工具函数

通用工具的测试:

- **GlacierDirectory**: 文件 I/O、路径解析、BASENAMES 查找。
- **月度时间序列**: `monthly_timeseries` DataFrame 的 创建、重采样和聚合。
- **几何函数**: 多边形操作、面积计算和坐标变换。
- **Config 函数**: `pack_config`、`unpack_config`、参数校验。
- **NetCDF I/O**: 读取和写入带有正确元数据的 NetCDF 文件。

21.2.6 test_graphics.py: 可视化测试

可视化测试使用 matplotlib 的测试基础设施来验证绘图函数能够无错误地执行 并生成预期的视觉元素:

- **图形生成**: 使用测试数据调用每个绘图函数, 并验证生成的图形包含预期的图形元素 (线条、多边形、标签)。
- **坐标轴标签和标题**: 验证坐标轴标签是否正确设置。
- **地图投影**: 测试基于 cartopy 的地图绘图, 验证其使用了正确的坐标参考系统。
- **文件输出**: 测试图形是否能够以 PNG 和 PDF 格式保存到磁盘。

21.2.7 test_benchmarks.py: 性能测试

基准测试确保性能关键的操作保持在可接受的时间范围内。关键指标:

- **预处理速度**: 端到端预处理单条冰川所需的时间。
- **反演速度**: 反演 100 个 flowline 点的冰厚度所需的时间。
- **演变速度**: 进行 100 年模拟时每秒可处理的模型年数。
- **内存使用**: 处理 100 条冰川清单时的峰值内存。

21.2.8 test_minimal.py: 冒烟测试 (Smoke Tests)

最小功能测试 (冒烟测试), 能够在 60 秒内运行完毕, 验证核心 API 是否可用:

```
def test_import_oggm():
    import oggm
    assert oggm.__version__ is not None

def test_cfg_initialize():
    from oggm import cfg
    cfg.initialize()
    assert cfg.PARAMS['continue_on_error'] is False

def test_get_demo_file():
    from oggm.utils import get_demo_file
    f = get_demo_file('Hintereisferner_RGI5.shp')
    assert os.path.exists(f)
```

21.3 测试 Fixture 与 GlacierDirectory 工厂

创建测试用 GlacierDirectory 是一个常见需求。OGGM 在 `funcs.py` 中提供了工厂函数:

```
# Create a minimal GlacierDirectory from a demo shapefile
def get_test_gdir(minimal=False, border=None):
    """Factory for test GlacierDirectory objects.

    Parameters
    -----
    minimal : bool
        If True, create a bare GlacierDirectory with just the RGI outline.
        If False, run the full preprocessing pipeline.
    border : int, optional
        Map border in pixels.

    Returns
    -----
    GlacierDirectory
    """
    from oggm.utils import get_demo_file, GlacierDirectory
    from oggm import workflow
    cfg.initialize()
    rgi_file = get_demo_file('Hintereisferner_RGI5.shp')
    gdir = GlacierDirectory(rgi_file)
    if not minimal:
        workflow.gis_prepro_tasks(gdir)
    return gdir
```

21.4 回归测试：与已知输出进行对比

OGGM 维护一组用于回归测试的参考输出。这些输出由已知正确的代码版本生成，并以压缩形式存储在测试套件中。当代码发生变更时，回归测试会验证输出结果是否保持一致（或在容差范围内一致）。

```
def test_inversion_regression():
    """Verify that the mass-conservation inversion produces consistent results."""
    from oggm.core.inversion import mass_conservation_inversion
    from oggm.tests.funcs import get_benchmark_data

    gdir = get_test_gdir()
    # Run inversion
    mass_conservation_inversion(gdir)
    result = gdir.read_pickle('inversion_output')

    # Compare against benchmark
    benchmark = get_benchmark_data('inversion_hintereis')
    for vn in ['volume', 'thickness']:
        v_new = result[vn]
        v_ref = benchmark[vn]
        np.testing.assert_allclose(
            v_new, v_ref, rtol=1e-3,
            err_msg=f"{vn} regression failure"
        )
```

21.5 持续集成：GitHub Actions

OGGM 使用 GitHub Actions 进行持续集成。CI 工作流定义在 `.github/workflows/` 目录中，通常包括以下作业：

作业	触发条件	用途
tests	Push、PR	在多个 Python 版本（3.9、3.10、3.11）上运行完整测试套件
minimal	Push、PR	冒烟测试；作为快速门禁最先运行
docs	Push 到 master	构建并部署 Sphinx 文档

作业	触发条件	用途
lint	Push、PR	代码风格检查 (flake8、black)
benchmarks	定时调度、PR 标签	性能回归检测

提示：在本地运行 CI 使用 tox 在本地运行与 GitHub Actions 执行的相同的测试矩阵：

```
# Run all test environments
tox

# Run just the tests for Python 3.10
tox -e py310

# Run linting only
tox -e lint
```

21.6 测试数据：oggm-sample-data 仓库

OGGM 维护一个独立的仓库 `OGGM/oggm-sample-data`，其中包含轻量级的测试数据集：

- **RGI shapefile**：用于测试的 RGI 冰川轮廓小规模子集。
- **DEM 瓦片**：覆盖测试冰川的低分辨率 DEM 子集。
- **气候数据**：适用于测试冰川的预处理气候时间序列。
- **预处理目录**：缓存的已处理 `GlacierDirectory`，用于加快测试执行速度。
- **基准输出**：用于回归测试的参考输出。

`oggm.utils` 中的 `get_demo_file()` 函数负责透明地 下载并缓存这些文件。

21.7 外部参考实现：ext/sia_fluxlim.py

`oggm/ext/sia_fluxlim.py` 模块包含了 SIA 通量限制器 (flux limiter) 算法的独立 参考实现。这段外部代码不在业务化运行环境中使用，而是作为验证靶标：

- 由不同的作者编写，对同一数学问题采用了不同的算法方法。
- 在测试中用于验证业务化运行环境实现是否产生相同的结果（在浮点容差范围内）。
- 为通量限制器 (flux limiter) 提供了一个独立于生产代码中任何优化或简化的“真实基准 (ground truth)”。

21.8 如何运行测试

21.8.1 基本命令

```
# Run the entire test suite
pytest

# Run a specific test file
pytest oggm/tests/test_models.py

# Run a specific test function
pytest oggm/tests/test_models.py::test_flux_based_model_init

# Run tests matching a keyword expression
pytest -k "inversion"

# Run tests with verbose output
pytest -v

# Run tests and stop on first failure
pytest -x

# Run tests in parallel (requires pytest-xdist)
pytest -n 4
```

21.8.2 测试标记

OGGM 使用 `pytest` 标记对测试进行分类:

```
# Run only slow tests
pytest -m slow

# Skip graphics tests (no display available)
pytest -m "not test_graphics"

# Run only the workflow integration tests
pytest -m workflow

# See all available markers
pytest --markers
```

标记	描述	典型耗时
@pytest.mark.slow	需要大量计算资源的测试	> 30 秒
@pytest.mark.workflow	端到端工作流集成测试	> 60 秒
@pytest.mark.test_graphics	需要显示环境 / matplotlib 的测试	< 10 秒
@pytest.mark.test_benchmarks	性能基准测试	不固定

21.9 编写新测试

21.9.1 模式: Entity Task 测试

```
import pytest
import numpy as np
import oggm
from oggm import workflow
from oggm.utils import get_demo_file, GlacierDirectory
from oggm.tests.funcs import get_test_gdir

class TestMyNewTask:
    """Test suite for my_new_custom_task."""

    @classmethod
    def setup_class(cls):
        """Create shared test resources once per class."""
        oggm.cfg.initialize()
        cls.gdir = get_test_gdir(minimal=True)

    def test_task_returns_expected_type(self):
        """The task should return a dict with required keys."""
        from my_module import my_new_task
        result = my_new_task(self.gdir)
        assert isinstance(result, dict)
        assert 'required_key_1' in result
        assert 'required_key_2' in result

    def test_task_output_written_to_disk(self):
        """The task should write output to the glacier directory."""
        from my_module import my_new_task
        my_new_task(self.gdir)
        output_path = self.gdir.get_filepath('my_new_output')
        assert os.path.exists(output_path)

    def test_task_numerical_accuracy(self):
        """Output values should be within physical bounds."""
        from my_module import my_new_task
        result = my_new_task(self.gdir)
        assert 0 <= result['volume_km3'] <= 1e6 # No glacier > 10^6 km³

    def test_task_edge_case_empty_glacier(self):
        """The task should handle glaciers with zero thickness."""
        # Create a pathological glacier directory
        # ...
        from my_module import my_new_task
        result = my_new_task(empty_gdir)
        assert result['volume_km3'] == 0.0

    def test_task_deterministic(self):
        """Same inputs should produce identical outputs."""
        from my_module import my_new_task
        r1 = my_new_task(self.gdir, seed=42)
        r2 = my_new_task(self.gdir, seed=42)
        assert r1['value'] == r2['value']
```

21.9.2 模式：数值验证

```
def test_my_numerical_function():
    """Verify numerical correctness against analytical reference."""
    from my_module import my_numerical_function

    # Known analytical case
    x = np.linspace(0, 1, 100)
    y_numerical = my_numerical_function(x)
    y_analytical = known_analytical_solution(x)

    # Relative error should be small
    rel_error = np.abs(y_numerical - y_analytical) / (np.abs(y_analytical) + 1e-10)
    assert np.max(rel_error) < 1e-6

    # Mass/energy should be conserved
    integral_numerical = np.trapz(y_numerical, x)
    integral_analytical = np.trapz(y_analytical, x)
    assert np.abs(integral_numerical - integral_analytical) < 1e-8
```

21.10 覆盖率与质量指标

OGGM 使用 `pytest-cov` 跟踪代码覆盖率：

```
# Run tests with coverage
pytest --cov=oggm --cov-report=html

# View coverage report
# open htmlcov/index.html
```

虽然 OGGM 不强制要求严格的覆盖率百分比，但遵循以下准则：

- **核心模块**：每个公共函数应至少有一个测试。
- **数值程序**：应在多个参数值下进行测试，并在可能的情况下与解析解进行比对。
- **Shop 模块**：至少需要一个冒烟测试，验证模块能够导入且其下载函数可以被调用。
- **错误路径**：测试应覆盖预期的失败模式（无效输入、缺失数据、损坏文件）。

额外的质量门禁包括：

- **代码检查 (Linting)**：使用 `flake8` 进行风格检查，在 `setup.cfg` 中维护项目特有的配置。
- **类型检查**：使用 `mypy` 逐步引入类型注解（按模块选择加入）。
- **文档**：Sphinx 构建必须无警告地完成。
- **安全**：使用 `Dependabot` 进行依赖漏洞扫描。

第22章 OGGM教程全集

初级

中级

高级

本章提供了一套完整的教程，涵盖OGGM的每个主要方面，从安装到高级专题（如动态预热（dynamic spinup）和入海型冰川建模）。教程中的代码分为三类：可以从空目录直接运行的完整脚本、需要沿用前面已经生成的 `gdir/gdirs` 对象的连续练习、以及用于解释思想的示意代码。每段关键代码前会说明依赖条件，读者不必把所有片段都视为可单独复制运行的脚本。

建议学习顺序

第一次接触OGGM：先跑教程1、4、5、6，目标是得到一条冰川的气候文件、物质平衡、反演厚度和历史模拟诊断。

需要做区域研究：再跑教程7，并把附录C作为输出文件速查表。

需要改模型或写论文方法部分：最后读教程8，并回到第8–13章核对物理假设和参数含义。

代码运行约定

本章默认你在Jupyter Notebook或一个干净的Python脚本中逐段运行。若跳读某个教程，请先检查该段是否已经定义了 `cfg`、`workflow`、`tasks`、`gdir` 或 `gdirs`。如果没有，请回到该教程开头重新初始化工作目录和冰川目录（GlacierDirectory）。

常用输出速查

研究问题	主要文件或对象	常用变量	推荐章节
冰川面积/长度变化	<code>model_diagnostics.nc</code>	<code>area_m2</code> ， <code>length_m</code>	教程6，第18章，附录C
冰储量和平均厚度	<code>inversion_output.pkl</code> ， <code>model_diagnostics.nc</code>	<code>volume</code> ， <code>volume_m3</code> ， <code>thickness</code>	教程5，第9章
物质平衡和ELA	<code>mb_calib.json</code> ，MB模型对象	<code>melt_f</code> ， <code>temp_bias</code> ， <code>get_specific_mb</code> ， <code>get_ela</code>	教程4，第8章
入海型冰川崩解	<code>diagnostics.json</code> ， <code>model_diagnostics.nc</code>	<code>calving_m3</code> ， <code>calving_rate</code> ， <code>calving_k</code>	教程8，第12章
区域批处理质量控制	<code>diagnostics.json</code> ，任务日志	<code>dem_source</code> ， <code>error_task</code> ， <code>run_dynamic_spinup_success</code>	教程7，第15–16章

教程一：入门指南

安装、基础设置与首次冰川模拟

初级

| 预计时间: 30分钟 | 前置条件: Python 3.9+

1.1 安装OGGM

OGGM可通过conda（推荐）和pip安装：

```
# Option A: conda (recommended – handles all C dependencies)
conda create -n oggm_env python=3.11
conda activate oggm_env
conda install -c conda-forge oggm-deps
pip install oggm

# Option B: pip-only (requires system GDAL and netCDF4 libs)
pip install oggm

# Option C: development install (for modifying OGGM source)
git clone https://github.com/OGGM/oggm.git
cd oggm
pip install -e .
```

安装与下载排错

现象	通常原因	建议处理
GDAL或netCDF4安装失败	系统库版本不匹配	优先使用conda-forge环境，不建议初学者使用纯pip安装地理空间依赖。
初始化或预处理长时间无输出	正在下载RGI、DEM或气候辅助文件	第一次运行需要联网和缓存数据；可先用单条示例冰川测试，确认缓存目录有写入权限。
下载失败或速度很慢	访问外部数据源不稳定	在稳定网络下先下载预处理数据；团队内可共享 ~/.oggm 缓存或预处理GlacierDirectory。
同一代码结果不同	OGGM版本、RGI版本、预处理级别 (prepro level) 或参数不同	在论文和脚本中记录OGGM版本、RGI版本、from_prepro_level、prepro_border 和关键参数。

1.2 基础设置

可直接运行：这段代码只初始化OGGM和工作目录，不会真正处理冰川。

```
import oggm
from oggm import cfg, workflow, utils

# Step 1 – Initialize OGGM (load configuration)
cfg.initialize()

# Step 2 – Set the working directory (where glacier data will be stored)
cfg.PATHS['working_dir'] = '/path/to/your/OGGM_data'

# Step 3 – Enable multiprocessing (optional but recommended)
cfg.PARAMS['use_multiprocessing'] = True
```

1.3 你的第一个冰川：Hintereisferner

可直接运行，但需要联网： from_prepro_level=0 表示从原始RGI和地形数据开始，第一次运行会下载数据。若只是学习后续流程，可把 from_prepro_level 改为官方已有的预处理级别 (prepro level)。

```
# Define a single glacier to work with
rgi_ids = ['RGI60-11.00897'] # Hintereisferner, Austrian Alps

# Create GlacierDirectory object(s)
gdirs = workflow.init_glacier_directories(
    rgi_ids,
    from_prepro_level=0, # start from scratch
    prepro_border=10     # map border in pixels
)

print(f"Created {len(gdirs)} glacier directory")
print(f"Glacier: {gdirs[0].rgi_id}")
print(f"Area: {gdirs[0].rgi_area_km2:.2f} km²")
print(f>Data directory: {gdirs[0].dir}")
```

预期输出:

```
Created 1 glacier directory
Glacier: RGI60-11.00897
Area: 8.45 km²
Data directory: /path/to/OGGM_data/per_glacier/RGI60-11/RGI60-11.00/RGI60-11.00897
```

1.4 运行预处理任务

```
# Run all GIS preprocessing tasks on the glacier
workflow.execute_entity_task(workflow.gis_prepro_tasks, gdirs)

# At this point, the glacier directory contains:
# - outlines.shp (glacier outline)
# - dem.tif (digital elevation model)
# - glacier_grid.json (grid definition)
# - gridded_data.nc (gridded topographic variables)
# - centerlines.pkl (computed centerlines)
# - model_flowlines.pkl (initialized flowlines)
```

1.5 理解输出文件

让我们检查一下生成了什么:

```
import os

gdir = gdirs[0]
for fname in os.listdir(gdir.dir):
    fpath = os.path.join(gdir.dir, fname)
    size_kb = os.path.getsize(fpath) / 1024
    print(f" {fname:<40s} {size_kb:>8.1f} KB")

# Inspect the gridded data
import xarray as xr
with xr.open_dataset(gdir.get_filepath('gridded_data')) as ds:
    print("\nGridded data variables:")
    for vn in ds.data_vars:
        print(f" {vn}: {ds[vn].shape}, {ds[vn].dtype}")
```

预期输出 (近似值):

```
outlines.shp                12.3 KB
dem.tif                    156.7 KB
glacier_grid.json           0.5 KB
gridded_data.nc            45.2 KB
centerlines.pkl            23.1 KB
model_flowlines.pkl        18.4 KB
diagnostics.json           1.2 KB
```

```
Gridded data variables:
  topo: (107, 134), float64
  glacier_mask: (107, 134), uint8
  glacier_topo: (107, 134), float64
  aspect: (107, 134), float64
  slope: (107, 134), float64
  dis: (107, 134), float64
  catchment_mask: (107, 134), uint8
```

你已经完成了第一次OGGM预处理运行！冰川目录（GlacierDirectory）已准备好进行气候处理、冰厚反演和动力学模拟。请参阅[第5章（GIS与DEM处理）](#)和[第6章（中心线与几何参数化）](#)，深入了解每个任务的具体作用。

教程二：使用GlacierDirectory

创建、读取和管理GlacierDirectory对象

初级 | 预计时间: 20 分钟

2.1 深入理解GlacierDirectory

```
import oggm
from oggm import cfg, utils, workflow

cfg.initialize()
cfg.PATHS['working_dir'] = '/tmp/oggm_tutorial'

# Create a GlacierDirectory from an RGI shapefile
rgi_file = utils.get_demo_file('Hintereisferner_RGI5.shp')
gdir = utils.GlacierDirectory(rgi_file)

# Key attributes
print(f"RGI ID: {gdir.rgi_id}")
print(f"RGI version: {gdir.rgi_version}")
print(f"Name: {gdir.rgi_name}")
print(f"Area (km²): {gdir.rgi_area_km2:.2f}")
print(f"CenLat: {gdir.cenlat:.3f}")
print(f"CenLon: {gdir.cenlon:.3f}")
print(f"TerminusType: {gdir.terminus_type}")
print(f>Data dir: {gdir.dir}")
```

2.2 BASENAMES系统

```
# BASENAMES provides standardized file names within a glacier directory
from oggm import cfg

print("Registered BASENAMES:")
for key, (filename, desc) in cfg.BASENAMES.items():
    print(f"  {key:<30s} → {filename:<30s} ({desc})")

# Get the full path to any registered file type
outline_path = gdir.get_filepath('outlines')
grid_path = gdir.get_filepath('gridded_data')
print(f"\nOutline file: {outline_path}")
print(f"Gridded data: {grid_path}")

# Check if a glacier has a particular file
if gdir.has_file('climate_historical'):
    print("Climate data is present")
else:
    print("Climate data not yet processed")
```

2.3 读取和写入冰川数据

```
# Reading gridded data (NetCDF)
import xarray as xr
ds = xr.open_dataset(gdir.get_filepath('gridded_data'))
glacier_topo = ds['glacier_topo'].values # numpy array
mask = ds['glacier_mask'].values.astype(bool)
print(f"Glacier elevation range: {glacier_topo[mask].min():.0f} - "
      f"{glacier_topo[mask].max():.0f} m a.s.l.")

# Reading pickled objects
centerlines = gdir.read_pickle('centerlines')
print(f"Number of centerlines: {len(centerlines)}")

# Reading JSON
diagnostics = gdir.read_json('diagnostics')
print(f"DEM source: {diagnostics.get('dem_source', 'unknown')}")

# Writing custom data
import json
my_results = {'custom_metric': 42.0, 'rgi_id': gdir.rgi_id}
gdir.write_json(my_results, 'my_custom_results')

# Reading it back
loaded = gdir.read_json('my_custom_results')
print(f"Custom metric: {loaded['custom_metric']}")
```

提示：目录结构 冰川/RGI60-11.00897的GlacierDirectory位于 `per_glacier/RGI60-11/RGI60-11.00/RGI60-11.00897/` 。三级层级结构（区域/子区域/冰川）映射了RGI的组织结构，便于高效的目录遍历。 详见第4章（GlacierDirectory）的完整规范。

2.4 查找GlacierDirectory

```
# Get all existing glacier directories in the working directory
all_gdirs = workflow.init_glacier_directories(
    rgi_ids=None, # None = all that exist on disk
    from_prepro_level=0
)
print(f"Found {len(all_gdirs)} glacier directories")

# Filter by RGI region
alps_gdirs = workflow.init_glacier_directories(
    rgi_ids=None,
    from_prepro_level=0,
    reset=False
)
alps_only = [g for g in alps_gdirs if g.rgi_region == '11']
print(f"Alps glaciers: {len(alps_only)}")
```

教程三：冰川预处理

GIS任务、中心线和流线—逐步详解

初级 | 预计时间：45分钟

3.1 预处理 workflow

函数 `gis_prepro_tasks(gdir)` 是一个便捷封装，可一次性运行所有预处理步骤。 为了深入理解，下面我们逐步运行每个步骤：

```
import oggm
from oggm import cfg, utils, workflow
from oggm.core import gis, centerlines

cfg.initialize()
cfg.PATHS['working_dir'] = '/tmp/oggm_tutorial_prepro'

# Start fresh
gdir = utils.GlacierDirectory(utils.get_demo_file('Hintereisferner_RGI5.shp'))

# Step 1: Define the glacier region on a local grid
gis.define_glacier_region(gdir)
print("1. Glacier region defined")

# Step 2: Process the DEM (reproject, crop, fill gaps)
gis.process_dem(gdir)
print("2. DEM processed")

# Step 3: Compute glacier masks
gis.glacier_masks(gdir)
print("3. Glacier masks computed")

# Step 4: Compute gridded attributes (slope, aspect, distance)
gis.gridded_attributes(gdir)
print("4. Gridded attributes computed")
```

3.2 中心线计算

```
# Step 5: Compute centerlines from the glacier mask
centerlines.compute_centerlines(gdir)
print("5. Centerlines computed")

# Visualize centerlines (requires matplotlib)
import matplotlib.pyplot as plt
centerlines.visualize_centerlines(gdir, figsize=(10, 8),
                                  use_flowlines=False)
plt.title(f"Centerlines for {gdir.rgi_id}")
plt.tight_layout()
plt.show()

# Step 6: Compute catchment areas for each centerline
centerlines.catchment_area(gdir)
print("6. Catchment areas computed")

# Step 7: Compute downstream centerline connections
centerlines.compute_downstream_line(gdir)
print("7. Downstream lines computed")

# Step 8: Determine flowline order and tributary structure
centerlines.compute_downstream_bedshape(gdir)
print("8. Downstream bed shape computed")
```

3.3 流线初始化

```
# Step 9: Initialize the flowlines
centerlines.initialize_flowlines(gdir)
print("9. Flowlines initialized")

# Inspect the flowlines
fls = gdir.read_pickle('model_flowlines')
for i, fl in enumerate(fls):
    print(f"\nFlowline {i}:")
    print(f"  Points: {len(fl.surface_h)}")
    print(f"  Length: {fl.dis_on_line[-1]:.0f} m")
    print(f"  Max width: {fl.widths.max():.0f} m")
    print(f"  Min elevation: {fl.surface_h.min():.0f} m")
    print(f"  Max elevation: {fl.surface_h.max():.0f} m")
    if hasattr(fl, 'is_tributary'):
        print(f"  Tributary: {fl.is_tributary}")
    if hasattr(fl, 'flows_to'):
        print(f"  Flows to: {fl.flows_to}")

# Visualize the flowlines
centerlines.visualize_centerlines(gdir, figsize=(10, 8),
                                  use_flowlines=True)
plt.title(f"Flowlines for {gdir.rgi_id}")
plt.tight_layout()
plt.show()
```

常见陷阱：中心线质量 对于某些冰川几何形状（非常平坦、被碎屑覆盖或跃动型冰川），自动中心线算法可能产生不理想的结果。在继续之前，请使用 `visualize_centerlines()` 检查输出。如果中心线质量不佳，优先检查DEM、RGI边界和 `border` 设置；必要时再谨慎调整 `cfg.PARAMS['min_centerline_distance']` 等中心线参数。问题排查详见第6章（中心线）。

教程四：气候数据与物质平衡

处理气候数据、校准和计算物质平衡

中级 | 预计时间: 40分钟

4.1 处理气候数据

```
import oggm
from oggm import cfg, workflow
from oggm.core import climate

cfg.initialize()
cfg.PATHS['working_dir'] = '/tmp/oggm_tutorial_climate'

# Get a preprocessed glacier (skip the GIS steps for brevity)
gdirs = workflow.init_glacier_directories(
    ['RGI60-11.00897'],
    from_prepro_level=2,
    prepro_border=10
)

# Process climate data for the glacier
workflow.execute_entity_task(climate.process_climate_data, gdirs)
print("Climate data processed!")

# Inspect the climate file
import xarray as xr
gdir = gdirs[0]
with xr.open_dataset(gdir.get_filepath('climate_historical')) as ds:
    print(f"Climate data time span: {ds.time.values[0]} to {ds.time.values[-1]}")
    print(f"Variables: {list(ds.data_vars)}")
    print(f"Temperature range: {ds['temp'].min().values:.1f} to "
          f"{ds['temp'].max().values:.1f} °C")
    print(f"Precipitation range: {ds['prcp'].min().values:.1f} to "
          f"{ds['prcp'].max().values:.1f} kg m-2")
    print(f"Ref height: {ds.ref_hgt.values:.0f} m")
    print(f"Ref pixel area: {ds.ref_pix_area.values*1e-6:.2f} km2")
```

4.2 计算物质平衡

连续练习：这段依赖4.1生成的 `climate_historical` 和 `mb_calib.json`。物质平衡模型内部常用冰当量速度 ($m\ ice\ s^{-1}$)，教程中的图轴用于教学展示，论文作图时请统一换算为 $mm\ w.e.\ yr^{-1}$ 或 $m\ w.e.\ yr^{-1}$ 。

```

# Compute the monthly mass balance at reference heights
mb_df = gdir.get_ref_mb_data()
print("Mass balance at reference heights:")
print(mb_df.describe())

# Plot the mass balance profile
import matplotlib.pyplot as plt
import numpy as np

# Get a typical year's mass balance profile
heights = np.linspace(
    gdir.gridded_data['glacier_topo'].min(),
    gdir.gridded_data['glacier_topo'].max(),
    50
)

# Create a mass balance model
from oggm.core.massbalance import PastMassBalance
mb_model = PastMassBalance(gdir)

# Compute annual MB profile for a specific year
year = 2000
annual_mb = mb_model.get_annual_mb(heights, year=year)

fig, ax = plt.subplots(figsize=(8, 6))
ax.plot(annual_mb, heights, 'b-', linewidth=2)
ax.axvline(x=0, color='k', linestyle='--')
ax.axhline(y=mb_model.get_ea(year=year), color='r', linestyle='--',
          label=f"ELA = {mb_model.get_ea(year=year):.0f} m")
ax.set_xlabel('Annual Mass Balance (kg m-2 yr-1)')
ax.set_ylabel('Elevation (m a.s.l.)')
ax.set_title(f'Mass Balance Profile - {year}')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

4.3 校准物质平衡模型

```

# OGGM automatically calibrates melt_f (temperature sensitivity)
# during process_climate_data, but you can inspect the results:

calib = gdir.read_json('mb_calib')
print("Mass balance calibration parameters:")
for key, val in calib.items():
    print(f"  {key}: {val}")

# The key parameters:
# - melt_f: temperature melt factor (kg m-2 K-1 day-1)
# - temp_bias: temperature correction applied to match geodetic MB
# - prcp_fac: precipitation correction factor

# Compute the specific mass balance for a period
from oggm.core.massbalance import MultipleFlowlineMassBalance
mb_multi = MultipleFlowlineMassBalance(gdir)

years = np.arange(1980, 2010)
s_mb_list = []
for yr in years:
    ela = mb_multi.get_ea(year=yr)
    s_mb = mb_multi.get_specific_mb(year=yr)
    s_mb_list.append(s_mb)

print(f"\nMean specific MB (1980-2010): {np.mean(s_mb_list):.2f} mm w.e. yr-1")
print(f"Mean ELA (1980-2010): {np.mean([mb_multi.get_ea(year=y) for y in years]):.0f} m")

```

高级：自定义校准 你可以通过 `tasks.mb_calibration_from_geodetic_mb`、`tasks.mb_calibration_from_scalar_mb` 或 `tasks.mb_calibration_from wgms_mb` 来提供自己的校准数据；默认的大地测量物质平衡 (geodetic mass balance) 时期由 `cfg.PARAMS['geodetic_mb_period']` 控制。详见第8章 (物质平衡模型)。

教程五：冰厚反演

基于质量守恒的冰厚反演

中级 | 预计时间：35分钟

5.1 运行反演

可直接运行：从预处理级别 (prepro level) 3开始，通常已经包含气候处理和物质平衡校准。反演结果主要写入 `inversion_output.pkl`，后续流线模型会用它初始化冰床和冰厚。

```
import oggm
from oggm import cfg, workflow
from oggm.core import inversion

cfg.initialize()
cfg.PATHS['working_dir'] = '/tmp/oggm_tutorial_inversion'

# Start from a preprocessed glacier with climate data
gdirs = workflow.init_glacier_directories(
    ['RGI60-11.00897'],
    from_prepro_level=3, # needs climate data
    prepro_border=10
)

# Run the inversion for each glacier
workflow.execute_entity_task(inversion.mass_conservation_inversion, gdirs)
print("Inversion complete!")

# Inspect the inversion output
gdir = gdirs[0]
inv_output = gdir.read_pickle('inversion_output')
print("\nInversion output keys:", list(inv_output.keys()))
print(f"Inverted volume: {inv_output['volume']*1e-9:.4f} km³")
print(f"Mean thickness: {inv_output['volume']/gdir.rgi_area_m2:.1f} m")
```

5.2 理解反演参数

```
# Key parameters affecting inversion results:
print("Inversion parameters:")
print(f" Glen A:      {cfg.PARAMS['glen_a']:.2e} s⁻¹ Pa⁻³")
print(f" Glen n:      {cfg.PARAMS['glen_n']}")
print(f" Sliding fs: {cfg.PARAMS['fs']}")
print(f" Inversion glen_a: {cfg.PARAMS['inversion_glen_a']:.2e}")
print(f" Inversion fs:   {cfg.PARAMS['inversion_fs']}")
print(f" mixed_min_shape: {cfg.PARAMS['mixed_min_shape']}")

# The inversion solves for ice thickness H such that:
# flux(H) = integrated_apparent_mass_balance
# where flux = u_bar * H * w and u_bar follows the SIA
# with deformation + sliding contributions.
```

5.3 可视化反演冰厚

```
import matplotlib.pyplot as plt
import numpy as np

# Plot the thickness profile along each flowline
fls = gdir.read_pickle('inversion_input')
inv = gdir.read_pickle('inversion_output')

fig, axes = plt.subplots(len(fls), 1, figsize=(12, 4 * len(fls)),
                          squeeze=False)
for i, fl in enumerate(fls):
    ax = axes[i, 0]
    x = fl.dis_on_line / 1000 # km
    thick = inv['thickness'][i]
    # Surface = bed + thickness
    bed = fl.bed_h
    surface = fl.surface_h
    computed_surface = bed + thick

    ax.fill_between(x, bed, surface, alpha=0.3, color='blue',
                    label='Measured surface')
    ax.fill_between(x, bed, computed_surface, alpha=0.5, color='red',
                    label='Inverted thickness')
    ax.plot(x, bed, 'k-', linewidth=1.5, label='Bed')
    ax.set_xlabel('Distance along flowline (km)')
    ax.set_ylabel('Elevation (m a.s.l.)')
    ax.set_title(f'Flowline {i}')
    ax.legend(loc='lower left')
    ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

5.4 过滤反演输出

```
# The raw inversion can have noise. Apply filtering:
inversion.filter_inversion_output(gdir)

# Re-read and compare
inv_filtered = gdir.read_pickle('inversion_output')
print(f"Volume before filter: {inv['volume']*1e-9:.4f} km³")
print(f"Volume after filter: {inv_filtered['volume']*1e-9:.4f} km³")

# Sensitivity test: vary inversion parameters
import copy
original_glen_a = cfg.PARAMS['inversion_glen_a']

results = {}
for factor in [0.5, 1.0, 2.0]:
    cfg.PARAMS['inversion_glen_a'] = original_glen_a * factor
    # In a real sensitivity experiment, reset or overwrite the inversion
    # output before re-running mass_conservation_inversion.
    vol = inv_filtered['volume'] * 1e-9 # placeholder
    results[factor] = vol
    print(f"  glen_a factor {factor}: volume = {vol:.4f} km³")

cfg.PARAMS['inversion_glen_a'] = original_glen_a # restore
```

重要：反演不确定性 质量守恒反演依赖表观物质平衡 (apparent mass balance)：沿流线积分后的通量需要与当前几何相容。对于强烈非平衡、跃动或几何资料误差很大的冰川，表观物质平衡 (apparent mass balance) 和反演厚度都会更不确定。默认反演使用表观物质平衡 (apparent mass balance) 来部分解决这一问题，但用户应注意反演体积存在10–30%的不确定性。误差来源的完整分析详见第9章 (冰厚反演)。

教程六：冰川演化

运行历史和未来预测

中级 | 预计时间: 50分钟

6.1 运行历史模拟

```
import oggm
from oggm import cfg, workflow
from oggm.core import flowline, massbalance

cfg.initialize()
cfg.PATHS['working_dir'] = '/tmp/oggm_tutorial_evolution'

# Get a fully preprocessed glacier (prepro_level >= 3)
gdirs = workflow.init_glacier_directories(
    ['RGI60-11.00897'],
    from_prepro_level=3,
    prepro_border=10
)

gdir = gdirs[0]

# Create a mass balance model (historical climate)
mb_model = massbalance.PastMassBalance(gdir)

# Initialize the flowline model with the inverted bed
fls = gdir.read_pickle('model_flowlines')
model = flowline.FluxBasedModel(
    flowlines=fls,
    mb_model=mb_model,
    y0=2003,          # start year (RGI date)
    glen_a=cfg.PARAMS['glen_a'],
    fs=cfg.PARAMS['fs'],
)

# Run from 2003 to 2020
print("Running historical simulation (2003-2020)...")
model.run_until(2020)

# Inspect results
diag = model.get_diagnostics()
print(f"\nFinal year diagnostics ({model.yr:.0f}):")
print(f"  Volume:      {model.volume_km3:.4f} km³")
print(f"  Area:        {model.area_km2:.2f} km²")
print(f"  Length:      {model.length_m:.0f} m")
print(f"  ELA:         {diag.get('ela', 'N/A')}")
print(f"  Spec. MB:    {model.get_specific_mb():.2f} mm w.e. yr⁻¹")
```

6.2 存储模型诊断数据

```
# Run and store diagnostics at each year
from oggm.core.flowline import flowline_model_run

# Re-initialize
model = flowline.FluxBasedModel(
    flowlines=fls,
    mb_model=mb_model,
    y0=2003,
    glen_a=cfg.PARAMS['glen_a'],
    fs=cfg.PARAMS['fs'],
)

# Run with diagnostics
diag_path = gdir.get_filepath('model_diagnostics')
flowline_model_run(
    gdir, model,
    ys=2003, ye=2020,
    store_diagnostics=True,
    output_filesuffix='_historical'
)

# Read the diagnostics
import xarray as xr
with xr.open_dataset(
    gdir.get_filepath('model_diagnostics',
                      filesuffix='_historical')
) as ds_diag:
    print(f"Diagnostic variables: {list(ds_diag.data_vars)}")
    print(f"Time steps: {len(ds_diag.time)}")

# Plot volume evolution
import matplotlib.pyplot as plt
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

ax1.plot(ds_diag.time, ds_diag.volume_m3 * 1e-9)
ax1.set_xlabel('Year')
ax1.set_ylabel('Volume (km³)')
ax1.set_title('Volume Evolution')
ax1.grid(True, alpha=0.3)

ax2.plot(ds_diag.time, ds_diag.area_m2 * 1e-6)
ax2.set_xlabel('Year')
ax2.set_ylabel('Area (km²)')
ax2.set_title('Area Evolution')
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

6.3 使用GCM数据进行未来预测

概念示意：真实GCM预测需要先准备或下载GCM/ISIMIP气候文件，并用 `process_gcm_data` 或官方预处理流程生成对应的气候强迫。下面代码只说明`delta`方法在OGGM中的位置，不能替代完整情景数据准备。


```

from oggm.core import climate

# Compute the climate delta (GCM - historical) for a future period
# This creates a "delta method" scenario
climate.historical_delta_method(gdir)

# Create a mass balance model for the future
mb_future = massbalance.ConstantMassBalance(gdir)

# Initialize a new model from the present-day state
model_future = flowline.FluxBasedModel(
    flowlines=fls,
    mb_model=mb_future,
    y0=2020,
    glen_a=cfg.PARAMS['glen_a'],
    fs=cfg.PARAMS['fs'],
    is_tidewater=gdir.is_tidewater,
)

# Run to 2100
print("Running future projection (2020-2100)...")
model_future.run_until(2100)

diag = model_future.get_diagnostics()
print(f"\nProjected state at 2100:")
print(f"  Volume:      {model_future.volume_km3:.4f} km³")
print(f"  Area:        {model_future.area_km2:.2f} km²")
print(f"  Length change: {model_future.length_m - gdir.rgi_length_m:.0f} m")

```

6.4 理解模型诊断数据

```

# Full list of available diagnostics from FluxBasedModel
print("Available diagnostics:")
common_diags = [
    'volume_m3', 'area_m2', 'length_m',
    'ela_m', 'calving_m3',
    'dmdtda_mmwea', # specific mass balance
]
for d in common_diags:
    print(f"  {d}")

# Annual mass balance breakdown
print("\nModel.get_diagnostics() return values:")
full_diag = model_future.get_diagnostics()
for k, v in full_diag.items():
    if isinstance(v, (int, float)):
        print(f"  {k}: {v:.2f}")
    else:
        print(f"  {k}: {type(v).__name__}")

```

提示：选择求解器 OGGM支持多种流线求解器：

- **SemiImplicitModel** —OGGM v1.6.3配置文件默认的半隐式流线求解器，具有自适应时间步长。
- **FluxBasedModel** —显式通量求解器，常用于多流线和崩解相关实验。
- **KarthaussModel** —较简单的显式格式，便于理解和调试数值概念。
- **FileModel** —从预先计算好的NetCDF输出中读取（用于后处理）。

每个求解器的数学细节详见第10章（流线模型）。

教程七：处理多个冰川

区域处理、并行执行和结果汇总

中级 | 预计时间：45分钟

7.1 处理区域冰川目录 (GlacierDirectory)

可直接运行，但建议先小规模测试：区域模拟最常见的问题不是代码语法，而是个别冰川的DEM、边界或气候数据异常。第一次运行请先选5-10条冰川，确认输出和诊断图合理后再扩大区域。

```
import oggm
from oggm import cfg, workflow, tasks

cfg.initialize()
cfg.PATHS['working_dir'] = '/tmp/oggm_tutorial_regional'

# Define a set of glaciers (e.g., a handful from the Alps)
rgi_ids = [
    'RGI60-11.00897', # Hintereisferner
    'RGI60-11.00778', # Vernagtferner
    'RGI60-11.00002', # Pasterze
    'RGI60-11.02692', # Findelengletscher
    'RGI60-11.02709', # Gornergletscher
]

print(f"Processing {len(rgi_ids)} glaciers...")

# Initialize all glacier directories
gdirs = workflow.init_glacier_directories(
    rgi_ids,
    from_prepro_level=0,
    prepro_border=10,
    reset=True
)
print(f"Initialized {len(gdirs)} glacier directories")

# Run preprocessing pipeline (enable parallel processing for speed)
cfg.PARAMS['use_multiprocessing'] = True
workflow.execute_entity_task(workflow.gis_prepro_tasks, gdirs)
print("GIS preprocessing complete!")

# Quick status check
for gdir in gdirs:
    print(f"  {gdir.rgi_id}: {gdir.rgi_name}, "
          f"n_flowlines={len(gdir.read_pickle('model_flowlines'))}")
```

7.2 使用execute_entity_task并行处理

```
import time

# Run climate processing in parallel
print("Processing climate data (parallel)...")
t0 = time.time()
workflow.execute_entity_task(tasks.process_climate_data, gdirs)
print(f"Climate processing done in {time.time() - t0:.1f}s")

# Run inversion in parallel
print("Running mass-conservation inversion (parallel)...")
t0 = time.time()
workflow.execute_entity_task(tasks.mass_conservation_inversion, gdirs)
print(f"Inversion done in {time.time() - t0:.1f}s")

# Monitor progress
workflow.execute_entity_task(
    tasks.mass_conservation_inversion, gdirs,
    print_log=True # print progress to stdout
)
```

7.3 汇总区域结果

```
import pandas as pd
import numpy as np

# Compile results from all glacier directories
results = []
for gdir in gdirs:
    try:
        inv = gdir.read_pickle('inversion_output')
        diag = gdir.read_json('diagnostics')
        calib = gdir.read_json('mb_calib')

        results.append({
            'rgi_id': gdir.rgi_id,
            'name': gdir.rgi_name,
            'area_km2': gdir.rgi_area_km2,
            'volume_km3': inv['volume'] * 1e-9,
            'mean_thick_m': inv['volume'] / gdir.rgi_area_m2,
            'n_flowlines': len(gdir.read_pickle('model_flowlines')),
            'melt_f': calib.get('melt_f', np.nan),
            'temp_bias': calib.get('temp_bias', np.nan),
            'prcp_fac': calib.get('prcp_fac', np.nan),
            'dem_source': diag.get('dem_source', 'unknown'),
        })
    except Exception as e:
        print(f"Failed for {gdir.rgi_id}: {e}")

df = pd.DataFrame(results)
print("\nRegional Summary:")
print(f"Total glaciers: {len(df)}")
print(f"Total area: {df['area_km2'].sum():.1f} km²")
print(f"Total volume: {df['volume_km3'].sum():.3f} km³")
print(f"Mean thickness: {df['mean_thick_m'].mean():.1f} m")
print(f"Mean melt_f: {df['melt_f'].mean():.2f}")
print("\nPer-glacier results:")
print(df.to_string(index=False))
```

7.4 错误处理和稳健处理

```
# Use "continue_on_error" for robust regional processing
cfg.PARAMS['continue_on_error'] = True

# This will log errors but not crash if one glacier fails
workflow.execute_entity_task(
    tasks.run_from_climate_data, gdirs,
    ys=2003, ye=2020,
)

# Check which glaciers succeeded
for gdir in gdirs:
    if gdir.has_file('model_diagnostics'):
        print(f" [OK] {gdir.rgi_id}")
    else:
        print(f" [FAIL] {gdir.rgi_id} - check log for details")

# Reset the parameter
cfg.PARAMS['continue_on_error'] = False
```

教程八：高级专题

动态预热 (dynamic spinup)、入海型冰川、崩解、支流合并与Delta方法

高级 | 预计时间：60分钟

8.1 动态预热 (dynamic spinup)

高级连续练习：动态预热 (dynamic spinup) 要求冰川目录 (GlacierDirectory) 已经完成反演和物质平衡校准。它用于减小初始化瞬态，不保证得到唯一的平衡态；区域批处理时应检查 `run_dynamic_spinup_success` 和匹配误差。

```
# Dynamic spinup searches for a temperature bias and recent-past
# initialization that matches the glacier area or volume near the RGI date.
from oggm.core import dynamic_spinup

gdir = gdirs[0] # use a preprocessed gdir

# Run a short dynamic spinup and store the initialized model geometry
model = dynamic_spinup.run_dynamic_spinup(
    gdir,
    spinup_period=20,
    min_spinup_period=10,
    minimise_for='area',
    precision_percent=1,
    output_filesuffix='_dynamic_spinup',
)

print("Running spinup...")
# The spinup runs automatically before the main simulation
model.run_until(2020)

print(f"Post-spinup volume: {model.volume_km3:.4f} km³")
diag = gdir.get_diagnostics()
print(f"Spinup period: {diag['dynamic_spinup_period']} years")
print(f"Temperature bias: {diag['temp_bias_dynamic_spinup']:.2f} °C")
```

8.2 入海型冰川与崩解

高级连续练习：这段依赖已有 `gdirs`、`cfg`、`workflow` 和 `tasks`。入海型冰川结果对水位、冰床和崩解参数非常敏感，不建议在没有诊断图的情况下直接用于区域结论。

```

import xarray as xr

# Tidewater glaciers require additional setup.
# Start from an already prepared list of GlacierDirectory objects, e.g.
# gdirs = workflow.init_glacier_directories(rgi_ids, from_prepro_level=3, ...)
tw_gdirs = [g for g in gdirs if g.is_tidewater]
if len(tw_gdirs) > 0:
    tw_gdir = tw_gdirs[0]
    print(f"Tidewater glacier: {tw_gdir.rgi_id} "
          f"{tw_gdir.rgi_name}")
    print(f"Terminus type: {tw_gdir.terminus_type}")

# Configure k-calving. OGGM v1.6.3 uses 0.6 yr-1 by default.
cfg.PARAMS['use_kcalving_for_run'] = True
cfg.PARAMS['calving_k'] = 0.6

# Recompute calving-aware inversion diagnostics, initialize the model,
# and run with the standard OGGM task interface.
workflow.execute_entity_task(
    tasks.find_inversion_calving_from_any_mb, [tw_gdir]
)
workflow.execute_entity_task(
    tasks.init_present_time_glacier, [tw_gdir]
)
workflow.execute_entity_task(
    tasks.run_from_climate_data, [tw_gdir], ys=2000, ye=2020
)

with xr.open_dataset(tw_gdir.get_filepath('model_diagnostics')) as ds:
    print(float(ds.volume_m3.isel(time=-1)) / 1e9, "km3")
    print(float(ds.length_m.isel(time=-1)), "m")

```

8.3 合并支流冰川

```

from oggm.core import flowline

# Complex glaciers have multiple centerlines with tributaries
# You can control how tributaries are handled:

# Option 1: Independent flowlines (default)
cfg.PARAMS['use_multiple_flowlines'] = True

# Option 2: Merge tributaries into one main flowline
cfg.PARAMS['use_multiple_flowlines'] = False

# Inspect the tributary structure
fls = gdir.read_pickle('model_flowlines')
for i, fl in enumerate(fls):
    info = []
    if hasattr(fl, 'is_tributary') and fl.is_tributary:
        info.append('tributary')
    if hasattr(fl, 'flows_to'):
        info.append(f'flows_to={fl.flows_to}')
    if hasattr(fl, 'is_rectangular'):
        info.append(f'rectangular={fl.is_rectangular}')
    print(f" Flowline {i}: {' '.join(info)}")

```

8.4 使用Delta方法进行气候预测

```
# The delta method computes climate change signals from GCMs
# and applies them as anomalies to the historical climate

from oggm.core import climate
import xarray as xr

# Step 1: Compute the delta for a given GCM and scenario
# (This is normally done during process_climate_data)
climate.historical_delta_method(gdir)

# Step 2: Inspect the delta
delta_file = gdir.get_filepath(
    'climate_historical',
    filesuffix='_delta'
)
print(f"Delta file: {delta_file}")

# Step 3: Create a future mass balance model
from oggm.core.massbalance import ConstantMassBalance
mb_future = ConstantMassBalance(
    gdir, y0=2003,
    halfsize=15,      # window half-size for climate averaging
    filename='climate_historical',
    input_filesuffix='_delta',
)

# The delta signal is already baked into the climate file,
# so the ConstantMassBalance model can use it directly.
# For GCM-driven runs, you compute deltas from raw GCM output:
# climate.process_gcm_data(gdir, filesuffix='_GCM_output')
```

8.5 交叉验证与敏感性分析

```
# Systematic parameter sensitivity test
import copy
from oggm.core.flowline import FluxBasedModel

baseline_params = {
    'glen_a': cfg.PARAMS['glen_a'],
    'fs': cfg.PARAMS['fs'],
}
fls = gdir.read_pickle('model_flowlines')
mb_model = massbalance.PastMassBalance(gdir)

sensitivity_results = {}
for glen_factor in [0.5, 1.0, 2.0]:
    for fs_value in [0.0, cfg.PARAMS['fs']]:
        # Clone flowlines to avoid in-place modification
        import copy
        fls_copy = copy.deepcopy(fls)

        model = FluxBasedModel(
            flowlines=fls_copy,
            mb_model=mb_model,
            y0=2003,
            glen_a=baseline_params['glen_a'] * glen_factor,
            fs=fs_value,
        )
        model.run_until(2020)
        key = f"glen={glen_factor:.1f}, fs={fs_value:.0f}"
        sensitivity_results[key] = model.volume_km3

print("\nSensitivity Analysis (final volume, km³):")
for key, vol in sensitivity_results.items():
    print(f"  {key}: {vol:.4f}")
```

教程九：常见错误与定位方法

从报错信息回到OGGM workflow

初级 | 预计时间：20分钟

报错或现象	通常含义	优先检查
<code>FileNotFoundError: climate_historical</code>	尚未运行气候处理，或filesuffix不一致。	确认是否执行 <code>tasks.process_climate_data</code> ，并检查 <code>gdir.has_file('climate_historical')</code> 。
<code>mb_calib.json</code> 不存在	物质平衡校准没有完成。	重新运行气候任务或 <code>tasks.mb_calibration_from_geodetic_mb</code> 。
中心线很短或方向异常	DEM、RGI 边界、冰川掩膜或边界像素设置不合适。	用 <code>plot_domain</code> 和 <code>visualize_centerlines</code> 检查几何，不要先调动力学参数。
反演体积异常大/小	表观物质平衡 (apparent mass balance)、Glen A、滑动、冰床形状或几何输入异常。	先查 <code>inversion_input</code> 和 <code>inversion_output</code> ，再做参数敏感性。
正演中冰川突然爆炸或消失	初始几何、物质平衡强迫或时间步长触发数值问题。	缩短运行时段，开启诊断输出，比较 <code>area_m2</code> 、 <code>volume_m3</code> 和 <code>length_m</code> 。
区域批处理中少数冰川失败	这是常见现象，通常由个别冰川几何或数据源导致。	使用 <code>continue_on_error=True</code> 记录失败清单，逐条检查失败任务和输入文件。

```
# Minimal diagnostic checklist for one glacier directory
for name in ['gridded_data', 'climate_historical', 'mb_calib',
            'inversion_output', 'model_flowlines', 'model_diagnostics']:
    print(f"{name:25s}", gdir.has_file(name))

print(gdir.rgi_id, gdir.rgi_area_km2, gdir.rgi_region)
print(gdir.get_diagnostics())
```

附录A API 参考

本附录提供了OGGM公共API的全面参考，按模块组织。每个条目包含函数/类的签名、关键参数、返回值和用途简述。本附录是参考资料而非教程—详细解释请参见相关章节。

A.1 oggm.cfg -- 配置与初始化

模块级函数

initialize()

oggm.cfg.initialize(file=None, logging_level='INFO')

参数: file (str, 可选) -- 用于覆盖默认参数的 params.cfg 文件路径。 logging_level (str) -- Python 日志级别。

返回值: None

从内置的 params.cfg 加载OGGM配置，应用用户提供的覆盖参数， 设置工作目录并配置日志系统。必须在大多数其他OGGM函数之前调用。

initialize_minimal()

oggm.cfg.initialize_minimal(file=None)

参数: file (str, 可选) -- params.cfg 覆盖文件。

返回值: None

轻量级初始化，仅加载参数但不设置日志或工作目录。 适用于只需要读取参数的脚本。

set_logging_config(logging_level='INFO')

oggm.cfg.set_logging_config(logging_level='INFO')

参数: logging_level (str) -- 可选值: 'DEBUG', 'INFO', 'WARNING', 'ERROR', 'CRITICAL'。

使用指定级别配置OGGM日志记录器。由 initialize() 自动调用。

pack_config()

oggm.cfg.pack_config()

返回值: str -- 包含所有非默认参数和工作目录路径的JSON字符串。

将当前配置序列化以便复现。可以将输出保存到模型结果旁边， 以记录所使用的确切参数集。

unpack_config(json_str)

oggm.cfg.unpack_config(json_str)

参数: json_str (str) -- 之前由 pack_config() 生成的JSON字符串。

从打包字符串恢复配置。用于重现之前运行的设置。

关键属性

属性	类型	描述
cfg.PARAMS	DocumentedDict	所有配置参数的有序字典，附带文档字符串
cfg.PATHS	dict	工作目录、缓存目录和示例数据目录的路径

属性	类型	描述
<code>cfg.BASENAMES</code>	<code>DocumentedDict</code>	从逻辑名称到（文件名，描述）的映射，用于冰川目录（ <code>GlacierDirectory</code> ）文件
<code>cfg.DATA</code>	<code>dict</code>	运行时数据，如RGI元数据、DEM数据源和气候参考

A.2 oggm.utils -- 工具函数与GlacierDirectory

`GlacierDirectory(rgi_entity, base_dir=None, reset=False)`

`class oggm.utils.GlacierDirectory(rgi_entity, base_dir=None, reset=False)`

参数: `rgi_entity` (`str`, `tuple`或`dict`) -- RGI ID 字符串、RGI shapefile路径或 RGI元数据字典。 `base_dir` (`str`, 可选) -- 覆盖默认工作目录。 `reset` (`bool`) -- 如为`True`，清空现有目录内容。

单个冰川的核心数据容器。管理文件路径，提供读写方法，并暴露冰川元数据。完整文档参见第7章。

`entity_task(logger, writes=[])`

`@oggm.utils.entity_task(logger, writes=[])`

参数: `logger` (`str` 或 `logging.Logger`) -- 日志记录器名称或实例。 `writes` (`str`列表) -- 该任务写入的 `BASENAMES` 键。

装饰器，用于包装单冰川处理函数。提供自动日志记录、错误处理（通过 `cfg.PARAMS['continue_on_error']`）、计时和依赖追踪。被装饰的函数 必须以 `GlacierDirectory` 作为其第一个位置参数。

`global_task(logger)`

`@oggm.utils.global_task(logger)`

装饰器，用于全局任务（`global task`；而非单冰川任务），例如下载参考数据 或汇总结果。与 `entity_task` 类似，但不期望 `GlacierDirectory` 参数。

`get_demo_file(fname)`

`oggm.utils.get_demo_file(fname)`

参数: `fname` (`str`) -- 文件名（如 `'Hintereisferner_RGI5.shp'`）。

返回值: `str` -- 已下载/缓存示例文件的绝对路径。

从 `oggm-sample-data` 仓库下载（如需要）并返回文件路径。教程和测试中不可或缺。

`monthly_timeseries(ini_date, end_date, cal=None)`

`oggm.utils.monthly_timeseries(ini_date, end_date, cal=None)`

参数: `ini_date` (`datetime-like`) -- 序列的起始时间。 `end_date` (`datetime-like`) -- 序列的结束时间（含）。 `cal` (`str`, 可选) -- 日历类型（`'standard'`, `'365_day'` 等）。

返回值: 按月频率的 `pandas.DatetimeIndex`。

创建月度时间索引，在OGGM中广泛用于气候和物质平衡时间序列。

A.3 oggm.core.gis -- GIS处理

`define_glacier_region(gdir, div_id=None)`

`oggm.core.gis.define_glacier_region(gdir, div_id=None)`

参数: `gdir` (`GlacierDirectory`)。 `div_id` (`int`, 可选) -- 用于具有多个流域分界的冰川的排水分界ID。

使用冰川轮廓定义冰川的局部地图投影和空间域。初始化 `Salem` 网格并写入 `glacier_grid.json`。

process_dem(gdir)

`oggm.core.gis.process_dem(gdir)`

下载并处理覆盖冰川的DEM瓦片。重投影到局部网格， 填充小型空洞，并写入 `dem.tif`。DEM数据源通过 `cfg.PARAMS['dem_source']` 配置。

glacier_masks(gdir)

`oggm.core.gis.glacier_masks(gdir)`

从RGI轮廓多边形计算冰川网格掩膜。将 `glacier_mask` 和 `glacier_topo`（限制在冰川区域内的DEM）写入 `gridded_data.nc`。

gridded_attributes(gdir)

`oggm.core.gis.gridded_attributes(gdir)`

计算冰川掩膜范围内每个网格单元的坡度、坡向和到轮廓的距离。 将多个变量写入 `gridded_data.nc`。

A.4 oggm.core.centerlines -- 中心线与流线处理

compute_centerlines(gdir, div_id=None)

`oggm.core.centerlines.compute_centerlines(gdir, div_id=None)`

使用最小成本路径算法从冰川掩膜计算几何中心线。 处理多分支冰川。写入 `centerlines.pkl`。

initialize_flowlines(gdir)

`oggm.core.centerlines.initialize_flowlines(gdir)`

将几何中心线转换为模型流线，包含计算宽度、面积-高度 分布和适用于数值模拟的网格划分。写入 `model_flowlines.pkl`。

catchment_area(gdir)

`oggm.core.centerlines.catchment_area(gdir)`

为每条中心线分支划分集水区。将 `catchment_mask` 写入 `gridded_data.nc` 和 `geometries.pkl`。

compute_downstream_line(gdir)

`oggm.core.centerlines.compute_downstream_line(gdir)`

计算冰川末端之外的下游中心线延伸段。用于可能超出 当前轮廓线前进的冰川。

compute_downstream_bedshape(gdir)

`oggm.core.centerlines.compute_downstream_bedshape(gdir)`

确定下游冰床形状和流线交汇处的配置。

A.5 oggm.core.climate -- 气候数据处理

process_climate_data(gdir, **kwargs)

`oggm.core.climate.process_climate_data(gdir, **kwargs)`

根据 `cfg.PARAMS['baseline_climate']` 分派到相应的数据商店模块。 处理冰川的气候数据并写入 `climate_historical.nc`。 同时校准物质平衡模型（写入 `mb_calib.json`）。

historical_delta_method(gdir, **kwargs)

`oggm.core.climate.historical_delta_method(gdir, **kwargs)`

计算冰川位置处的温度和降水气候增量（GCM减去历史值）。用于未来预估。写入带 `_delta` 后缀的气候文件。

`mb_calibration(gdir, **kwargs)`

```
oggm.core.climate.mb_calibration(gdir, **kwargs)
```

校准物质平衡模型参数 (`melt_f`, `temp_bias`, `prcp_fac`)，使其匹配 观测的大地测量物质平衡 (geodetic mass balance) 或用户指定的目标值。

`process_gcm_data(gdir, **kwargs)`

```
oggm.core.climate.process_gcm_data(gdir, **kwargs)
```

处理冰川位置处的原始GCM输出，计算未来情景的月温度和 降水时间序列。

`A.6 oggm.core.massbalance -- 物质平衡模型`

`MassBalanceModel`

```
class oggm.core.massbalance.MassBalanceModel
```

所有物质平衡模型的抽象基类。子类必须实现 `get_monthly_mb(heights, year, ...)` 和 `get_annual_mb(heights, year, ...)`。

`MonthlyTModel(gdir, **kwargs)`

```
class oggm.core.massbalance.MonthlyTModel(gdir, **kwargs)
```

标准的月度温度指数物质平衡模型。将积累量计算为固态降水比例， 将消融量通过度日法计算。

`ConstantMassBalance(gdir, **kwargs)`

```
class oggm.core.massbalance.ConstantMassBalance(gdir, **kwargs)
```

用于未来预估的物质平衡模型。使用由历史数据叠加GCM异常 所导出的恒定（或增量调整的）气候信号。

`PastMassBalance(gdir, **kwargs)`

```
class oggm.core.massbalance.PastMassBalance(gdir, **kwargs)
```

用于历史时期的物质平衡模型。使用来自 `climate_historical.nc` 的时变气候数据和校准后的温度指数模型 (temperature-index model)。

`MultipleFlowlineMassBalance(gdir, **kwargs)`

```
class oggm.core.massbalance.MultipleFlowlineMassBalance(gdir, **kwargs)
```

管理多条流线（支流）物质平衡的包装器。将物质平衡计算 分派给每条流线并处理支流几何。

`RandomMassBalance(gdir, **kwargs)`

```
class oggm.core.massbalance.RandomMassBalance(gdir, **kwargs)
```

在校准均值周围带有随机变异性的物质平衡模型。用于 不确定性量化和集合模拟。

`A.7 oggm.core.inversion -- 冰厚反演`

`mass_conservation_inversion(gdir, **kwargs)`

```
oggm.core.inversion.mass_conservation_inversion(gdir, **kwargs)
```

通过求解质量守恒方程计算沿流线的冰厚度： 通量散度等于表观物质平衡 (apparent mass balance)。写入 `inversion_input.pkl` 和 `inversion_output.pkl`。关键参数由 `cfg.PARAMS['glen_a']`、`cfg.PARAMS['fs']` 和 `cfg.PARAMS['inversion_glen_a']` 控

制。

filter_inversion_output(gdir)

`oggm.core.inversion.filter_inversion_output(gdir)`

对原始反演输出应用后处理滤波：去除负厚度值， 使用高斯核平滑（宽度由 `cfg.PARAMS['inversion_smoothing_radius']` 控制），并确保末端附近的厚度单调递减。

prepare_for_inversion(gdir, **kwargs)

`oggm.core.inversion.prepare_for_inversion(gdir, **kwargs)`

准备质量守恒反演的输入数据：计算表观物质平衡（apparent mass balance）， 建立通量计算的线性方程组，并写入 `inversion_input.pkl`。

A.8 oggm.core.flowline -- 冰川动力学

FluxBasedModel(flowlines, mb_model, y0, glen_a, fs, ...)

`class oggm.core.flowline.FluxBasedModel(flowlines, mb_model=None, y0=0, glen_a=None, fs=None, ...)`

主要的冰川演化模型。使用半隐式有限差分格式和 自适应时间步长求解浅冰近似方程以实现冰川厚度演化。 支持多流线、动态预热（dynamic spinup）和入海型冰川崩解。

SemiImplicitModel(flowlines, mb_model, y0, glen_a, fs, ...)

`class oggm.core.flowline.SemiImplicitModel(flowlines, mb_model=None, y0=0, glen_a=None, fs=None, ...)`

简化的半隐式求解器，用于教学和调试目的。不如 `FluxBasedModel` 优化，但算法更透明。

flowline_model_run(gdir, model, ys, ye, ...)

`oggm.core.flowline.flowline_model_run(gdir, model, ys, ye, store_diagnostics=True, ...)`

便捷函数，从年份 `ys` 运行流线模型到 `ye`，并可选择 将诊断变量存储为NetCDF格式。处理完整生命周期：初始化、启动（如已配置）、主运行和清理。

FileModel(fpath)

`class oggm.core.flowline.FileModel(fpath)`

从预先计算的NetCDF文件读取模型状态，而非运行求解器。适用于 对先前保存的运行进行后处理和分析。

A.9 oggm.workflow -- workflow编排

init_glacier_directories(rgi_ids, ...)

`oggm.workflow.init_glacier_directories(rgi_ids=None, from_prepro_level=None, prepro_border=10, ...)`

参数： `rgi_ids` (list 或 str, 可选) -- 要初始化的RGI ID。如为None，查找所有现有的。 `from_prepro_level` (int, 可选) -- 所需预处理级别 (prepro level) 的位掩码 (0=原始, 1=GIS完成, 2=中心线完成, 3=气候完成, 4=反演完成)。
`prepro_border` (int) -- 地图边界像素数。

返回值： `GlacierDirectory`列表

为一组RGI ID创建或加载冰川目录 (`GlacierDirectory`)。这是任何OGGM工作流的标准入口点。

execute_entity_task(task, gdirs, **kwargs)

`oggm.workflow.execute_entity_task(task, gdirs, **kwargs)`

参数: `task` (可调用对象) -- 以 `entity_task` 装饰的函数。 `gdirs` (`GlacierDirectory`列表)。 `**kwargs` -- 转发给任务函数的参数。

在冰川列表上执行单冰川任务。如果 `cfg.PARAMS['use_multiprocessing']` 为True, 则在多个CPU核心上分配执行。 处理错误收集和日志记录。

gis_prepro_tasks(gdir, **kwargs)

`oggm.workflow.gis_prepro_tasks(gdir, **kwargs)`

便捷函数, 按顺序运行所有GIS预处理任务: `define_glacier_region`, `process_dem`, `glacier_masks`, `gridded_attributes`, `compute_centerlines`, `initialize_flowlines`, `catchment_area`, `compute_downstream_line`, `compute_downstream_bedshape`。

climate_tasks(gdir, **kwargs)

`oggm.workflow.climate_tasks(gdir, **kwargs)`

便捷函数, 用于气候相关任务: `process_climate_data` 和 `historical_delta_method`。

inversion_tasks(gdir, **kwargs)

`oggm.workflow.inversion_tasks(gdir, **kwargs)`

便捷函数, 用于反演相关任务: `prepare_for_inversion`、`mass_conservation_inversion` 和 `filter_inversion_output`。

A.10 类层次结构

`MassBalanceModel` (ABC) |— `MonthlyTIModel` |— `ConstantMassBalance` |— `PastMassBalance` |—
`RandomMassBalance` `Flowline` |— `MixedBedFlowline` `FlowlineModel` |— `FluxBasedModel` |— `SemiImplicitModel`
|— `FileModel` `GlacierDirectory` |— (管理单冰川文件I/O) `MassBalanceModel` —包装—>
`MultipleFlowlineMassBalance` `MultipleFlowlineMassBalance` —使用—> `GlacierDirectory` `FluxBasedModel` —使用—> `MultipleFlowlineMassBalance` `FluxBasedModel` —包含—> `MixedBedFlowline[]`

附录B 参数参考手册

本附录汇总OGGM `params.cfg` 与常用运行参数中的核心配置项。参数控制着从文件路径和网格分辨率到 冰动力学常数和崩解法则等各个方面。每个条目包含参数名称、类型、默认值、中文描述以及常见使用模块；少数只作为函数调用参数出现的条目会在描述中说明。

参数继承 参数可以从三个位置设置，优先级如下：(1) 环境变量 (`OGGM_PARAMNAME`)，(2) 用户提供的 `params.cfg` 文件，(3) OGGM内置默认值。具有最高优先级的值胜出。

哪些参数可以先不碰 初学者通常只需要设置 `working_dir`、`use_multiprocessing`、`border`、`baseline_climate` 和运行函数的时间范围。`glen_a`、`fs`、`calving_k`、`cfl_number` 等会直接改变物理或数值行为，应作为敏感性实验的一部分修改，并在结果中报告。

B.1 输入输出与路径

参数	类型	默认值	描述	使用者
<code>working_dir</code>	<code>str</code>	<code>~/ .oggm</code>	所有OGGM数据的根目录。冰川目录 (GlacierDirectory) 位于 <code>working_dir/per_glacier/</code> 下。	<code>cfg</code> , <code>GlacierDirectory</code> , <code>workflow</code>
<code>dl_verify</code>	<code>bool</code>	<code>True</code>	如为 <code>True</code> ，验证下载文件的校验和。	<code>utils</code> , <code>shop</code> 模块
<code>use_sample_data</code>	<code>bool</code>	<code>False</code>	如为 <code>True</code> ，使用 <code>oggm-sample-data</code> 进行测试和演示，而非下载完整数据集。	<code>tests</code> , <code>utils</code>
<code>tmp_dir</code>	<code>str</code>	<code>~/ .oggm_tmp</code>	用于中间处理文件的临时目录。	<code>cfg</code> , <code>gis</code>
<code>cache_dir</code>	<code>str</code>	<code>~/ .oggm_cache</code>	下载的DEM瓦片、气候数据和RGI文件的缓存目录。	<code>cfg</code> , <code>shop</code> , <code>gis</code>
<code>rgi_version</code>	<code>str</code>	<code>62</code>	使用的RGI版本 (62表示RGI 6.2；RGI 7在预处理脚本中通常用70G/70C等标识)。	<code>workflow</code> , <code>GlacierDirectory</code>
<code>use_rgi_area</code>	<code>bool</code>	<code>True</code>	如为 <code>True</code> ，使用RGI报告的面积。如为 <code>False</code> ，从局部网格计算面积。	<code>gis</code> , <code>centerlines</code>
<code>use_intersects</code>	<code>bool</code>	<code>True</code>	如为 <code>True</code> ，在处理中包含相交冰川 (针对具有相交关系的RGI区域)。	<code>gis</code>
<code>clip_intersects</code>	<code>bool</code>	<code>False</code>	如为 <code>True</code> ，将相交冰川轮廓裁剪到主冰川的集水区。	<code>gis</code>
<code>min_glacier_area_km2</code>	<code>float</code>	<code>0.01</code>	处理的最小冰川面积 (km ²)。更小的冰川将被跳过。	<code>workflow</code>

B.2 多进程

参数	类型	默认值	描述	使用者
<code>use_multiprocessing</code>	<code>bool</code>	<code>False</code>	启用实体任务 (entity task) 的并行执行；正式批处理时通常显式开启。	<code>workflow</code> , <code>execute_entity_task</code>
<code>mp_processes</code>	<code>int</code>	<code>-1</code>	多进程的进程数。 <code>-1</code> 表示使用所有可用CPU。	<code>workflow</code>

参数	类型	默认值	描述	使用者
<code>continue_on_error</code>	<code>bool</code>	<code>False</code>	如为 <code>True</code> ，当某个冰川失败时继续处理其他冰川。如为 <code>False</code> ，立即抛出异常。	<code>entity_task</code>
<code>log_history_level</code>	<code>str</code>	<code>INFO</code>	任务历史记录的日志级别（工作流日志）。	<code>cfg, workflow</code>
<code>show_progress</code>	<code>bool</code>	<code>True</code>	在实体任务（ <code>entity task</code> ）执行期间显示进度条。	<code>workflow</code>

B.3 网格与地图

参数	类型	默认值	描述	使用者
<code>grid_dx_method</code>	<code>str</code>	<code>square</code>	从冰川面积计算网格分辨率的方法。可选值包括' <code>fixed</code> '、' <code>by_bin</code> '、' <code>linear</code> '、' <code>square</code> '。	<code>gis</code> (<code>define_glacier_region</code>)
<code>map_proj</code>	<code>str</code>	<code>tmerc</code>	局部地图投影类型：' <code>tmerc</code> '（横轴墨卡托）或' <code>utm</code> '。	<code>gis</code>
<code>border</code>	<code>int</code>	<code>80</code>	局地地图在冰川边界外保留的像素宽度。过去模拟或冰川前进实验通常需要更大的边界。	<code>gis, workflow</code>
<code>clip_tidewater_border</code>	<code>bool</code>	<code>True</code>	对入海型冰川限制过大的局地地图边界，以减少不必要的海域网格。	<code>gis</code>

B.4 中心线与流线

参数	类型	默认值	描述	使用者
<code>use_multiple_flowlines</code>	<code>bool</code>	<code>True</code>	如为 <code>True</code> ，分别对每条中心线分支建模。如为 <code>False</code> ，将支流合并为一条。	<code>centerlines, flowline</code>
<code>localmax_window</code>	<code>float</code>	<code>500.</code>	寻找冰川头部局地高程极大值时使用的半窗口大小（m）。	<code>centerlines</code>
<code>smooth_window</code>	<code>float</code>	<code>251.</code>	DEM平滑窗口大小（m）；设为0表示不平滑。	<code>gis, centerlines</code>
<code>q1, q2, rmax</code>	<code>float</code>	<code>2e-6, 500., 1000.</code>	Kienholz等中心线算法中的头部候选点和路线代价参数。	<code>centerlines</code>
<code>f1, f2, a, b</code>	<code>float</code>	<code>1000., 3000., 4.25, 3.7</code>	Kienholz等中心线算法中的代价函数参数。	<code>centerlines</code>
<code>kbuffer</code>	<code>float</code>	<code>2.5</code>	在汇流处裁剪支流中心线的缓冲距离（像素）。	<code>centerlines</code>
<code>terminus_search_percentile</code>	<code>int</code>	<code>10</code>	水体终止冰川寻找末端高程时使用的百分位数；0表示不启用特殊处理。	<code>centerlines</code>
<code>terminus_search_altitude_range</code>	<code>float</code>	<code>100</code>	水体终止冰川末端搜索允许的高程范围（m）。	<code>centerlines</code>
<code>flowline_dx</code>	<code>int</code>	<code>2</code>	流线网格间距，以局地地图像素为单位。	<code>centerlines, flowline</code>
<code>flowline_junction_pix</code>	<code>int</code>	<code>3</code>	在流线汇合处任意裁剪的像素数，用于避免交汇处几何重叠。	<code>centerlines</code>

参数	类型	默认值	描述	使用者
<code>flowline_height_smooth</code>	<code>float</code>	<code>1</code>	沿流线高程的高斯平滑尺度（像素）。	<code>centerlines</code>
<code>filter_min_slope</code>	<code>bool</code>	<code>True</code>	是否过滤过小坡度；与 <code>min_slope</code> 共同影响流线坡度处理。	<code>centerlines</code>
<code>elevation_band_flowline_binsize</code>	<code>float</code>	<code>30</code>	高程带流线 (<code>elevation-band flowline</code>) 定义使用的海拔分段大小 (m)。	<code>centerlines</code>
<code>width_alt_range_thres</code>	<code>float</code>	<code>250.</code>	宽度计算中用于筛选高程范围的阈值。	<code>centerlines</code>
<code>min_n_per_bin</code>	<code>int</code>	<code>2</code>	高度-面积分布每个分箱所需的最少元素数。	<code>centerlines</code>
<code>base_binsize</code>	<code>float</code>	<code>50.</code>	高度-面积分布的基准分箱大小 (m)。	<code>centerlines</code>
<code>smooth_widths_window_size</code>	<code>int</code>	<code>1</code>	宽度平滑窗口；0表示不平滑，1表示默认平滑。	<code>centerlines</code>
<code>downstream_line_shape</code>	<code>str</code>	<code>trapezoidal</code>	冰川下游无冰延伸线的冰床形状，可为 <code>parabola</code> 或 <code>trapezoidal</code> 。	<code>centerlines</code> , <code>flowline</code>
<code>trapezoid_min_bottom_width</code>	<code>float</code>	<code>50.</code>	梯形下游线允许的最小底宽 (m)。	<code>flowline</code>

B.5 气候

参数	类型	默认值	描述	使用者
<code>baseline_climate</code>	<code>str</code>	<code>GSWP3_W5E5</code>	基线气候数据来源: 'CRU', 'HISTALP', 'ERA5', 'ERA5L', 'W5E5', 'GSWP3_W5E5' 等。	<code>climate</code> (分派器)
<code>temp_default_gradient</code>	<code>float</code>	<code>-0.0065</code>	默认大气温度递减率 (K/m, 等价于-6.5 K/km)。	<code>climate</code>
<code>temp_melt</code>	<code>float</code>	<code>-1.0</code>	融化计算使用的温度阈值 (°C)。	<code>massbalance</code>
<code>temp_all_solid</code>	<code>float</code>	<code>0.0</code>	低于此温度阈值 (°C) 的所有降水为固态。	<code>massbalance</code>
<code>temp_all_liq</code>	<code>float</code>	<code>2.0</code>	高于此温度阈值 (°C) 的所有降水为液态。	<code>massbalance</code>
<code>melt_f</code>	<code>float</code>	<code>5</code>	温度指数融化因子 (<code>temperature-index melt factor</code> ; $\text{kg m}^{-2} \text{K}^{-1} \text{day}^{-1}$), 校准时可更新。	<code>massbalance</code>
<code>melt_f_min</code>	<code>float</code>	<code>1.5</code>	校准融化因子时允许的下界。	<code>massbalance</code>
<code>melt_f_max</code>	<code>float</code>	<code>17</code>	校准融化因子时允许的上界。	<code>massbalance</code>
<code>temp_bias_min</code>	<code>float</code>	<code>-15</code>	温度偏差校准允许的下界 (°C)。	<code>massbalance</code>
<code>temp_bias_max</code>	<code>float</code>	<code>15</code>	温度偏差校准允许的上界 (°C)。	<code>massbalance</code>
<code>prcp_fac</code>	<code>float/None</code>	<code>None</code>	降水缩放因子；为空时由相关校准逻辑决定。	<code>massbalance</code>
<code>prcp_fac_min</code>	<code>float</code>	<code>0.1</code>	降水缩放因子校准允许的下界。	<code>massbalance</code>
<code>prcp_fac_max</code>	<code>float</code>	<code>10</code>	降水缩放因子校准允许的上界。	<code>massbalance</code>

参数	类型	默认值	描述	使用者
<code>geodetic_mb_period</code>	<code>str</code>	<code>2000-01-01_2020-01-01</code>	逐冰川匹配大地测量物质平衡 (geodetic mass balance) 时使用的参考期。	<code>massbalance</code>

B.6 冰动力学

参数	类型	默认值	描述	使用者
<code>glen_a</code>	<code>float</code>	<code>2.4e-24</code>	Glen流动定律的速率因子A ($\text{s}^{-1} \text{ Pa}^{-3}$)。控制冰的变形能力。	<code>flowline</code> , <code>inversion</code>
<code>glen_n</code>	<code>int</code>	<code>3</code>	Glen流动定律的指数n (无量纲)。	<code>flowline</code> , <code>inversion</code>
<code>fs</code>	<code>float</code>	<code>0</code>	滑动参数 <code>f_s</code> (无量纲)。0 = 无滑动, 1 = 最大滑动。	<code>flowline</code> , <code>inversion</code>
<code>inversion_glen_a</code>	<code>float</code>	<code>2.4e-24</code>	反演中专门使用的Glen A值 (可与演化计算不同)。	<code>inversion</code>
<code>inversion_fs</code>	<code>float</code>	<code>0</code>	反演中专门使用的滑动参数。	<code>inversion</code>
<code>ice_density</code>	<code>float</code>	<code>900</code>	冰密度 (kg m^{-3})。	<code>flowline</code> , <code>inversion</code>
<code>mixed_min_shape</code>	<code>float</code>	<code>0.001</code>	混合冰床横截面 (bed cross-section) 的最小形状参数, 用于避免反演中出现退化横截面。	<code>inversion</code>
<code>cfl_number</code>	<code>float</code>	<code>0.02</code>	流线求解器中自适应时间步长的CFL数。	<code>flowline</code>
<code>cfl_min_dt</code>	<code>float</code>	<code>60</code>	CFL自适应步长允许的最小子步长 (秒)。	<code>flowline</code>

B.7 崩解与入海型冰川

参数	类型	默认值	描述	使用者
<code>use_kalving_for_run</code>	<code>bool</code>	<code>False</code>	为入海型冰川启用k-崩解参数化。默认关闭, 需在入海型冰川实验中显式开启。	<code>flowline</code>
<code>use_kalving_for_inversion</code>	<code>bool</code>	<code>False</code>	在反演中对入海型冰川应用崩解校正。默认关闭, 开启后需配合入海型冰川预处理和水位约束。	<code>inversion</code>
<code>calving_k</code>	<code>float</code>	<code>0.6</code>	水深依赖型崩解法则中的崩解速率常数k (yr^{-1})。	<code>flowline</code>
<code>inversion_calving_k</code>	<code>float</code>	<code>0.6</code>	厚度反演阶段使用的崩解速率常数 (yr^{-1})。	<code>inversion</code>
<code>calving_use_limiter</code>	<code>bool</code>	<code>True</code>	对崩解通量使用限制器, 以避免末端冰厚和通量出现数值不稳定。	<code>flowline</code>
<code>free_board_marine_terminating</code>	<code>tuple</code>	<code>10, 50</code>	入海型冰川反演中允许的出水高度 (freeboard) 范围 (m)。	<code>inversion</code>
<code>free_board_lake_terminating</code>	<code>float</code>	<code>10</code>	湖泊终止冰川使用的出水高度 (freeboard) (m)。	<code>inversion</code>
<code>calving_line_extension</code>	<code>int</code>	<code>30</code>	为崩解前缘向水体方向延伸的网格点数。	<code>centerlines</code> , <code>inversion</code>

参数	类型	默认值	描述	使用者
<code>calving_front_slope</code>	<code>float</code>	<code>0.05</code>	崩解前缘外侧水下床面坡度，按 $\tan(\alpha)$ 表示。	<code>centerlines</code> , <code>inversion</code>

B.8 输出控制

参数	类型	默认值	描述	使用者
<code>store_diagnostic_variables</code>	<code>list</code>	<code>volume</code> , <code>volume_bsl</code> , ...	模型运行期间要写入诊断文件的变量列表。	<code>flowline</code>
<code>store_model_geometry</code>	<code>bool</code>	<code>False</code>	将完整模型几何（所有网格点的表面、冰床、厚度）保存为NetCDF格式。	<code>flowline</code>
<code>store_fl_diagnostics</code>	<code>bool</code>	<code>False</code>	在模型运行期间存储每条流线的诊断变量。该输出很占磁盘，通常只在诊断或教学中开启。	<code>flowline</code>
<code>store_fl_diagnostic_variables</code>	<code>list</code>	<code>area</code> , <code>thickness</code> , ...	启用流线诊断输出时写入的变量列表。	<code>flowline</code>
<code>evolution_model</code>	<code>str</code>	<code>SemiImplicit</code>	配置文件默认使用的流线演化求解器。参见第20章自定义模型注册。	<code>flowline</code>
<code>store_monthly_step</code>	<code>bool</code>	<code>False</code>	运行函数参数；如为True，以月度分辨率写出诊断变量，输出量会显著增大。	<code>run_* task</code>
<code>dynamic_spinup_min_ice_thick</code>	<code>float</code>	<code>2.0</code>	动态预热（dynamic spinup）中允许保留的最小冰厚阈值（m），用于避免极薄冰带导致的数值问题。	<code>flowline</code> , <code>dynamic_spinup</code>

B.9 诊断变量（详细）

模型运行期间存储的标准诊断变量集（启用时）：

变量	描述	单位	是否按流线？
<code>volume_m3</code>	冰川冰总体积	m^3	是
<code>area_m2</code>	冰川表面积	m^2	是
<code>length_m</code>	冰川长度（沿主流线）	m	是
<code>ela_m</code>	平衡线高度	m a.s.l.	否
<code>calving_m3</code>	年度崩解通量	$m^3\ yr^{-1}$	是
<code>dmdtda_mmwea</code>	比物质平衡速率	$mm\ w.e.\ yr^{-1}$	否
<code>terminus_thick_m</code>	末端冰厚度	m	是
<code>calving_rate_myr</code>	末端崩解速率	$m\ yr^{-1}$	是
<code>surface_h</code>	表面高程剖面（如果 <code>store_model_geometry</code> ）	m a.s.l.	是
<code>bed_h</code>	冰床高程剖面（如果 <code>store_model_geometry</code> ）	m a.s.l.	是

变量	描述	单位	是否按流线?
<code>thickness_m</code>	冰厚度剖面 (如果 <code>store_model_geometry</code>)	<i>m</i>	是
<code>ice_velocity_myr</code>	表面冰流速 (如果 <code>store_model_geometry</code>)	<i>m yr⁻¹</i>	是

B.10 交叉引用：参数与模块

模块	关键参数
<code>oggm.cfg</code>	<code>working_dir</code> , <code>tmp_dir</code> , <code>cache_dir</code> , <code>use_sample_data</code> , <code>dl_verify</code> , <code>dl_retries</code>
<code>oggm.workflow</code>	<code>use_multiprocessing</code> , <code>mp_processes</code> , <code>continue_on_error</code> , <code>show_progress</code> , <code>rgi_version</code> , <code>min_glacier_area_km2</code>
<code>oggm.core.gis</code>	<code>grid_dx_method</code> , <code>dem_source</code> , <code>border_km</code> , <code>map_proj</code> , <code>use_intersects</code> , <code>use_rgi_area</code> , <code>use_local_dem_cache</code>
<code>oggm.core.centerlines</code>	<code>use_multiple_flowlines</code> , <code>localmax_window</code> , <code>q1</code> , <code>q2</code> , <code>rmax</code> , <code>f1</code> , <code>f2</code> , <code>flowline_dx</code> , <code>downstream_line_shape</code>
<code>oggm.core.climate</code> / <code>massbalance</code>	<code>baseline_climate</code> , <code>temp_default_gradient</code> , <code>temp_melt</code> , <code>temp_all_solid</code> , <code>temp_all_liq</code> , <code>melt_f</code> , <code>prcp_fac</code> , <code>geodetic_mb_period</code>
<code>oggm.core.massbalance</code>	<code>temp_melt</code> , <code>temp_all_solid</code> , <code>temp_all_liq</code>
<code>oggm.core.inversion</code>	<code>glen_a</code> , <code>glen_n</code> , <code>fs</code> , <code>inversion_glen_a</code> , <code>inversion_fs</code> , <code>mixed_min_shape</code> , <code>use_kcalving_for_inversion</code> , <code>inversion_calving_k</code>
<code>oggm.core.flowline</code>	<code>glen_a</code> , <code>glen_n</code> , <code>fs</code> , <code>ice_density</code> , <code>cfl_number</code> , <code>cfl_min_dt</code> , <code>use_kcalving_for_run</code> , <code>calving_k</code> , <code>calving_use_limiter</code> , <code>evolution_model</code>

附录C 文件格式参考

每个 OGGM 冰川目录 (GlacierDirectory) 包含一组由预处理、气候、反演和模拟 workflow 生成的标准化文件。 本附录记录了每个文件的用途、格式、内部结构以及变量/键清单。 在读取、写入或调试冰川目录 (GlacierDirectory) 内容时，请使用此参考。

按研究问题查文件

研究问题	优先读取	核心变量或键	备注
冰川面积、长度、体积时间序列	model_diagnostics.nc	area_m2 , length_m , volume_m3	用于历史模拟、未来情景和区域统计。
初始冰厚和冰储量	inversion_output.pkl	volume , thick / thickness	反演结果，不等同于直接观测。
物质平衡校准	mb_calib.json	melt_f , temp_bias , prcp_fac	解释不同冰川响应差异时必须检查。
气候输入	climate_historical.nc	temp , prcp , ref_hgt	降水是月总量，不是瞬时速率。
质量控制和溯源	diagnostics.json	dem_source , 任务诊断键	区域批处理后应首先汇总。

C.1 冰川目录 (GlacierDirectory) 文件清单

文件名	BASENAMES 键	格式	生成来源	用途
outlines.shp	outlines	ESRI Shapefile	init_glacier_directories	所有任务
dem.tif	dem	GeoTIFF	process_dem	GIS 任务
glacier_grid.json	glacier_grid	JSON	define_glacier_region	初始化之后的所有任务
gridded_data.nc	gridded_data	NetCDF	glacier_masks, gridded_attributes, catchment_area	中心线、反演
climate_historical.nc	climate_historical	NetCDF	process_climate_data	物质平衡、校准
centerlines.pkl	centerlines	gzip pickle	compute_centerlines	流线初始化
inversion_input.pkl	inversion_input	pickle	prepare_for_inversion	反演
inversion_output.pkl	inversion_output	pickle	mass_conservation_inversion	流线模型
model_flowlines.pkl	model_flowlines	pickle	initialize_flowlines	反演、演化
model_diagnostics.nc	model_diagnostics	NetCDF	flowline_model_run	后处理
mb_calib.json	mb_calib	JSON	mb_calibration	物质平衡模型
geometries.pkl	geometries	pickle	catchment_area	可视化

文件名	BASENAMES 键	格式	生成来源	用途
<code>hypsonetry.csv</code>	<code>hypsonetry</code>	CSV	<code>initialize_flowlines</code>	体积-面积 标度
<code>diagnostics.json</code>	<code>diagnostics</code>	JSON	贯穿整个工作流	调试、溯 源

C.2 逐文件规格说明

`outlines.shp` -- RGI 冰川轮廓

格式: ESRI Shapefile | BASENAMES 键: `outlines`

用途: 包含冰川轮廓多边形和 RGI 元数据。这是所有其他数据的起始来源。

字段:

字段	类型	描述
<code>RGIId</code>	<code>str</code> (14 字符)	唯一的 RGI 冰川标识符
<code>GLIMSIId</code>	<code>str</code>	GLIMS 冰川标识符
<code>Name</code>	<code>str</code>	冰川名称 (如果已知)
<code>Area</code>	<code>float</code>	RGI 报告的面积 (km ²)
<code>CenLat</code>	<code>float</code>	质心纬度 (度)
<code>CenLon</code>	<code>float</code>	质心经度 (度)
<code>TermType</code>	<code>int</code>	末端类型代码 (0=陆地, 1=海洋, 2=湖泊等)
<code>Connect</code>	<code>int</code>	连通性代码
<code>Status</code>	<code>int</code>	冰川状态代码

`dem.tif` -- 数字高程模型

格式: GeoTIFF (Float32) | BASENAMES 键: `dem`

用途: 经过重投影和裁剪的 DEM, 覆盖冰川及其缓冲区 (即"边界")。

属性: 单波段 Float32 栅格。使用投影坐标参考系 (通常以冰川为中心的横轴墨卡托投影)。NoData 值: -9999。分辨率由参数中的 `grid_dx_method` 决定。

尺寸: 网格尺寸取决于冰川面积和边界。通常在 30-100 米分辨率下, 每边 50-200 像素。

`glacier_grid.json` -- Salem 网格定义

格式: JSON | BASENAMES 键: `glacier_grid`

用途: 定义冰川的局部坐标系和网格。

结构:

```
{
  "proj": "+proj=tmerc +lat_0=46.80 +lon_0=10.78 +k=1.0 +x_0=0 +y_0=0 +ellps=WGS84",
  "x0": 0,
  "y0": 0,
  "nx": 134,
  "ny": 107,
  "dx": 100.0,
  "dy": 100.0,
  "origin": "lower-left",
  "pixel_ref": "corner",
  "topo_valid": true
}
```

关键字段: `proj` (`PROJ.4` 字符串), `dx / dy` (网格间距, 单位米), `nx / ny` (`x` 和 `y` 方向的网格单元数)。

gridded_data.nc -- 网格化变量

格式: NetCDF4 | BASENAMES 键: `gridded_data`

用途: 所有 2D 网格化数据的中央存储库。随着预处理工作流的推进, 变量会逐步添加。

维度: `(y, x)`, 与 `glacier_grid` 维度匹配。

变量清单:

变量	数据类型	单位	添加者	描述
<code>topo</code>	<code>float64</code>	<code>m a.s.l.</code>	<code>process_dem</code>	完整 DEM 高程 (包括冰川外地形)
<code>glacier_mask</code>	<code>uint8</code>	布尔型 (0/1)	<code>glacier_masks</code>	掩膜: 1 = 冰川内, 0 = 冰川外
<code>glacier_topo</code>	<code>float64</code>	<code>m a.s.l.</code>	<code>glacier_masks</code>	被冰川区域掩膜后的 DEM (外部为 NaN)
<code>aspect</code>	<code>float64</code>	度	<code>gridded_attributes</code>	地表坡向 (0–360, 0 = 北)
<code>slope</code>	<code>float64</code>	度	<code>gridded_attributes</code>	地表坡度角
<code>dis</code>	<code>float64</code>	<code>m</code>	<code>gridded_attributes</code>	每个网格单元到冰川轮廓的距离
<code>catchment_mask</code>	<code>uint8</code>	类别	<code>catchment_area</code>	每个网格单元的流域 ID (0 表示无流域)
<code>distributed_thickness</code>	<code>float64</code>	<code>m</code>	<code>distribute_2d</code> (<code>sandbox</code> 模块)	从流线分布得到的 2D 冰厚度

全局属性: `rgi_id`、`dem_source`、`proj`、`grid_dx`、`grid_dy`。

climate_historical.nc -- 气候时间序列

格式: NetCDF4 | BASENAMES 键: `climate_historical`

用途: 存储冰川的月均温度和降水时间序列, 从气候数据源插值到冰川的高程范围。

维度: `time` (逐月, 通常从 1901 年至今), `height` (离散高程层)。

变量清单:

变量	维度	单位	描述
<code>temp</code>	<code>(time, height)</code>	<code>deg C</code>	各高程层的月平均温度
<code>prcp</code>	<code>(time, height)</code>	<code>kg m⁻²</code>	各高程层的月总降水量
<code>gradient</code>	<code>(time)</code>	<code>deg C / 100m</code>	月温度递减率

变量	维度	单位	描述
ref_hgt	标量	m a.s.l.	原始气候数据的参考高程
ref_pix_lat	标量	度	参考气候像素的纬度
ref_pix_lon	标量	度	参考气候像素的经度
ref_pix_dis	标量	km	冰川中心到参考像素的距离
ref_pix_area	标量	m ²	参考气候像素的面积

时间覆盖范围: 起止日期取决于气候数据源。CRU TS 通常覆盖 1901 年至今（存在 1-2 年的滞后）。

高程层: 通常为 30 个离散层，覆盖从冰川最低点到最高点的高程范围，外加安全余量。

centerlines.pkl -- 中心线对象

格式: gzip 压缩 pickle | BASENAMES 键: centerlines

用途: 存储从冰川掩膜计算得到的几何中心线对象。

内容: 一个 Centerline 对象列表，每个对象包含: line (shapely LineString -- 投影坐标下的中心线几何), dis_on_line (ndarray -- 沿线距离, 单位米), heads (Point 列表 -- 上游端点), tail (Point -- 下游末端点), flows_to (Centerline 或 None -- 该中心线汇入的下游中心线), inflows (Centerline 列表 -- 汇入该中心线的上游中心线), order (int -- 河流级别)。

inversion_input.pkl -- 反演输入

格式: pickle | BASENAMES 键: inversion_input

用途: 为质量守恒反演准备的数据。

内容: dict, 包含以下键: flux (ndarray 列表 -- 每条流线的表观物质平衡 (apparent mass balance) 通量), width (ndarray 列表 -- 每个网格点处的流线宽度), dx (float -- 网格间距, 单位米), surface_h (ndarray 列表 -- 每条流线的表面高程), bed_h (ndarray 列表 -- 每条流线的初始冰床估计), glen_a (float -- 反演使用的 Glen's A 参数), fs (float -- 反演使用的滑动参数)。

inversion_output.pkl -- 反演结果

格式: pickle | BASENAMES 键: inversion_output

用途: 质量守恒反演得到的冰厚度分布。

内容: dict, 包含以下键: volume (float -- 冰川总体积, m³), volume_bsl (float -- 海平面以下体积, m³, 用于入海型冰川), volume_bsl_max (float -- 最大可能的海平面以下体积), thickness (ndarray 列表 -- 每条流线的反演厚度, m), is_tidewater (bool -- 冰川末端是否到达海/湖), bed_h (ndarray 列表 -- 每条流线的最终冰床高程), glen_a (float -- 使用的 Glen's A 值), fs (float -- 使用的滑动参数)。

model_flowlines.pkl -- MixedBedFlowline 对象

格式: pickle | BASENAMES 键: model_flowlines

用途: 准备用于数值模拟的流线对象。包含模型网格分辨率下的完整几何信息（表面、反演后的冰床、宽度）。

内容: 一个 MixedBedFlowline 对象列表 (Flowline 的子类)，每个对象包含: dis_on_line (ndarray -- 沿流线距离, m), surface_h (ndarray -- 来自 DEM 的表面高程, m), bed_h (ndarray -- 来自反演或初始估计的冰床高程, m), widths (ndarray -- 各网格点处的冰川宽度, m), widths_m (ndarray -- 数值格式中交错网格上的宽度, m), dx (float -- 沿流线网格间距), dx_meter (float -- 实际米数网格间距), is_tidewater (bool), is_rectangular (bool -- 冰床类型标志), is_trapezoid (bool -- 冰床类型标志), is_parabolic (bool -- 冰床类型标志), flows_to (MixedBedFlowline 或 None -- 支流连接)。

model_diagnostics.nc -- 模型输出诊断

格式: NetCDF4 | BASENAMES 键: model_diagnostics

用途： 流线模型运行期间存储的模型诊断变量时间序列。

维度： `time` （逐年，每个存储年份一个条目）， `flowlines` （每条流线一个，用于每条流线的独立变量）。

变量清单：

变量	维度	单位	描述
<code>volume_m3</code>	<code>(time, flowlines)</code>	m^3	每条流线每年的冰体积
<code>area_m2</code>	<code>(time, flowlines)</code>	m^2	每条流线每年的表面积
<code>length_m</code>	<code>(time, flowlines)</code>	m	每条流线每年的长度
<code>calving_m3</code>	<code>(time, flowlines)</code>	$m^3\ yr^{-1}$	每条流线的崩解通量（陆地终止冰川为 0）
<code>ela_m</code>	<code>(time)</code>	$m\ a.s.l.$	冰川整体平衡线高度
<code>dmdtda_mmwea</code>	<code>(time)</code>	$mm\ w.e.\ yr^{-1}$	比物质平衡率

可选（如果 `store_model_geometry=True`）： `surface_h`、`bed_h`、`thickness_m`、`ice_velocity_myr` -- 这些变量会增加一个额外的 `dis_along_flowline` 维度。

`mb_calib.json` -- 物质平衡校准

格式： JSON | **BASENAMES 键：** `mb_calib`

用途： 存储冰川的校准物质平衡参数。

结构：

```
{
  "melt_f": 4.2,
  "temp_bias": 0.3,
  "prcp_fac": 1.0,
  "reference_mb": -850.0,
  "reference_period": "2000-01-01_2015-12-31",
  "calibration_method": "geodetic_mb_consensus",
  "t_star": 1920.5,
  "bias": 0.03
}
```

关键字段： `melt_f` -- 度日融化因子 ($kg\ m^{-2}\ K^{-1}\ day^{-1}$)， `temp_bias` -- 温度偏差校正 (K)， `prcp_fac` -- 降水校正因子 (1.0 = 无校正)， `t_star` -- 校准参考年份（物质平衡近似等于参考值的年份）。

`geometries.pkl` -- 多边形几何

格式： pickle | **BASENAMES 键：** `geometries`

用途： 存储冰川的 GIS 多边形几何（流域多边形、相交多边形、流线流域）。

内容： dict, 包含以下键： `polygon_pix` (list -- 以像素坐标表示的冰川掩膜)， `polygon_hr` (shapely Polygon -- 以 shapely 几何表示的冰川轮廓)， `catchments` (dict -- 将流线索引映射到流域多边形，如果已计算)， `intersects` (Polygon 列表 -- 相交的冰川多边形，如果存在)。

`hypsometry.csv` -- 测高统计

格式： CSV | **BASENAMES 键：** `hypsometry`

用途： 测高表：高程区间及其对应的冰川面积。

列： `elevation` (m a.s.l. -- 高程区间中心)， `area` (m^2 -- 该高程区间内的冰川面积)， `width` (m -- 该高程处的冰川总宽度)。

该 CSV 由 `initialize_flowlines` 生成，并在物质平衡计算和可视化中内部使用。

diagnostics.json -- 处理诊断信息

格式: JSON | BASENAMES 键: diagnostics

用途: 记录处理元数据和诊断信息, 用于溯源和调试。

典型内容:

```
{
  "dem_source": "SRTM",
  "dem_resolution": 30.0,
  "grid_dx": 100,
  "grid_dy": 100,
  "map_proj": "tmerc",
  "rgi_area_km2": 8.45,
  "glacier_area_on_grid_km2": 8.43,
  "n_centerlines": 3,
  "centerlines_total_length_m": 12450,
  "n_flowlines": 3,
  "prepro_level": 3,
  "has_climate": true,
  "climate_source": "CRU",
  "mb_calibration_done": true,
  "inversion_done": true
}
```

C.3 读写模式

OGGM 在 `GlacierDirectory` 上为每种文件类型提供了标准化的读写方法:

```
# NetCDF 文件
gdir.write_netcdf(data_dict, basename_key, filesuffix='')
ds = xr.open_dataset(gdir.get_filepath(basename_key, filesuffix=''))

# Pickle 文件
gdir.write_pickle(obj, basename_key, filesuffix='')
obj = gdir.read_pickle(basename_key, filesuffix='')

# JSON 文件
gdir.write_json(obj, basename_key, filesuffix='')
obj = gdir.read_json(basename_key, filesuffix='')

# Shapefile 文件
gdir.write_shapefile(gdf, basename_key, filesuffix='')
gdf = gpd.read_file(gdir.get_filepath(basename_key, filesuffix=''))

# 文本/CSV 文件
gdir.write_text(text, basename_key, filesuffix='')
text = gdir.read_text(basename_key, filesuffix='')
```

附录D 术语表 (Glossary)

本术语表定义了整个OGGM中使用的重要术语——包括冰川科学概念、 OGGM特有抽象概念、数值方法和气候建模术语。每个条目提供英文术语、 其中文翻译、定义以及其在OGGM中的使用背景。

最容易混淆的四组术语

术语	一句话区别
气候物质平衡 vs. 表观物质平衡 (apparent mass balance)	前者来自气候强迫和固定几何，后者为反演服务，把几何变化对应的通量散度也纳入考虑。
大地测量物质平衡 (geodetic mass balance) vs. 原位物质平衡	前者由DEM/ 卫星高程变化估算，覆盖广；后者来自地面观测，时间细但空间少。
动态预热 (dynamic spinup) vs. 平衡态实验	动态预热 (dynamic spinup) 匹配近期观测面积或体积，不保证严格平衡；平衡态实验要求长期净变化接近零。
GlacierDirectory vs. Flowline	GlacierDirectory是整条冰川的数据文件夹；Flowline是其中用于动力学的一维几何对象。

D.1 快速索引

A -- Ablation 至 AAR B -- Basal sliding 至 Border C -- Calving 至 CFL condition D -- Deformation 至 Dynamic spinup E -- ELA 至 Entity task F -- Flowline 至 fs G -- gdir 至 Gridded attributes H -- Hypsometry I -- Ice dynamics 至 Inversion L -- Lapse rate M -- Mass balance 至 Multiple flowlines O -- OGGM P -- Prepro level 至 PRCP R -- RGI 至 RGIId S -- SIA 至 Surge T -- Temperature index model 至 Tributary W -- Widths 至 Workflow

A

AAR 积累区比率

积累区比率 (Accumulation Area Ratio): 积累区面积与冰川总面积之比。在OGGM中, AAR由物质平衡模型计算, 用作冰川健康状况的诊断指标。AAR约为0.6通常对应于收支平衡 (均衡) 状态。

参见: ELA (平衡线高度)、Mass balance (物质平衡)、Accumulation area (积累区)

Ablation 消融

冰川通过融化、升华、崩解或风蚀而损失冰雪的过程。在OGGM中, 消融由温度指数 (度日) 模型计算, 为`melt_f`与 0°C 以上正温度异常的乘积。

参见: Mass balance (物质平衡)、Accumulation (积累)、Melt factor (融化因子)

Accumulation 积累

冰川通过降雪、雪崩或再冻结获得质量的过程。在OGGM的温度指数模型 (temperature-index model) 中, 积累主要由固态降水部分表示; 固态比例由 `temp_all_solid` 和 `temp_all_liq` 控制。

参见: Mass balance (物质平衡)、Ablation (消融)、Solid precipitation (固态降水)

Accumulation area 积累区

冰川在平衡线高度以上、年物质平衡为正的部分。OGGM通过冰川表面地形与ELA的交点来计算积累区面积。

参见: ELA (平衡线高度)、AAR (积累区比率)、Ablation area (消融区)

Apparent mass balance 表观物质平衡

物质平衡剖面, 当从末端向上积分时, 能够再现冰川各点处的观测冰通量。用作质量守恒反演的输入。与真实气候物质平衡不同, 因为它包含了冰川几何变化 (dH/dt) 的影响。

参见: Inversion (反演)、Mass-conservation inversion (质量守恒反演)、Flux (通量)

B

Basal sliding 基底滑动

冰川在冰-基岩界面处因冰床融水润滑而产生的滑移运动。在OGGM中, 滑动速度参数化为 $u_s = f_s * A * \tau^n * H$, 其中 `f_s` 是滑动参数 (`cfg.PARAMS['fs']`)。

参见: Deformation (变形)、fs (滑动参数)、SIA (浅冰近似)、Glen's law (格伦定律)

BASENAMES 文件基础名注册表

OGGM的内部注册表 (`cfg.BASENAMES`), 将逻辑文件名 (如'`gridded_data`') 映射到物理文件名 (如'`gridded_data.nc`') 及描述。由 `gdir.get_filepath()` 用于解析冰川目录 (GlacierDirectory) 中的文件路径。

参见: GlacierDirectory (冰川目录对象)、params.cfg (参数配置文件)

Bed topography (bed_h) 冰床地形

冰川冰床 (基岩表面) 的海拔高程 (m)。在反演前, `bed_h`是初始估计值 (通常是从表面进行抛物线外推的结果)。反演后, 它是求解出的基岩高程。

参见: Flowline (流线)、Surface elevation (表面高程)、Inversion (反演)

Border (prepro_border) 地图边界

定义局部地图时, 在冰川轮廓周围添加的网格像素数。较大的边界允许冰川在未来模拟中超出当前轮廓前进。典型值: 10像素 (相当于数百米)。

参见: Glacier grid (冰川格网)、define_glacier_region

C

Calving 冰川崩解

海洋或湖泊终止型冰川末端冰块脱离并形成冰山的过程。在OGGM核心模型中, 入海型冰川可使用水深依赖的`k-calving`参数化; `sandbox`中还包含若干替代方案。

参见: [Tidewater glacier](#) (入海型冰川)、[calving_k](#)、[calving_use_limiter](#)

Calving front 崩解前缘

入海型冰川末端的临海/临湖面, 冰山在此脱离。OGGM追踪崩解前缘位置, 并在末端网格单元处应用崩解法则。

参见: [Calving](#) (崩解)、[Tidewater glacier](#) (入海型冰川)、[Terminus](#) (末端)

calving_k 崩解系数

水深依赖型崩解参数化中的比例常数 (yr^{-1})。OGGM v1.6.3默认值为 0.6 yr^{-1} 。更高的值通常产生更强的崩解通量。

参见: [Calving](#) (崩解)、[params.cfg](#) (参数配置文件)

Catchment 集水区/流域

冰川表面上通过特定流线截面排水的区域。在OGGM中, 每条中心线/流线都有一个相关联的集水多边形。集水区划分是流线初始化的关键步骤。

参见: [Centerline](#) (中心线)、[Flowline](#) (流线)、[catchment_area](#)

Centerline 中心线

从冰川末端到源头的几何线条, 沿冰川流动方向追踪。中心线使用最小成本路径算法从DEM和冰川掩膜计算得出。它们构成了流线的基础骨架。

参见: [Flowline](#) (流线)、[compute_centerlines](#)、[Tributary](#) (支流)

CFL condition CFL 条件

Courant–Friedrichs–Lewy条件: 在显式 (或半隐式) 数值格式中对时间步长的稳定性约束。在OGGM中, CFL数控制自适应时间步长: dt 必须满足 $u * \text{dt} / \text{dx} \leq \text{CFL_number}$ 。默认CFL数: **0.02**。

参见: [cfl_number](#)、[FluxBasedModel](#)、[Adaptive time-stepping](#) (自适应时间步长)

D

Deformation (ice) 冰变形

冰川因应力作用下冰晶体内部蠕变而产生的运动。由格伦流动定律描述。在OGGM中, 变形速度是冰通量的主要成分, 基底滑动为补充。

参见: [Glen's law](#) (格伦定律)、[SIA](#) (浅冰近似)、[Basal sliding](#) (基底滑动)

Degree-day model 度日模型

一种经验性冰川融化模型: $\text{melt} = \text{melt_f} * \max(T - T_{\text{threshold}}, 0)$ 。它对输入数据要求较低, 适合大尺度长期模拟; 能量平衡模型物理过程更完整, 但需要更多气象变量。OGGM使用月度温度指数 (度日) 模型。

参见: [Temperature index model](#) (温度指数模型)、[melt_f](#) (融化因子)、[Mass balance model](#) (物质平衡模型)

Delta method 增量方法

未来气候预估的一种技术: 计算GCM未来期与GCM历史期之间的异常 (增量), 然后将此增量叠加到观测到的历史气候上。OGGM的 `historical_delta_method()` 实现此方法。

参见: [GCM](#) (全球气候模式)、[Climate projection](#) (气候预估)、[process_gcm_data](#)

DEM (Digital Elevation Model) 数字高程模型

地表高程的栅格化 (rasterization) 网格数据。OGGM使用DEM获取冰川表面地形, 从中导出坡度、坡向、中心线和初始冰床估计。支持多种DEM数据源: SRTM、ASTER、AW3D30、COP-DEM和自定义源。

参见: [process_dem](#)、[dem_source](#)、[glacier_grid](#) (冰川格网)

Diffusivity 扩散系数

在SIA中, 冰川演化被描述为一个非线性扩散方程: $\text{dH}/\text{dt} = \text{d}/\text{dx} (D * \text{dS}/\text{dx}) + \text{b_dot}$, 其中 D 是冰扩散系数, 正比于 $A * \tau^{(n-1)} * H^{(n+2)}$ 。OGGM的流线模型数值求解此方程。

参见: [SIA](#) (浅冰近似)、[FluxBasedModel](#)、[Glen's law](#) (格伦定律)

Dynamic spinup 动态预热

一种动力学初始化方法: 在近期历史气候下调整温度偏差和预热 (spinup) 时段, 使模拟冰川在目标年份匹配RGI附近的面积或体积。它用于减小直接从观测几何启动正演时的初始化冲击; 结果不是唯一解。

参见: [run_dynamic_spinup](#)、[Dynamic melt_f calibration](#)、[Equilibrium](#) (平衡状态)

E

ELA (Equilibrium Line Altitude) 平衡线高度

年度积累量与消融量相等（净物质平衡=0）的高程。ELA将积累区与消融区分开。在OGGM中，ELA由 `MassBalanceModel.get_ela()` 计算，即物质平衡剖面的零交点。

参见：AAR（积累区比率）、Mass balance（物质平衡）、Accumulation（积累）

Entity task 实体任务（entity task，冰川级任务）

以 `@entity_task` 装饰的单冰川处理函数。实体任务（entity task）是OGGM工作流的构建模块：它们以 `GlacierDirectory` 作为第一个参数，执行特定的处理步骤，并（可选）写入输出文件。装饰器提供自动日志记录、错误处理和依赖追踪。

参见： `GlacierDirectory`（冰川目录对象）、`execute_entity_task`、Global task（全局任务）、workflow（工作流）

Equilibrium 平衡状态

冰川表面积分的物质平衡为零且冰川几何不再变化的理想状态。OGGM中的质量守恒反演并不简单假设现代冰川严格平衡，而是通过表观物质平衡（apparent mass balance）把几何变化对应的通量散度纳入反演。动态预热（dynamic spinup）用于减小初始化瞬态，使模型状态更接近观测面积或体积，但不保证得到严格平衡态。

参见：Dynamic spinup（动态预热）、Mass-conservation inversion（质量守恒反演）、Steady state（稳态）

F

Flowline 流线

冰川沿流动方向的一维表示。流线由中心线导出，携带数值模拟所需的全部几何信息：表面高程、冰床高程、冰川宽度和网格间距。OGGM的动力学模型在流线（1D）上运行，而非在完整二维网格上运行。

参见：Centerline（中心线）、MixedBedFlowline（混合基底流线）、Flowline model（流线模型）、Multiple flowlines（多流线）

Flowline model 流线模型

沿一维流线求解冰川演化方程的数值模型。OGGM提供多种求解器：`SemiImplicitModel`（v1.6.3配置文件默认，半隐式，自适应时间步长）、`FluxBasedModel`（显式通量方案，支持多流线和崩解相关处理）和 `FileModel`（读取预计算输出）。

参见： `FluxBasedModel`、`SemiImplicitModel`、Flowline（流线）、Step（时间步进）

Flux (ice flux) 冰通量

单位时间通过冰川截面的冰体积，常以 $\text{m}^3 \text{ yr}^{-1}$ 或 $\text{m}^3 \text{ s}^{-1}$ 表示，具体取决于输出文件或源码上下文。冰通量是截面积与深度平均速度的乘积。在SIA中，通量与表面坡度和冰厚度的 $(n+2)$ 次方成正比。

参见：SIA（浅冰近似）、Diffusivity（扩散系数）、Mass conservation（质量守恒）、Inversion（反演）

Flux limiter 通量限制器

一种数值方法，通过将冰通量限制在物理合理值范围内，防止SIA通量计算中出现非物理振荡。OGGM在 `FluxBasedModel` 中使用通量限制器（flux limiter），以维持末端附近薄冰的数值稳定性。

参见： `FluxBasedModel`、CFL condition（CFL条件）、sia_fluxlim

fs (Sliding parameter) 滑动参数

控制基底滑动对总冰速度贡献的无量纲参数（0到1）。 $fs=0$ 表示无滑动（全部变形）， $fs=1$ 表示最大滑动。默认值为0（仅变形），对大多数冰川而言是保守的。

参见：Basal sliding（基底滑动）、Glen's law（格伦定律）、Flowline model（流线模型）

G

gdir (GlacierDirectory) 冰川目录对象

表示单个冰川及其关联数据文件的中央Python对象。由 `workflow.init_glacier_directories()` 创建或直接通过

`utils.GlacierDirectory()` 创建。提供读取、写入和访问冰川元数据及数据文件的方法。在OGGM代码中通常缩写为 `gdir`。

参见：BASENAMES（文件基础名注册表）、Entity task（实体任务）、Working directory（工作目录）

GCM (Global Climate Model) 全球气候模式

大气环流模式 (General Circulation Model): 基于物理学原理的全球气候系统数值模型。OGGM使用GCM输出 (温度与降水预估) 驱动未来冰川演化模拟, 通常通过增量方法实现。

参见: [Delta method](#) (增量方法)、[Climate projection](#) (气候预估)、[process_gcm_data](#)

Glacier grid 冰川格网

冰川的局部坐标系, 由 `define_glacier_region()` 定义并存储在 `glacier_grid.json` 中。一个以冰川为中心的投影网格 (通常为横轴墨卡托投影), 分辨率由冰川面积决定。

参见: [define_glacier_region](#)、[DEM](#) (数字高程模型)、[Grid resolution](#) (网格分辨率)

Glen's law / Glen's A / Glen's n 格伦流动定律

冰蠕变的本构关系: $\text{strain_rate} = A * \tau^n$, 其中A是速率因子 (温度相关), τ 是剪切应力, $n \sim 3$ 是幂律指数。OGGM在整个流程中使用格伦定律: 反演中 (用于关联厚度与通量) 和流线模型中 (用于计算冰速度)。默认值: $A = 2.4 \times 10^{-24} \text{ s}^{-1} \text{ Pa}^{-3}$, $n = 3$ 。

参见: [glen_a](#)、[glen_n](#)、[Deformation](#) (变形)、[SIA](#) (浅冰近似)

Global task 全局任务 (global task)

以 `@global_task` 装饰的任务, 对所有冰川集中操作而非逐个处理。示例包括下载参考数据集、汇总区域统计信息和初始化多进程池。

参见: [Entity task](#) (实体任务)、[workflow](#) (工作流)、[execute_entity_task](#)

Gridded attributes 格网属性

从DEM计算的逐像素地形变量: 坡度、坡向和到轮廓的距离。这些数据存储在 `gridded_data.nc` 中, 用于中心线计算和物质平衡分布。

参见: [DEM](#) (数字高程模型)、[gridded_data.nc](#)、[gridded_attributes](#)

H

Hypsometry 高度-面积分布

冰川面积作为海拔函数的分布。OGGM在流线初始化时计算高度-面积分布曲线, 并存储在 `hypsometry.csv` 中。高度-面积分布用于将气候数据分布到各高程, 并计算冰川整体物质平衡。

参见: [Flowline](#) (流线)、[hypsometry.csv](#)、[Mass balance](#) (物质平衡)

I

Ice dynamics 冰动力学

冰川流动的物理学, 由质量守恒、动量守恒和能量守恒控制。在OGGM中, 冰动力学被简化为浅冰近似 (SIA), 这是一种在水平尺度远大于冰厚 (纵横比远小于1) 的冰体上有效的简化。

参见: [SIA](#) (浅冰近似)、[Flowline model](#) (流线模型)、[Deformation](#) (变形)、[Basal sliding](#) (基底滑动)

Ice thickness / Ice thickness distribution 冰厚/ 冰厚分布

冰川冰的垂直维度。冰厚度是质量守恒反演的主要未知量。沿冰川和跨冰川的厚度分布决定了体积, 是动力学模拟的起点。

参见: [Inversion](#) (反演)、[Mass-conservation inversion](#) (质量守恒反演)、[volume](#) (体积)

Inversion 反演

从可观测物理量 (表面地形、现有表面速度观测和物质平衡) 推求冰厚度的过程。OGGM的主要反演方法是质量守恒反演, 该方法求解满足通量连续性的冰厚度分布, 给定表观物质平衡 (apparent mass balance)。

参见: [Mass-conservation inversion](#) (质量守恒反演)、[Apparent mass balance](#) (表观物质平衡)、[inversion_output.pkl](#)

L

Lapse rate 递减率

大气温度随海拔降低的速率。在OGGM中, 默认温度递减率为 -6.5 K/km (`temp_default_gradient`)。递减率用于将气候数据从参考高度降尺度到冰川的海拔范围。

参见: [Climate](#) (气候)、[temp_default_gradient](#)、[Refreezing](#) (再冻结)

M

Mass balance (specific, annual, monthly) 物质平衡 (比、年度、月度)

特定时段内冰川质量的净变化 (kg m^{-2})。比物质平衡是单位面积的质量变化。年度物质平衡是全年积分。月度物质平衡解析季节循环。OGGM的MassBalanceModel计算全部三种。

参见: [MassBalanceModel](#)、[get_monthly_mb](#)、[get_annual_mb](#)、[get_specific_mb](#)

Mass balance gradient 物质平衡梯度

物质平衡随海拔的变化率 (kg m^{-2} 每米高度)。更陡的梯度意味着低海拔处更多的消融和/或高海拔处更多的积累。该梯度是校准后温度指数模型 (temperature-index model) 的一个涌现属性。

参见: [Mass balance](#) (物质平衡)、[Temperature index model](#) (温度指数模型)、[ELA](#) (平衡线高度)

Mass-conservation inversion 质量守恒反演

OGGM计算冰厚度的主要方法。它求解连续性方程: $d(\text{flux})/dx = (\text{apparent mass balance}) * \text{width}$, 求解冰厚H, 利用SIA将通量与H关联。结果是一个满足质量守恒的物理一致性厚度剖面。

参见: [Inversion](#) (反演)、[Apparent mass balance](#) (表观物质平衡)、[mass_conservation_inversion](#)

melt_f (Melt factor) 融化因子

温度指数融化因子 (temperature-index melt factor; $\text{kg m}^{-2} \text{K}^{-1} \text{day}^{-1}$): 每个正温度日对应的融化量。melt_f是OGGM物质平衡模型中的主要可校准参数, 通常通过大地测量或原位物质平衡观测约束。

参见: [Temperature index model](#) (温度指数模型)、[Degree-day model](#) (度日模型)、[mb_calib.json](#)

MixedBedFlowline 混合基底流线

Flowline的子类, 处理混合冰床形状: 矩形、梯形和抛物线形截面。冰床横截面形状 (bed cross-section geometry) 影响冰厚度与截面积之间的关系, 进而影响通量和动力学响应。

参见: [Flowline](#) (流线)、[model_flowlines.pkl](#)、[Bed topography](#) (冰床地形)

Multiple flowlines 多流线

OGGM对有多个支流的冰川建模方法: 每个支流被建模为单独的一维流线, 在交汇处进行质量交换。由

`cfg.PARAMS['use_multiple_flowlines']` 控制。替代方案是单个合并流线。

参见: [Flowline](#) (流线)、[Tributary](#) (支流)、[use_multiple_flowlines](#)

O

OGGM (Open Global Glacier Model) 开放全球冰川模型

一个开源、社区驱动的冰川建模框架。OGGM提供从RGI轮廓到动力学模拟的完整管线, 采用模块化设计, 可在每个层面进行定制。用Python编写, 基于xarray、salem和rasterio构建。

P

params.cfg 参数配置文件

存储OGGM所有可调参数的中央配置文件。按章节组织 (输入输出、网格、气候、动力学、崩解等)。用户可以通过提供自己的params.cfg或通过 `cfg.PARAMS['param_name'] = value` 编程方式覆盖默认值。

参见: [cfg.initialize](#)、[cfg.PARAMS](#)、[附录B](#)

Prepro level (Preprocessing level) 预处理级别

表示冰川目录 (GlacierDirectory) 已完成的处理步骤的位掩码: 0=原始 (仅轮廓), 1=GIS任务完成, 2=中心线完成, 3=气候完成, 4=反演完成。由 `init_glacier_directories(from_prepro_level=N)` 用于从某一阶段恢复。

参见: [GlacierDirectory](#) (冰川目录对象)、[init_glacier_directories](#)、[diagnostics.json](#)

PRCP (Precipitation) 降水

在OGGM中，降水以月总量形式存储在气候文件中 ($\text{kg m}^{-2} \text{ month}^{-1}$)。温度决定固液比例，固态降水贡献于积累，液态降水假设立即流出。

参见: Accumulation (积累)、Climate (气候)、temp_melt、prcp_fac

R

RGI (Randolph Glacier Inventory) 伦道夫冰川编目

全球完整的冰川轮廓清单，为OGGM模拟提供边界条件。每个冰川是一个多边形，包含面积、质心坐标和末端类型等元数据。OGGM支持 RGI 6.0和7.0版本。

参见: RGIId、outlines.shp、rgi_version

RGIId 伦道夫冰川编目标识符

RGI中每个冰川的14字符唯一标识符。格式: RGI60-RR.SSSSS，其中RR是区域代码，SSSSS是该区域内的5位序列号。示例:

RGI60-11.00897 (阿尔卑斯山的Hintereisferner冰川)。

参见: RGI (伦道夫冰川编目)、GlacierDirectory.rgi_id

S

SIA (Shallow Ice Approximation) 浅冰近似

对冰川流动完整Stokes方程的一种简化，当水平尺度远大于冰厚 (纵横比远小于1) 时有效。在SIA中，冰流仅由局部表面坡度驱动，纵向应力梯度被忽略。OGGM的流线模型基于SIA构建。

参见: Ice dynamics (冰动力学)、Flowline model (流线模型)、Diffusivity (扩散系数)、Glen's law (格伦定律)

Solid precipitation 固态降水

以雪形式降落的降水，贡献于积累。在OGGM的温度指数模型 (temperature-index model) 中，固态比例在temp_all_solid和temp_all_liq之间从1 (全部固态) 过渡到0 (全部液态)。

参见: Accumulation (积累)、PRCP (降水)、temp_all_solid、temp_all_liq

Specific mass balance (SMB) 比物质平衡

冰川整体平均物质平衡，通常以每年毫米水当量 (water equivalent, w.e.; mm w.e. yr^{-1}) 或 $\text{kg m}^{-2} \text{ yr}^{-1}$ 表示。SMB通过对冰川高度-面积分布积分物质平衡剖面计算。

参见: Mass balance (物质平衡)、get_specific_mb、dmdtda_mmwea

Spinup 预热

模型初始化或预热过程。参见Dynamic spinup (动态预热)。

参见: Dynamic spinup (动态预热)、run_dynamic_spinup

Step (model step) 时间步进

流线模型的一个时间步长。FluxBasedModel.step(dt) 方法将冰川几何向前推进dt年，求解SIA方程。自适应时间步长意味着实际dt可能小于请求值以满足CFL条件。

参见: FluxBasedModel、CFL condition (CFL条件)、Adaptive time-stepping (自适应时间步长)

Surface elevation 表面高程

冰川表面的海拔 (m a.s.l.)，由DEM测量获得。surface_h是每个Flowline对象中的关键数组，是约束反演和模型的主要观测测量。

参见: Flowline (流线)、DEM (数字高程模型)、Bed topography (冰床地形)、surface_h

Surge 冰川跃动

冰川流速在较短时间 (数月到数年) 内急剧加速的现象，通常以多个十年为周期发生在跃动型冰川上。OGGM不明确建模冰川跃动；跃动型冰川可能需要特殊处理 (例如从物质平衡校准中排除)。

参见: Tidewater glacier (入海型冰川)、Terminus (末端)

T

Temperature index model (TI model) 温度指数模型 (temperature-index model)

一类将融化与温度相关联的物质平衡模型。OGGM的 `MonthlyTIModel` 是月度温度指数模型 (`MonthlyTIModel`, `temperature-index model`)，融化与月均正温度成正比，积累是固态降水部分。该模型有一个主要可校准参数: `melt_f`。

参见: `Degree-day model` (度日模型)、`MonthlyTIModel`、`melt_f` (融化因子)、`Mass balance` (物质平衡)

temp_bias 温度偏差

校准参数: 施加于气候时间序列的温度偏差 (K)，使模拟的物质平衡与观测一致。`temp_bias`用于校正气候数据集中的系统偏差。

参见: `mb_calib.json`、`melt_f` (融化因子)、`prcp_fac`

temp_melt 融化温度阈值

融化计算使用的温度阈值 (°C)。OGGM v1.6.3默认值为-1.0°C。降水固液相比例由 `temp_all_solid` 和 `temp_all_liq` 控制，而不是由 `temp_melt` 控制。

参见: `Solid precipitation` (固态降水)、`temp_all_solid`、`temp_all_liq`

Terminus 末端/冰舌

冰川的下游终点。末端类型 (陆地、海洋、湖泊、冰架) 决定适用哪些过程: 陆地终止型冰川仅经历融化; 海洋终止型冰川还经历崩解和海底融化。末端类型记录在RGI中。

参见: `Tidewater glacier` (入海型冰川)、`Calving` (崩解)、`TermType`、`terminus_type`

Tidewater glacier 入海型冰川

其末端到达海洋 (或大型湖泊) 的冰川，通过崩解和海底融化失去质量。OGGM对入海型冰川进行特殊处理: 激活崩解参数化，反演中考虑海平面以下冰床的可能性。

参见: `Calving` (崩解)、`Terminus` (末端)、`is_tidewater`、`calving_k`

Tributary 支流/支冰川

汇入较大主干冰川的较小冰川分支。在OGGM中，支流在中心线计算期间被检测到，可以建模为在交汇点处向主流线贡献质量的独立流线。

参见: `Centerline` (中心线)、`Flowline` (流线)、`Multiple flowlines` (多流线)、`flows_to`

t_star 参考年份

物质平衡校准的参考年份: 历史记录中模拟物质平衡等于参考 (大地测量) 物质平衡的年份。`t_star`在校准期间计算，用于锚定时间序列。

参见: `mb_calib.json`、`Mass balance calibration` (物质平衡校准)、`temp_bias`

W

Widths (glacier widths) 冰川宽度

冰川沿流线各点的截面宽度 (m)。宽度从面积-海拔分布计算得出，对质量守恒至关重要: 给定厚度下，更宽的截面承载更多的通量。存储为 `fl.widths` (在网格中心) 和 `fl.widths_m` (在交错网格点上)。

参见: `Flowline` (流线)、`initialize_flowlines`、`Hypsometry` (高度-面积分布)

Workflow 工作流

OGGM的编排层 (`oggm.workflow`)，协调多个冰川上实体任务 (`entity task`) 的执行。提供初始化、任务执行和结果汇总等函数。工作流模块透明地处理多进程、日志和错误恢复。

参见: `Entity task` (实体任务)、`execute_entity_task`、`init_glacier_directories`

Working directory 工作目录

所有OGGM数据的根目录，通过 `cfg.PATHS['working_dir']` 设置。包含各单个冰川数据 (`per_glacier/`)、气候数据、DEM瓦片和日志文件的子目录。

参见: `cfg.initialize`、`cfg.PATHS`、`GlacierDirectory` (冰川目录对象)

D.2 补充术语（快速参考）

术语	中文	简要定义
<i>Absolute tolerance</i>	绝对容差	数值比较中允许的最大绝对误差 (<i>atol</i>)
<i>Adaptive time-stepping</i>	自适应时间步长	自动调整 <i>dt</i> 以满足CFL条件
<i>Boundary condition</i>	边界条件	域边界的约束；OGGM在末端使用零厚度边界
<i>Divergence</i>	散度	$d(\text{flux})/dx$ ：冰通量沿流线的空间变化
<i>Fjord</i>	峡湾	入海型冰川通常终止的狭窄海洋入口
<i>Flotation</i>	浮力/ 漂浮	当冰厚小于浮力阈值时 ($H < \rho_w/\rho_i * D$)
<i>Geodetic mass balance</i>	大地测量物质平衡 (<i>geodetic mass balance</i>)	通过重复DEM差分测量得到的质量变化
<i>GeoTIFF</i>	地理标记TIFF	嵌入地理参考信息的TIFF栅格 (<i>CRS</i> , <i>geotransform</i>)
<i>Glacier mask</i>	冰川掩膜	表示哪些单元属于冰川 (1) 或不属于 (0) 的二值网格
<i>Interpolation</i>	插值	从离散采样估计中间点值 (如不同高度上的气候值)
<i>Land-terminating</i>	陆地终止型	末端位于陆地上的冰川 (RGI中 <i>TermType</i> =0)
<i>LEGOS</i>	LEGOS 格式	崩解模块中使用的内部类 (<i>sandbox</i>)
<i>Logging</i>	日志	OGGM使用Python的 <i>logging</i> 模块；级别由 <i>cfg.set_logging_config</i> 设置
<i>Marine-terminating</i>	海洋终止型	末端到达海洋的冰川 (RGI中 <i>TermType</i> =1)
<i>NetCDF</i>	网络通用数据格式	自描述二进制科学网格数据格式；OGGM广泛使用
<i>Pickle</i>	序列化	Python的二进制序列化格式，用于存储任意对象
<i>prcp_fac</i>	降水修正因子	校准期间降水的乘法校正因子
<i>PROJ.4</i>	坐标参考系统	坐标参考系规范字符串格式
<i>Relaxation</i>	松弛	用于稳定迭代解的数值技术
<i>Relative tolerance</i>	相对容差	数值比较中允许的最大相对误差 (<i>rtol</i>)
<i>Salem</i>	Salem库	OGGM用于网格和地图操作的地理空间库
<i>Shapefile</i>	矢量文件	ESRI 地理空间矢量数据格式 (多边形、点、线)
<i>Shop</i>	数据商店	OGGM的模块化数据提供系统 (<i>oggm.shop.*</i>)
<i>Smoothing</i>	平滑	应用空间滤波器去除噪声；用于反演输出滤波
<i>SRTM</i>	航天雷达地形测绘任务	航天雷达地形任务DEM；OGGM的默认高程数据源
<i>Staggered grid</i>	交错网格	变量定义在单元边缘 (<i>widths_m</i>) 与单元中心 (<i>widths</i>) 的网格
<i>Steady state</i>	稳态	不随时间变化的系统状态 ($dH/dt = 0$)
<i>Thermal regime</i>	热力状态	冰川内部温度分布；在OGGM中进行了简化

术语	中文	简要定义
Volume (glacier volume)	冰川体积	冰总体积 (m^3 或 km^3); volume_km3是关键模型输出
w.e. (water equivalent)	水当量 (water equivalent, w.e.)	以等效水深表示的质量; 换算: $1\text{ kg m}^{-2} = 1\text{ mm w.e.}$
xarray	xarray库	标记多维数组的Python库; OGGM的主要数据容器
Zero-thickness boundary	零厚度边界	冰川末端 $H=0$; 关键的模型边界条件

致谢与版权许可

致谢

本书《OGGM 源码深度解析》的写作建立在OGGM (Open Global Glacier Model) 项目长期开放、透明和高质量的软件开发基础之上。作者谨向OGGM团队、OGGM e.V. 以及所有OGGM贡献者表示诚挚感谢。

OGGM不仅提供了一套可复现的开源冰川演化模拟框架，也为全球冰川学界展示了开放科学、可审查模型和社区协作在冰川变化研究中的重要价值。本书所讨论的源码结构、工作流、物质平衡校准、冰厚反演、流线动力学、配置系统与诊断工具，均受益于OGGM开发者多年积累的科学设计和工程实现。

作者也感谢OGGM官方文档、教程、示例和社区讨论所提供的学习资源。本书希望在中文语境中补充这些官方资源，帮助中国冰川研究人员更准确地理解OGGM的模型假设、代码逻辑和适用边界，并促进OGGM在中国及高亚洲冰川研究中的规范使用。

OGGM项目版权与许可

OGGM源码由OGGM e.V. 和OGGM贡献者持有版权。根据OGGM官方仓库中的许可证文件，其版权声明为：**Copyright (c) 2014–2021, OGGM e.V. and OGGM Contributors**。

截至本书写作和校订时，OGGM官方源码采用三条款BSD许可证 (BSD 3-Clause License)。该许可证允许在满足许可证条件的前提下，以源码或二进制形式使用、复制、修改和再分发OGGM。许可证的核心要求包括：保留原版权声明、许可证条件和免责声明；以二进制形式再分发时在文档或随附材料中包含相应声明；未经书面许可，不得使用版权持有人或贡献者名称为衍生产品背书或推广。

OGGM许可证还明确说明，软件按“原样”提供，版权持有人和贡献者不对适销性、特定用途适用性等作出明示或默示担保，也不对因使用软件导致的直接或间接损失承担责任。完整、准确和具有法律效力的许可文本请以OGGM官方仓库中的LICENSE文件为准：

<https://github.com/OGGM/oggm/blob/master/LICENSE.txt>

本书与OGGM官方项目的关系

本书是作者面向中文读者撰写的学习和解析材料，不是OGGM官方文档，也不代表OGGM团队或OGGM e.V. 的官方立场。书中对OGGM源码、模型流程和冰川学概念的解释，旨在帮助读者学习和研究；若本书内容与OGGM官方文档、源码或许可证存在差异，应以OGGM官方发布内容为准。

本书中涉及的OGGM名称、函数名、模块名、配置项、文件名和少量代码片段均用于技术说明、教学和学术交流目的。读者在科研、教学或软件开发中使用OGGM时，应同时遵守OGGM官方许可证、引用规范和相关数据源许可要求。

本书版权说明

除明确归属于OGGM项目、第三方数据源、第三方软件包或其他文献资料的内容外，本书中文解析文字、章节组织和学习路径设计由作者徐强强撰写和整理。引用、转载或改编本书内容时，请注明作者、书名和在线地址：

徐强强：《OGGM 源码深度解析》

<https://jonhxu.github.io/oggm-source-code-deep-dive/>

本书难免仍有疏漏。若读者或OGGM开发者发现术语、物理解释、代码理解或版权说明中存在问题，欢迎提出修改建议。作者将持续修订，以便让本书更好地服务于中国冰川学界对OGGM的学习、使用和贡献。